

УНИВЕРСАЛЬНОЕ ПОСОБИЕ ПО АЛГОРИТМАМ И СТРУКТУРАМ
ПОЛНЫЙ ИСХОДНЫЙ КОД ПРОЕКТА (ВСЕ ФАЙЛЫ, ВСЕ

Дата генерации: 15.09.2026, 17:04:16
Файлов исходного кода: 111
Суммарный объём кода: 1.97 MB
Глав/билетов в пособии: 36

ОГЛАВЛЕНИЕ: ГЛАВЫ И БИЛЕТЫ

1. 1. Дерево отрезков с операциями на отрезках [id: ...]
2. 2. Двумерная разреженная матрица (Sparse Table)
3. 3. Декартово дерево (Treap) [id: treap]
4. 4. Splay-дерево [id: splay-tree]
5. 5. Двумерная матрица префиксных сумм [id: pref]
6. 6. Графы. Представление графов, DFS, BFS [id: ...]
7. 7. Графы. Планарные. Покраска [id: graph-plana]
8. 7. Графы. Планарные. Покраска [id: planarity-e]
9. 8. Графы. Компоненты связности [id: graph-comp]
10. 9. Графы. Топологическая сортировка [id: top-s]
11. 10. Графы. Компоненты сильной связности (SCC)
12. 11. Графы. Мосты [id: graph-bridges]
13. 11. Графы. Мосты [id: bridges-code]
14. 12. Графы. Точки сочленения [id: graph-articul]
15. 13. Графы. Эйлеров цикл [id: graph-euler]
16. 13. Графы. Эйлеров цикл [id: euler-path-vs-cyc]
17. Билет 14. Дейкстра [id: dijkstra]
18. Билет 15. Форд-Беллман [id: bellman-ford]
19. Билет 16. Флойд-Уоршелл [id: floyd]
20. 17. Графы. Ускорения поиска [id: pathfinding-a]
21. 17. Графы. Поиск кратчайшего пути. Ускорения [id: ...]
22. 18. Графы. Остовное дерево. Краскал [id: mst-k]
23. 18–20. Графы. Остовное дерево: Краскал, Прима, Б
24. 19. Графы. Остовное дерево. Прима [id: mst-pri]
25. 20. Графы. Остовное дерево. Борувка [id: mst-b]

- 18. src/data/content_temp.ts
- 19. src/data/content.ts
- 20. src/data/debugRunner.ts
- 21. src/data/glossary.ts
- 22. src/data/graphContent.ts
- 23. src/data/quizzes.ts
- 24. src/data/vizStepBus.ts
- 25. src/data/vizSync.ts

Билеты (учебный контент)

- 26. src/data/tickets/ticket1_segtree.ts
- 27. src/data/tickets/ticket14_20.ts
- 28. src/data/tickets/ticket18_20_mst.ts
- 29. src/data/tickets/ticket21_24.ts
- 30. src/data/tickets/ticket6_13.ts

Интерактивные компоненты и симуляторы

- 31. src/components/ArticulationPointsViz.tsx
- 32. src/components/BellmanFordViz.tsx
- 33. src/components/BfsViz.tsx
- 34. src/components/ChapterImage.tsx
- 35. src/components/ChapterNav.tsx
- 36. src/components/ChapterQuiz.tsx
- 37. src/components/ChapterView.tsx
- 38. src/components/ComplexityViz.tsx
- 39. src/components/DfsBridgesSimulator.tsx
- 40. src/components/DijkstraViz.tsx
- 41. src/components/DPVisualizer.tsx
- 42. src/components/EulerSimulator.tsx
- 43. src/components/FloydViz.tsx
- 44. src/components/GraphComponentsViz.tsx
- 45. src/components/GraphTraversalViz.tsx
- 46. src/components/HeapViz.tsx
- 47. src/components/hints/demoKit.tsx
- 48. src/components/hints/demosExtra.tsx
- 49. src/components/hints/demosGraph.tsx
- 50. src/components/hints/demosStruct.tsx
- 51. src/components/hints/MiniDemo.tsx

86. src/layout/header.html

Утилиты

87. src/utils/cn.ts

88. src/utils/exportHtml.ts

89. src/utils/ticketLinks.ts

Скрипты сборки и экспорта

90. scripts/audit-coverage.mjs

91. scripts/audit-terms.mjs

92. scripts/export/build-guide-pdf.mjs

93. scripts/export/build-pdf.mjs

94. scripts/export/capture-viz-shots.mjs

95. scripts/export/collect-code.mjs

96. scripts/export/common.mjs

97. scripts/export/html-to-blocks.mjs

98. scripts/export/render-pages.mjs

99. scripts/verify/browser-checks.tsx

100. scripts/verify/check-links.mjs

101. scripts/verify/check-reference-code.mjs

102. scripts/verify/dom-env.mjs

103. scripts/verify/live-matrix.test.mjs

104. scripts/verify/probe-compiler.tsx

105. scripts/verify/README.md

106. scripts/verify/run-browser-checks.mjs

107. scripts/verify/run-probe.mjs

CI / автоматизация

108. .github/workflows/deploy-pages.yml

109. .github/workflows/rebuild-pdf.yml

110. .github/workflows/verify.yml

Прочее

111. .ai_skills/universal_guide_authoring/SKILL.md

```
contents: read
pages: write
id-token: write
```

```
concurrency:
  group: pages
  cancel-in-progress: true
```

```
jobs:
  build:
    name: Build static site
    runs-on: ubuntu-latest
    steps:
      - name: Checkout
        uses: actions/checkout@v6

      - name: Setup Node.js
        uses: actions/setup-node@v7
        with:
          node-version: 24
          cache: npm

      - name: Install dependencies
        run: npm ci --no-audit --no-fund

      - name: Build for the repository subpath
        env:
          VITE_BASE: /algv0/
        run: npm run build

      - name: Make GitHub Pages fallback
        run: cp dist/index.html dist/404.html

      - name: Upload Pages artifact
        uses: actions/upload-pages-artifact@v5
        with:
          path: ./dist
```

```
+ group: pages
+ cancel-in-progress: true
+
+ jobs:
+   build:
+     name: Build static site
+     runs-on: ubuntu-latest
+     steps:
+       - name: Checkout
+         uses: actions/checkout@v6
+
+       - name: Setup Node.js
+         uses: actions/setup-node@v7
+         with:
+           node-version: 24
+           cache: npm
+
+       - name: Install dependencies
+         run: npm ci --no-audit --no-fund
+
+       - name: Build for the repository subpath
+         env:
+           VITE_BASE: /algv0/
+         run: npm run build
+
+       - name: Make GitHub Pages fallback
+         run: cp dist/index.html dist/404.html
+
+       - name: Upload Pages artifact
+         uses: actions/upload-pages-artifact@v5
+         with:
+           path: ./dist
+
+   deploy:
+     name: Publish to GitHub Pages
+     needs: build
+     runs-on: ubuntu-latest
+     environment:
```

```
      required: false
      default: "150"

concurrency:
  group: rebuild-pdf-`${{ github.ref }}
  cancel-in-progress: true

permissions:
  contents: write

jobs:
  rebuild-pdf:
    name: код → txt → png → pdf
    runs-on: ubuntu-latest
    # не реагируем на собственный коммит бота
    if: "!contains(github.event.head_commit.message, '[
    steps:
      - name: Checkout
        uses: actions/checkout@v6
        with:
          fetch-depth: 0
          persist-credentials: true

      - name: Setup Node.js
        uses: actions/setup-node@v7
        with:
          node-version: "24"
          cache: npm

      - name: Install ImageMagick + DejaVu fonts
        run: |
          sudo apt-get update -qq
          sudo apt-get install -y --no-install-recommen
          # На Ubuntu политика ImageMagick иногда запре
          # разрешаем то, что нужно пайплайну (text -> p
          for policy in /etc/ImageMagick-6/policy.xml /
            if [ -f "$policy" ]; then
```

```
      name: full-code-export
      path: |
        public/export/full_code_all_pages.pdf
        public/export/full_code_all_pages.txt
      if-no-files-found: error
      retention-days: 30

- name: Upload artifacts (PNG-страницы)
  uses: actions/upload-artifact@v7
  with:
    name: full-code-pages-png
    path: tmp/export-pages/*.png
    if-no-files-found: error
    retention-days: 7

- name: Commit rebuilt export back to the branch
  run: |
    git config user.name "github-actions[bot]"
    git config user.email "41898282+github-action@users.noreply.github.com"
    git add -- public/export/full_code_all_pages.*
    if git diff --cached --quiet; then
      echo "Экспорт не изменился — коммит не нужен"
      exit 0
    fi
    git commit -m "chore(export): пересборка full-code-export"
    git push origin "HEAD:${GITHUB_REF_NAME}"
  ....

### Patch относительно `main`

```diff
diff --git a/.github/workflows/rebuild-pdf.yml b/.github/workflows/rebuild-pdf.yml
index bdc3c33..ff49c0c 100644
--- a/.github/workflows/rebuild-pdf.yml
+++ b/.github/workflows/rebuild-pdf.yml
@@ -42,15 +42,15 @@ jobs:

 steps:
```

Универсальное пособие • полный исходный код

-----  
### Полное содержимое после изменений

```
````html
<!doctype html>
<html lang="ru" class="dark bg-slate-950 text-slate-100" >
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, height=device-height" />
    <link rel="icon" type="image/svg+xml" href="%BASE_URL%/public/favicon.svg" />
    <title> Дискретка для тупых – Интерактивный гид по дискреткам
  </head>
  <body class="bg-slate-950 text-slate-100 min-h-screen" >
    <div id="root"></div>
    <script type="module" src="/src/main.tsx"></script>
  </body>
</html>
````
```

### Patch относительно `main`

```
````diff
diff --git a/index.html b/index.html
index cdea4c7..a44c135 100644
--- a/index.html
+++ b/index.html
@@ -3,6 +3,7 @@
   <head>
     <meta charset="UTF-8" />
     <meta name="viewport" content="width=device-width, height=device-height" />
+    <link rel="icon" type="image/svg+xml" href="%BASE_URL%/public/favicon.svg" />
     <title> Дискретка для тупых – Интерактивный гид по дискреткам
   </head>
   <body class="bg-slate-950 text-slate-100 min-h-screen" >
````
```

## `public/favicon.svg`

### Полное содержимое после изменений



```

import { SimulatorHub } from "../components/SimulatorHub
import { PythonCompiler } from "../components/PythonComp

export default function App() {
 const [activeTab, setActiveTab] = useState<"guide" |
 const [activeChapterId, setActiveChapterId] = useStat
 const [modalVizId, setModalVizId] = useState<string |

 const index = Math.max(
 0,
 chapters.findIndex((c) => c.id === activeChapterId)
);
 const activeChapter = chapters[index] ?? chapters[0];
 const prev = index > 0 ? chapters[index - 1] : undefin
 const next = index < chapters.length - 1 ? chapters[in

 const goTo = useCallback((id: string) => {
 setActiveChapterId(id);
 window.scrollTo({ top: 0, behavior: "smooth" });
 }, []);

 // Стрелки ← → листают темы (как на обучающих сайтах)
 useEffect(() => {
 const onKey = (e: KeyboardEvent) => {
 const target = e.target as HTMLInputElement | null;
 if (target && /^(INPUT|TEXTAREA|SELECT)$/.test(target))
 if (e.metaKey || e.ctrlKey || e.altKey) return;
 if (e.key === "ArrowRight" && next) goTo(next.id);
 if (e.key === "ArrowLeft" && prev) goTo(prev.id);
 };
 window.addEventListener("keydown", onKey);
 return () => window.removeEventListener("keydown", onKey);
 }, [goTo, next, prev]);

 const progress = useMemo(() => ((index + 1) / chapters.length) * 100, [index, chapters.length]);

 return (
 <div className="min-h-screen bg-slate-950 text-slate-50"

```

```

 <ChapterView
 key={activeChapter.id}
 chapter={activeChapter}
 prev={prev}
 next={next}
 index={index}
 total={chapters.length}
 onGo={goTo}
 onOpenSimulator={setModalVizId}
 />
 </main>
</div>
</>
) : activeTab === "simulator" ? (
 <main className="px-4 sm:px-6 lg:px-10 py-6 lg:py-8" >
 <SimulatorHub
 onOpen={setModalVizId}
 onGoChapter={({id}) => {
 setActiveTab("guide");
 goTo(id);
 }}
 />
 </main>
) : (
 <PythonCompiler />
)
}

<SimulatorModal vizId={modalVizId} onClose={() => {
 setModalVizId(null);
}} />

<footer className="p-8 text-center text-slate-600" >
 © 2026 Universal Educational Guide. Интерактивное
</footer>
</div>
);
}
....

```

## `src/components/Navbar.tsx`

### Полное содержимое после изменений

```
````tsx
import React, { useState } from 'react';
import { BookOpen, Cpu, Sparkles, FileDown, FileText, Cpu } from 'lucide-react';
import { chapters } from '../data/content';
import { downloadBookHtml } from '../utils/exportHtml';

interface NavbarProps {
  activeTab: 'guide' | 'simulator' | 'compiler';
  setActiveTab: (tab: 'guide' | 'simulator' | 'compiler') => void;
}

export const Navbar: React.FC<NavbarProps> = ({ activeTab }) => {
  const [busy, setBusy] = useState(false);

  const saveBook = async () => {
    setBusy(true);
    try {
      await downloadBookHtml(chapters);
    } finally {
      setBusy(false);
    }
  };

  return (
    <header className="bg-slate-900 border-b border-slate-800">
      <div className="max-w-7xl mx-auto px-4 sm:px-6 lg:px-8">
        <div className="flex items-center gap-3 w-full">
          <div className="bg-gradient-to-tr from-indigo-500 to-purple-500 p-2">
            <Sparkles className="w-6 h-6" />
          </div>
          <div className="min-w-0 flex-1">
            <h1>Универсальное пособие • полный исходный код</h1>
            <h2>Содержимое после изменений</h2>
          </div>
        </div>
      </div>
    </header>
  );
};
```

```

        className={`flex-1 lg:flex-none flex items-
uration-200 whitespace-nowrap ${
            activeTab === 'compiler'
                ? 'bg-emerald-600 text-white shadow-lg
                : 'text-slate-400 hover:text-white'
        }}
    >
    <Terminal className="w-4 h-4" /> Python
</button>
<a
    id="download-pdf-btn"
    href={` ${import.meta.env.BASE_URL}export/full
download="full_code_all_pages.pdf"
    target="_blank"
    rel="noreferrer"
    className="flex items-center justify-center
over:bg-emerald-600/20 border border-emerald
    title="Скачать полный PDF сборник всех стра
    >
    <FileDown className="w-4 h-4" /> Скачать PD
</a>
<a
    id="download-txt-btn"
    href={` ${import.meta.env.BASE_URL}export/full
download="full_code_all_pages.txt"
    target="_blank"
    rel="noreferrer"
    className="flex items-center justify-center
: bg-sky-600/20 border border-sky-500/30 tra
    title="Скачать полный TXT файл со всем кодом
    >
    <FileText className="w-4 h-4" /> TXT
</a>
<button
    id="download-html-btn"
    onClick={saveBook}
    disabled={busy}
    className="flex items-center justify-center

```

```

+         <button
+             id="tab-compiler-btn"
+             onClick={() => setActiveTab('compiler')}
+             className={`flex-1 lg:flex-none flex items
duration-200 whitespace-nowrap ${
+                 activeTab === 'compiler'
+                 ? 'bg-emerald-600 text-white shadow-lg
+                 : 'text-slate-400 hover:text-white'
+             }`}
+         >
+         <Terminal className="w-4 h-4" /> Python
+     </button>
+     <a
+         id="download-pdf-btn"
+         href="/export/full_code_all_pages.pdf"
+         href={`${import.meta.env.BASE_URL}export/full
+         download="full_code_all_pages.pdf"
+         target="_blank"
+         rel="noreferrer"
@@ -73,7 +84,7 @@ export const Navbar: React.FC<NavbarP
    </a>
    <a
        id="download-txt-btn"
        href="/export/full_code_all_pages.txt"
        href={`${import.meta.env.BASE_URL}export/full
        download="full_code_all_pages.txt"
        target="_blank"
        rel="noreferrer"
    ....

```

`src/components/PythonCompiler.tsx`

Полное содержимое после изменений

```

````tsx
import { useCallback, useState } from "react";
import { CheckCircle2, ExternalLink, Info, Loader2, Play

```

```

export function PythonCompiler() {
 const [code, setCode] = useState(STARTER_CODE);
 const [stdin, setStdin] = useState("");
 const [output, setOutput] = useState("Нажмите «Запустить»");
 const [running, setRunning] = useState(false);
 const [runtimeReady, setRuntimeReady] = useState(false);
 const [runtimeMessage, setRuntimeMessage] = useState("");

 const runCode = useCallback(async () => {
 if (running) return;

 setRunning(true);
 setOutput("");
 setRuntimeMessage(runtimeReady ? "Выполняю код..." : "Готово: программа ничего не выполнила");

 const stdout: string[] = [];
 const stderr: string[] = [];
 const inputLines = stdin.split(/\r?\n/);

 try {
 const runtime = await getPythonRuntime();
 setRuntimeReady(true);
 setRuntimeMessage("Python готов — код выполняется");

 runtime.setStdout({ batched: (message) => stdout.push(message) });
 runtime.setStderr({ batched: (message) => stderr.push(message) });
 runtime.setStdin({ stdin: () => (inputLines.length > 0 ? inputLines.shift() : null) });
 await runtime.runPythonAsync(code);

 const text = [...stdout, ...stderr].join("");
 setOutput(text || "Готово: программа ничего не выполнила");
 } catch (error) {
 const captured = [...stdout, ...stderr].join("");
 setOutput(`${captured}${captured && !captured.endsWith('.') ? '.' : ''}`);
 setRuntimeMessage("Не удалось выполнить код — подробности в консоли");
 } finally {
 setRunning(false);
 }
 }, [code, stdin]);
}

```

```


 </div>
 <div className="flex items-center gap-2">
 <button
 type="button"
 onClick={reset}
 className="inline-flex items-center gap-2 text-slate-800 transition-colors"
 >
 <RotateCcw className="w-3.5 h-3.5" />
 </button>
 <button
 type="button"
 onClick={() => void runCode()}
 disabled={running}
 className="inline-flex items-center gap-2 text-white text-xs font-bold transition-colors"
 >
 {running ? <Loader2 className="w-3.5 h-3.5" /> : "Запустить"}
 </button>
 </div>
 </div>
 </div>

 <textarea
 value={code}
 onChange={(event) => setCode(event.target.value)}
 onKeyDown={(event) => {
 if ((event.ctrlKey || event.metaKey) && event.key === "Enter") {
 event.preventDefault();
 void runCode();
 }
 }}
 spellCheck={false}
 aria-label="Код Python"
 className="block w-full min-h-[360px] resize-none focus:ring-2 focus:ring-inset focus:ring-slate-800"
 placeholder="Напишите Python-код..."
 />

```

```
 <div className="flex gap-2 rounded-xl border border-gray-200 p-4">
 <Info className="w-4 h-4 shrink-0 text-indigo-600">
 Ctrl или Cmd + Enter запускает код. Введите код в поле ниже
 </div>
 <div className="flex gap-2 rounded-xl border border-gray-200 p-4">
 <Info className="w-4 h-4 shrink-0 text-amber-600">
 Первый запуск скачивает примерно несколько килобайт кода
 </div>
 <div className="flex gap-2 rounded-xl border border-gray-200 p-4">
 <Info className="w-4 h-4 shrink-0 text-emerald-600">
 Код выполняется в браузере и не отправляется на сервер
 </div>
 </div>
 </div>
 </div>
 </main>
);
 }

```

### Patch относительно `main`

```
````diff
diff --git a/src/components/PythonCompiler.tsx b/src/components/PythonCompiler.tsx
new file mode 100644
index 00000000..1113e0f
--- /dev/null
+++ b/src/components/PythonCompiler.tsx
@@ -0,0 +1,205 @@
+import { useCallback, useState } from "react";
+import { CheckCircle2, ExternalLink, Info, Loader2, Plus } from "lucide-react";
+
+const PYODIDE_VERSION = "0.27.7";
+const PYODIDE_MODULE_URL = `https://cdn.jsdelivr.net/pyodide/v${PYODIDE_VERSION}/bin/pyodide.js`;
+const PYODIDE_INDEX_URL = `https://cdn.jsdelivr.net/pyodide/v${PYODIDE_VERSION}/index.html`;
+
+const STARTER_CODE = `# Первый запуск загрузит Python и библиотеки
+name = "алгоритмы"
+
+def main():
+    print("Привет, мир!")
+    return "Ваш код выполнен успешно!"
+
+main()
+`;
```



```

+
+ const runCode = useCallback(async () => {
+   if (running) return;
+
+   setRunning(true);
+   setOutput("");
+   setRuntimeMessage(runtimeReady ? "Выполняю код..." :
+
+   const stdout: string[] = [];
+   const stderr: string[] = [];
+   const inputLines = stdin.split(/\r?\n/);
+
+   try {
+     const runtime = await getPythonRuntime();
+     setRuntimeReady(true);
+     setRuntimeMessage("Python готов – код выполняется");
+
+     runtime.setStdout({ batched: (message) => stdout
+     runtime.setStderr({ batched: (message) => stderr
+     runtime.setStdin({ stdin: () => (inputLines.length
+     await runtime.runPythonAsync(code);
+
+     const text = [...stdout, ...stderr].join("");
+     setOutput(text || "Готово: программа ничего не выв
+   } catch (error) {
+     const captured = [...stdout, ...stderr].join("")
+     setOutput(`${captured}${captured && !captured.en
+     setRuntimeMessage("Не удалось выполнить код – пр
+   } finally {
+     setRunning(false);
+   }
+ }, [code, stdin, running, runtimeReady]);
+
+ const reset = () => {
+   setCode(STARTER_CODE);
+   setStdin("");
+   setOutput("Нажмите «Запустить», чтобы увидеть резу.
+   };

```

```

late-800 transition-colors"
+
+       >
+       <RotateCcw className="w-3.5 h-3.5" />
+       </button>
+       <button
+       type="button"
+       onClick={() => void runCode()}
+       disabled={running}
+       className="inline-flex items-center
-60 text-white text-xs font-bold transition-colors"
+       >
+       {running ? <Loader2 className="w-3.5
+       {running ? "Запускаю..." : "Запустить"}
+       </button>
+       </div>
+     </div>
+
+     <textarea
+     value={code}
+     onChange={(event) => setCode(event.target
+     onKeyDown={(event) => {
+       if ((event.ctrlKey || event.metaKey) &
+       event.preventDefault();
+       void runCode();
+     }
+     }}
+     spellCheck={false}
+     aria-label="Код Python"
+     className="block w-full min-h-[360px] re
e-none focus:ring-2 focus:ring-inset focus:ring-emera
+     placeholder="Напишите Python-код..."
+     />
+
+     <div className="border-t border-slate-800
+     <label className="block px-4 pt-3 text-[
+       Ввод для input() • по одной строке на
+     </label>
+     <textarea

```

```

+         </div>
+         <div className="flex gap-2 rounded-xl border
+             <Info className="w-4 h-4 shrink-0 text-eme
+             <span>Код выполняется в браузере и не отпра
+         </div>
+     </div>
+ </div>
+ </main>
+ );
+}
+...

```

`vite.config.ts`

Полное содержимое после изменений

```

````typescript
import path from "path";
import { fileURLToPath } from "url";
import tailwindcss from "@tailwindcss/vite";
import react from "@vitejs/plugin-react";
import { defineConfig } from "vite";
import { viteSingleFile } from "vite-plugin-singlefile"

const __filename = fileURLToPath(import.meta.url);
const __dirname = path.dirname(__filename);

// https://vite.dev/config/
export default defineConfig({
 // На GitHub Pages приложение живёт в /algv0/, локальн
 // VITE_BASE можно переопределить, если репозиторий б
 base: process.env.VITE_BASE || "/",
 plugins: [react(), tailwindcss(), viteSingleFile()],
 resolve: {
 alias: {
 "@": path.resolve(__dirname, "src"),
 },
 },
},

```

Универсальное пособие • полный исходный код

-----  
КАТЕГОРИЯ: Конфигурация проекта | СТРОК: 12 | РАЗМЕР: 7  
-----

# AI Agent Custom Instructions

You are the author of a Universal Educational Guide (Универсальное

**\*\*CRITICAL DIRECTIVE\*\***:

Before you start creating new chapters, editing content or using the `view_file` tool to read the skill instructions located at `.ai_skills/universal_guide_authoring/SKILL.md``

This file contains the mandatory rich HTML templates, styles and components required to maintain the high quality of this universal guide.

Do NOT generate plain markdown chapters or invent new HTML tags.

-----  
ФАЙЛ 3/111: GEMINI.md

КАТЕГОРИЯ: Конфигурация проекта | СТРОК: 16 | РАЗМЕР: 2  
-----

# Gemini App Canvas Instructions

Ты — AI-ассистент, работающий в Gemini App Canvas (или Gemini App Canvas).  
При запросе пользователя создать или сгенерировать новую страницу, ты **\*\*строго\*\*** придерживаешься готовых шаблонов разметки, стилей и компонентов.

## Правила для малых страниц (Билетов / Глав)

- \*\*Единый стиль:\*\*** Всегда используй Tailwind CSS и HTML5.
- \*\*Структура JSON/TS:\*\*** Каждая новая малая страница должна быть в формате `{ "id": "...", "content": ... }` для вставки в ``src/data/content.json``.
- \*\*Строгая структура контента:\*\***
  - Заголовок с цветным бейджем (например, ``bg-indigo-100``).
  - Определение/правило в центре (``border-blue-500`` или ``border-purple-500``).
  - Обязательная сетка аналогий (2 колонки: неформальный язык / строгий язык).

---

ФАЙЛ 6/111: package.json

КАТЕГОРИЯ: Конфигурация проекта | СТРОК: 54 | РАЗМЕР: 2

---

```
{
 "name": "algv0-exam-guide",
 "private": true,
 "version": "0.0.0",
 "type": "module",
 "scripts": {
 "dev": "vite",
 "build": "vite build",
 "preview": "vite preview",
 "audit:content": "node scripts/audit-coverage.mjs",
 "audit:terms": "node scripts/audit-terms.mjs",
 "export:txt": "node scripts/export/collect-code.mjs",
 "export:png": "node scripts/export/render-pages.mjs",
 "export:pdf": "node scripts/export/build-pdf.mjs",
 "export": "npm run export:txt && npm run export:png",
 "export:guide": "node scripts/export/build-guide-pdf.mjs",
 "export:shots": "node scripts/export/capture-viz-shots.mjs",
 "verify:unit": "node scripts/verify/live-matrix.test.mjs",
 "verify:code": "node scripts/verify/check-reference.mjs",
 "verify:links": "node scripts/verify/check-links.mjs",
 "verify:dom": "node scripts/verify/run-browser-check.mjs",
 "verify:probe": "node scripts/verify/run-probe.mjs",
 "verify": "npm run verify:unit && npm run verify:links",
 },
 "dependencies": {
 "clsx": "2.1.1",
 "lucide-react": "^1.21.0",
 "motion": "^12.40.0",
 "pdfkit": "^0.20.1",
 "react": "19.2.6",
 "react-dom": "19.2.6",
 }
}
```

Универсальное пособие • полный исходный код

```

npm run dev # предпросмотр на http://0.0.0.0:5
npm run build # сборка в dist/ (single-file)
````
```

Экспорт кода: код → txt → png → pdf

Пайплайн собирает ****весь**** исходный код репозитория в о

| Команда | Шаг | Результат |
|----------------------|-----------|---|
| `npm run export:txt` | код → txt | `public/export/full_code_all_pages.txt` |
| `npm run export:png` | txt → png | `tmp/export-pages/` |
| `npm run export:pdf` | png → pdf | `public/export/full_code_all_pages.pdf` |
| `npm run export` | всё сразу | txt + png + pdf |

Скрипты лежат в `scripts/export/`. Требуется ImageMagick

```
```bash  
sudo apt-get install -y imagemagick fonts-dejavu-core
````
```

Настройки через переменные окружения:

- * `EXPORT\_DENSITY=200` – DPI растеризации (по умолчанию 300)
- * `EXPORT\_GRAYSCALE=1` – оттенки серого вместо 1-битного

Автоматизация (CI)

`github/workflows/rebuild-pdf.yml` запускается ****на каж**
пересобирает TXT → PNG → PDF, кладёт PNG-страницы и док
и коммитит обновлённые `public/export/full\_code\_all\_pages`
(с меткой `[skip ci]`, чтобы не заиклиться).

Приложение отдаёт эти файлы по ссылкам `/export/full\_code`

Аудит контента

```
```bash
```

## ## Заголовки

- \* Заголовки секций: Используют стандартный тег `<h2>` и
- \* Подзаголовки: Используют тег `<h3>`.

## ## Текст и параграфы

- \* Обычный текст содержится в тегах `<p>`.
- \* Блоки "Шиза" (Аналогии) и другая текстовая инфографика параграфы `<p>`.

## ## Математические формулы

В приложении используется MathJax.

- \* В самом исходном коде страницы (и внутри тегов) формулы.
- \* Таким образом, при извлечении текстового содержимого математически остаются в валидном Markdown-формате латеха (и

## ## Картинки и визуализации (Схемы)

Вместо обычных `<img>` в приложении используются векторы

- \* Они помещены внутрь оберток с классом `<div class="img">`
- \* Все отрисовки находятся внутри векторного тега `<svg>`
- \* Подпись к картинке, если она есть, идет сразу после `<img>`

## ## Списки

- \* Стандартные списки оформлены через `<ul class="list-n">`

## ## Валидный экспорт в Markdown (встроенный)

Экспорт перехватывает основные структурные компоненты HTML

1. Заголовки `<h2>` и `<h3>` заменяются на `##` и `###`
2. Элементы `.analogy-badge` получают выделение маркера
3. Теги `<svg>` заменяются строкой-заглушкой `[ Иллюстрация ]`.
4. Элементы списка `<li>` получают маркер `-`.
5. Результат извлекается в виде текста и чистится от лишнего

Универсальное пособие • полный исходный код

```

* `stack-dfs` → `StackViz.tsx` — **Стек и DFS**
* `queue-bfs` → `QueueViz.tsx` + `BfsViz.tsx` — **Очередь и BFS**
* `heap-beam-search` → `HeapViz.tsx` — **Куча и лучевой поиск**
* `segment-trees` → `SegmentTreeVisualizer.tsx` — **Деревья отрезков**
* `dynamic-programming` → `DPVisualizer.tsx` — **Динамическое программирование**

```

## Уровень 3. Есть 2D, но НЕТ бытовой аналогии («сухари»)

Глава	id	Комментарий
15. Планарность: K5, K3,3 и формула Эйлера	`planarity-euler`	т «на пальцах».
16. Флойд (Флойд–Уоршелл)	`floyd`	Есть матричный алгоритм
17. Алгоритм Джонсона	`johnson-algo`	Перевзвешивание
Обзор: Кратчайшие пути и Графы	`intro`	Вводная страница
Эйлер: Путь ≠ Цикл	`euler-path-vs-cycle`	Ключевая концепция

## Уровень 4. Есть аналогия, но НЕТ 2D-визуализации («сухари»)

> Замечание: «5. Двумерная матрица префиксных сумм» пока закрыто

Глава	id	Чего не хватает
3. Декартово дерево (Treap)	`treap`	Нет ни визуализации, ни кода
8. Графы. Компоненты связности	`graph-components`	Нет кода
24. Классы сложности, сведение задач	`complexity-classes`	Нет кода

## Сводная таблица по всем главам

#	Глава	Аналогий	2D-виз.	SVG	Картинка
---	---	---	---	---	---



ФАЙЛ 9/111: tsconfig.json

КАТЕГОРИЯ: Конфигурация проекта | СТРОК: 32 | РАЗМЕР: 6

---

```
{
 "compilerOptions": {
 "target": "ES2020",
 "useDefineForClassFields": true,
 "lib": ["ES2020", "DOM", "DOM.Iterable"],
 "module": "ESNext",
 "skipLibCheck": true,
 "types": ["node"],

 /* Bundler mode */
 "moduleResolution": "bundler",
 "allowImportingTsExtensions": true,
 "resolveJsonModule": true,
 "isolatedModules": true,
 "noEmit": true,
 "jsx": "react-jsx",

 /* Path mapping */
 "baseUrl": ".",
 "paths": {
 "@/*": ["src/*"]
 },

 /* Linting */
 "strict": true,
 "noUnusedLocals": true,
 "noUnusedParameters": true,
 "noFallthroughCasesInSwitch": true
 },
 "include": ["src", "vite.config.ts"]
}
```

```
 },
 });
}
```

---

## РАЗДЕЛ: ЯДРО ПРИЛОЖЕНИЯ

---

ФАЙЛ 11/111: src/App.tsx

КАТЕГОРИЯ: Ядро приложения | СТРОК: 218 | РАЗМЕР: 11.3 КБ

```
import { useCallback, useEffect, useMemo, useRef, useState } from "react";
import { PanelLeftClose, PanelLeftOpen } from "lucide-react";
import { chapters } from "../data/content";
import { Navbar } from "../components/Navbar";
import { Sidebar } from "../components/Sidebar";
import { MobileToc } from "../components/MobileToc";
import { ChapterView } from "../components/ChapterView";
import { ChapterNav } from "../components/ChapterNav";
import { SimulatorModal } from "../components/hints/SimulatorModal";
import { LiteGuide } from "../components/LiteGuide";
import { PythonCompiler } from "../components/PythonCompiler";
```

```
/**
```

```
 * «Лёгкий» режим для печати и внешних парсеров: `?lite`
```

```
 * или `?lite=1&topic=mst` (одна тема). Без навигации, без поиска.
```

```
 */
```

```
function readLiteMode(): { lite: boolean; topic?: string } {
 if (typeof window === "undefined") return { lite: false };
 const q = new URLSearchParams(window.location.search);
 const lite = q.has("lite") || q.get("view") === "lite";
 return { lite, topic: q.get("topic") ?? undefined };
}
```

```
export default function App() {
 const liteMode = useMemo(readLiteMode, []);
```

```

 setSidebarMode((m) => {
 const next = m === "wide" ? "compact" : m === "co
 window.localStorage.setItem("sidebarMode", next);
 return next;
 });
 }, []));
const sidebarCollapsed = sidebarMode === "hidden";

// Десктопный отступ контента = ширине панели компиля
const contentRef = useRef<HTMLDivElement>(null);
useEffect(() => {
 const apply = () => {
 const el = contentRef.current;
 if (!el) return;
 el.style.paddingRight = compilerOpen && window.in
 };
 apply();
 window.addEventListener("resize", apply);
 return () => window.removeEventListener("resize", a
}, [compilerOpen, compilerWidth]);

const index = Math.max(
 0,
 chapters.findIndex((c) => c.id === activeChapterId)
);
const activeChapter = chapters[index] ?? chapters[0];
const prev = index > 0 ? chapters[index - 1] : undefi
const next = index < chapters.length - 1 ? chapters[i

const goTo = useCallback((id: string) => {
 setActiveChapterId(id);
 window.scrollTo({ top: 0, behavior: "smooth" });
}, []);

// Открыть компилятор и подогнать визуализацию вверх:
const openCompilerWithViz = useCallback(() => {
 setCompilerOpen(true);
 requestAnimationFrame(() => {

```

```

<div className="flex flex-col lg:flex-row max-w-
 /* Сайдбар только на десктопе — на мобильном
 <div
 id="sidebar-root"
 className={`hidden lg:block shrink-0 border
 -all duration-200 print:hidden ${sidebarCol
 " : "lg:w-56"}` }
 >
 {!sidebarCollapsed && (
 <Sidebar selectedChapterId={activeChapterId}
)}
 </div>
 /* Вкладка-переключатель содержания у левого
 <button
 type="button"
 onClick={toggleSidebar}
 className="hidden lg:flex items-center gap-1
 border-slate-700 bg-slate-900/90 px-1 py-3
 aria-label={sidebarMode === "wide" ? "Сжать
 ание"}
 title={sidebarMode === "wide" ? "Сжать содер
 }
 >
 {sidebarCollapsed ? <PanelLeftOpen className="
 <span className="text-[10px] font-bold [wri
 </button>

 <main id="main-content" className="flex-1 min
 /* Шапка главы с быстрыми переходами */
 <div className="flex items-center justify-b
 <div className="min-w-0">
 <div className="text-[10px] uppercase t
 {activeChapter.category || "Универсал
 </div>
 <h2 className="text-base sm:text-xl fon
 {activeChapter.title}
 </h2>
 </div>
 </div>

```

```
 onHeightChange={setCompilerHeight}
 onClose={() => setCompilerOpen(false)}
 />
)}
 </div>
);
}
```

---

ФАЙЛ 12/111: src/hooks/useTermHints.ts

КАТЕГОРИЯ: Ядро приложения | СТРӨК: 129 | РАЗМЕР: 5.1 K

---

```
import { useEffect } from "react";
import { GLOSSARY_LABELS } from "../data/glossary";

/**
 * Ограничители, чтобы текст не рябил, но и чтобы подсказки
 *
 * Считаем не «сколько раз встретился термин», а «сколько
 * написание». Иначе в главе про splay первые же «Splay
 * лимит, и слова «Zig-Zag», «вращений», «AVL-дереве» о
 */
const MAX_PER_SURFACE = 2;
const MAX_PER_TERM = 8;

const SKIP_SELECTOR = "code, pre, script, style, a, text";

/** Числовые константы TreeWalker: глобального `NodeFile
const SHOW_TEXT = 4;
const FILTER_ACCEPT = 1;
const FILTER_REJECT = 2;

const escapeRe = (s: string) => s.replace(/[\.*\+\?\^\$\{\}\(\)]/g, "\\$1");

/** Одна большая регулярка из всех меток: длинные раньше
function buildRegex(): RegExp {
```

```

 re.lastIndex = 0;
 if (!re.test(text)) continue;
 re.lastIndex = 0;

 const frag = document.createDocumentFragment();
 let last = 0;
 let m: RegExpExecArray | null;

 while ((m = re.exec(text)) !== null) {
 const surface = m[1].toLowerCase();
 const id = labelToId.get(surface);
 if (!id) continue;

 const usedTerm = perTerm.get(id) ?? 0;
 const usedSurface = perSurface.get(surface) ?? 0;
 if (usedTerm >= MAX_PER_TERM || usedSurface >= MAX_PER_SURFACE) continue;
 perTerm.set(id, usedTerm + 1);
 perSurface.set(surface, usedSurface + 1);

 if (m.index > last) frag.appendChild(document.createElement("span"));
 const span = document.createElement("span");
 span.className = "term-hint";
 span.dataset.termId = id;
 span.tabIndex = 0;
 span.setAttribute("role", "button");
 span.setAttribute("aria-label", `Подсказка: ${m[1]}`);
 span.textContent = m[1];
 frag.appendChild(span);
 last = m.index + m[1].length;
 }

 if (last === 0) continue;
 if (last < text.length) frag.appendChild(document.createElement("span"));
 textNode.parentNode?.replaceChild(frag, textNode);
}

/**

```

```
.neon-glow-blue {
 box-shadow: 0 0 15px rgba(59, 130, 246, 0.5);
}
.neon-glow-emerald {
 box-shadow: 0 0 15px rgba(16, 185, 129, 0.5);
}
.neon-glow-rose {
 box-shadow: 0 0 15px rgba(244, 63, 94, 0.5);
}
.neon-glow-yellow {
 box-shadow: 0 0 20px rgba(234, 179, 8, 0.8);
}
.no-scrollbar::-webkit-scrollbar {
 display: none;
}
.no-scrollbar {
 scrollbar-width: none;
}
}

@keyframes pulse-gold {
 0%,
 100% {
 stroke: #eab308;
 stroke-width: 5;
 filter: drop-shadow(0 0 2px rgba(234, 179, 8, 0.5));
 }
 50% {
 stroke: #fef08a;
 stroke-width: 8;
 filter: drop-shadow(0 0 12px rgba(234, 179, 8, 0.9));
 }
}

.animate-pulse-gold {
 animation: pulse-gold 1.5s infinite;
}
```

```
 color: #fff;
 outline: none;
}
@media (hover: none) {
 .term-hint {
 border-bottom-style: solid;
 background: rgba(99, 102, 241, 0.12);
 }
}

/* Ссылка на другой билет («...берите Форда–Беллмана (билет
.ticket-link {
 color: #a5b4fc;
 border-bottom: 1px dotted rgba(165, 180, 252, 0.7);
 text-decoration: none;
 border-radius: 3px;
 padding: 0 1px;
 cursor: pointer;
 transition:
 background-color 0.15s ease,
 color 0.15s ease;
}
.ticket-link:hover,
.ticket-link:focus-visible {
 background: rgba(99, 102, 241, 0.25);
 color: #fff;
 outline: none;
}

/* Контент главы приходит готовым HTML – чиним его под
.chapter-body {
 max-width: 100%;
 overflow-wrap: anywhere;
}

.chapter-body :where(section, div, p, li) {
 min-width: 0;
}
```



```
 line-height: 1.3 !important;
 }
 .chapter-body :where(h3) {
 font-size: 1.05rem !important;
 line-height: 1.35 !important;
 }
 .chapter-body :where(.p-6, .p-8) {
 padding: 0.9rem !important;
 }
 .chapter-body :where(.text-3xl) {
 font-size: 1.4rem !important;
 }
 .chapter-body :where(.text-2xl) {
 font-size: 1.2rem !important;
 }
 .chapter-body :where(.text-xl) {
 font-size: 1.05rem !important;
 }
}

/* Custom scrollbar styling for dark theme */
::-webkit-scrollbar {
 width: 8px;
 height: 8px;
}
::-webkit-scrollbar-track {
 background: #0f172a;
}
::-webkit-scrollbar-thumb {
 background: #334155;
 border-radius: 4px;
}
::-webkit-scrollbar-thumb:hover {
 background: #475569;
}

/* Индикатор длительности кадра в пошаговых визуализаци.
@keyframes demoProgress {
```

```
main {
 padding: 0 !important;
 max-width: none !important;
}

/* текст темы — чёрным по белому, без теней и плашек
article.chapter-body .prose,
article.chapter-body .prose * {
 color: #111827 !important;
 background-color: transparent !important;
 background-image: none !important;
 box-shadow: none !important;
 text-shadow: none !important;
 border-color: #d1d5db !important;
}

article.chapter-body .prose pre,
article.chapter-body .prose code {
 background-color: #f3f4f6 !important;
 color: #111827 !important;
 font-size: 9pt;
}

article.chapter-body .prose table {
 border-collapse: collapse;
}

article.chapter-body .prose th,
article.chapter-body .prose td {
 border: 1px solid #9ca3af !important;
}

article.chapter-body .prose h2,
article.chapter-body .prose h3,
article.chapter-body .prose h4 {
 page-break-after: avoid;
}
```

Универсальное пособие • полный исходный код

---

ФАЙЛ 14/111: src/main.tsx

КАТЕГОРИЯ: Ядро приложения | СТРОК: 11 | РАЗМЕР: 230 В

---

```
import { StrictMode } from "react";
import { createRoot } from "react-dom/client";
import "./index.css";
import App from "./App";

createRoot(document.getElementById("root")!).render(
 <StrictMode>
 <App />
 </StrictMode>
);
```

---

ФАЙЛ 15/111: src/types.ts

КАТЕГОРИЯ: Ядро приложения | СТРОК: 9 | РАЗМЕР: 153 В

---

```
export interface Chapter {
 id: string;
 title: string;
 type: "html" | "markdown";
 content: string;
 description?: string;
 category?: string;
}
```

---

ФАЙЛ 16/111: src/vite-env.d.ts

КАТЕГОРИЯ: Ядро приложения | СТРОК: 19 | РАЗМЕР: 324 В

---

```
/// <reference types="vite/client" />
```

```
<div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-white">
 </div>
```

```
<div class="space-y-8">
```

```
 <div class="bg-slate-700/50 p-6 rounded-xl border">
 <h3 class="text-xl font-bold text-blue-400">
```

```
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-blue-300 italic">
 <p class="text-xl font-mono text-white">
 <p class="text-sm font-mono text-slate-300">
] [k] </p>
 </div>
```

```
<p class="text-slate-300 text-sm mb-4">Ячейка
лины
">i. Любой отрезок накрывается двумя
перация идемпотентна: min,
, побитовые И и ИЛИ.</p>
```

```
<div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-indigo-300 italic">
 <p class="text-xl font-mono text-white">
 <p class="text-sm font-mono text-slate-300">
вет = op из четырёх блоков kx × ky, накрыва
</div>
```

```
<p class="text-slate-300 text-sm mb-4">Двумя
по столбцам. Отсюда и цена: уровней становится
s="font-mono text-emerald-300">log N
```

```
<div class="grid grid-cols-1 sm:grid-cols-3">
 <div class="bg-slate-900 border border-slate-700">
 <p class="text-[10px] uppercase tracking-wider">
```

```

 <div class="absolute -top-3 left-4 border-rose-500 shadow-md">
 Аналогия 2: Торт-пирамидка
 </div>
 <p class="text-slate-300 text-sm mb-4">
 ия, зато красивый, как пирамидка: каждый из
 сех блоков $2^{kx} \times 2^{ky}$), и ответ на запрос
 ой операции нет: крем обратно в торт не запы
 $n(x, x) = x$), и цена $O(1)$ – сто коржей вмес
 <div class="rounded-lg bg-slate-950">
 <div>
 $n \ O(1)$ </div>
 <div>
 = 100 копий матрицы</div>
 </div>
 <p class="text-[12px] text-slate-400">
 шает). Менять элементы тоже нельзя. Сумму ч
 </div>
 </div>
 <p class="text-slate-400 text-sm mt-4">Поря
 ерению) → 2D: построение (уровни <span
 ос (прямоугольник раскладывается на чет
 </div>

<div class="bg-slate-700/50 p-6 rounded-xl border">
 <h3 class="text-xl font-bold text-rose-400">
 <p class="text-slate-300 text-sm mb-4">Эти
 за
 они решают разные задачи.</p>

<div class="bg-slate-800/70 p-5 rounded-lg">
 <p class="text-sm font-bold text-white">
 <p class="text-slate-300 text-sm mb-3">
 цы, у разреженной таблицы – размер, умножен
 <table class="w-full text-xs text-left">
 <thead>
 <tr class="text-slate-500 border">
 <th class="py-1.5 pr-2 font
```

Почему префиксная тонкая

</div>

<p class="text-slate-300 text-sm mb-4" style="border: 1px solid #000; padding: 5px; display: inline-block; width: 100%;">P[i][j]</p>  
ругой прямоугольник выражается через четыре

</div>

<div class="bg-slate-900 p-6 rounded-lg">

<div class="absolute -top-3 left-4 border-rose-500 shadow-md">

Почему разреженная жирная

</div>

<p class="text-slate-300 text-sm mb-4" style="border: 1px solid #000; padding: 5px; display: inline-block; width: 100%;">ум на его части не вычислишь. Приходится хранить матрицу. log N × log M</p>. На ка

</div>

</div>

<p class="text-slate-300 text-sm mt-4">Вывод: **лишние** – перекрывающиеся блоки посчитают дважды. **лишние** – у минимума нет обратной операции. Стрелки

</div>

<div class="bg-slate-700/50 p-6 rounded-xl border-rose-500/50">

<h3 class="text-xl font-bold text-amber-400">Проблема разреженных матриц

<div class="bg-rose-950/30 p-5 rounded-lg border-rose-500/50">

<ul class="list-disc pl-5 space-y-2 text-slate-300">

<li><b>Сумма через разреженную таблицу. Ограничение: префиксные (билет 5).</b></li>

<li><b>Память.</b> N = M = 1000 даём матрицу, считайте ячейки, потом пишите код.</li>

<li><b>Один log вместо двух.</b> В матрице st[0][0] st[0][1] st[1][0] st[1][1] st[2][0] st[2][1] st[3][0] st[3][1] st[4][0] st[4][1] st[5][0] st[5][1] st[6][0] st[6][1] st[7][0] st[7][1] st[8][0] st[8][1] st[9][0] st[9][1] st[10][0] st[10][1] st[11][0] st[11][1] st[12][0] st[12][1] st[13][0] st[13][1] st[14][0] st[14][1] st[15][0] st[15][1] st[16][0] st[16][1] st[17][0] st[17][1] st[18][0] st[18][1] st[19][0] st[19][1] st[20][0] st[20][1] st[21][0] st[21][1] st[22][0] st[22][1] st[23][0] st[23][1] st[24][0] st[24][1] st[25][0] st[25][1] st[26][0] st[26][1] st[27][0] st[27][1] st[28][0] st[28][1] st[29][0] st[29][1] st[30][0] st[30][1] st[31][0] st[31][1] st[32][0] st[32][1] st[33][0] st[33][1] st[34][0] st[34][1] st[35][0] st[35][1] st[36][0] st[36][1] st[37][0] st[37][1] st[38][0] st[38][1] st[39][0] st[39][1] st[40][0] st[40][1] st[41][0] st[41][1] st[42][0] st[42][1] st[43][0] st[43][1] st[44][0] st[44][1] st[45][0] st[45][1] st[46][0] st[46][1] st[47][0] st[47][1] st[48][0] st[48][1] st[49][0] st[49][1] st[50][0] st[50][1] st[51][0] st[51][1] st[52][0] st[52][1] st[53][0] st[53][1] st[54][0] st[54][1] st[55][0] st[55][1] st[56][0] st[56][1] st[57][0] st[57][1] st[58][0] st[58][1] st[59][0] st[59][1] st[60][0] st[60][1] st[61][0] st[61][1] st[62][0] st[62][1] st[63][0] st[63][1] st[64][0] st[64][1] st[65][0] st[65][1] st[66][0] st[66][1] st[67][0] st[67][1] st[68][0] st[68][1] st[69][0] st[69][1] st[70][0] st[70][1] st[71][0] st[71][1] st[72][0] st[72][1] st[73][0] st[73][1] st[74][0] st[74][1] st[75][0] st[75][1] st[76][0] st[76][1] st[77][0] st[77][1] st[78][0] st[78][1] st[79][0] st[79][1] st[80][0] st[80][1] st[81][0] st[81][1] st[82][0] st[82][1] st[83][0] st[83][1] st[84][0] st[84][1] st[85][0] st[85][1] st[86][0] st[86][1] st[87][0] st[87][1] st[88][0] st[88][1] st[89][0] st[89][1] st[90][0] st[90][1] st[91][0] st[91][1] st[92][0] st[92][1] st[93][0] st[93][1] st[94][0] st[94][1] st[95][0] st[95][1] st[96][0] st[96][1] st[97][0] st[97][1] st[98][0] st[98][1] st[99][0] st[99][1] st[100][0] st[100][1] st[101][0] st[101][1] st[102][0] st[102][1] st[103][0] st[103][1] st[104][0] st[104][1] st[105][0] st[105][1] st[106][0] st[106][1] st[107][0] st[107][1] st[108][0] st[108][1] st[109][0] st[109][1] st[110][0] st[110][1] st[111][0] st[111][1] st[112][0] st[112][1] st[113][0] st[113][1] st[114][0] st[114][1] st[115][0] st[115][1] st[116][0] st[116][1] st[117][0] st[117][1] st[118][0] st[118][1] st[119][0] st[119][1] st[120][0] st[120][1] st[121][0] st[121][1] st[122][0] st[122][1] st[123][0] st[123][1] st[124][0] st[124][1] st[125][0] st[125][1] st[126][0] st[126][1] st[127][0] st[127][1] st[128][0] st[128][1] st[129][0] st[129][1] st[130][0] st[130][1] st[131][0] st[131][1] st[132][0] st[132][1] st[133][0] st[133][1] st[134][0] st[134][1] st[135][0] st[135][1] st[136][0] st[136][1] st[137][0] st[137][1] st[138][0] st[138][1] st[139][0] st[139][1] st[140][0] st[140][1] st[141][0] st[141][1] st[142][0] st[142][1] st[143][0] st[143][1] st[144][0] st[144][1] st[145][0] st[145][1] st[146][0] st[146][1] st[147][0] st[147][1] st[148][0] st[148][1] st[149][0] st[149][1] st[150][0] st[150][1] st[151][0] st[151][1] st[152][0] st[152][1] st[153][0] st[153][1] st[154][0] st[154][1] st[155][0] st[155][1] st[156][0] st[156][1] st[157][0] st[157][1] st[158][0] st[158][1] st[159][0] st[159][1] st[160][0] st[160][1] st[161][0] st[161][1] st[162][0] st[162][1] st[163][0] st[163][1] st[164][0] st[164][1] st[165][0] st[165][1] st[166][0] st[166][1] st[167][0] st[167][1] st[168][0] st[168][1] st[169][0] st[169][1] st[170][0] st[170][1] st[171][0] st[171][1] st[172][0] st[172][1] st[173][0] st[173][1] st[174][0] st[174][1] st[175][0] st[175][1] st[176][0] st[176][1] st[177][0] st[177][1] st[178][0] st[178][1] st[179][0] st[179][1] st[180][0] st[180][1] st[181][0] st[181][1] st[182][0] st[182][1] st[183][0] st[183][1] st[184][0] st[184][1] st[185][0] st[185][1] st[186][0] st[186][1] st[187][0] st[187][1] st[188][0] st[188][1] st[189][0] st[189][1] st[190][0] st[190][1] st[191][0] st[191][1] st[192][0] st[192][1] st[193][0] st[193][1] st[194][0] st[194][1] st[195][0] st[195][1]</

```

 else
 st[r][c][kx][ky] = min(

```

```

// запрос минимума на прямоугольнике (R1,C1)..(R2,C2)
int kx = log2(R2 - R1 + 1);
int ky = log2(C2 - C1 + 1);
int ans1 = min(st[R1][C1][kx][ky], st[R1][C2 - (1 << ky) + 1][C1][kx][ky]);
int ans2 = min(st[R2 - (1 << kx) + 1][C1][kx][ky], st[R2][C1][kx][ky]);
int minimum = min(ans1, ans2);

```

Четыре запроса к четырем углам запроса и вместе с ним дают ответ. Но.

Сколько это памяти в числах

- 100 × 100: уровни 7 × 7 ≈ 49
- 1000 × 1000: уровни 10 × 10 = 100
- Префиксная сумма для той же матрицы

Почему? Потому что для каждого уровня нужно хранить только один бит (или несколько сотен на сторону), а запросов очень много.

- [Префиксная сумма](#) – вторая половина этого сравнения:
- [Суммирование](#)

```
<div class="text-center my-6">
 <p class="text-slate-400 text-xs uppercase">
 <p class="my-1 font-bold leading-none">
 И
 </p>
 <p class="text-slate-300 font-mono text">
</div>
```

```
<div class="grid grid-cols-1 xl:grid-cols-2">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 l
rder border-emerald-500 shadow-md">
 Аналогия 1: Split – таможня
 </div>
 <p class="text-slate-300 text-sm">S
х». Спуск от корня: узел, который вместе с
скрытия – досматривается только одна ве
 </div>
```

```
 <div class="bg-slate-900 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 l
border-rose-500 shadow-md">
 Аналогия 2: Merge – стыковка
 </div>
 <p class="text-slate-300 text-sm">M
аг – одно сравнение: чей у у корня больше,
сохраняет порядок x). Итого $O(h)$ сравнений
 </div>
 </div>
```

```
<div class="bg-slate-900/40 rounded-lg bord
 <p class="text-slate-300 text-sm mb-2">
 <p class="text-slate-400 text-sm mb-1">
быстрее $n \cdot \log n$.</p>
 <p class="text-slate-400 text-sm mb-1">
 <p class="text-slate-400 text-sm">• «У6
```



```
<summary class="p-4 font-bold text-slate-700 border-slate-700/50">
```

Код: вставка, merge, erase (Скрыто)

```
</summary>
```

```
<div class="p-5 text-sm text-slate-300">
```

```
<pre class="bg-slate-950 p-4 rounded">
```

```
def insert(root, key, pri):
```

```
 if root is None:
```

```
 return Node(key, pri)
```

```
 if key <= root.key: # равные x — левый
```

```
 root.left = insert(root.left, key, pri)
```

```
 else:
```

```
 root.right = insert(root.right, key, pri)
```

```
 return root
```

```
for key, pri in pairs_sorted_by_pri_desc: # порядок э
```

```
 root = insert(root, key, pri)</pre>
```

```
<pre class="bg-slate-950 p-4 rounded">
```

```
def merge(a, b): # все x в a < все x в b
```

```
 if not a or not b: return a or b
```

```
 if a.pri > b.pri:
```

```
 a.right = merge(a.right, b); return a
```

```
 b.left = merge(a, b.left); return b
```

```
def erase(root, key): # удалить произвольный элемент
```

```
 if key < root.key: root.left = erase(root.left, key)
```

```
 elif key > root.key: root.right = erase(root.right, key)
```

```
 else: root = merge(root.left, root.right)
```

```
 return root</pre>
```

```
</div>
```

```
</details>
```

```
</div>
```

```
</div>
```

```
</section>`,
```

```
},
```

```
{
```

```
 id: "splay-tree",
```

```
 title: "4. Splay-дерево",
```

по факту

: ничего не хранит, дереву разрё

```
<div class="grid grid-cols-1 xl:grid-cols-2"
 <div class="bg-slate-800 p-6 rounded-lg"
 <div class="absolute -top-3 left-4 l
rder border-emerald-500 shadow-md">
```

Аналогия 1: Стопка бумаг

```
</div>
<p class="text-slate-300 text-sm">С
тировку не ведёшь, но через неделю самое ну
ерх».</p>
```

```
</div>
<div class="bg-slate-900 p-6 rounded-lg"
 <div class="absolute -top-3 left-4 l
border-rose-500 shadow-md">
```

Аналогия 2: Тропинка через га.

```
</div>
<p class="text-slate-300 text-sm">И,
рно вдвое. Первый проход дорогой, зато опла
ы.</p>
```

```
</div>
</div>
```

```
<h4 class="text-lg font-bold text-white mt-3">
<p class="text-slate-300 text-sm mb-3">Повор
Среднее поддерево β (между x и p) – единств
 x , и под p .</p>
```

```
<pre class="bg-slate-950 p-4 rounded-lg ove
ading-relaxed">
```

```

 / \ \
 x y
 / \ \
 α β
 / \ \
 x y
 / \ \
 α β

 x выше
 -----> α
 / \ \
 p r
 / \ \
 β γ

 p выше
 <-----
 / \ \
 p r
 / \ \
 β γ
 </pre>
```

```
<p class="text-slate-300 text-sm mt-3">Обхо
омать дерево поиска поворотами невозможно в
омент «отстёжки», а врезка «Механика одного
```

```
<h4 class="text-lg font-bold text-white mt-3">
```

```
<pre class="bg-slate-950 p-4 rounded-lg">
 амбук из 8 узлов 8→7→6→5→4→3→2→1, ищем 1 (ч
наивно (x, потом x): 3 × «два зига» + Zig → высота 8
Zig-Zig (p, потом x): 3 × Zig-Zig + Zig → высота 8
бамбук из 16 узлов: наивно 16 → 16, Zig-Zig 16 → 10<
```

```
 <p class="text-slate-300 text-sm mt-3">
де: <code class="bg-slate-950 px-2 py-0.5 r
s="bg-slate-950 px-2 py-0.5 rounded font-mon
изации не будет.</p>
</div>
```

```
<p class="text-slate-300 text-sm mt-6">Полн
м 1. Спуск 8 узлов, затем три Zig-Zig по
8 до 6 – линия перестала быть линией. Наивно
о перевёрнутой. Важно не врать себе: на кор
ван не на одном запросе, а в сумме. Следующ
по пути. Отсюда амортизация: дорогой за
дом потенциала: потенциал Φ = сумма по узла
ь его; отсюда же теорема о статической опти
ь запросов).</p>
```

```
<div class="bg-slate-900/40 rounded-lg bord
 <p class="text-slate-300 text-sm mb-2">
 <p class="text-slate-400 text-sm mb-1">
га» – просто переворот.</p>
 <p class="text-slate-400 text-sm mb-1">
иза к верху; в симуляторе оно подсвечиваетс
 <p class="text-slate-400 text-sm">• «Sp
о амортизированная.</p>
</div>
```

```
<p class="text-slate-300 text-sm mt-4"><b c
ое <code class="bg-slate-950 px-2 py-0.5 ro
одиночная операция – до <code class="bg-sla
>O(n)</code>; n вставок подряд – амортизиро
-300 border border-slate-800">O(n log n)</c
```

```
<p class="text-slate-300 text-sm mt-4">Все
```

```

 rotate_up(x)
 else: # ZIG-ZAG: НИЖНЯЯ
 rotate_up(x)
 rotate_up(x)
 return x

def find(root, key): # поиск: спуск + сп
 cur, last = root, None
 while cur:
 last = cur
 if key == cur.key: break
 cur = cur.left if key < cur.key else cur.right
 return splay(last) if last else None</pre>
 <p class="text-xs text-slate-400">В
 Одна строка – и вся амортизация.</p>
 </div>
</details>
</div>
</div>
</section>`,
 },
 {
 id: "prefix-sums-2d",
 title: "5. Двумерная матрица префиксных сумм",
 type: "html",
 description: "Сумма подматрицы за O(1): 1D и 2D, дв
 category: "Алгоритмы матрицы",
 content: `
<section id="prefix-sums-2d" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-v
 </div>

 <div class="space-y-8">

 <div class="bg-slate-700/50 p-6 rounded-xl bord
 <h3 class="text-xl font-bold text-emerald-4

```

```

 <td class="py-2 pr-3 font-mono">
 </td>
 </tr>
 </tbody>
</table>
<p class="text-slate-400 text-xs mt-2">
 ышли за границу» – поэтому в 2D почти всегда
</div>

<div class="grid grid-cols-1 sm:grid-cols-3">
 <div class="bg-slate-900 border border-
 <p class="text-[10px] uppercase tra
 <p class="font-mono text-emerald-300
 <p class="text-[11px] text-slate-500
 </div>
 <div class="bg-slate-900 border border-
 <p class="text-[10px] uppercase tra
 <p class="font-mono text-emerald-300
 <p class="text-[11px] text-slate-500
 </div>
 <div class="bg-slate-900 border border-
 <p class="text-[10px] uppercase tra
 <p class="font-mono text-rose-300 t
 <p class="text-[11px] text-slate-500
 </div>
</div>
</div>
</div>

<div class="bg-slate-700/50 p-6 rounded-xl bord
 <h3 class="text-xl font-bold text-blue-400 m
 <div class="grid grid-cols-1 xl:grid-cols-2">
 <div class="bg-slate-800
 <div class="absolute -top-3 left-4 b
 rder border-emerald-500 shadow-md">
 Аналогия 1: Сетка из вафельных
 </div>
 <p class="text-slate-300 text-sm mb
 о накопилось от угла (0,0) до меня». Хочешь
 у и верни дважды отрезанный угол. Вычитание

```

```

<div class="bg-slate-800 p-6 rounded-lg" style="border: 1px solid #000; border-radius: 10px; padding: 10px;">
 <div class="absolute -top-3 left-4 right-4 bottom-4 border border-emerald-500 shadow-md">
 Тонкая: одна матрица
 </div>
 <p class="text-slate-300 text-sm mb-2">
 одной матрицы (плюс рамка из нулей). Никаких вычитаний, только сложение.
 прямоугольника от угла. Всё остальное получается так же.
 </div>
 <div class="bg-slate-900 p-6 rounded-lg" style="border: 1px solid #000; border-radius: 10px; padding: 10px;">
 <div class="absolute -top-3 left-4 right-4 bottom-4 border border-rose-500 shadow-md">
 Не умеет: всё, что нельзя вычислить.
 </div>
 <p class="text-slate-300 text-sm mb-2">
 максимума или gcd вычитания не существует:
 е запросы – это
 платит она за это памятью
 </div>
 </div>

<div class="bg-slate-800/70 p-5 rounded-lg" style="border: 1px solid #000; border-radius: 10px; padding: 10px;">
 <p class="text-sm font-bold text-white">
 <table class="w-full text-xs text-left" style="width: 100%; border-collapse: collapse;">
 <thead>
 <tr class="text-slate-500 border-b">
 <th class="py-1.5 pr-2 font-normal">
 <th class="py-1.5 pr-2 font-normal">
 <th class="py-1.5 pr-2 font-normal">
 </tr>
 </thead>
 <tbody>
 <tr class="border-b border-slate-700">
 <td class="py-2 pr-3 font-normal">
 <td class="py-2 pr-3 font-normal">
 <td class="py-2 pr-3 font-normal">
 </tr>
 <tr class="border-b border-slate-700">

```

```

<details class="bg-slate-900/40 rounded-lg l
 <summary class="p-4 font-bold text-slate
-b border-slate-700/50">
 Код: 1D (тот же, что выполняет пане
 </summary>
 <div class="p-5 text-sm text-slate-300
 <pre class="bg-slate-950 p-4 rounde
 = [3, 1, 4, 1, 5, 9, 2, 6] # исходный ма
n = len(a) # префиксов будет n + 1
P = [0] # P[0] = 0 – пустой преф
for i in range(1, n + 1):
 P.append(P[i - 1] + a[i - 1])
print("n =", n, "| P =", P)</code></pre>
 <p class="text-slate-300 text-sm">П
- часть демонстрации: лента подсвечивает, из
 </div>
</details>

<details class="bg-slate-900/40 rounded-lg l
 <summary class="p-4 font-bold text-slate
-b border-slate-700/50">
 Код: 2D – построение и запрос прямо
 </summary>
 <div class="p-5 text-sm text-slate-300
 <pre class="bg-slate-950 p-4 rounde
 = [[1, 2, 3, 4], [5, 6, 7, 8], [9, 1, 2, 3
n, m = 4, 4

S = [] # рамка из нуле
for rr in range(n + 1):
 S.append([0] + [0] * m)
for i in range(1, n + 1):
 for j in range(1, m + 1):
 S[i][j] = A[i-1][j-1] + S[i-1][j] + S[i][j-1] -

r1, c1, r2, c2 = 1, 1, 2, 2 # прямоугольник
ans = S[r2+1][c2+1] - S[r1][c2+1] - S[r2+1][c1] + S[r1]
print("итого:", ans)</code></pre>

```

```

 },
 {
 id: "graph-planar-colors",
 title: "7. Графы. Планарные. Покраска",
 type: "html",
 description: "Формула Эйлера и теорема о 4 красках.",
 category: "Графы. Теория",
 content: `
<section id="graph-planar-colors" class="mb-12 scroll-m-12">
 <div class="flex items-center mb-6 flex-wrap gap-3">
 7
 <h2 class="text-2xl sm:text-3xl font-bold text-blue-400">Графы. Планарные. Покраска
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border border-slate-600">
 <h3 class="text-xl font-bold text-blue-400">Теорема о 4 красках
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-blue-300 italic">Любой планарный граф можно раскрасить 4 цветами.
 <p class="text-xl font-mono text-white">
 1. Если граф не планарный, то его нельзя раскрасить 4 цветами.
 2. Если граф планарный, то его можно раскрасить 4 цветами.
 </p>
 </div>
 <div class="grid grid-cols-1 gap-6">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
 Аналогия 1: Карта мира
 </div>
 <p class="text-slate-300 text-sm mb-4">
 Границы не пересекаются в одной точке, поэтому
 можно раскрасить карты 4 цветами. Всегда можно всего 4
 цвета.
 </p>
 </div>
 </div>
 </div>
 </div>
</section>`,
 },
 {
 id: "top-sort",

```



```

 </div>
</section>`,
 },
 {
 id: "scc-kosaraju",
 title: "10. Графы. Компоненты сильной связности (SCC)",
 type: "html",
 description: "Разбиение орграфа на макро-вершины. Алгоритм Косарaju",
 category: "Графы. Продвинутое",
 content: `
<section id="scc-kosaraju" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">
 Графы
 <h2 class="text-2xl sm:text-3xl font-bold text-emerald-400">10. Графы. Компоненты сильной связности (SCC)
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border border-emerald-500">
 <h3 class="text-xl font-bold text-emerald-400">10.1. Компоненты сильной связности (SCC)
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-emerald-300 italic">Сильная связность графа — это свойство, при котором для любых двух вершин u и v графа существует путь из u в v и из v в u. Другими словами, граф является сильно связным, если для любых двух вершин u и v графа существует путь из u в v и из v в u.
 <p class="text-xl font-mono text-white">Алгоритм Косарaju позволяет найти все компоненты сильной связности графа, инвертированному в порядке top-sort.</p>
 </div>
 <div class="grid grid-cols-1 gap-6">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
 Аналогия 1: Улицы с односторонним движением
 </div>
 <p class="text-slate-300 text-sm mb-4">Представьте себе карту города, где улицы имеют одностороннее движение. Если вы можете пройти по улицам в обоих направлениях между любыми двумя точками, то город является сильно связным. В реальных городах это редко встречается, но понимание сильной связности помогает анализировать структуру графов, представляющих транспортные сети, социальные сети и другие системы.
 <p class="text-slate-300 text-sm mb-4">В графе с односторонними улицами (ориентированном графе) сильная связность означает, что для любых двух точек A и B на карте существует путь из A в B и из B в A.
 </div>
 </div>
 </div>
 </div>
</section>`,
 },

```

```

 description: "Вершины, удаление которых убивает свя
 category: "Графы. Продвинутые",
 content: `
<section id="graph-articulation" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-rose-400">
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border">
 <h3 class="text-xl font-bold text-rose-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-rose-300 italic">
 <p class="text-xl font-mono text-white">
 теи > 1).</p>
 </div>
 <div class="grid grid-cols-1 gap-6">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border
 order border-emerald-500 shadow-md">
 Аналогия 1: Центральный роутер
 </div>
 <p class="text-slate-300 text-sm mb-4">
 правильный роутер - и два сегмента офиса б
 </div>
 </div>
 </div>
 </div>
 </div>
</section>`,
 },
 {
 id: "graph-euler",
 title: "13. Графы. Эйлеров цикл",
 type: "html",
 description: "Пройти по всем ребрам ровно 1 раз.",
 category: "Графы",
 content: `
<section id="graph-euler" class="mb-12 scroll-mt-10">

```

```

<div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border"
 <h3 class="text-xl font-bold text-blue-400"
 <div class="bg-slate-900 p-4 rounded-lg mb-1"
 <p class="text-lg text-blue-300 italic"
 <p class="text-xl font-mono text-white"
d-длина.</p>
 </div>
 <div class="grid grid-cols-1 gap-6">
 <div class="bg-slate-800 p-6 rounded-lg"
 <div class="absolute -top-3 left-4 l
rder border-emerald-500 shadow-md">
 Аналогия: Копаем туннель под .
 </div>
 <p class="text-slate-300 text-sm mb
ней земли. Но если начать копать одновременно
в 2 РАЗА меньше работы (геометрически: площ
 </div>
 </div>
 </div>
 </div>
</section>`,
 },
 {
 id: "mst-kruskal",
 title: "18. Графы. Остовное дерево. Краскал",
 type: "html",
 description: "",
 category: "Графы. Остовные",
 content: `
<section id="mst-kruskal" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">
 <span class="bg-blue-600 text-white px-4 py-1 r
 <h2 class="text-2xl sm:text-3xl font-bold text-
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl bord
 <h3 class="text-xl font-bold text-emerald-4

```

```

 которое торчит из посещенных в непосещенных
 </div>
 <div class="grid grid-cols-1 gap-6">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
 ✦ Аналогия: Заражение вирусом
 </div>
 <p class="text-slate-300 text-sm mb-2">
 (или вирус). Мы стартуем из одного города, и
 Сеть растет только из одного связного куска
 </p>
 </div>
 </div>
</div>
</section>`,
 },
 {
 id: "mst-boruvka",
 title: "20. Графы. Остовное дерево. Борувка",
 type: "html",
 description: "",
 category: "Графы. Остовные",
 content: `
<section id="mst-boruvka" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-white">
 </h2>

 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border border-emerald-500">
 <h3 class="text-xl font-bold text-emerald-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-emerald-300 italic">
 <p class="text-xl font-mono text-white">
 ается с соседом.</p>
 </p>
 </div>
 </h3>
 <div class="grid grid-cols-1 gap-6">

```

```

 rder border-emerald-500 shadow-md">
 Аналогия 1: Бумеранг (Префикс
 </div>
 <p class="text-slate-300 text-sm mb
идишь "АБРА". Если ты ошибешься при поиске
концовка "АБРА" совпадает с началом "АБРА"
меня перепроверок!</p>
 </div>
</div>
<details class="bg-slate-900/40 rounded-lg
 <summary class="p-4 font-bold text-slate
-b border-slate-700/50">
 Код (Скрыто)
 </summary>
 <div class="p-5 text-sm text-slate-300
 <pre class="bg-slate-950 p-4 rounde
vector<int> pi(s.length());
for (int i = 1; i < s.length(); i++) {
 int j = pi[i-1];
 while (j > 0 && s[i] != s[j]) j = pi[j-1];
 if (s[i] == s[j]) j++;
 pi[i] = j;
}</pre>
 </div>
 </details>
</div>
</div>
</section>`,
 },
 {
 id: "string-z-func",
 title: "22. Z-функция",
 type: "html",
 description: "",
 category: "Строки",
 content: `
<section id="string-z-func" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

```

```

 </div>
</section>`,
 },
 {
 id: "complexity-classes",
 title: "24. Классы сложности, сведение задач",
 type: "html",
 description: "",
 category: "Теория сложности",
 content: `
<section id="complexity-classes" class="mb-12 scroll-mt-12">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-white">
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border border-slate-600">
 <h3 class="text-xl font-bold text-purple-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-purple-300 italic">
 <p class="text-base sm:text-lg font-mono">
 P: класс decision-задач, решаемый за полиномиальное время.
 NP: класс decision-задач с сертификатом.
 </p>
 </div>
 <p>
 Сведение (Reduction) A ≤p B: задача A сводится к задаче B.
 </p>
 </div>
 <div class="grid grid-cols-1 xl:grid-cols-2 gap-4">
 <!-- Аналогия 1 -->
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
 Аналогия 1: Судоку (P vs NP)
 </div>
 <p class="text-slate-300 text-sm mb-4">
 Пример сортировка чисел). Класс NP – класс задач, для которых существует
 енную сетку (ответ), он БЫСТРО (за класс P)
 </div>
 </div>
 </div>
 </h3>
 </div>
 </div>
 </div>
</section>`
 }
]

```

```

<div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border"
 <h3 class="text-xl font-bold text-blue-400"

 <div class="bg-slate-900 p-4 rounded-lg mb-4"
 <p class="text-lg text-blue-300 italic"
 <div class="text-left font-mono text-sm"
 <p><strong class="text-white">Эйлер
ин раз. Вершины повторять можно.</p>
 <p><strong class="text-white">Эйлер
 <div class="p-3 bg-slate-950 rounded"
 <p class="text-sm text-blue-300"
 <p><strong class="text-white">П
 <p><strong class="text-white">Ц
 </div>
 </div>
 </div>
</div>

<p class="text-slate-300 text-sm">
 Билет 13 (Различие пути и цикла

• <span class="text-green-400 font-
шел-вошёл-вышел).

• <span class="text-yellow-400 font
финиш).
</p>
</div>

<div class="grid grid-cols-1 xl:grid-cols-2 gap"
 <div class="bg-slate-800 p-6 rounded-lg bor"
 <div class="absolute -top-3 left-4 bg-s
order-green-500 shadow-md">
 Аналогия 1: Путь – из дома в бар,
 </div>
 <p class="text-slate-300 text-sm mb-4">
 Ты вышел из дома (нечётная)
 талый пришёл в бар (вторая нечётная
 Домой вернуться не смог, потому что
 </p>

```

```

 </div>
</section>`,
 },
 {
 id: "bridges-code",
 title: "11. Графы. Мосты",
 type: "html",
 description: "Мосты через DFS Тарьяна: tin/low, усл",
 category: "Графы. Продвинутые",
 content: `<section id="bridges-code" class="mb-20 s
<div class="flex items-center mb-6">
 <span class="bg-emerald-600 text-white px-4 py-
 <h2 class="text-3xl font-bold text-white">Мосты
</div>

<div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl bord
 <h3 class="text-xl font-bold text-emerald-4

 <div class="bg-slate-900 p-4 rounded-lg mb-
 <p class="text-lg text-emerald-300 ital
 <div class="text-left font-mono text-sm
 <p><strong class="text-white">tin[v
 <p><strong class="text-white">low[v
 <div class="p-3 bg-slate-950 rounde
 <span class="text-rose-400 font
 <p class="text-xl font-bold tex
 <p class="text-xs text-slate-40
 </div>
 </div>
 </div>
 </div>
 <div class="bg-slate-900 p-4 rounded-lg mb-
 <p class="text-slate-300 text-sm">
 Если из сына u и всех его потомков <str
 Единственная ниточка. Порвётся – граф р
 </p>
 </div>
</div>

```



```

<pre class="text-xs font-mono text-
#include <vector>;
#include <algorithm>;

using namespace std;

int n; // число вершин
vector<vector<int>>> adj; // список смежност
vector<bool> visited;
vector<int> tin, low;
int timer;

void dfs(int v, int p = -1) {
 visited[v] = true;
 tin[v] = low[v] = timer++; // устанавливаем

 for (int to : adj[v]) {
 if (to == p) continue; // не идём назад

 if (visited[to]) {
 // обратное ребро: обновляем low
 low[v] = min(low[v], tin[to]);
 } else {
 // ребро дерева DFS
 dfs(to, v); // рекурсивный вызов
 low[v] = min(low[v], low[to]);

 if (low[to] > tin[v]) {
 // ЭТО МОСТ!
 // bridge_list.push_back({v, to});
 }
 }
 }
}

void find_bridges() {
 timer = 0;
 visited.assign(n, false);
}
```

```

 <p><span class="text-emerald-400 font-
 <p><span class="text-rose-400 font-l
 </div>
</div>

<p class="text-slate-300 text-sm">
 Следствие: Для планарн
ode>.
 Если ребер больше – граф гарантированно
</p>
</div>

<div class="grid grid-cols-1 xl:grid-cols-2 gap
 <div class="bg-slate-800 p-6 rounded-lg bor
 <div class="absolute -top-3 left-4 bg-s
order-green-500 shadow-md">
 K5: Полный граф на 5 вершинах
 </div>
 <p class="text-slate-300 text-sm mb-4">
 У K5: <code class="text-white">V=5,
-white">10 > 9</code> – БАХ, непланарен!
 Все 5 вершин соединены со всеми. На
 Теорема Куратовского: K5 и K3,3 – д
 Если внутри графа спрятано подра
 – плоской раскладки не будет.
 </p>
 </div>

<div class="bg-slate-900 p-6 rounded-lg bor
 <div class="absolute -top-3 left-4 bg-r
der-rose-500 shadow-md">
 K3,3: 3 дома и 3 колодца
 </div>
 <p class="text-slate-300 text-sm mb-4">
 Три дома хотят соединиться с тремя
 Нарисовать без пересечений невозможно
 У него <code class="text-white">V=6
xt-white">9 ≤ 12</code> – проходит, но он в
```

```
<div class="grid grid-cols-1 xl:grid-cols-2"
 <div class="bg-slate-800 p-6 rounded-lg"
 <div class="absolute -top-3 left-4 l
rder border-emerald-500 shadow-md">
```

Аналогия 1: Расписание экзамен

```
</div>
```

```
<p class="text-slate-300 text-sm">В
слот времени. Раскрасить граф = составить р
«берём предметы в каком-то порядке и ставим
</div>
```

```
<div class="bg-slate-900 p-6 rounded-lg"
 <div class="absolute -top-3 left-4 l
border border-fuchsia-500 shadow-md">
```

Аналогия 2: Частоты вышек

```
</div>
```

```
<p class="text-slate-300 text-sm">С
цвет, интерференция = ребро. Диапазон частот
регистров в компиляторе: «живые» одновременно
</div>
```

```
</div>
```

```
<div class="overflow-x-auto my-6">
```

```
<table class="w-full text-xs text-left"
 <thead>
```

```
<tr class="text-slate-500 border-b"
 <th class="py-2 pr-3 font-b"
 <th class="py-2 pr-3 font-b"
 <th class="py-2 font-bold">1
```

```
</tr>
```

```
</thead>
```

```
<tbody class="text-slate-300">
```

```
<tr class="border-b border-slate-800"
><td class="py-2 pr-3 font-mono"> $\chi \leq \Delta + 1$ <
а свободен</td></tr>
```

```
<tr class="border-b border-slate-800"
d><td class="py-2 pr-3 font-mono"> $\chi \leq \Delta$ </td>
/td></tr>
```

```

 веркой цветов соседей; проверка двудольности
border border-slate-800">0(V + E)</code>; т
-300 border border-slate-800">NP-трудно</co

<details class="bg-slate-900/40 rounded-lg l
 <summary class="p-4 font-bold text-slate
-b border-slate-700/50">
 Код: жадная раскраска и проверка
 </summary>
 <div class="p-5 text-sm text-slate-300
 <pre class="bg-slate-950 p-4 rounded
я раскраска: не больше $\Delta + 1$ цвета
color = [-1] * n # -1 = ещё не покрашен
for v in order: # порядок влияет на чи
 used = set()
 for to in graph[v]:
 if color[to] != -1:
 used.add(color[to]) # цвета уже покрашены
 c = 0
 while c in used: # минимальный свободны
 c += 1
 color[v] = c
chi = max(color) + 1 # χ , который выдал жад

Проверка двудольности = раскраска в 2 цвета (BFS)
from collections import deque
col = [-1] * n
bipartite = True
for start in range(n):
 if col[start] != -1:
 continue
 col[start] = 0
 dq = deque([start])
 while dq:
 v = dq.popleft()
 for to in graph[v]:
 if col[to] == -1:
 col[to] = col[v] ^ 1 # второй цвет

```

```
<p class="text-slate-300 text-sm">
```

```
 Важно: Это работает то-
 еплятся к точкам сочленения. То есть,
 ег - это тупик со степенью 1).
```

```
</p>
```

```
</div>
```

```
<div class="grid grid-cols-1 xl:grid-cols-2 gap-
```

```
 <div class="bg-slate-800 p-6 rounded-lg border-
```

```
 <div class="absolute -top-3 left-4 bg-slate-800
 rder-rose-500 shadow-md">
```

```
 Аналогия: Главный перекресток
```

```
 </div>
```

```
 <p class="text-slate-300 text-sm mb-4">
```

```
 Представь город, где два района соединены
 её перекроют (удалят вершину) — районы будут
 g>хрупкость системы.
```

```
 </p>
```

```
 </div>
```

```
<div class="bg-slate-900 p-6 rounded-lg border-
```

```
 <div class="absolute -top-3 left-4 bg-emerald-900
 er border-emerald-500 shadow-md">
```

```
 Телеком: Центральный свитч
```

```
 </div>
```

```
 <p class="text-slate-300 text-sm mb-4">
```

```
 У тебя несколько компьютеров в одной сети,
 абелем (мостом). Сами свитчи — это точки со-
```

```
 </p>
```

```
</div>
```

```
</div>
```

```
<details class="bg-slate-900/40 rounded-lg border-
```

```
 <summary class="p-4 font-bold text-slate-400
 order-slate-700/50">
```

```
 Душный мод: Код и корень DFS (Скрыто)
```

```
 </summary>
```

```

 <ul class="list-disc list-inside text-sm text-rose-400">
 <strong class="text-rose-400">Точки
ong> и показывают физическую уязвимость свя
 <strong class="text-emerald-400">SCC
ависимые "конгломераты".
 Как точка сочленения рушит
Косарайю, свернув каждую SCC в одну супер-в
(сделаем неориентированным), то любая <strong
о и есть критическая SCC! Если удалить эту
. Одно слабое звено изолирует целые касты S

 </div>
</div>
</section>`,
 },
];

```

ФАЙЛ 19/111: src/data/content.ts

КАТЕГОРИЯ: Контент и данные пособия | СТРОК: 145 | РАЗМ

```

import { Chapter } from "../types";
import { graphChapters } from "../graphContent";
import { additionalTickets } from "../additionalTickets";
import { tickets14to20 } from "../tickets/ticket14_20";
import { tickets21to24 } from "../tickets/ticket21_24";
import { tickets6to13 } from "../tickets/ticket6_13";
import { ticket1Segtree } from "../tickets/ticket1_segtr
import { tickets18to20Mst } from "../tickets/ticket18_20
import { chapters as newChapters } from "../content_temp

/**
 * Содержание пособия = билеты 1–24 из списка тем.
 *
 * Порядок массивов важен: при одинаковом id побеждает
 * версия главы. `newChapters` (content temp) перетирае

```

```
merged.forEach((c) => {
 if (!toRemove.includes(c.id)) {
 dedup.set(c.id, c);
 }
});

// Самые свежие версии страниц перетирают старые.
if (newChapters) {
 newChapters.forEach((c) => {
 if (!toRemove.includes(c.id)) dedup.set(c.id, c);
 });
}

const finalChapters = Array.from(dedup.values());

finalChapters.sort((a, b) => {
 // Главы без номера билета (введение) стоят ДО билетов
 const numOf = (t: string) => {
 const m = t.match(/(\d+)/);
 return m ? parseInt(m[0], 10) : -1;
 };
 return numOf(a.title) - numOf(b.title);
});

export const chapters: Chapter[] = finalChapters;

/**
 * Билеты, закрытые каждой страницей (для шапки, экспор
 * Страница `mst`, например, закрывает сразу 18, 19 и 20
 */
export const chapterTopics: Record<string, number[]> = {
 "segment-trees": [1],
 "sparse-table": [2],
 treap: [3],
 "splay-tree": [4],
 "prefix-sums-2d": [5],
 "graph-dfs-bfs": [6],
 "planarity-euler-formula": [7],

```

```

const tickets = chapterTopics[chapterId] ?? [];
// Глав без билетов (введение) относим к первому разд
if (!tickets.length) return SECTIONS[0];
const first = Math.min(...tickets);
return SECTIONS.find((sec) => first >= sec.from && fi
}

export const activeVizIds = [
 "euler-path-vs-cycle", "bridges-code", "planarity-euler
 "graph-dfs-bfs", "stack-dfs", "queue-bfs", "heap-beam-se
 "dijkstra", "bellman-ford", "floyd", "aho-corasick",
 "mst", "string-kmp", "string-z-func", "segment-trees", "s
 "dynamic-programming", "johnson-algo", "graph-articulat
 "complexity-classes", "treap", "splay-tree", "prefix-sum
];

```

-----  
ФАЙЛ 20/111: src/data/debugRunner.ts

КАТЕГОРИЯ: Контент и данные пособия | СТРOK: 207 | РАЗМ

```

/**
 * Пошаговый дебаггер поверх Pyodide: CPython выполняет
 * под sys.settrace и на каждой строке записывает фрейм
 * как панель Variables в обычном отладчике.
 */

```

```

export type DebugValue = null | boolean | number | string

```

```

export interface DebugStep {
 /** Номер строки в коде пользователя (1-based). */
 line: number;
 /** Имя функции-фрейма ("<module>" – верхний уровень)
 func: string;
 /** Локальные переменные: имя → герг (усечённый), для
 locals: Record<string, string>;
 /**

```



```

__steps = []
__final = {}
__final_values = {}
__CAP = ${DEBUG_STEP_CAP}
__was_truncated = False

class __TraceLimit(Exception):
 pass

def __safe(v):
 try:
 r = repr(v)
 except Exception:
 r = "<...>"
 return r if len(r) <= 80 else r[:77] + "..."

def __snapshot(v, depth=0):
 """Небольшой JSON-снимок: UI использует его без парсера"""
 if v is None or isinstance(v, (bool, int, str)):
 return v
 if isinstance(v, float):
 return v if v == v and v not in (float("inf"), float("-inf")) else None
 if depth >= 4:
 return __safe(v)
 limit = 80 if depth < 2 else 32
 if isinstance(v, (list, tuple)):
 return [__snapshot(x, depth + 1) for x in list(v)]
 if isinstance(v, (set, frozenset)):
 return [__snapshot(x, depth + 1) for x in list(v)]
 if isinstance(v, dict):
 # 8x8 Sparse Table содержит 64+49+25+1 = 139 элементов
 # предел 80 незаметно обрезал уровни 4x4 и 8x8
 dict_limit = 192 if depth == 0 else limit
 return {str(k): __snapshot(x, depth + 1) for k, x in list(v.items())}
 # deque, heap-подобные и другие итерируемые учебные
 if type(v).__name__ == "deque":
 return [__snapshot(x, depth + 1) for x in list(v)]
 return __safe(v)

```

```

 # PEP 709: с 3.12 компрекеншны встроены во фрейм
 # на своей строке каждую итерацию. Склеиваем по
 if __steps and __steps[-1]["line"] == frame.f_lineno:
 __steps[-1]["locals"] = loc
 __steps[-1]["values"] = values
 else:
 __steps.append({"line": frame.f_lineno, "func": frame.f_code.co_name})
 if len(__steps) >= __CAP:
 # Нельзя просто отключить trace: бесконечный цикл
 # тогда продолжит работать вечно. Мягко прерываем
 __was_truncated = True
 raise __TraceLimit()
 # Финальные локалы программы (после последней строки)
 if event == "return" and frame.f_code.co_filename == __FILE__:
 loc, values = __capture(frame)
 __final.update(loc)
 __final_values.update(values)
 return __tracer

__err = ""
try:
 __sys.settrace(__tracer)
 exec(compile(__src, "<user-code>", "exec"), {"__name__": "__main__"})
except __TraceLimit:
 pass
except Exception:
 import traceback as __tb
 __err = __tb.format_exc(limit=3)
finally:
 __sys.settrace(None)

__OUT = __json.dumps({
 "steps": __steps,
 "finalLocals": __final,
 "finalValues": __final_values,
 "error": __err,
 "truncated": __was_truncated,
})

```

---

ФАЙЛ 21/111: src/data/glossary.ts

КАТЕГОРИЯ: Контент и данные пособия | СТРОК: 639 | РАЗМ

---

```
import type { DemoKind } from "../components/hints/Mini"
```

```
/**
```

```
 * Глоссарий «википедийных» подсказок.
```

```
 *
```

```
 * Любое слово из `labels`, встреченное в тексте главы,
```

```
 * в подчёркнутый термин: при наведении (или тапе на мо
```

```
 * с объяснением на пальцах и мини-визуализацией.
```

```
 */
```

```
export interface GlossaryEntry {
```

```
 id: string;
```

```
 /** Варианты написания в тексте (регистр не важен). */
```

```
 labels: string[];
```

```
 title: string;
```

```
 /** Объяснение «на пальцах» – 1–2 фразы. */
```

```
 short: string;
```

```
 /** Строгая формулировка / что важно не забыть. */
```

```
 formal?: string;
```

```
 complexity?: string;
```

```
 /** Какая мини-анимация показывается в карточке. */
```

```
 demo?: DemoKind;
```

```
 /** id полного симулятора (см. vizRegistry) – кнопка
```

```
 simulator?: string;
```

```
}
```

```
export const GLOSSARY: GlossaryEntry[] = [
```

```
 {
```

```
 id: "bfs",
```

```
 labels: ["BFS", "обход в ширину", "поиск в ширину",
```

```
 title: "BFS – обход в ширину",
```

```
 short:
```

"Не отсортированный список, а «мешок, из которого  
formal:

"Полное бинарное дерево в массиве: дети узла  $i$  ле

complexity: "вставка/извлечение  $O(\log n)$ , «посмотре

demo: "heap",

simulator: "heap-beam-search",

},

{

id: "stack",

labels: ["стек", "стека", "стеке", "стеком", "LIFO"]

title: "Стек (LIFO)",

short: "Стопка тарелок: кладёшь сверху и берёшь свер

formal: "push / pop / top за  $O(1)$ . На стеке живёт р

demo: "stack",

simulator: "stack-dfs",

},

{

id: "queue",

labels: ["очередь", "очереди", "очередью", "FIFO"],

title: "Очередь (FIFO)",

short: "Очередь в столовой: кто первым встал, тот п

formal: "push\_back / pop\_front за  $O(1)$ . Основа BFS

demo: "queue",

simulator: "queue-bfs",

},

{

id: "dsu",

labels: ["DSU", "СНМ", "система непересекающихся мн

title: "DSU — система непересекающихся множеств",

short:

"«Кто твой староста?» Каждый элемент знает своего

formal:

"find со сжатием путей + union по рангу/размеру. I

complexity: "почти  $O(1)$  — обратная функция Аккерман

demo: "dsu",

simulator: "mst",

},

{

```

 short:
 "«Куда откатиться, если следующая буква не подошла",
 строки.",
 formal: "link(v) = go(link(parent(v)), ch). Считает",
 demo: "suffix-link",
 simulator: "aho-corasick",
},
{
 id: "toposort",
 labels: ["топологическая сортировка", "топсорт", "топ",
 title: "Топологическая сортировка",
 short:
 "Порядок «сначала штаны, потом ботинки»: каждая с",
 циклов.",
 formal: "DFS + вершины в обратном порядке выхода, л",
 complexity: "O(V + E)",
 demo: "toposort",
 simulator: "top-sort",
},
{
 id: "relaxation",
 labels: ["релаксация", "релаксацию", "релаксации",
 title: "Релаксация ребра",
 short:
 "«А через соседа не быстрее?» Если известный путь",
 ,
 formal: "if d[v] > d[u] + w(u,v) → d[v] = d[u] + w(u,v)",
 ан и Флойд.",
 demo: "relax",
 simulator: "dijkstra",
},
{
 id: "prefix-sum",
 labels: ["префиксные суммы", "префиксная сумма", "пре",
 title: "Префиксные суммы",
 short:
 "Заранее посчитанные «сколько всего набежало от н",
 formal: "P[0]=0, P[i]=P[i-1]+a[i-1]. Сумма a[l..r] = P[r] - P[l-1]"

```

```

 complexity: "O(V + E) одним DFS",
 demo: "bridge",
 simulator: "bridges-code",
},
{
 id: "articulation",
 labels: ["точка сочленения", "точки сочленения", "то"],
 title: "Точка сочленения",
 short:
 "Вершина-незаменимый-сотрудник: уволь её — и коллапс",
 formal: "Для не-корня: v — точка сочленения Э ребро",
 demo: "articulation",
 simulator: "graph-articulation",
},
{
 id: "mst",
 labels: [
 "остовное дерево", "остовного дерева", "остовное",
 "минимальное остовное дерево", "MST",
],
 title: "Минимальное остовное дерево (MST)",
 short:
 "Связать все точки проводами так, чтобы суммарная",
 formal: "V-1 ребро, никаких циклов. Краскал (жадно и",
 "но).",
 demo: "mst",
 simulator: "mst",
},
{
 id: "amortized",
 labels: [
 "амортизированная", "амортизированный", "амортизир",
 "амортизированное", "амортизированного", "амортизир",
],
 title: "Амортизированная сложность",
 short:
 "Средняя цена операции «по абонементу»: одна опер",
 formal: "Метод потенциалов: дорогая операция оплачи
```

```
{
 id: "prefix-function",
 labels: [
 "префикс-функция", "префикс-функции", "префикс-фу",
 "префикс", "префикса", "суффиксом", "суффикс", "с",
 "подстроки", "подстроку", "подстрока",
],
 title: "Префикс-функция π (КМП)",
 short:
 "Для каждой позиции запоминаем: какой кусок начался",
 "от кусок.",
 formal:
 " $\pi[i]$ — длина наибольшего собственного префикса s ",
 "возвращая указатель по тексту.",
 complexity: "O(n + m)",
 demo: "prefix-function",
 simulator: "string-kmp",
},
{
 id: "z-function",
 labels: ["Z-функция", "Z-функции", "Z-функцию", "z-"],
 title: "Z-функция",
 short:
 "Для каждой позиции: сколько символов подряд, начи",
 "аждой букве.",
 formal:
 " $z[i]$ = длина наибольшего общего префикса s и $s[i$ ",
 complexity: "O(n)",
 demo: "prefix-function",
 simulator: "string-z-func",
},
{
 id: "dp",
 labels: [
 "динамическое программирование", "динамики", "дин",
 "мемоизация", "мемоизацию", "подзадач", "подзадач",
],
 title: "Динамическое программирование",
```

```

 short:
 "На каждом шаге хватаем то, что лучше прямо сейчас"
 formal:
 "Корректность обычно доказывается «леммой о разрезе»"
 demo: "mst",
 simulator: "mst",
},
{
 id: "euler",
 labels: ["эйлеров цикл", "эйлеров путь", "эйлерова теорема"],
 title: "Эйлеров путь и цикл",
 short:
 "Пройти по каждому ребру ровно один раз, не отрываясь"
 formal:
 "Связный граф: эйлеров цикл все степени чётные; эйлеров путь все степени нечётные"
 simulator: "euler-path-vs-cycle",
},
{
 id: "planarity",
 labels: ["планарный", "планарность", "планарного", "планаризация"],
 title: "Планарность",
 short:
 "Граф планарен, если его можно нарисовать на листе бумаги"
 formal:
 "V - E + F = 2 для связного планарного графа. Курсивом выделены вершины, рёбра и области"
 simulator: "planarity-euler-formula",
},
{
 id: "splay",
 labels: [
 "splay", "splay-дерево", "AVL", "AVL-дерево", "всё остальное"
],
 title: "Splay и повороты",
 short:
 "Каждый раз, когда к узлу обратились, его «вытягивают» к корню"
 formal:

```



```

 "После k-й итерации гарантированно верны все кратчайшие пути, которые не содержат отрицательный цикл.",
 complexity: "O(V · E)",
 demo: "relax",
 simulator: "bellman-ford",
 },
 {
 id: "floyd",
 labels: [
 "Флойд", "Флойда", "Флойд-Уоршелл", "Флойда-Уоршелла",
 "промежуточная вершина", "промежуточную вершину",
],
 title: "Флойд-Уоршелл: разрешаем вершину k",
 short:
 "Внешний цикл — это не «шаг», а «какие пересадки сделать, чтобы пересадку в k?",
 formal:
 "D[i][j] = min(D[i][j], D[i][k] + D[k][j]). Порядок пересадки — k.",
 complexity: "O(V³) времени, O(V²) памяти",
 demo: "floyd",
 simulator: "floyd",
 },
 {
 id: "prim",
 labels: ["Прима", "Прим", "Prim"],
 title: "Алгоритм Прима",
 short:
 "Остов растёт как плесень из одной точки: на каждом шаге добавляем ребро, не создавая циклов.",
 formal:
 "Множество посещённых + куча рёбер на границе. Добавляем ребро с минимальным весом, не создавая циклов.",
 complexity: "O(E log V)",
 demo: "mst",
 simulator: "mst",
 },
 {
 id: "boruvka",

```

```

{
 id: "graph",
 labels: [
 "граф", "графа", "графе", "графы", "графов", "граф",
 "вершины", "вершина", "вершин", "ребра с весом",
],
 title: "Граф",
 short:
 "Точки (вершины) и связи между ними (рёбра). Города и дороги.",
 formal:
 " $G = (V, E)$. Ребро бывает ориентированным (улицы с односторонним движением) и неориентированным (двустороннее движение).",
 demo: "graph",
 simulator: "graph-dfs-bfs",
},
{
 id: "coord-compress",
 labels: ["сжатие координат", "сжатия координат", "сжатие координат"],
 title: "Сжатие координат",
 short:
 "Координаты бывают гигантские и дробные, а нам нужно хранить их в виде целых чисел. Сжимаем координаты в один порядковый номер.",
 formal:
 "sort + unique + binary search: значение → индекс",
 complexity: " $O(n \log n)$ ",
 demo: "coord-compress",
},
{
 id: "treap",
 labels: ["Treap", "декартово дерево", "декартова д-во"],
 title: "Treap = Tree + Heap",
 short:
 "Одно дерево живёт по двум правилам сразу: по ключу и по приоритету. Случайность и держит его сбалансированным.",
 formal:
 "Базис — split(t, x) (разрезать по ключу) и merge(t1, t2) (соединить). Ожидаемая высота $O(\log n)$.",
 complexity: "ожидаемо $O(\log n)$ ",
}

```

```

 demo: "np",
 },
 {
 id: "range",
 labels: ["отрезок", "отрезка", "отрезке", "отрезки"],
 title: "Запрос на отрезке",
 short:
 "Спрашиваем не про один элемент, а про кусок массива

 предподсчёты.",
 formal:
 "Суммы — префиксные суммы $O(1)$. Минимум без изменений.",
 demo: "prefix-sum",
 simulator: "prefix-sums-2d",
 },
 {
 id: "string",
 labels: ["строки", "строке", "строку", "строка", "строкам"],
 title: "Строка как массив символов",
 short:
 "Строковые алгоритмы держатся на трёх вопросах: где

 (Z-функция) и где встречаются слова словаря (бордер-лист).",
 formal:
 "Наивный поиск подстроки — $O(N \cdot M)$. Префикс-функция — $O(N)$.",
 demo: "prefix-function",
 simulator: "string-kmp",
 },
 {
 id: "rotation",
 labels: ["поворот", "повороты", "поворота", "поворотом"],
 title: "Поворот (rotation) в дереве",
 short:
 "Локальная перевеска двух узлов: ребёнок встанет на

 дившемся месту. Порядок ключей при этом не ломается.",
 formal:
 "Правый поворот вокруг p: $x = p.left; p.left = x$.

 g и Zig-Zag.",
 complexity: "O(1)",
 demo: "zig",
 },

```

Универсальное пособие • полный исходный код

```

/** Быстрый доступ по id. */
export const GLOSSARY_BY_ID = new Map(GLOSSARY.map((e) => {

/** Все метки, отсортированные от длинных к коротким (ч
export const GLOSSARY_LABELS: { label: string; id: string }[] =
 e.labels.map((label) => ({ label, id: e.id }))
}).sort((a, b) => b.label.length - a.label.length);

```

ФАЙЛ 22/111: src/data/graphContent.ts

КАТЕГОРИЯ: Контент и данные пособия | СТРОК: 411 | РАЗМЕР: 1.5 КБ

```

import { Chapter } from "../types";

export const graphChapters: Chapter[] = [
 {
 id: "intro",
 title: "Введение. Базовая база: куча, очередь, бинарное
 type: "html",
 category: "Введение",
 content: `
 <section id="intro-content" class="mb-12">
 <div class="flex items-center mb-6 flex-wrap gap-2">
 Базовая база
 <h2 class="text-3xl font-bold text-white">Базовая база</h2>
 </div>

 <div class="bg-slate-900 border border-indigo-500/40 p-6">
 <p class="text-slate-200 text-sm leading-relaxed">
 <b class="text-indigo-300">Базовая база – это
 дерево как массив, куча, очередь и стек, бинарное
 Кодим по минимуму: там, где за нас уже написали.
 </p>
 </div>

 <div class="space-y-8">
```

```
h = []
heapq.heappush(h, 5); heapq.heappush(h, 1); heapq.heappop(h)
h[0] # 1 – минимум виден без всякого обхода
heapq.heappop(h) # 1, куча починилась за $O(\log n)$
```

<p class="text-slate-300 text-sm">Куча – движок, который поддерживает кучу, раз, и список с <span class="font-mono text-sm">heapq</span>.</p>

<div class="bg-slate-700/50 p-6 rounded-xl border-l-4 border-r-4 border-t-4 border-b-4">  
<h3 class="text-xl font-bold text-sky-400 mb-4">Очереди и стеки</h3>  
<p class="text-slate-300 text-sm mb-4"><b>Очереди</b> (BFS) отличаются от стеков (DFS) тем, что первый вышел) отличаются одной операцией снятия элемента</span>: оба конца работают за  $O(1)$ . Обычный список сдвигает весь хвост и стоит  $O(n)$ .</p>

```
<pre class="bg-slate-950 border border-slate-700">from collections import deque
q = deque([1, 2])
q.append(3); q.popleft() # 1 – очередь (BFS)
s = [1, 2]; s.append(3); s.pop() # 3 – стек (DFS)</pre>
```

<p class="text-slate-300 text-sm">Вся разница между BFS (волнами), стек – DFS (вглубь).</p>

<div class="bg-slate-700/50 p-6 rounded-xl border-l-4 border-r-4 border-t-4 border-b-4">  
<h3 class="text-xl font-bold text-amber-400 mb-4">Бинарный поиск</h3>  
<p class="text-slate-300 text-sm mb-4">Если массив отсортирован, то можно найти элемент за  $O(\log n)$  с помощью бинарного поиска. Если массив не отсортирован, то нужно сначала отсортировать массив, что займет  $O(n \log n)$ .</p>

```
<pre class="bg-slate-950 border border-slate-700">from bisect import bisect_left
a = [10, 20, 30, 40, 50]
bisect_left(a, 30) # 2 – индекс первого элемента, не меньшего 30</pre>
```

<p class="text-slate-300 text-sm">Без сортировки бинарный поиск не работает.</p>

<div class="bg-slate-700/50 p-6 rounded-xl border-l-4 border-r-4 border-t-4 border-b-4">  
<h3 class="text-xl font-bold text-rose-400 mb-4">Сортировка</h3>  
<p class="text-slate-300 text-sm mb-4">Писать сортировку можно вручную, но лучше использовать встроенные функции.</p>

```

left(x) = z # z становится левым р
высоты пересчитываются только у z и x — ост
</div>
</details>
</div>

<div class="bg-slate-700/50 p-6 rounded-xl border-l
 <h3 class="text-xl font-bold text-teal-400 mb-4
 <p class="text-slate-300 text-sm mb-4">Оба обхо
 тару». Вся разница в таре: стек уходит вглуби
 ext-emerald-300">0(V + E): каждая верш
 <pre class="bg-slate-950 border border-slate-700
def go(graph, start, tank): # tank: список-стек
 seen, tank = {start}, tank([start])
 while tank:
 v = tank.pop() if isinstance(tank, list) else t
 for to in graph[v]:
 if to not in seen:
 seen.add(to); tank.append(to)
 return seen</pre>
 <p class="text-slate-300 text-sm">Дальше эта па
 надстройки над этими восемью строками.</p>
</div>

<div class="bg-slate-700/50 p-6 rounded-xl border-l
 <h3 class="text-xl font-bold text-slate-300 mb-
 <div class="grid grid-cols-1 xl:grid-cols-2 gap
 <div class="bg-slate-800 p-6 rounded-lg bord
 <div class="absolute -top-3 left-4 bg-s
 border-indigo-500 shadow-md">
 Аналогия 1: Схема кинозала
 </div>
 <p class="text-slate-300 text-sm">Дерев
 азателей-«usher'ов» не нужно. Место n, дети
 рху вниз и слева направо.</p>
 </div>
 <div class="bg-slate-800 p-6 rounded-lg bord
 <div class="absolute -top-3 left-4 bg-s

```

```

title: "Билет 14. Дейкстра",
type: "html",
category: "Графы",
content: `<section id="dijkstra-content" class="mb-
<div class="flex items-center mb-6">
 <span class="bg-blue-600 text-white px-4 py-1 rounded
 <h2 class="text-3xl font-bold text-white">Дейкст
</div>

<div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border
 <h3 class="text-xl font-bold text-emerald-400">Аналогия 1:
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-emerald-300 italic">Аналогия 1:
 <p class="text-xl font-mono text-white">Аналогия 1:
 </div>

 <div class="grid grid-cols-1 xl:grid-cols-2">
 <!-- Аналогия 1 -->
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
 Аналогия 1: Позитивное планирование
 </div>
 <p class="text-slate-300 text-sm mb-4">Аналогия 1: Позитивное планирование
 (нельзя поехать и заработать). Он всегда выбирает кратчайший путь.
 <div id="slot-chapter-image-dijkstra" class="h-400px">
 </div>

 <!-- Аналогия 2 -->
 <div class="bg-slate-900 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border border-rose-500 shadow-md">
 Ограничения
 </div>
 <p class="text-slate-300 text-sm mb-4">Ограничения
 (неожиданное ограничение: нельзя ехать по одной и той же улице дважды), Дейкстра сломается, так как

```

```

<div class="grid grid-cols-1 xl:grid-cols-2"
 <!-- Аналогия 1 -->
 <div class="bg-slate-800 p-6 rounded-lg"
 <div class="absolute -top-3 left-4 l
rder border-emerald-500 shadow-md">
 ♂ Аналогия 1: Параноик
 </div>
 <p class="text-slate-300 text-sm mb
еряет ВСЕ возможные дороги V-1 раз подряд.
 <div id="slot-chapter-image-bellman
 </div>

 <!-- Аналогия 2 -->
 <div class="bg-slate-900 p-6 rounded-lg"
 <div class="absolute -top-3 left-4 l
border-rose-500 shadow-md">
 Отрицательный цикл
 </div>
 <p class="text-slate-300 text-sm mb
г станет ЕЩЕ короче – значит мы попали во в
 </div>
 </div>

 <div id="slot-bellman-viz" class="mt-8"></div>
</div>
</section>`
},
{
 id: "floyd",
 title: "Билет 16. Флойд-Уоршелл",
 type: "html",
 category: "Графы",
 content: `<section id="floyd-content" class="mb-12"
<div class="flex items-center mb-6">
 <span class="bg-blue-600 text-white px-4 py-1 r
 <h2 class="text-3xl font-bold text-white">Флойд

```



```

 },
 {
 id: "aho-corasick",
 title: "Билет 23. Алгоритм Ахо-Корасик",
 type: "html",
 category: "Строки",
 content: `<section id="aho-corasick" class="mb-12 s
<div class="flex items-center mb-6 flex-wrap gap-3">
 <span class="bg-indigo-600 text-white px-4 py-1 round
 <h2 class="text-2xl sm:text-3xl font-bold text-white
</div>

<div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border-l
 <h3 class="text-xl font-bold text-blue-400 mb-4">

 <div class="bg-slate-900 p-4 rounded-lg mb-6 text
 <p class="text-sm text-blue-300 italic mb-2">0с
 <p class="text-lg font-mono text-white">Бор (Tr
 </div>
 <p class="text-slate-300 text-sm">Позволяет найти
 -mono text-amber-300 bg-black/30 px-1 rounded">
 y.</p>
 </div>

<!-- Аналогии -->
<div class="grid grid-cols-1 xl:grid-cols-2 gap-6">
 <div class="bg-slate-800 p-6 rounded-lg border bo
 <div class="absolute -top-3 left-4 bg-slate-700
 emerald-500 shadow-md">
 Аналогия 1: Паутина (Бор)
 </div>
 <p class="text-slate-300 text-sm mb-4">Представ
 мы двигаемся по веткам. Если нужного продолж
 ("нити страховки") в самый длинный из извест
 </div>

 <div class="bg-slate-900 p-6 rounded-lg border-2

```

```

while (!q.empty()) {
 int v = q.front(); q.pop();
 for (auto const& [ch, child] : nodes[v].trie) {
 // Суффиксная ссылка ребёнка — это переход
 nodes[child].link = nodes[nodes[v].link].go

 // Если суффиксная ссылка сама является сло
 // Иначе наследуем terminal_link
 if (nodes[nodes[child].link].is_terminal) {
 nodes[child].term_link = nodes[child].l
 } else {
 nodes[child].term_link = nodes[nodes[ch
 }

 q.push(child);
 }
}

</div>
</div>
</details>
</div>
</section>`
},
{
 id: "alg-map",
 title: "Алгоритмы на карте",
 type: "html",
 category: "Графы",
 content: `<section id="alg-map-content" class="mb-1
<div class="flex items-center mb-6">
 <span class="bg-purple-600 text-white px-4 py-1
 <h2 class="text-3xl font-bold text-white">Алгори
</div>

<div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl bord

```

```

 options: [
 "Да: минимум и сумма считаются одним приёмом",
 "Нет: блоки перекрываются, а вычесть перекрытие",
 "Да, если заранее хранить суммы в нулевом уровне",
],
 correctIndex: 1,
 explanation: "Два (в 2D – четыре) блока накрывают
 обратная операция, но применить её к перекрытию
 ммы, билет 5."
},
{
 question: "Какие операции подходят разреженной та
 options: [
 "Любые ассоциативные",
 "Идемпотентные: min, max, gcd, побитовые И и ИЛИ",
 "Только сумма и произведение"
],
 correctIndex: 1,
 explanation: "Нужно $op(x, x) = x$: тогда дважды на
 приём и уходит в префиксные суммы."
},
{
 question: "Между запросами элементы матрицы меняю
 options: [
 "Разреженную таблицу: она перестраивается за $O(n^2)$ ",
 "Префиксную матрицу: достаточно поправить одну
 "Дерево отрезков: и таблица, и префиксы статичны",
],
 correctIndex: 2,
 explanation: "Обе структуры статичны: изменение э
 ксной. С обновлениями работает дерево отрезков,
 }
},
"prefix-sums-2d": [
 {
 question: "Сколько памяти занимает двумерная преф
 options: [
 "N × M – как исходная матрица, плюс рамка из ну

```

```

 ной операции нет. Для min, max и gcd берут разр
 },
 {
 question: "Изменили один элемент матрицы. Что ста
 options: [
 "Поправится одна клетка за $O(1)$ ",
 "Нужен пересчёт $O(N \cdot M)$: от этой клетки зависит
 "Ничего не станет: таблица остаётся верной"
],
 correctIndex: 1,
 explanation: "Каждая клетка $S[i][j]$ включает в се
 я частые, это задача дерева отрезков (билет 1).
 }
],
"euler-path-vs-cycle": [
 {
 question: "В связном графе ровно 2 вершины с нечёт
 options: [
 "Есть эйлеров цикл (вернись домой!)",
 "Есть эйлеров путь (из одной нечётной в другую)
 "Ни пути, ни цикла нет — это полный провал"
],
 correctIndex: 1,
 explanation: "Если нечётных вершин ровно 2 — это
 ётной, заканчиваем в другой."
 },
 {
 question: "Почему гамильтонов цикл — это NP-полна
 options: [
 "Потому что Гамильтон — болезнь, и лечить её т
 "Потому что для эйлерова цикла есть простой кри
 "Потому что Эйлер был умнее Гамильтона"
],
 correctIndex: 1,
 explanation: "Для эйлерова цикла работает критери
 ого критерия. Это NP-полная задача, которую не
 },
 {

```

```

 "O(2^V) – полный перебор"
],
 correctIndex: 1,
 explanation: "DFS обходит каждую вершину и каждое
}
],
"planarity-euler-formula": [
{
 question: "Планарный граф имеет V=6, E=12. Возможн
 options: [
 "Да, если это двудольный граф",
 "Нет, нарушается неравенство $E \leq 3V - 6$ ",
 "Да, если нет треугольных граней"
],
 correctIndex: 1,
 explanation: " $E \leq 3V - 6 \Rightarrow 12 \leq 12$ – на грани пр
 Так что такой граф непланарен."
 },
 {
 question: "Сколько граней у планарного графа с V=
 options: [
 "3 грани",
 "5 граней",
 "10 граней"
],
 correctIndex: 1,
 explanation: " $F = E - V + 2 = 10 - 7 + 2 = 5$. Не
 },
 {
 question: "Почему K5 непланарен?",
 options: [
 "Потому что $10 > 3*5 - 6 = 9$ – нарушение",
 "Потому что Эйлер был против",
 "Потому что 5 вершин – несчастливое число"
],
 correctIndex: 0,
 explanation: "У K5: V=5, E=10, $3V - 6 = 9$. $10 > 9$
 }
}

```

```

/** Транслировать реальные переменные визуализациям тек
export function emitVizState(
 chapterId: string,
 snapshot: Omit<VizStateEventDetail, "chapterId">
) {
 if (typeof window === "undefined") return;
 const detail: VizStateEventDetail = { chapterId, ...s
 latestByChapter.set(chapterId, detail);
 window.dispatchEvent(new window.CustomEvent<VizStateE
}

/** Вернуть демонстрацию в ручной режим при закрытии/ре
export function clearVizState(chapterId: string) {
 if (typeof window === "undefined") return;
 latestByChapter.delete(chapterId);
 window.dispatchEvent(new window.CustomEvent<VizStateE
}

/** Текущий снимок компилятора для прямой отрисовки мас
export function useVizRuntime(): VizStateEventDetail | n
 const chapterId = useContext(VizChapterContext);
 const [runtime, setRuntime] = useState<VizStateEventD
 chapterId ? latestByChapter.get(chapterId) ?? null
);

 useEffect(() => {
 setRuntime(chapterId ? latestByChapter.get(chapterId
 if (!chapterId) return;
 const handler = (e: Event) => {
 const detail = (e as CustomEvent<VizStateEventDet
 if (detail?.chapterId === chapterId) {
 setRuntime(detail.variables || detail.line !==
 }
 };
 window.addEventListener(CHANNEL, handler);
 return () => window.removeEventListener(CHANNEL, ha
 }, [chapterId]);

```

```

 }, [chapterId, currentStep, maxStep, goToStep, selectStep]
 }
}

```

```

/** Утилиты без небезопасных cast-ов для визуализаторов
export const vizNumber = (value: DebugValue | undefined) => {
 typeof value === "number" && Number.isFinite(value) ? value : null;
}

```

```

export const vizString = (value: DebugValue | undefined) => {
 typeof value === "string" ? value : null;
}

```

```

export const vizArray = (value: DebugValue | undefined) => {
 Array.isArray(value) ? value : null;
}

```

```

export const vizRecord = (value: DebugValue | undefined) => {
 value !== null && typeof value === "object" && !Array.isArray(value)
 ? (value as Record<string, DebugValue>)
 : null;
}

```

```

/* — Матрицы из компилятора: СВОЯ размерность, а не дефолтная

```

```

/**
 * Нормализованная матрица, пришедшая из выполненного Рунима
 *
 * Размерность берётся из самих данных: если пользователь
 * ввёл 5 вершин, визуализация рисует 5x5, а не подставляет его
 * 7x7 (раньше ячейки «не той» размерности превращались в 0)
 *
 * Поддерживаются все учебные способы хранения:
 * list[list] d = [[0, 4], [999, 0]]
 * list[dict] d = [{0: 0, 1: 4}, {1: 0}]
 * dict[dict] d = {0: {0: 0, 1: 4}}
 * dict[(i, j)] d = {(0, 1): 4} — ключ после запятой
 */

```

```

export interface LiveMatrix {
 rows: number;
 cols: number;
 /** Значение ячейки; null — таких данных в коде пользователя нет
 at: (i: number, j: number) => number | null;
}

```

```

 if (n === null) return;
 cells.set(`${i}:${j}`, n);
 rows = Math.max(rows, i + 1);
 cols = Math.max(cols, j + 1);
};

const asArray = vizArray(value);
if (asArray) {
 asArray.slice(0, MAX_MATRIX_SIDE).forEach((row, i) => {
 const inner = vizArray(row);
 if (inner) {
 inner.slice(0, MAX_MATRIX_SIDE).forEach((cell, j) => {
 return;
 });
 }
 const record = vizRecord(row);
 if (record) {
 for (const [key, cell] of Object.entries(record)) {
 const j = Number(key);
 if (Number.isInteger(j)) put(i, j, cell);
 }
 }
 });
} else {
 const record = vizRecord(value);
 if (!record) return null;
 for (const [key, cell] of Object.entries(record)) {
 const pair = parsePairKey(key);
 if (pair) {
 put(pair[0], pair[1], cell);
 continue;
 }
 const i = Number(key);
 if (!Number.isInteger(i)) continue;
 const inner = vizArray(cell);
 if (inner) {
 inner.slice(0, MAX_MATRIX_SIDE).forEach((v, j) => {
 continue;
 });
 }
 }
}

```



-----  
/\* — Выбор демонстрации внутри страницы (вкладки 1D / 2D)

```
const DEMO_CHANNEL = "algo:viz-demo";
const latestDemoByChapter = new Map<string, string>();

export interface VizDemoEventDetail {
 chapterId: string;
 /** base-запись страницы – demoId пустой; подписанные */
 demoId: string;
}

/** Демонстрация сообщает, какая её вкладка сейчас открыта */
export function emitVizDemo(chapterId: string, demoId: string) {
 if (typeof window === "undefined") return;
 latestDemoByChapter.set(chapterId, demoId);
 window.dispatchEvent(new window.CustomEvent<VizDemoEventDetail>({
 detail: { chapterId, demoId }
 }));
}

/**
 * Подписка: панель компилятора следует за активной вкладкой
 * Сразу отдаём последнее значение: компилятор часто отстает
 * как пользователь переключился с 1D на 2D.
 */
export function onVizDemo(chapterId: string, cb: (demoId: string) => void) {
 if (typeof window === "undefined") return () => {};
 const latest = latestDemoByChapter.get(chapterId);
 if (latest !== undefined) cb(latest);
 const handler = (e: Event) => {
 const detail = (e as CustomEvent<VizDemoEventDetail>).detail;
 if (detail && detail.chapterId === chapterId) cb(detail.demoId);
 };
 window.addEventListener(DEMO_CHANNEL, handler);
 return () => window.removeEventListener(DEMO_CHANNEL, handler);
}
```

## Универсальное пособие • полный исходный код

```

tree = [0] * (2 * n)
i = 0 # лист: tree[n + i]

v, l, r = 1, 0, n - 1 # вершина и её отрезок
m = (l + r) // 2 # граница детей 2v, 2v+1
print(f"v={v} [{l}..{r}] mid={m}")`,
 variables: [
 { name: "n", role: "длина исходного массива; листы"},
 { name: "i", role: "индекс элемента массива; его левый и правый дети"},
 { name: "v", role: "текущая вершина дерева отрезка"},
 { name: "l, r", role: "границы отрезка, за которыми лежат левый и правый дети"},
 { name: "m", role: "середина отрезка m = (l + r) // 2"},
],
},
treap: {
 vizTitle: "Декартово дерево: build → split → merge → erase",
 stepNote: "Шаг = один спуск в split/merge/erase (перезапись массива целиком.",
 code: `points = [[45, 8], [3, 6], [6, 5], [2, 4], [1, 3], [5, 7], [4, 2], [7, 9]]
n = len(points) # сколько пар (x; y) – столько точек
mid = 7 # разрез для split: x <= mid y > mid
key = 9 # какую точку удаляем в erase
step = 0 # номер шага – листает кадры декартова дерева
height = 0 # высота построенного дерева
root = None

--- тело алгоритма ---
class Node:
 def __init__(s, x, y):
 s.x = x; s.y = y; s.l = None; s.r = None

def build(pts):
 """Декартово дерево из отсортированных по x точек:
 pts = sorted(pts, key=lambda p: p[0])
 stack = []
 for x, y in pts:
 cur = Node(x, y)
 last = None
```



```

while x.p:
 p = x.p; g = p.p
 step += 1
 if g is None:
 case = "zig" # родители
 rot(x)
 elif (g.l is p) == (p.l is x):
 case = "zig-zig" # x и p -
 rot(p); rot(x)
 else:
 case = "zig-zag" # змейка
 rot(x); rot(x)
 xk, pk, gk = x.k, p.k, (g.k if g else 0)
 print("шаг", step, "->", case, "| x =", xk, "p = ", pk)
 root = x
 print(" корень теперь", x.k, "| обход:", inorder(root))
 return x

```

# 1) Zig: родитель x – уже корень (дерево 40 -> 20)  
 root = N(40); root.l = N(20); root.l.p = root  
 splay(root.l)

# 2) Zig-Zig: бамбук 60 -> 40 -> 20, оба сына левые  
 root = N(60); root.l = N(40); root.l.p = root; root.l.l = N(20)  
 splay(root.l.l)

# 3) Zig-Zag: змейка 60 -> 20 -> 40 (40 – правый сын 20)  
 root = N(60); root.l = N(20); root.l.p = root; root.l.r = N(40)  
 splay(root.l.r),

```

variables: [
 { name: "case", role: "какой случай сработал: \"zig-zig\", \"zig-zag\", \"zig\""},
 { name: "step", role: "номер применения случая – 1, 2, 3"},
 { name: "xk / x", role: "ключ узла, который поднимается"},
 { name: "pk / p", role: "ключ родителя x перед поворотом"},
 { name: "gk / g", role: "ключ деда – нужен в Zig-Zig"},
 { name: "root", role: "корень дерева после расшейки"},
]

```

```

 stepNote: "Шаг = заполнение очередного $P[i] = P[i-1]$
code: `a = [3, 1, 4, 1, 5, 9, 2, 6]` # исходный массив
n = len(a) # длина: префиксов будет n
i = 0 # текущая граница префикса

P = [0] # $P[0] = 0$ – пустой префикс
for i in range(1, n + 1):
 P.append(P[i - 1] + a[i - 1])
print("n =", n, "| P =", P)` ,
 variables: [
 { name: "a", role: "исходный массив 1D-демонстрация" },
 { name: "n", role: "длина массива a; префиксов будет n" },
 { name: "P", role: "префиксные суммы: $P[i] = a[0] + \dots + a[i-1]$ " },
 { name: "i", role: "граница префикса: на шаге считаем до i" },
],
},
"dynamic-programming": {
 vizTitle: "Динамическое программирование (мемоизация)",
 stepNote: "Шаг = событие внутри fibМемо(k): вызов, возврат",
 code: `n = 5
memo = {}

def fib(k):
 if k in memo:
 return memo[k]
 if k <= 2:
 return 1
 print(f"считаем fib({k-1}) + fib({k-2})")
 r1 = fib(k - 1)
 r2 = fib(k - 2)
 memo[k] = r1 + r2
 return memo[k]

print("итог:", fib(n))`,
 variables: [
 { name: "n", role: "целевой номер числа Фибоначчи" },
 { name: "k", role: "аргумент текущего рекурсивного вызова" },
 { name: "memo", role: "кэш уже посчитанных значений" },
],
},

```

```

code: `from collections import deque

q = deque(["A"]) # очередь BFS
u = "B" # сосед, найденный из текущ
q.append(u) # enqueue соседа – в конец
front = q[0] # голова очереди: её обрабо
v = q.popleft() # шаг: достали голову и обр
print("front =", front, "| v =", v, "| очередь:", list(
 variables: [
 { name: "front", role: "голова очереди – вершина,
 { name: "v", role: "вершина, извлечённая из очере
 { name: "u", role: "сосед вершины v, которого кла
],
},
"everyday-basics": {
 vizTitle: "Бытовой тренажёр: стек, очередь, куча",
 stepNote: "Шаг = одна бытовая операция: тарелка на
 code: `from collections import deque

st, q = [], deque() # тарелки (стек) и очередь
st.append(" "); top = st.pop()
q.append(" "); q.append(" "); first = q.popleft()

i = 0 # счётчик выполненных опера
for op in ("push", "pop", "enqueue", "dequeue"):
 i += 1
print("тарелка:", top, "| первый из очереди:", first, "
 variables: [
 { name: "top / front", role: "верх стопки (стек)
 { name: "i", role: "индекс в массиве кучи; родите
],
},
"graph-dfs-bfs": {
 vizTitle: "DFS и BFS на графе",
 stepNote: "Шаг = переход из текущей вершины v к неп
 code: `from collections import deque

adj = {"A": ["B", "C"], "B": ["D"], "C": [], "D": []}
```

## Универсальное пособие • полный исходный код

```

v = -1 # вершина, в которой обход начался
stack = [] # фронт обхода (стек для DFS)
i = 0 # номер шага — двигает по какому-то ребру

--- тело алгоритма ---
for start in range(n):
 if comp[start] != -1:
 continue
 comps += 1 # непокрашенная вершина => новая компонента
 c = comps
 stack = [start]
 comp[start] = c
 used[start] = True
 v = start
 while stack:
 v = stack.pop() # для BFS: v = stack.pop(0)
 i += 1
 for to in adj[v]:
 if comp[to] == -1:
 comp[to] = c
 used[to] = True
 stack.append(to)
print("компонент:", comps)
print("разметка comp:", comp)`
variables: [
 { name: "n", role: "число вершин; в демонстрации задано 10" },
 { name: "adj", role: "списки смежности — тот же граф, что в примере" },
 { name: "comp", role: "разметка: номер компоненты", range: "-1 ... comps" },
 { name: "used", role: "булев вариант разметки — для BFS не нужен" },
 { name: "comps", role: "счётчик найденных компонент" },
 { name: "c", role: "номер компоненты, которую крашим сейчас" },
 { name: "v", role: "вершина, где обход находится" },
 { name: "stack", role: "фронт обхода: стек (DFS) или очередь (BFS)" },
 { name: "i / step", role: "номер шага — перематываем шаг" },
],
},
"top-sort": {
```

```
def dfs1(v):
 used[v] = True
 for to in adj[v]:
 if not used[to]:
 dfs1(to)
 order.append(v) # выходим из вершины — она

for v in adj:
 if not used[v]:
 dfs1(v)
 print("DFS из", v, "закончен | order:", order)

транспонирование: каждое ребро разворачиваем
adjT = {}
for v in adj:
 adjT[v] = []
for v in adj:
 for to in adj[v]:
 adjT[to].append(v)
print("обратный граф:", adjT)

фаза 2: идём по order С КОНЦА, DFS по adjT красит одн
comp = {}
c = 0
for i in range(len(order) - 1, -1, -1):
 v = order[i]
 if v in comp:
 continue
 stack = [v]
 comp[v] = c
 while stack:
 u = stack.pop()
 for to in adjT[u]:
 if to not in comp:
 comp[to] = c
 stack.append(to)
 print("SCC", c, "собрана из", v, ":", comp)
```



```
},
 "bridges-code": {
 vizTitle: "Мосты: DFS + tin/low",
 stepNote: "Шаг = строка псевдокода слева; ребро (v,
 code: `tin, low = {"A": 1, "B": 2, "G": 3}, {"A": 1
timer = 3 # как на демо

v, to = "B", "G" # шаг: возврат из G
is_bridge = low[to] > tin[v] # 3 > 2 → мост!
print(f"ребро {v}-{to}: мост? {is_bridge}")`,
 variables: [
 { name: "v", role: "текущая вершина DFS", range:
 { name: "to", role: "сосед; ребро (v, to) проверяе
 { name: "tin[v]", role: "время входа в вершину – m
 { name: "low[v]", role: "минимум tin, куда можно ,
 { name: "timer", role: "глобальный счётчик времени
],
 },
 "euler-path-vs-cycle": {
 vizTitle: "Эйлеров маршрут: алгоритм Иера",
 stepNote: "Шаг = переход по неиспользованному ребру
 идти нельзя.",
 code: `n = 5 # вершин: 0...4 (на
edges = [(0, 1), (0, 2), (1, 3), (2, 4), (1, 2), (3, 4)]
adj = {v: [] for v in range(n)}
for i, (u, v) in enumerate(edges):
 adj[u].append((v, i))
 adj[v].append((u, i))
deg = {v: len(adj[v]) for v in range(n)} # степени ве
odd = [v for v in range(n) if deg[v] % 2 == 1] # нечёт
start = odd[0] if odd else 0 # путь начинаем
used = [False] * len(edges) # какие рёбра :
path = [start] # маршрут: пос.
last = start # где мы сейчас
stack = [start] # стек для отк
v = start
step = 0
```

```

V = n # вершин для формулы
E = sum(len(a) for a in adj) // 2 # рёбер: в списке смежн
F = 2 - V + E # граней: из $V - E + F = 2$

--- тело алгоритма ---
print("V =", V, "| E =", E, "| F =", F, "| $V - E + F = 2$ ")
for v in range(n):
 used = set()
 for u in adj[v]:
 if color[u] != 0:
 used.add(color[u])
 c = 1
 while c in used: # наименьший свободный цвет
 c += 1
 color[v] = c
 chi = max(chi, c)
 i += 1
 print("v =", v, "занято:", sorted(used), "-> цвет", color[v])
print("chi =", chi, "| delta =", delta, "| chi <= delta")
variables: [
 { name: "n", role: "число вершин графа на демонстрации" },
 { name: "adj", role: "списки смежности — тот же граф" },
 { name: "color", role: "массив цветов вершин (0 = не покрашено)" },
 { name: "v", role: "вершина, которую красим сейчас" },
 { name: "u", role: "сосед вершины v, чей цвет уже занят" },
 { name: "delta", role: "максимальная степень $\Delta = \max \deg v$ " },
 { name: "chi", role: "сколько цветов реально понадобится" },
 { name: "i / step", role: "номер шага — перематываем таймер" },
 { name: "V, E, F", role: "для вкладки «Планарность»" },
]
},
dijkstra: {
 vizTitle: "Дейкстра",
 stepNote: "Шаг = старт, извлечение ближайшей вершины",
 code: `import heapq

```

## Универсальное пособие • полный исходный код

```

nodes = ["S", "A", "B", "C", "D", "E", "F"]
n = len(nodes) # 7 вершин => n - 1 = 6 работ
m = len(E) # 9 рёбер
```

```
dist = {v: INF for v in nodes}
dist["S"] = 0 # старт – База
```

```
--- тело алгоритма ---
```

```
last_changed = ""
for i in range(1, n): # итерация i = 1 ... n-1
 last_changed = ""
 for u, v, w in E: # шаг: релаксация ребра (u, v)
 if dist[u] != INF and dist[u] + w < dist[v]:
 dist[v] = dist[u] + w
 last_changed = v
 if last_changed == "":
 print(f"итерация {i}: изменений нет – сошлось для всех вершин")
 break
 print(f"итерация {i}: " + ", ".join(f"{k}={dist[k]}:" for k in nodes))
```

```
контрольный (n-й) проход: улучшение здесь означает отрицательный цикл
cycle_edge = None
for u, v, w in E:
 if dist[u] != INF and dist[u] + w < dist[v]:
 cycle_edge = (u, v, w)
 break
```

```
if cycle_edge:
 print("отрицательный цикл: улучшается ребро", cycle_edge)
else:
```

```
 print("отрицательных циклов нет:", {k: dist[k] for k in nodes})
 variables = [
 { name: "n", role: "число вершин графа – ровно n-1"},
 { name: "m", role: "число рёбер, по которым проходят"},
 { name: "i", role: "номер итерации (после i итераций)"},
 { name: "u, v, w", role: "текущее ребро: откуда, куда"},
 { name: "dist[v]", role: "расстояние; если обновилось"},
],
```

```

 на-Форда" },
 { name: "u, v", role: "ребро (u, v) перевзвешивает",
 { name: "d[v]", role: "расстояние Дейкстры в новом графе"
 },
],
},
mst: {
 vizTitle: "Остовное дерево: Краскал и DSU",
 stepNote: "Шаг = следующее ребро из отсортированных",
 code: `n = 9 # вершин: A..I — те, что в графе
edges = [(2, "A", "B"), (3, "D", "E"), (4, "B", "C"), (5, "A", "C"),
 (7, "E", "F"), (8, "D", "G"), (9, "C", "F"), (10, "D", "F"),
 (12, "H", "I"), (13, "F", "I")]
edges.sort() # Краскал: рёбра по возрастанию веса
parent = {v: v for v in "ABCDEFGH I"} # DSU: представителю
rank = {v: 0 for v in "ABCDEFGH I"} # высота дерева
mst = [] # рёбра, взятые в остов
total = 0 # вес остова (на этом графе)
i = 0 # номер текущего ребра
u, v, w = None, None, None # концы и вес ребра, которое

--- тело алгоритма ---
def find(x):
 while parent[x] != x:
 parent[x] = parent[parent[x]] # сжатие путей
 x = parent[x]
 return x

for i, (w, a, b) in enumerate(edges):
 u, v = a, b
 ra, rb = find(a), find(b)
 if ra != rb: # разные компоненты
 parent[ra] = rb
 if rank[ra] == rank[rb]:
 rank[rb] += 1
 mst.append((w, a, b))
 total += w
print("вес остова:", total, "| рёбер:", len(mst))

```

```

vizTitle: "Z-функция",
stepNote: "Шаг = вычисление z[i]: внутри Z-блока [l, r]",
code: `s = "abacaba"
n = len(s)
i, l, r = 1, 0, 0 # правый Z-блок [l, r]
z = [None] * n # пустые клетки: значения по умолчанию
z[0] = 0

for i in range(1, n): # шаг: вычисляем z[i]
 if i <= r:
 z[i] = min(r - i + 1, z[i - l]) # из блока
 else:
 z[i] = 0 # свежая клетка: с нуля
 while i + z[i] < n and s[z[i]] == s[i + z[i]]:
 z[i] += 1 # досчёт в лоб
 if i + z[i] - 1 > r:
 l, r = i, i + z[i] - 1
print(z)`
variables: [
 { name: "n", role: "длина строки s", range: "n = len(s)" },
 { name: "i", role: "позиция, для которой считаем z[i]" },
 { name: "l, r", role: "правый крайний Z-блок: отрезок [l, r]" },
 { name: "z[i]", role: "длина наибольшего общего префикса s[i:] и s[0:z[i]]" }
],
},
"aho-corasick": {
 vizTitle: "Ахо-Корасик: бор, fail-ссылки и автомат",
 stepNote: "Шаг = один символ текста: state = go(state, c). Если не найдено, то переходят в корень. Если найдено, то переходят с картинкой «водопада».",
 code: `words = ["HERS", "HIS", "SHE", "HE"] # образцы
text = "USHERS" # текст, в котором ищем
n = len(text) # длина текста
state = 0 # текущее состояние автомата
i = 0 # индекс текущего символа
c = "" # сам символ
nodes = 1 # сколько вершин в автомате
queue = [] # очередь BFS, к которой добавляем новые вершины
fail = {} # fail[v] = вершина, в которую переходим, если не найдено

```

```

 queue.append(u)
print("вершин в боре:", nodes)
print("fail-ссылки:", fail)

def go(v, ch):
 """Переход автомата: сначала по бору, иначе спускаем
 while v != 0 and (v, ch) not in nxt:
 v = fail[v]
 return nxt.get((v, ch), 0)

for i, c in enumerate(text):
 state = go(state, c)
 v = state
 while v != 0: # собираем все образцы
 if v in term:
 matches.append((i, term[v]))
 v = fail[v]
 print(f"i={i} c='{c}' state={state} найдено={matches}")

print("всего вхождений:", len(matches), matches),
variables: [
 { name: "words", role: "набор образцов – по нему",
 },
 { name: "text", role: "сканируемый текст – подста",
 { name: "state", role: "текущее состояние автомата",
 { name: "i / c", role: "индекс и сам символ текста",
 { name: "nxt", role: "переходы бора: nxt[(v, ch)]",
 { name: "fail", role: "fail[v] – вершина наибольшей",
 { name: "term", role: "в каких вершинах заканчивае",
 { name: "queue", role: "очередь BFS, которой натя",
 { name: "matches", role: "найденные вхождения: (ин",
 { name: "nodes", role: "сколько вершин в боре (ко",
],
},
"complexity-classes": {
 vizTitle: "Классы сложности: рост по n и ops",
 stepNote: "Шаг = одна точка (n, ops): визуализация",
 code: `n = 1 # размер входа

```

```

 },
 intro: {
 // мнемодкарточки без шагов алгоритма
 variables: [],
 },

 /* — Дополнительные демонстрации внутри страниц (вкл.)

 "planarity-euler-formula#greedy": {
 vizTitle: "Жадная покраска вершин ($\chi \leq \Delta + 1$)",
 stepNote: "Шаг = вершина v получает наименьший цвет",
 code: `n = 8 # вершин в графе
adj = [[1, 2], [0, 2, 3], [0, 1, 4], [1, 4, 5], [2, 3, 4]]
delta = max(len(a) for a in adj) # максимальная степень
color = [0] * n # 0 = ещё не покрашена
chi = 0 # сколько цветов понадобилось
v = -1 # вершина, которую красим сейчас
i = 0 # номер шага

--- тело алгоритма ---
for v in range(n):
 used = set()
 for u in adj[v]:
 if color[u] != 0:
 used.add(color[u])
 c = 1
 while c in used: # наименьший свободный цвет
 c += 1
 color[v] = c
 chi = max(chi, c)
 i += 1
 print("v =", v, "занято:", sorted(used), "-> цвет", color[v])
print("chi =", chi, "| delta =", delta, "| chi <= delta")
 variables: [
 { name: "color", role: "цвета вершин (0 = не покрашено)", range: "0 ... n-1" },
 { name: "v", role: "текущая вершина", range: "0 ... n-1" },
 { name: "delta / chi", role: "максимальная степень / количество цветов", range: "0 ... n" },
 { name: "i / step", role: "номер кадра", range: "0 ... n" },
]
 }
}
 `

```

```

],
},
"planarity-euler-formula#edges": {
 vizTitle: "Покраска рёбер (Визинг: $\Delta \leq \chi' \leq \Delta + 1$)",
 stepNote: "Шаг = ребро e получает наименьший цвет, <= \Delta",
 code: `n = 8
edges = [(0, 1), (0, 2), (1, 2), (1, 3), (2, 4), (3, 4), (4, 5)]
adj = [[1, 2], [0, 2, 3], [0, 1, 4], [1, 4, 5], [2, 3, 4], [3, 4, 5]]
delta = max(len(a) for a in adj)
ecolor = [0] * len(edges) # цвет каждого ребра
inc = [dict() for _ in range(n)] # inc[v][цвет] = ребро, которое имеет этот цвет
e = -1 # номер текущего ребра
v = u = -1 # его концы
chi_e = 0

--- тело алгоритма ---
for e, (a, b) in enumerate(edges):
 v, u = a, b
 used = set(inc[a].values()) | set(inc[b].values())
 c = 1
 while c in used:
 c += 1
 ecolor[e] = c
 inc[a][e] = c
 inc[b][e] = c
 chi_e = max(chi_e, c)
 print("ребро", (a, b), "занято:", sorted(used), "-> ", c)
print("chi' =", chi_e, "| delta =", delta, "| delta <= chi'")
variables: [
 { name: "ecolor", role: "цвета рёбер – рёбра на графе", range: "0..delta" },
 { name: "e", role: "номер текущего ребра", range: "0..len(edges)-1" },
 { name: "v, u", role: "концы текущего ребра – подмножества вершин", range: "0..n-1" },
 { name: "inc", role: "для каждой вершины: какие цвета имеют инцидентные рёбра", range: "0..delta" },
 { name: "chi_e / delta", role: "χ' – сколько цветов нужно", range: "0..delta+1" }
],
},
"graph-components#dfs": {
 vizTitle: "Компоненты связности: DFS-покраска",

```



```

 },
 "graph-components#dsu": {
 vizTitle: "Компоненты связности: DSU по рёбрам",
 stepNote: "Шаг = одно ребро (u, v): union склеивает",
 code: `n = 10 # вершин в графе
edges = [(0, 1), (1, 2), (2, 3), (0, 3), (1, 3), (4, 5)]
parent = list(range(n)) # DSU: parent[v] = представи
rank = [0] * n # высота дерева – по ней реш
comps = n # сначала каждая вершина – о
e = -1 # номер обрабатываемого ребра
u, v = -1, -1 # концы текущего ребра
step = 0 # номер шага – двигает по ка

--- тело алгоритма ---
def find(x):
 while parent[x] != x:
 parent[x] = parent[parent[x]] # сжатие путей
 x = parent[x]
 return x

def union(a, b):
 global comps
 ra, rb = find(a), find(b)
 if ra == rb:
 return False # ребро внутри
 if rank[ra] < rank[rb]:
 ra, rb = rb, ra
 parent[rb] = ra
 if rank[ra] == rank[rb]:
 rank[ra] += 1
 comps -= 1
 return True

for e, (a, b) in enumerate(edges):
 u, v = a, b
 step = e + 1
 union(a, b)
print("компонент:", comps)

```

```

k = 3 # размер клики, которую ищем

--- тело алгоритма ---
Сведение 3-SAT <=p КЛИКА: вершина = {номер дизъюнкта,
ребро = два литерала из РАЗНЫХ дизъюнктов, не противоречивые
clauses = [(1, 2, -3), (-1, 3, 4), (2, -3, -4)]
V = [(i, l) for i, cl in enumerate(clauses) for l in cl]
E = [(a, b) for a in V for b in V if a[0] < b[0] and a[1] != b[1]]
for step in range(7):
 print("кадр", step, "| вершин:", len(V), "| рёбер:", len(E))
print("формула выполнима <=> в графе есть клика размера k")
variables: [
 { name: "step", role: "номер кадра сведения (0..6)" },
 { name: "cls", role: "класс задачи A, которую сводят к клике" },
 { name: "n", role: "размер входа x; после сведения к клике" },
 { name: "k", role: "размер клики, которую ищет рекурсия" },
],
},
"complexity-classes#verify": {
 vizTitle: "Верификатор 3-SAT: проверка сертификата",
 stepNote: "Шаг = один дизъюнкт: верификатор подставляет значения в формулу",
 code: `clauses = [(1, 2, -3), (-1, 3, 4), (2, -3, -4)]
cert = [1, 0, 1, 0] # сертификат: x1=1, x2=0, x3=1
k = 0 # номер проверяемого дизъюнкта
ok = True # вердикт верификатора

--- тело алгоритма ---
for k, cl in enumerate(clauses):
 hit = False
 for l in cl:
 val = cert[l - 1] if l > 0 else 1 - cert[-l - 1]
 if val == 1:
 hit = True
 ok = ok and hit
 print("дизъюнкт", k + 1, cl, "выполнен" if hit else "не выполнен")
print("сертификат", "принят" if ok else "отвергнут")`,
 variables: [
 { name: "cert", role: "сертификат – назначение переменным значений 0 или 1" },
],
},

```

```

edges.sort() # Краскал: рёбра по возрастанию веса
parent = {v: v for v in "ABCDEFGHI"} # DSU: представител
rank = {v: 0 for v in "ABCDEFGHI"} # высота дерева
mst = [] # рёбра, взятые в остов
total = 0 # вес остова (на этом графе 51)
i = 0 # номер текущего ребра
u, v, w = None, None, None # концы и вес ребра, которое рассматриваем

--- тело алгоритма ---
def find(x):
 while parent[x] != x:
 parent[x] = parent[parent[x]] # сжатие путей
 x = parent[x]
 return x

for i, (w, a, b) in enumerate(edges):
 u, v = a, b
 ra, rb = find(a), find(b)
 if ra != rb: # разные компоненты
 parent[ra] = rb
 if rank[ra] == rank[rb]:
 rank[rb] += 1
 mst.append((w, a, b))
 total += w
print("вес остова:", total, "| рёбер:", len(mst))
print(mst)`
 variables: [
 { name: "i", role: "номер текущего ребра – перебираем"},
 { name: "u, v, w", role: "концы и вес рассматриваемого ребра"},
 { name: "parent", role: "DSU-массив: цвета компонент"},
 { name: "mst", role: "рёбра, уже взятые в остов", "51"},
 { name: "total", role: "вес остова (51 на этом графе)"},
],
},
"mst#prim": {
 vizTitle: "Остовное дерево: Прим (рост из вершины E)",
 stepNote: "Шаг = из кучи достаётся самое лёгкое ребро",
 code: `import heapq

```

```

vizTitle: "Остовное дерево: Борувка (фазы-«пятiletки»)"
stepNote: "Шаг = одна фаза: каждая компонента ОДНОВЕРХОВА"
code: `n = 9 # вершин: A...I
edges = [(2, "A", "B"), (3, "D", "E"), (4, "B", "C"), (5, "C", "D"),
 (7, "E", "F"), (8, "D", "G"), (9, "C", "F"), (10, "F", "G"),
 (12, "H", "I"), (13, "F", "I")]
comp = {v: v for v in "ABCDEFGH I"} # DSU: корень компоненты
cheapest = {} # корень компоненты -> самое дешёвое ребро
mst = [] # рёбра остова
total = 0 # вес остова (= 51)
phase = 0 # номер фазы («пятiletки») — от 0 до 4
u, v, w = None, None, None # ребро-кандидат

--- тело алгоритма ---
def find(x):
 while comp[x] != x:
 comp[x] = comp[comp[x]]
 x = comp[x]
 return x

while len(mst) < n - 1:
 phase += 1
 cheapest = {}
 for (w, a, b) in edges: # КАЖДАЯ компонента ОДНОВЕРХОВА
 ra, rb = find(a), find(b)
 if ra == rb:
 continue
 u, v = a, b
 if ra not in cheapest or w < cheapest[ra][0]:
 cheapest[ra] = (w, a, b)
 if rb not in cheapest or w < cheapest[rb][0]:
 cheapest[rb] = (w, a, b)
 if not cheapest:
 break # граф несвязен
 for (w, a, b) in cheapest.values():
 ra, rb = find(a), find(b)
 if ra == rb:
 continue # две компоненты

```

```

 { name: "k", role: "уровень: блоки $2^k \times 2^k$ (шаг
 { name: "st2[{r,c,k,k}]", role: "минимум квадратн
 },
],
},
"sparse-table#2d-query": {
 vizTitle: "Разреженная таблица 2D: запрос",
 stepNote: "Демонстрация кликабельна вручную: выбери
 code: `# st2[{r, c, kx, ky}] из вкладки «построение
r1, c1, r2, c2 = 1, 1, 6, 6

h, w = r2 - r1 + 1, c2 - c1 + 1
kx, ky = h.bit_length() - 1, w.bit_length() - 1
print(f"прямоугольник {h}x{w} кроется блоками уровня ({
 variables: [
 { name: "r1, c1, r2, c2", role: "углы запрашиваем
 { name: "kx, ky", role: "уровень самого крупного
],
},
"prefix-sums-2d#2d": {
 vizTitle: "Префиксные суммы 2D (прямоугольники)",
 stepNote: "Демонстрация кликабельна вручную: наведи
 code: `A = [[1, 2, 3, 4], [5, 6, 7, 8], [9, 1, 2, 3
n, m = 4, 4
i, j = 0, 0
S = [] # строки таблицы: рамка и
for rr in range(n + 1):
 S.append([0] + [None] * m)
S[0] = [0] * (m + 1)
for i in range(1, n + 1):
 for j in range(1, m + 1):
 S[i][j] = A[i - 1][j - 1] + S[i - 1][j] + S[i][j - 1]

r1, c1, r2, c2 = 1, 1, 2, 2
ans = S[r2 + 1][c2 + 1] - S[r1][c2 + 1] - S[r2 + 1][c1]
print("итого:", ans)`
 variables: [
 { name: "i, j", role: "текущая ячейка таблицы S (

```

```

const keep: string[] = [];
for (const line of code.split("\n")) {
 const t = line.trim();
 if (t === "# --- тело алгоритма ---" || /^(for|while)/.test(t)) {
 keep.push(line);
 }
}
// убрать пустые строки с конца
while (keep.length > 0 && keep[keep.length - 1].trim() === "") {
 keep.pop();
}

/**
 * Дефолтное содержимое редактора: шапка + ТОЛЬКО инициализация
 * переменные демо (без всего кода алгоритма).
 */
export function buildInitTemplate(chapterId: string, chapterTitle: string, sync: PageSync) {
 const sync = getPageSync(chapterId, demoId);

 const head = sync.vizTitle
 ? `# «${chapterTitle}»\n# демо: ${sync.vizTitle}\n#`
 : `# «${chapterTitle}»\n# демо на странице нет – св

 if (!sync.code) return head;
 const init = extractInit(sync.code);
 return `${head}\n${init}\n`;
}

```

## РАЗДЕЛ: БИЛЕТЫ (УЧЕБНЫЙ КОНТЕНТ)

ФАЙЛ 26/111: src/data/tickets/ticket1\_segtree.ts

КАТЕГОРИЯ: Билеты (учебный контент) | СТРОК: 217 | РАЗМЕР: 1.5 КБ

```
import { Chapter } from '../../../types';
```

```

span class="font-mono text-emerald-300">[0, 1
тям <span class="font-mono text-emerald-300
– это одиночные элементы массива. В узле л
чество единиц, И/ИЛИ – любая операция, кото
>).</p>

```

```

<p class="text-slate-300 text-sm mb-4">Отсю
а каждом уровне встречает один из трёх случ
не пересекается – возвращаем нейтральный э
склеиваем. Частичных пересечений на уровне
одного элемента – спуск к листу и пересчёт

```

```

<div class="grid grid-cols-1 sm:grid-cols-3
 <div class="bg-slate-900 border border-
 <p class="text-[10px] uppercase tra
 <p class="font-mono text-emerald-300
 </div>
 <div class="bg-slate-900 border border-
 <p class="text-[10px] uppercase tra
 <p class="font-mono text-emerald-300
 </div>
 <div class="bg-slate-900 border border-
 <p class="text-[10px] uppercase tra
 <p class="font-mono text-emerald-300
 </div>
</div>

```

```

<div class="grid grid-cols-1 xl:grid-cols-2
 <div class="bg-slate-800 p-6 rounded-lg
 <div class="absolute -top-3 left-4 l
 <div class="border border-emerald-500 shadow-md">

```

Аналогия 1: Корпоративная пре

```

 </div>
 <p class="text-slate-300 text-sm">Б
икам департаментов: «внутри всем +5 %». Нач
nt-mono text-emerald-300">lazy[v])</div>
свой баланс. Это и есть отложенное обнов
</div>

```

```
<p class="text-slate-500 text-xs mt-3">
 кивают в строгом порядке: сначала присваива
</div>
```

```
<div class="bg-slate-900/40 rounded-lg border
 <p class="text-slate-300 text-sm mb-2">
 <p class="text-slate-400 text-sm mb-1">
за $O(1)$, но строятся один раз и умирают при
ми.</p>
```

```
 <p class="text-slate-400 text-sm mb-1">
идемпотентные запросы (min, gcd, И) по ста
 <p class="text-slate-400 text-sm mb-1">
n ячеек (не $2n!$) – иначе на $n = 6$ поймать п
n ... $2n-1$] и n – степень двойки (или использ
 <p class="text-slate-400 text-sm">•
ть лень во всё дерево – это $O(n)$ и весь смы
</div>
```

```
<p class="text-slate-300 text-sm mt-4"><b c
-2 py-0.5 rounded text-xs font-mono text-em
ся один раз), запрос на отрезке <code class=
-slate-800"> $O(\log n)$ </code>, одиночное обно
-300 border border-slate-800"> $O(\log n)$ </code>
s font-mono text-emerald-300 border border-
xt-xs font-mono text-emerald-300 border bord
ounded text-xs font-mono text-emerald-300 b
```

```
<p class="text-slate-500 text-xs mt-3 leading
 в редакторах и базах, сжатие координат + «
оить», дерево отрезков по дереву (HLD), пер
</p>
```

```
<details class="bg-slate-900/40 rounded-lg l
 <summary class="p-4 font-bold text-slate
-b border-slate-700/50">
```

Код: построение и запрос суммы (C

```
</summary>
```

```
<div class="p-5 text-sm text-slate-300
```



```

 y[4 * MAXN]; // «обещание»: прибавит

void push(int v, int l, int r) {
 if (lazy[v] == 0) return;
 int m = (l + r) / 2;
 tree[2 * v] += lazy[v] * (m - l + 1); // п
 lazy[2 * v] += lazy[v];
 tree[2 * v + 1] += lazy[v] * (r - m); // и
 lazy[2 * v + 1] += lazy[v];
 lazy[v] = 0; // о
}

void add(int v, int l, int r, int L, int R, int x) { /
 if (r < L || R < l) return;
 if (L <= l && r <= R) {
 tree[v] += x * (r - l + 1);
 lazy[v] += x; // д
 return;
 }
 push(v, l, r); // о
 int m = (l + r) / 2;
 add(2 * v, l, m, L, R, x);
 add(2 * v + 1, m + 1, r, L, R, x);
 tree[v] = tree[2 * v] + tree[2 * v + 1];
}

```

<p class="text-slate-400 text-xs">Пр  
 г накопиться. Забыл push – получил правильн  
 </div>  
 </details>

<details class="bg-slate-900/40 rounded-lg l  
 <summary class="p-4 font-bold text-slate  
 -b border-slate-700/50">  
 Код: итеративное дерево (2n, снизу)  
 </summary>  
 <div class="p-5 text-sm text-slate-300  
 <pre class="bg-slate-950 p-4 rounded  
 # для удобств

```
</section>
```

```
`,
 },
];
```

---

ФАЙЛ 27/111: src/data/tickets/ticket14\_20.ts

КАТЕГОРИЯ: Билеты (учебный контент) | СТР0К: 1235 | РАЗМ:

---

```
import { Chapter } from '../..types';
```

```
export const tickets14to20: Chapter[] = [
 {
```

```
 id: "dijkstra",
 title: "14. Графы. Поиск кратчайшего пути. Дейкстра",
 type: "html",
 category: "Графы. Пути",
 content: `
```

```
<section id="dijkstra-content" class="mb-12 scroll-mt-12">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-emerald-400">
 </div>
```

```
<div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border">
 <h3 class="text-xl font-bold text-emerald-400">

 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-emerald-300 italic">
 <p class="text-xl font-mono text-white">
 </div>
```

```
<p class="text-slate-300 text-sm mb-4">Дейк
ершины
требование к графу — неотрицательные вес
```

```
ext-emerald-300">w ≥ 0}, зафиксирова
ию расстояния!</p>
</div>
```

```
<div class="bg-slate-900 p-6 rounded-lg
 <div class="absolute -top-3 left-4
border-rose-500 shadow-md">
```

Ограничения: минус ломает жадно

```
</div>
```

```
<p class="text-slate-300 text-sm mb-4"
class="font-mono text-emerald-300">s→a = 2</
ono text-emerald-300">b→a = -2. Верн
начала вытащит <span class="font-mono text-
o text-emerald-300">3 - 2 = 1 придёт
Джонсона (билет 17).</p>
```

```
</div>
```

```
</div>
```

```
</div>
```

```
<div class="bg-slate-700/50 p-6 rounded-xl bord
<h3 class="text-xl font-bold text-blue-400 m
```

```
<p class="text-slate-300 text-sm mb-4">Демон
D, Дом 2 E, ТЦ F, Метро G) и 11 одностор
ршины <span class="font-mono text-emerald-3
(u → v): переменные <span class="fon
>,
аут картинку напрямую.</p>
```

```
<ol class="list-decimal pl-5 space-y-2 text
 Инициализация. <span class="
text-emerald-300">∞; в кучу кладём <
emerald-300">prev пуст.
```

```
Извлечь минимум. Достаём пар
"font-mono text-emerald-300">du. Если
xt-emerald-300">done – пара устарела
```

```
Зафиксировать. <span class="
меняется.
```

```
</div>
```

```
<div class="bg-slate-800/70 p-5 rounded-lg"
 <p class="text-sm font-bold text-white"
 <table class="w-full text-xs text-left"
 <thead>
 <tr class="text-slate-500 border"
 <th class="py-1.5 pr-2 font"
 <th class="py-1.5 pr-2 font"
 <th class="py-1.5 font-bold"
 </tr>
 </thead>
 <tbody class="text-slate-300">
 <tr class="border-b border-slate-
y-2 pr-2">лучший известный путь $s \rightarrow v$ </td><td class="py-2 pr-2">предок v на текущем лучшем пути</td><tr class="border-b border-slate-
y-2 pr-2">предок v на текущем лучшем пути</td><tr class="border-b border-slate-
lass="py-2 pr-2">вершины с окончательным $dist[v]$ </td></tr>
 <tr class="align-top"><td class="py-2 pr-2">вершина, вершина)</td><td class="py-2">извлечения</td></tr>
 </tbody>
 </table>
</div>
```

```
</div>
```

```
<div class="bg-slate-700/50 p-6 rounded-xl border"
 <h3 class="text-xl font-bold text-indigo-400">Извлечение из кучи</h3>
 <p class="text-slate-300 text-sm mb-4">Инициализация: $dist[s] = 0$, $dist[v] = \infty$ для всех $v \in V \setminus \{s\}$.
 $s \rightarrow u$. Отсюда $dist[u] = \delta(s, u)$.
 $dist[v] = \delta(s, v)$ для всех достижимых v .
 $dist[v] = \infty$ для недостижимых v .
```

```
<details class="bg-slate-900/40 rounded-lg"
 <summary class="p-4 font-bold text-slate-300">Извлечение из кучи</summary>
 <p class="text-slate-300 text-sm mb-4">Инициализация: $dist[s] = 0$, $dist[v] = \infty$ для всех $v \in V \setminus \{s\}$.
 $s \rightarrow u$. Отсюда $dist[u] = \delta(s, u)$.
 $dist[v] = \delta(s, v)$ для всех достижимых v .
 $dist[v] = \infty$ для недостижимых v .
```

```

 </div>
 </details>
</div>

<div class="bg-slate-700/50 p-6 rounded-xl border-2 border-slate-800">
 <h3 class="text-xl font-bold text-emerald-400">Дейкстра на графе
 <div class="overflow-x-auto my-4">
 <table class="w-full text-xs text-left">
 <thead>
 <tr class="text-slate-500 border-b">
 <th class="py-2 pr-3 font-b">
 <th class="py-2 pr-3 font-b">
 <th class="py-2 font-bold">
 </tr>
 </thead>
 <tbody class="text-slate-300">
 <tr class="border-b border-slate-700">
 <td class="text-slate-300">priority_queue (C++)</td><td class="py-2 pr-3 font-mono">
 й выбор</td></tr>
 <tr class="border-b border-slate-700">
 <td class="text-slate-300">ease-key</td><td class="py-2 pr-3 font-mono">
 очереди</td></tr>
 <tr class="border-b border-slate-700">
 <td class="text-slate-300">поиск минимума</td><td class="py-2 pr-3 font-mono">
 </tr>
 <tr class="border-b border-slate-700">
 <td class="text-slate-300">><td class="py-2 pr-3 font-mono">0(V + E)</td></tr>
 <tr class="align-top"><td class="text-slate-300 font-mono">0(E + V · C)</td><td class="py-2">ц
 </tbody>
 </table>
 </div>

 <details class="bg-slate-900/40 rounded-lg border-2 border-slate-800">
 <summary class="p-4 font-bold text-slate-300 border-b border-slate-700/50">» Код: Дейкстра на
 <div class="p-5 text-sm text-slate-300"

```

```

 "E": [{"F", 1}],
 "F": [],
 "G": [{"D", 1}, {"E", 2}],
}
dist, prev = dijkstra(g, "A")
print(dist) # A 0, B 3, C 2, D 6, E 7,
print(restore(prev, "A", "F")) # ['A', 'C', 'G', 'E',
 <p class="text-slate-400 text-xs">П
 же вершины, потому что читает ваши <span c
nt-mono">done.</p>
 </div>
</details>

<details class="bg-slate-900/40 rounded-lg l
 <summary class="p-4 font-bold text-slate
-b border-slate-700/50">> Код: C++ (priority
 <div class="p-5 text-sm text-slate-300
<pre class="bg-slate-950 p-4 rounded overflow-x-auto te
 t;
#include <queue>;
using namespace std;
const long long INF = 4e18; // не INT_MAX:

int n;
vector<vector<pair<int,int>>>> g;
vector<long long> d;
vector<int> p;

void dijkstra(int s) {
 d.assign(n, INF); p.assign(n, -1); d[s] = 0;
 priority_queue<pair<long long,int>;,
 vector<pair<long long,int>>&
 greater<pair<long long,int>>
 pq.push({0, s});
 while (!pq.empty()) {
 auto [du, u] = pq.top(); pq.pop();
 if (du > d[u]) continue; // устарев
 for (auto [v, w] : g[u]) {

```



```

<div class="grid grid-cols-1 sm:grid-cols-3">
 <div class="bg-slate-900 border border-
 <p class="text-[10px] uppercase tra
 <p class="font-mono text-emerald-300
 </div>
 <div class="bg-slate-900 border border-
 <p class="text-[10px] uppercase tra
 <p class="font-mono text-emerald-300
 </div>
 <div class="bg-slate-900 border border-
 <p class="text-[10px] uppercase tra
 <p class="font-mono text-emerald-300
 </div>
</div>

```

```

<div class="grid grid-cols-1 xl:grid-cols-2">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
 Аналогия 1: Патруль Бэтмена на м
 </div>
 <div class="my-3 rounded-lg overflow">
 все ули
овно 1 выходной в году, когда он отдр
-ю ночь Бэтмен делает контрольный проезд: е
о-то района снова уменьшилось – значит, Джо
</div>

```

```

 <div class="bg-slate-900 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border border-rose-500 shadow-md">
 Отрицательный цикл: временная п
 </div>

```



```

 </table>
 <p class="text-slate-400 text-xs mt-3">
бном» порядке: S→A,
порядок рёбер местами – и понадобится до ше
</div>

<div class="bg-slate-800/70 p-5 rounded-lg"
 <p class="text-sm font-bold text-white"
 <p class="text-slate-300 text-sm mb-3">
тава A имеет суммарный вес <span cla
веет» на 2.</p>
 <table class="w-full text-xs text-left"
 <thead>
 <tr class="text-slate-500 borde
 <th class="py-1.5 pr-2 font
 <th class="py-1.5 pr-2 font
 <th class="py-1.5 pr-2 font
 <th class="py-1.5 pr-2 font
 <th class="py-1.5 font-bold
 </tr>
 </thead>
 <tbody class="text-slate-300">
 <tr class="border-b border-slate
-2 font-mono">0</td><td class="py-2 pr-2 fo
td></tr>
 <tr class="border-b border-slate
-2 font-mono">-2</td><td class="py-2 pr-2 f
/td></tr>
 <tr class="border-b border-slate
-2 font-mono">-6</td><td class="py-2 pr-2 f
</td></tr>
 <tr class="border-b border-slate
"py-2 pr-2 font-mono">-10</td><td class="py
nt-mono">-2</td></tr>
 <tr class="align-top"><td class
12</td><td class="py-2 pr-2 font-mono">-9</
ont-bold">всё ещё уменьшается → отрицательны
 </tbody>

```

```

<div class="overflow-x-auto">
 <table class="w-full text-xs text-l
 <thead>
 <tr class="text-slate-500 b
 <th class="py-1.5 pr-2
 <th class="py-1.5 pr-2
 <th class="py-1.5 pr-2
 <th class="py-1.5 font-l
 </tr>
 </thead>
 <tbody class="text-slate-300">
 <tr class="border-b border-
2">одна вершина → все</td><td class="py-2 p
 <tr class="border-b border-
font-mono">0((V+E) log V)</td><td class="p
 <tr class="border-b border-
ass="py-2 pr-2 text-rose-300">нет</td><td c
 </tr>
 <tr class="border-b border-
ass="py-2 pr-2 text-rose-300">молча ошибётся
emerald-300">обнаруживает (d[i][i] < 0)<
 <tr class="border-b border-
ass="py-2 pr-2">список смежности</td><td cl
 <tr class="align-top"><td c
φ, веса ≥ 0</td><td class="py-2 pr-2">минус
 </tbody>
 </table>
</div>
</div>
</div>

<div class="bg-slate-700/50 p-6 rounded-xl bord
 <h3 class="text-xl font-bold text-emerald-4

 <div class="bg-slate-900/60 p-4 rounded-lg l
 <p class="text-xs font-bold text-amber-:
 :</p>
 <pre class="bg-slate-950 p-3 rounded text-xs font-mono

```

```

if changed_vertex == -1:
 return dist, prev, None # отрицательны

changed_vertex улучшен на n-й итерации:
поднимаемся по prev на n шагов – гарантированно з
y = changed_vertex
for _ in range(n):
 y = prev[y]

cycle = [y]
v = prev[y]
while v != y:
 cycle.append(v)
 v = prev[v]
cycle.append(y)
cycle.reverse()
return dist, prev, cycle

демо-граф страницы (9 рёбер, режим с отрицательным ци
nodes = ["S", "A", "B", "C", "D", "E", "F"]
idx = {name: i for i, name in enumerate(nodes)}
E = [{"S", "A", 4}, {"S", "B", 5}, {"B", "C", -2}, {"A", "C", 1},
 {"C", "D", 3}, {"C", "E", 4}, {"D", "F", 2}, {"E", "F", 1}]
E = [(idx[u], idx[v], w) for u, v, w in E]

dist, prev, cyc = bellman_ford(len(nodes), E + [(idx["D",
print([dist[i] for i in range(len(nodes))]) # [0, 4,
print(cyc) # None –

dist2, _, cyc2 = bellman_ford(len(nodes), E + [(idx["D",
print([nodes[i] for i in cyc2]) # ['D',
print([dist2[i] for i in range(len(nodes))]) # все
 <p class="text-slate-400 text-xs">П
 имая вершина с INF<
 ля float('inf')</sp
 </div>
</details>

```

```

 vector<int> cycle;
 for (int v = y;; v = p[v]) {
 cycle.push_back(v);
 if (v == y && cycle.size() > 1) break;
 }
 reverse(cycle.begin(), cycle.end());
 return cycle;
 }

```

```

int main() {
 int s = 0;
 int x = bellman_ford(s);
 if (x == -1) cout <<< "отрицательных циклов не";
 else {
 cout <<< "цикл: ";
 for (int v : negative_cycle(x)) cout <<< v << " ";
 cout <<< "\n";
 }
}
</code></pre>

```

</div>

</details>

<details class="bg-slate-900/40 rounded-lg">

<summary class="p-4 font-bold text-slate-300"> Оптимизации: раннее завершение, использование деки

<div class="p-5 text-sm text-slate-300">

```

<pre class="bg-slate-950 p-4 rounded overflow-x-auto text-slate-300">
 rt deque

```

```

def spfa(n, adj, s):
 """adj[u] = [(v, w), ...]. Релаксируем только «живые» вершины.
 INF = float("inf")
 dist = [INF] * n
 in_queue = [False] * n
 counter = [0] * n # сколько раз вершина попала в очередь
 dist[s] = 0
 q = deque([s])
 in_queue[s] = True

```



```

 Суть фокуса (по шагам)
 </div>
 <p class="text-slate-300 text-sm mb-4 lg:mb-6">
 Главный герой – внешний цикл
 k т.е.
 , если в качестве промежуточных
 , ..., k}?». Каждый шаг добавляет в ра
 </p>
 <ul class="text-sm text-slate-300 space-y-4">
 Шаг k = -1 (старт): разрешено перемещаться
 Шаг k = 0: разрешаем перемещаться по
 i → 0 → j, чем текущий i
 Шаг k = 1: разрешены перемещения по
 l → j уже может стоять
 Шаг k = V - 1: разрешены перемещения по

</div>

<div class="grid grid-cols-1 sm:grid-cols-3 lg:grid-cols-4">
 <div class="bg-slate-900 border border-emerald-500 shadow-md">
 <p class="text-[10px] uppercase tracking-wider">Шаг k = -1 (старт):</p>
 <p class="font-mono text-emerald-300">i → 0 → j</p>
 </div>
 <div class="bg-slate-900 border border-emerald-500 shadow-md">
 <p class="text-[10px] uppercase tracking-wider">Шаг k = 0:</p>
 <p class="font-mono text-emerald-300">i → 0 → j</p>
 </div>
 <div class="bg-slate-900 border border-emerald-500 shadow-md">
 <p class="text-[10px] uppercase tracking-wider">Шаг k = 1:</p>
 <p class="font-mono text-emerald-300">l → j</p>
 </div>
</div>

<div class="grid grid-cols-1 xl:grid-cols-2">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 right-4 bottom-4 border border-emerald-500 shadow-md">
 Аналогия 1: Таблица пересадок в

```

```

 <th class="py-1.5 pr-2 font
 <th class="py-1.5 font-bold
 </tr>
 </thead>
 <tbody class="text-slate-300">
 <tr class="border-b border-slate
-2 font-mono">0</td><td class="py-2">в вершн
 <tr class="border-b border-slate
-2 font-mono">4</td><td class="py-2">открыва
 <tr class="border-b border-slate
-2 font-mono">3</td><td class="py-2">2 – вх
 <tr class="border-b border-slate
-2 font-mono">2</td><td class="py-2">3 – вх
 <tr class="border-b border-slate
-2 font-mono">4</td><td class="py-2">через
 <tr class="border-b border-slate
-2 font-mono">0</td><td class="py-2">из 5 в
 <tr class="align-top"><td class=
300">10</td><td class="py-2">ребро 6 → 1 за
 </tbody>
 </table>
 <p class="text-slate-400 text-xs mt-3">
mono text-emerald-300">0 → {0, 4, 7, 2, 6,
"font-mono">0 → 3 → 4 → 6 (2 + 4 + 1
всего в финальной матрице 11 таких пар. Диа
</div>

<div class="bg-slate-800/70 p-5 rounded-lg
 <p class="text-sm font-bold text-white
 <table class="w-full text-xs text-left
 <thead>
 <tr class="text-slate-500 borde
 <th class="py-1.5 pr-2 font
 <th class="py-1.5 pr-2 font
 <th class="py-1.5 font-bold
 </tr>
 </thead>
 <tbody class="text-slate-300">

```

```

- mono">d[k][k] < 0 и рассуждение
</div>
</details>

```

```

<details class="bg-slate-900/40 rounded-lg
 <summary class="p-4 font-bold text-slate
 -b border-slate-700/50">> Эксперимент: пере
 <div class="p-5 text-sm text-slate-300
<pre class="bg-slate-950 p-4 rounded overflow-x-auto te
M = [[0,4,INF,2,INF,INF,INF],
 [INF,0,3,INF,INF,3,INF],
 [INF,INF,0,INF,2,INF,INF],
 [INF,INF,INF,0,4,2,INF],
 [INF,INF,INF,INF,0,INF,1],
 [INF,INF,INF,INF,INF,0,5],
 [INF,1,INF,INF,INF,INF,0]]

```

```

def floyd(M, order="kij"):
 n = len(M)
 d = [row[:] for row in M]
 for a in range(n):
 for b in range(n):
 for c in range(n):
 k, i, j = {"kij": (a, b, c),
 "ijk": (c, a, b),
 "ikj": (c, a, b)}[order]
 if d[i][k] + d[k][j] < d[i][j]:
 d[i][j] = d[i][k] + d[k][j]
 return d

```

```

good = floyd(M, "kij")
bad = floyd(M, "ijk")
print(good[2][1], bad[2][1]) # 4 и 999 – путь 2→4→6→1
print(sum(1 for i in range(7) for j in range(7) if good
4 клетки отличаются; при порядке (i,k,j) их 6
<p class="text-slate-400 text-xs">В
угих «почти работают» (большинство клеток с
</div>

```



```

 return d, nxt, negative

def path(nxt, i, j):
 if nxt[i][j] == -1:
 return None # j недостижим
 p = [i]
 while p[-1] != j:
 p.append(nxt[p[-1]][j])
 return p

nodes = ["S", "A", "B", "C", "D", "E", "F"]
idx = {name: i for i, name in enumerate(nodes)}
E = [{"S", "A", 4}, {"S", "B", 5}, {"B", "C", -2}, {"A", "C", 1},
 {"C", "D", 3}, {"C", "E", 4}, {"D", "F", 2}, {"E", "F", 1}]
E = [(idx[u], idx[v], w) for u, v, w in E]

d, nxt, neg = floyd_warshall(len(nodes), E)
print([d[0][j] for j in range(len(nodes))]) # [0, 4, 5, 3, 2, 1]
print(path(nxt, idx["S"], idx["F"])) # [0, 2, 1]
print(neg) # [] — отрицательного цикла нет

```

</div>  
</details>

```

<details class="bg-slate-900/40 rounded-lg p-4">
 <summary class="p-4 font-bold text-slate-300">
 -b border-slate-700/50">» Код: C++ (матрица
 <div class="p-5 text-sm text-slate-300">
<pre class="bg-slate-950 p-4 rounded overflow-x-auto text-slate-300">
 t;
#include <algorithm>
using namespace std;
const long long INF = 4e18;

int n;
vector<vector<long long>>> d(n, vector<long long>(n, INF));

void floyd() {
 for (int i = 0; i < n; ++i) d[i][i] = 0;

```

```

<details class="bg-slate-900/40 rounded-lg l
 <summary class="p-4 font-bold text-slate
 -b border-slate-700/50">» Та же динамика дл
 <div class="p-5 text-sm text-slate-300
 <table class="w-full text-xs text-l
 <thead>
 <tr class="text-slate-500 b
 <th class="py-1.5 pr-2
 <th class="py-1.5 pr-2
 <th class="py-1.5 font-l
 </tr>
 </thead>
 <tbody class="text-slate-300">
 <tr class="border-b border-
имость)</td><td class="py-2 pr-2 font-mono">
ь ребро; r[i][i] = true</td></tr>
 <tr class="border-b border-
пути)</td><td class="py-2 pr-2 font-mono">d
= w(i,j); d[i][i] = 0</td></tr>
 <tr class="border-b border-
ой сети</td><td class="py-2 pr-2 font-mono">
и отсутствии положительных циклов</td></tr>
 <tr class="align-top"><td c
o">if d[i][k]+d[k][j] == d[i][j]: cnt[i][j]
а переполнением</td></tr>
 </tbody>
 </table>
 <p class="text-slate-400 text-xs mt
> при r[i][k]
V³).</p>
 </div>
 </details>
</div>

<div class="bg-slate-700/50 p-6 rounded-xl bord
 <h3 class="text-xl font-bold text-amber-400

 <div class="bg-rose-950/30 p-5 rounded-lg b

```

```

title: "17. Графы. Поиск кратчайшего пути. Ускорени
type: "html",
description:
 "Джонсон (перевзвешивание потенциалов), встречный
category: "Графы. Пути",
content: `
<section id="johnson-algo" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-slate-900">
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border border-slate-800">
 <h3 class="text-xl font-bold text-amber-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-amber-300 italic">
 <p class="text-xl font-mono text-white">
 е посещённых вершин (встречный поиск, ранни
 contraction hierarchies).</p>
 </div>

 <p class="text-slate-300 text-sm mb-4">Дейк
 в лоб. На экзамене спрашивают, что сдела
 строки перевзвешивания» до промышленных у

 <div class="grid grid-cols-1 xl:grid-cols-2">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
 Аналогия 1: Такси с навигатором
 </div>
 <p class="text-slate-300 text-sm mb-4">
 ы одинаково. А* — тот же таксист, но с нави
 /span> прибавляется оценка остатка пути по
 т туда, где сумма меньше. Навигатор обязан
 /span>): занижать можно, завышать нельзя —
 0">h = 0 это снова Дейкстра.</p>
 </div>

```

```

 лько 0 и 1</td></tr>
 <tr class="border-b border-slate-500">
<td class="py-2 pr-3">куча → массив корзинок
 С небольшое</td></tr>
 <tr class="border-b border-slate-500">
<td class="py-2 pr-3">decrease-key за $O(1)$
 y-2">теоретически; на практике двоичная/d-ар
 <tr class="border-b border-slate-500">
<td class="py-2 pr-3">не релаксируем все рёбра
 , на практике быстро</td><td class="py-2">о
 <tr class="align-top"><td class="py-2 pr-3">3">переносит работу в предобработку</td><td class="py-2">
 иллионы запросов (карты)</td></tr>
 </tbody>
 </table>
</div>

```

```

<h3 class="text-lg font-bold text-white mt-6">
<div class="bg-slate-900 p-4 rounded-lg mb-6">
 <p class="text-xl font-mono text-white">
</div>
<div class="grid grid-cols-1 gap-6 mb-6">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border border-amber-500 shadow-md">
 ☞ Суть перевзвешивания
 </div>
 <p class="text-slate-300 text-sm mb-6">
 Дейкстра (билет 14) быстрая, но не оптимальный. Алгоритм Джонсона делает финт: пере-
 ь теми же.
 </p>
 <ol class="text-sm text-slate-300 space-y-2">
 Добавляем фиктивную вершину
 Запускаем от s Форда-Беллмана

 <p>важный факт:</i> Джонсон НЕ «удаляет» отрицательные
 кой и прекращает работу, так как при отрицательных весах
 Перевзвешиваем:

```

оторых расстояние только что уменьшилось

ческого прогона, но асимптотика та же,  $O(V \cdot E)$ .  
пользуют осторожно, с эвристиками SLF/LLL).  
) уменьшает высоту и на практике быстрее, к

### Навигация по графу

в **предобработку**. Сначала – **сжатие** индексами  $0 \dots N$ , и уже по ним строят граф и сетевые потоки. **Contraction hierarchies** (так и называются) и запрос идёт двунаправленным поиском вверх по дереву. В вершине заранее приписывают «метки», и запрос

тро, но не кратчайший. Переоценка = неверный ответ.

именимой к отрицательным весам и даёт все возможные пути.

in\_f + min\_b ≥ μ. Остановка по встрече даёт кратчайший путь.

ок сломается. Там нужен Dial.

практике меньше бесполезных релаксаций.

**Алгоритмы поиска кратчайшего пути**

м числом раскрытий, 0-1 BFS  $O(V + E)$ , Дейкстра с кучей  $O(E + V \cdot \log V)$ , Dial  $O(E + V \cdot C)$ , где  $C$  – максимальная длина ребра.

```

 for to, w in g[v]:
 if d[s][to] > dist + w:
 d[s][to] = dist + w
 heapq.heappush(heap, (d[s][to], to))
 for v in range(n):
 if d[s][v] < INF:
 d[s][v] = d[s][v] - h[s] + h[v] # откорректировать
 return d

```

</div>

</details>

<details class="bg-slate-900/40 rounded-lg border-slate-700/50">

    <summary class="p-4 font-bold text-slate-300">

        Код: встречный Дейкстра, A\*, 0-1

        Код: встречный Дейкстра, A\*, 0-1

    </summary>

    <div class="p-5 text-sm text-slate-300">

        <pre class="bg-slate-950 p-4 rounded">

        встречный Дейкстра: остановка по min\_f + min\_g

def bidirectional(g, rg, s, t): # rg – граф с обратными ребрами

INF = float("inf")

df = {s: 0}; db = {t: 0}

qf = [(0, s)]; qb = [(0, t)]

mu = INF

while qf and qb:

if qf[0][0] + qb[0][0] >= mu: # вот правый ответ

return mu

dist, v = heapq.heappop(qf)

if dist > df.get(v, INF): continue

for to, w in g[v]:

nd = dist + w

if nd < df.get(to, INF):

df[to] = nd; heapq.heappush(qf, (nd, to))

if to in db:

mu = min(mu, nd + db[to]) # встреча:

dist, v = heapq.heappop(qb)

if dist > db.get(v, INF): continue

for to, w in rg[v]:



MST. Алгоритмы отличаются только тем, **<b>к**

Прим – «выросшее дерево против остального графа»

```
<div class="bg-slate-800 p-5 rounded-lg border">
 <h4 class="text-amber-400 font-bold text-sm">Пример
 <p class="text-slate-200 text-sm font-sans">
 <p class="text-slate-300 text-sm mb-3">
 адает буква Р – остаётся остов.</p>
 <p class="text-slate-300 text-sm">Остов
 т графа: минимум костей, чтобы всё держалось
 </div>
```

**<p class="text-slate-300 text-sm mb-4">Второй**

никогда не входит в MST. Именно его использовали в

образовавшемся цикле (рёбра-то мы берём по возрастанию)

```
<div class="overflow-x-auto my-6">
 <table class="w-full text-xs text-left border-collapse">
 <thead>
 <tr class="text-slate-500 border-b">
 <th class="py-2 pr-3 font-bold">Веса
 <th class="py-2 pr-3 font-bold">Веса
 <th class="py-2 pr-3 font-bold">Веса
 <th class="py-2 pr-3 font-bold">Веса
 <th class="py-2 font-bold">Веса
 </tr>
 </thead>
 <tbody class="text-slate-300">
 <tr class="border-b border-slate-700">
 <td class="py-2 pr-3 font-bold">Веса
 <td class="py-2 pr-3">Рёбра
 <td class="py-2 pr-3 font-mono">Веса
 <td class="py-2 pr-3 font-mono">Веса
 <td class="py-2">рёбра даны
 </tr>
 <tr class="border-b border-slate-700">
 <td class="py-2 pr-3 font-bold">Веса
 <td class="py-2 pr-3">Растёт
```



```
<div class="absolute -top-3 left-4 l
r border-rose-500 shadow-md">
```

Аналогия к билету 20 • Борува

```
</div>
```

```
<p class="text-slate-300 text-sm">С
айшему чужому племени: к вечеру компонент в
log2 V дней образуется единая федер
```

```
</div>
```

```
</div>
```

```
<div class="bg-slate-900/40 rounded-lg bord
```

```
<p class="text-slate-300 text-sm mb-2">
```

```
<p class="text-slate-400 text-sm mb-1">
```

состояние от одной вершины до каждой; MST мин  
куче другой: у Прима – вес ребра к дереву,   
тся.</p>

```
<p class="text-slate-400 text-sm mb-1">
```

вных весах деревьев может быть несколько, н

```
<p class="text-slate-400 text-sm mb-1">
```

дом вместо почти-константного find/union.</p>

```
<p class="text-slate-400 text-sm mb-1">
```

и Краскал построил минимальный остовный <i>

```
<p class="text-slate-400 text-sm">• P
```

ин раз, DSU спасает от дубля (find(u) == fi  
</div>

```
<p class="text-slate-300 text-sm mt-4">с
py-0.5 rounded text-xs font-mono text-emera
```

бесплатный); Прим с двоичной кучей <code>с  
rder-slate-800">O(E log V)</code>, на плотн  
text-xs font-mono text-emerald-300 border b  
d text-xs font-mono text-emerald-300 border  
ex <code>class="bg-slate-950 px-2 py-0.5 ro  
</p>

```
<details class="bg-slate-900/40 rounded-lg l
```

```
<summary class="p-4 font-bold text-slate
```

```
-b border-slate-700/50">
```

```

key = [INF] * n # минимальный вес ребра, ведущ
used = [False] * n
key[0] = 0
heap = [(0, 0)] # (вес ребра, вершина) ← вот
total = 0

```

```

while heap:
 w, v = heapq.heappop(heap)
 if used[v]:
 continue # устаревшая запись в куче
 used[v] = True
 total += w
 for to, weight in graph[v]:
 if not used[to] and weight < key[to]:
 key[to] = weight
 heapq.heappush(heap, (weight, to))
print(total)

```

(накапливаем путь от старта), здесь  
 а – другая задача.

Код 20: Боровка (Скрыто)

Код 20: Боровка (Скрыто)

```

def find(v):
 while parent[v] != v:
 parent[v] = parent[parent[v]]
 v = parent[v]
 return v

comps = n

```

```
 <p>• Что если граф несвязный? — все
 <p>
 <p>• Как найти второе по величине о
 ребро на образовавшемся цикле (LCA + максим
 </div>
 </details>
 </div>
</div>
</section>`,
 },
];
```

---

ФАЙЛ 29/111: src/data/tickets/ticket21\_24.ts

КАТЕГОРИЯ: Билеты (учебный контент) | СТРӨК: 444 | РАЗМ

---

```
import { Chapter } from '../types';

export const tickets21to24: Chapter[] = [
 {
 id: "string-kmp",
 title: "21. Алгоритм Кнута-Морриса-Пратта (КМР)",
 type: "html",
 category: "Строки",
 content: `
<section id="string-kmp" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl bord
 <h3 class="text-xl font-bold text-blue-400 r
 <div class="bg-slate-900 p-4 rounded-lg mb-
 <p class="text-lg text-blue-300 italic r
 <p class="text-xl font-mono text-white">
```

```

<p>Тот же пример: abcabcd</p>
<ul class="list-disc pl-5 space-y-2">
 a: Префиксов нет. <code>
 ab: Из начала ("a")
>
 abc: Опять мимо. <code>
 abca: 0! Начало ("a"
</code>
 abcab: Начало ("ab")
/li>
 abcabc: Начало ("abc
e>
 abcabcd: «d» всё исп
последняя буква). Длина 3: abc проти
digo-300">π[6] = 0</code>

</div>
</details>
<details class="bg-slate-900/40 rounded-lg l
 <summary class="p-4 font-bold text-slate
-b border-slate-700/50">
 Код C++ (Скрыто)
</summary>
<div class="p-5 text-sm text-slate-300
 <pre class="bg-slate-950 p-4 rounded
vector<int> pi(s.length());
for (int i = 1; i < s.length(); i++) {
 int j = pi[i-1];
 while (j > 0 && s[i] != s[j]) j = pi[j-1];
 if (s[i] == s[j]) j++;
 pi[i] = j;
}</pre>
</div>
</details>

<h3 class="text-lg font-bold text-white mt-
<div class="bg-slate-900 p-4 rounded-lg mb-
 <p class="text-xl font-mono text-white">

```



```

 </p>
 </div>
</div>

```

```

<details class="bg-slate-900/40 rounded-lg l
 <summary class="p-4 font-bold text-slate
 -b border-slate-700/50">

```

Пример вычисления (Скрыто)

```

 </summary>
 <div class="p-5 text-sm text-slate-300
 <p>Тот же пример: abcabcd</p>
 <ul class="list-disc pl-5 space-y-2
 <code>Z[0]</code> – не счита
 <code>Z[1]</code> (bcabcd
[1] = 0</code>
 <code>Z[2]</code> (cabcd

 <code>Z[3]</code> (abcd<
а дальше s[6]='d' против s[3]='a' – мимо.
 <code>Z[4], Z[5], Z[6]</code>

 </div>
</details>

```

```

<details class="bg-slate-900/40 rounded-lg l
 <summary class="p-4 font-bold text-slate
 -b border-slate-700/50">

```

Код C++ (Скрыто)

```

 </summary>
 <div class="p-5 text-sm text-slate-300
 <pre class="bg-slate-950 p-4 rounde
int l = 0, r = 0;
for (int i = 1; i < n; i++) {
 if (i <= r) z[i] = min(r - i + 1, z[i - 1]);
 while (i + z[i] < n && s[z[i]] == s[i + z[i]]) z
 if (i + z[i] - 1 > r) { l = i; r = i + z[i] - 1; }
}</pre>
 </div>
</details>

```

```

 <div class="bg-slate-900 p-6 rounded-lg border-2"
 <div class="absolute -top-3 left-4 bg-rose-900
 -500 shadow-md">
 Аналогия 2: Матёшка (Терминальные)
 </div>
 <p class="text-slate-300 text-sm mb-4">Представ
 нашли и "he", потому что оно спрятано внутри
 </div>
 </div>

 <details class="bg-slate-900/40 rounded-lg border b
 <summary class="p-4 font-bold text-slate-400 ou
 r-slate-700/50">
 Код (Скрыто)
 </summary>
 <div class="p-5 text-sm text-slate-300 space-y-
 <pre class="bg-slate-950 p-4 rounded overfl

int get_link(int v) {
 if (t[v].link == -1) {
 if (v == 0 || t[v].p == 0) t[v].link = 0;
 else t[v].link = go(get_link(t[v].p), t[v].pch)
 }
 return t[v].link;
}

int go(int v, char c) {
 if (t[v].go[c] == -1) {
 if (t[v].next[c] != -1) t[v].go[c] = t[v].next[
 else t[v].go[c] = v == 0 ? 0 : go(get_link(v),
 }
 return t[v].go[c];
}</pre>
 </div>
 </details>
</div>
</section>`
 },

```

```

 </div>
 <div class="bg-slate-900 p-6 rounded-lg" style="border: 1px solid #4b4b9b; border-radius: 10px; padding: 10px; margin-top: 10px;">
 <div class="absolute -top-3 left-4 right-4 bottom-4 border border-purple-500 shadow-md" style="border: 1px solid #4b4b9b; border-radius: 5px; padding: 5px; margin: -3px -4px 5px -4px;">
 Аналогия 2: Переводчик (сведение)
 </div>
 <p class="text-slate-300 text-sm">Сложность перевода с русского на английский (сведение за полином) и друг-англичанин. Отсюда главный инструмент: если к В сводится А, то А не стоит.</p>
 </div>
 </div>

 <h3 class="text-lg font-bold text-white mt-4">Иерархия сложности
 <p class="text-slate-300 text-sm mb-3">Известно, что сложность перевода в цепочке доказывается теоремой об иерархии сложности.
 <div class="bg-slate-950 border border-slate-800 padding-4" style="background-color: #0a0a1a; border: 1px solid #1a1a3a; padding: 10px; margin-top: 10px;">
 <p class="font-mono text-sm sm:text-base text-white">
 $P \subseteq NP \subseteq PSPACE \subseteq EXP \subseteq NEXP$

 NP-полнота (не доказано)
 </p>
 </div>

 <div class="overflow-x-auto">
 <table class="w-full text-xs text-left font-mono" style="width: 100%; border-collapse: collapse; font-family: monospace; font-size: 0.8em;">
 <thead>
 <tr class="text-slate-500 border-bottom" style="background-color: #1a1a3a; color: #a0a0ff; border-bottom: 1px solid #3a3a6a;">
 <th class="py-2 pr-3 font-bold">Сложность
 <th class="py-2 pr-3 font-bold">Описание
 <th class="py-2 font-bold">Примечания
 </tr>
 </thead>
 <tbody class="text-slate-300">
 <tr class="border-b border-slate-800" style="border-bottom: 1px solid #2a2a5a;">
 <td class="py-2 pr-3">детерминированно за $O(n^k)$
 <td class="py-2 pr-3">удольности, максимальный поток
 </tr>
 <tr class="border-b border-slate-800" style="border-bottom: 1px solid #2a2a5a;">
 <td class="py-2 pr-3">сертификат проверяется за полиномиальное время
 </tr>
 </tbody>
 </table>
 </div>

```



```

 ke.
 Параметризованная сложность
n).
 Специальные классы входов
полиномиальными (независимое множество на д
 Готовые решатели: SAT/ILP
всего алгоритма.

</div>

```

```

<div class="bg-slate-900/40 rounded-lg border
 <p class="text-slate-300 text-sm mb-2">
 <p class="text-slate-400 text-sm mb-1">
NP: любая задача из P автоматически в NP.
 <p class="text-slate-400 text-sm mb-1">
становки – неразрешима, зато NP-трудна). NP
 <p class="text-slate-400 text-sm mb-1">
стной трудной A к B ($A \leq_p B$), а не н
 <p class="text-slate-400 text-sm mb-1">
 <p class="text-slate-400 text-sm mb-1">
дит по времени на $n = 40$. NP-полнота запрещ
 <p class="text-slate-400 text-sm">•
ит RSA. «Раз она в NP, значит NP-полная» –
</div>

```

```

<p class="text-slate-300 text-sm mt-4">с
ass="bg-slate-950 px-2 py-0.5 rounded text-
еребор сертификатов <code class="bg-slate-9
^poly(n))</code>; сведение обязано работать
0 border border-slate-800">0(poly(n))</code>

```

```

<details class="bg-slate-900/40 rounded-lg
 <summary class="p-4 font-bold text-slate
-b border-slate-700/50">

```

Код: верификатор сертификата и пр

```

</summary>
<div class="p-5 text-sm text-slate-300
 <pre class="bg-slate-950 p-4 rounded

```

```

 if col[to] == -1:
 col[to] = col[v] ^ 1
 dq.append(to)
 elif col[to] == col[v]:
 return False
 return True</pre>
 <p class="text-slate-400 text-xs">О
товый ответ, но не находить его; для задачи
 </div>
</details>

<details class="bg-slate-900/40 rounded-lg l
 <summary class="p-4 font-bold text-slate
 -b border-slate-700/50">
 Вопросы, которые любят задавать (
 </summary>
 <div class="p-5 text-sm text-slate-300
 <p>• Дайте определение NP через вер
нтны.</p>
 <p>• Что такое NP-полная задача и к
 </p>
 <p>• Почему первая NP-полная задача
 <p>• Если $P = NP$, что станет с крип
их задач исчезнет граница approximability)<
 <p>• Пример задачи в NP и co-NP, пр
– затем AKS доказал P ></p>
 <p>• Чем NP-трудная задача отличаетс
облема остановки)</p>
 </div>
</details>
</div>
</div>
</section>`
 }
};

```

---

```

 <div class="absolute -top-3 left-4 l
border-rose-500 shadow-md">
 Аналогия: Списки = Контакты
 </div>
 <p class="text-slate-300 text-sm mb
т. Оптимально для почти всех задач.</p>
 </div>
</div>

{/* DFS vs BFS */}
<div class="bg-slate-700/50 p-6 rounded-xl bord
 <h3 class="text-xl font-bold text-indigo-400">

 <div class="grid grid-cols-1 xl:grid-cols-2
 <!-- Аналогия DFS -->
 <div class="bg-slate-800 p-6 rounded-lg
 <div class="absolute -top-3 left-4 l
rder border-indigo-500 shadow-md">
 ♂ DFS (Поиск в глубину) = 0
 </div>
 <p class="text-slate-300 text-sm mb
 Что делает: Поиск в глуб
да попадает в тупик – делает шаг назад по с
 </p>
 <ul class="text-xs text-indigo-300
 Где используется:
 Топологическая сортировка (l
 Поиск мостов и точек сочлен
 Компоненты сильной связности
 Детектирование циклов (чере

 </div>

 <!-- Аналогия BFS -->
 <div class="bg-slate-900 p-6 rounded-lg
 <div class="absolute -top-3 left-4 l
border border-emerald-500 shadow-md">
```

```
vis[start_node] = true;
```

```
while (!q.empty()) {
 int v = q.front(); q.pop();
 for (int u : adj[v]) {
 if (!vis[u]) {
 vis[u] = true;
 q.push(u);
 }
 }
}
```

```
</pre>
</div>
</details>
</div>
</div>
```

```
</section>`,
```

```
{,
```

```
{
 id: "graph-planar-colors",
 title: "7. Графы. Планарные. Покраска",
 type: "html",
 description: "Планарные графы: формула Эйлера, K5 и",
 category: "Графы. Теория",
 content: `
```

```
<section id="graph-planar-colors" class="mb-12 scroll-m-12">
 <div class="flex items-center mb-6 flex-wrap gap-3">
 7
 <h2 class="text-2xl sm:text-3xl font-bold text-slate-900">Графы. Планарные. Покраска</h2>
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border">
 <h3 class="text-xl font-bold text-blue-400">Планарные графы</h3>
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-blue-300 italic">Планарный граф — граф, который можно нарисовать в плоскости так, чтобы никакие две его ребра не пересекались.</p>
 <p class="text-2xl font-mono text-white">Графы, которые можно нарисовать в плоскости так, чтобы никакие две его ребра не пересекались, называются планарными.</p>
 <p class="text-xs text-slate-400 mt-1">Планарные графы имеют много интересных свойств. Например, для любого планарного графа справедлива формула Эйлера:</p>
 </div>
 </div>
 </div>
</div>
```

```

<div class="space-y-3 text-slate-300 text-s
 <p>Сложность задачи раскраски:</p>
 <ul class="list-disc list-inside space-
 k = 2 (2-раскрашиваемость):
) обычным BFS/DFS обходом.
 k ≥ 3 (k-colorability):

 <p class="mt-3">Теоремы о раскраске:
 <ul class="list-disc list-inside space-
 Жадная раскраска: дает в
 максимальная степень вершины.
 Теорема Брукса: <span cl
 ных графов K_n и нечётных циклов C_{2k+1} .
 Теорема о 4 красках: Люб
 no text-emerald-300"> $\chi(G) \leq 4$.

</div>
</div>
</div>
</section>`,
 },
 {
 id: "graph-components",
 title: "8. Графы. Компоненты связности",
 type: "html",
 description:
 "Компоненты связности: обходами DFS/BFS за $O(V + E)$ ",
 category: "Графы. База",
 content: `
<section id="graph-components" class="mb-12 scroll-mt-12">
 <div class="flex items-center mb-6 flex-wrap gap-3">
 8
 <h2 class="text-2xl sm:text-3xl font-bold text-slate-800">8. Графы. Компоненты связности
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border">

```

```

<div class="overflow-x-auto">
 <table class="w-full text-xs text-left">
 <thead>
 <tr class="text-slate-500 border-b">
 <th class="py-2 pr-3 font-b">
 <th class="py-2 pr-3 font-b">
 <th class="py-2 pr-3 font-b">
 <th class="py-2 font-bold">
 </tr>
 </thead>
 <tbody class="text-slate-300">
 <tr class="border-b border-slate-200">
 <td class="py-2 pr-3">забыли про стрелку
 <td class="py-2 font-mono">0(V + E)
 </tr>
 <tr class="border-b border-slate-200">
 <td class="py-2 pr-3">из любой вершины
 <td class="py-2 font-mono">0(V + E)
 </tr>
 <tr class="align-top">
 <td class="py-2 pr-3">до кого дойдёт старт
 <td class="py-2 font-mono">0(V + E)
 </tr>
 </tbody>
 </table>
</div>
<p class="text-slate-400 text-xs mt-3">Классический алгоритм SCC. Обход из s даёт только вершины,
</p>
</div>
</div>
</section>`,
 },
 {
 id: "top-sort",
 title: "9. Графы. Топологическая сортировка",
 type: "html",
 description: "Топологическая сортировка ДАГ и детектирование циклов",
 category: "Графы",
 content: `
<section id="top-sort" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

```

```


*

*

Если во время обхода из серой вершины
и гарантированный цикл!
 </p>
</div>

```

```

<details class="bg-slate-900/40 rounded-lg border-1
 <summary class="p-4 font-bold text-slate-300
 -b border-slate-700/50">

```

Корректный код C++ с детекцией цикла

```

</summary>

```

```

<div class="p-5 text-sm text-slate-300" style="border: 1px solid #444; padding: 10px;">

```

```

 <pre class="bg-slate-950 p-4 rounded" style="background-color: #1a202c; color: #e2e3e5;">

```

```

vector<int> adj[MAXN];
int color[MAXN]; // 0=белый, 1=серый (в стеке), 2=чёрный
vector<int> ans;
bool has_cycle = false;

```

```

void dfs(int v) {
 color[v] = 1; // Зашли – вершина серая
 for (int to : adj[v]) {
 if (color[to] == 0) {
 dfs(to);
 if (has_cycle) return;
 } else if (color[to] == 1) {
 // Переход в серую вершину = НАЙДЕН ЦИКЛ!
 has_cycle = true;
 return;
 }
 }
 color[v] = 2; // Вышли – вершина чёрная
 ans.push_back(v); // Сохраняем в момент ВЫХОДА (post-order)
}

```

```

bool topological_sort(int n) {
 fill(color, color + n + 1, 0);
 has_cycle = false;
}

```

```

<p class="text-slate-300 text-sm mb-4">
 «Это просто цикл?» – Нет!
</p>

<details class="bg-slate-900/40 rounded-lg mb-4" >
 <summary class="p-4 font-bold text-slate-300 border-slate-700/50">
 Код: Алгоритм Косарайю (Скрыто)
 </summary>
 <div class="p-5 text-sm text-slate-300">
 <p class="text-emerald-400 font-bold">
 <ol class="list-decimal list-inside">
 Запускаем DFS на исходном графе

 Разворачиваем этот список (и делаем топологическую сортировку самого графа)

 Переворачиваем все ребра в графе

 Идем по <code>order</code>: делаем DFS по каждому узлу. Все циклы, которые
 образуются в этот запуск – образуют очередную SCC!

 </div>
 </details>
</div>
</div>
</section>`,
 },
 {
 id: "graph-bridges",
 title: "11. Графы. Мосты",
 type: "html",
 description: "Рёбра, удаление которых увеличивает количество компонент связности графа",
 category: "Графы. Продвинутое",
 content: `
<section id="graph-bridges" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">


```





```

 }
 }</pre>
</div>
</details>
</div>
</div>
</section>`,
 },
 {
 id: "graph-euler",
 title: "13. Графы. Эйлеров путь и цикл",
 type: "html",
 description: "Условия существования и построение Эйлерова",
 category: "Графы",
 content: `
<section id="graph-euler" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">
 Графы
 <h2 class="text-2xl sm:text-3xl font-bold text-indigo-400">13. Графы. Эйлеров путь и цикл
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border border-slate-700">
 <h3 class="text-xl font-bold text-indigo-400">Условия существования
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-indigo-300 italic">Теорема 13.1 (Эйлер):
 <p class="text-base sm:text-lg font-monospace">Пусть G — связный граф. Тогда существуют следующие условия:
 </div>
 <div class="grid grid-cols-1 md:grid-cols-2 gap-4">
 <div class="bg-slate-800 p-5 rounded-lg">
 <p class="text-indigo-300 font-bold">Эйлеров цикл:
 <ul class="list-disc list-inside space-y-2">
 Эйлеров цикл: связан с условием
 deg[v] % 2 == 0).
 Эйлеров путь: связан с условием
 deg[v] % 2 == 0).

 <p class="text-base">о 2 — путь начинается в одной нечётной и заканчивается в другой нечётной.
 </div>
 </div>
 </div>
 </div>
 </div>
`
 }
}

```

```
 path.push_back(v); // Добавляем в путь на выходе!
 }</pre>
 </div>
 </details>
</div>
</div>
</section>`,
 },
];
```

---

## РАЗДЕЛ: ИНТЕРАКТИВНЫЕ КОМПОНЕНТЫ И СИМУЛЯТОРЫ

---

---

ФАЙЛ 31/111: src/components/ArticulationPointsViz.tsx  
КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

---

```
import React, { useState, useEffect } from "react";
import { useVizStepSync, vizNumber } from '../data/vizS
import { Play, Pause, RotateCcw, Info } from "lucide-re
```

```
interface Node {
 id: number;
 x: number;
 y: number;
 label: string;
}
```

```
interface Edge {
 u: number;
 v: number;
}
```

```
const nodes: Node[] = [
 { id: 0, x: 250, y: 50, label: "0" },
```

```

export const ArticulationPointsViz: React.FC = () => {
 const [steps, setSteps] = useState<StepInfo[]>([]);
 const [currentStepIndex, setCurrentStepIndex] = useState(0);
 useVizStepSync(currentStepIndex, setCurrentStepIndex, () => {
 const v = vizNumber(vars.v);
 if (v === null) return null;
 return steps.map((step, index) => ({ step, index }));
 });
 const [isPlaying, setIsPlaying] = useState(false);

 useEffect(() => {
 const newSteps: StepInfo[] = [];
 let timer = 0;
 const visited = new Set<number>();
 const ap = new Set<number>();
 const currentTin = Array(nodes.length).fill(-1);
 const currentLow = Array(nodes.length).fill(-1);

 const recordStep = (desc: string, u: number | null) => {
 newSteps.push({
 desc,
 visited: new Set(visited),
 ap: new Set(ap),
 currentNode: u,
 tin: [...currentTin],
 low: [...currentLow],
 });
 };

 recordStep("Начало DFS (Тарьян) для поиска точек соarticulation", null);

 const dfs = (v: number, p: number = -1) => {
 visited.add(v);
 currentTin[v] = currentLow[v] = timer++;
 let children = 0;

 recordStep(`Заходим в вершину ${v}. tin[${v}]=${currentTin[v]}`, v);
 };
 }, []);
};

```

```

 v,
);
 }
 };

 for (let i = 0; i < nodes.length; i++) {
 if (!visited.has(i)) {
 dfs(i);
 }
 }

 recordStep("Алгоритм завершен. Найдены все точки со
 setSteps(newSteps);
}, []));

const currentStep = steps[currentStepIndex] || {
 desc: "",
 visited: new Set(),
 ap: new Set(),
 currentNode: null,
 tin: [],
 low: [],
};

useEffect(() => {
 let interval: NodeJS.Timeout;
 if (isPlaying && currentStepIndex < steps.length -
 interval = setInterval(() => {
 setCurrentStepIndex((prev) => prev + 1);
 }, 1500);
 } else if (currentStepIndex >= steps.length - 1) {
 setIsPlaying(false);
 }
 return () => clearInterval(interval);
}, [isPlaying, currentStepIndex, steps.length]);

const togglePlay = () => setIsPlaying(!isPlaying);
const reset = () => {

```

```

<div className="grid grid-cols-1 lg:grid-cols-3 g
 <div className="lg:col-span-2 relative h-96 lg:l
 <svg className="w-full h-full" viewBox="0 0 8
 {/* Edges */}
 {edges.map((edge, i) => {
 const u = nodes.find((n) => n.id === edge
 const v = nodes.find((n) => n.id === edge

 const isBridge =
 (edge.u === 3 && edge.v === 4) ||
 (edge.u === 4 && edge.v === 3);

 return (
 <line
 key={"edge-" + i}
 x1={u.x}
 y1={u.y}
 x2={v.x}
 y2={v.y}
 stroke={isBridge ? "#f43f5e" : "#475568"}
 strokeWidth={isBridge ? "4" : "2"}
 strokeDasharray={isBridge ? "8,4" : "1"}
 />
);
 })}

 {/* Nodes */}
 {nodes.map((node) => {
 const isCurrent = currentStep.currentNode
 const isVisited = currentStep.visited.has
 const isAp = currentStep.ap.has(node.id);

 const fill = isCurrent
 ? "#3b82f6"
 : isAp
 ? "#e11d48"
 : isVisited
 ? "#10b981"

```

```

 y={node.y - r - 15}
 textAnchor="middle"
 fill="#94a3b8"
 fontSize="10px"
 >
 tin:{" "}
 {currentStep.tin[node.id] >= 0
 ? currentStep.tin[node.id]
 : "-"}
 </text>
 <text
 x={node.x}
 y={node.y + r + 20}
 textAnchor="middle"
 fill="#94a3b8"
 fontSize="10px"
 >
 low:{" "}
 {currentStep.low[node.id] >= 0
 ? currentStep.low[node.id]
 : "-"}
 </text>
 </>
 </g>
</svg>
</div>

```

```

<div className="bg-slate-950 p-6 border-t lg:bo
 <h4 className="text-sm font-bold text-slate-4
 Статус алгоритма
 </h4>

```

```

<div className="bg-slate-900 border border-sl
 <p className="text-white font-medium min-h-
 {currentStep.desc}

```





```

 if (cycle) {
 return [...defaultEdges, { u: 'D', v: 'A', w: -6 }];
 } else {
 return [...defaultEdges, { u: 'D', v: 'A', w: -2 }];
 }
 };

 const edges = getEdges(hasCycle);

 const getAdjList = (ed: Edge[]) => {
 const list: { [key: string]: { v: string; w: number } } = {};
 nodes.forEach(n => (list[n.id] = []));
 ed.forEach(e => list[e.u].push({ v: e.v, w: e.w }));
 return list;
 };
 const adjList = getAdjList(edges);

 const codeLines = [
 { line: 1, text: 'function bellman_ford(graph, start)' },
 { line: 2, text: ' dist = {v: ∞}; dist[start] = 0;' },
 { line: 3, text: ' for i from 1 to V - 1: // |V| - 1 iterations' },
 { line: 4, text: ' for (u, v, weight) in E:' },
 { line: 5, text: ' if dist[u] + weight < dist[v]' },
 { line: 6, text: ' dist[v] = dist[u] + weight;' },
 { line: 7, text: ' for (u, v, weight) in E: // Проверка на отрицательный цикл' },
 { line: 8, text: ' if dist[u] + weight < dist[v]' },
 { line: 9, text: ' return "ОБНАРУЖЕН ОТРИЦАТЕЛЬНЫЙ ЦИКЛ"' },
 { line: 10, text: ' return dist' },
];

 useEffect(() => {
 const simulationSteps: Step[] = [];
 let dist: { [key: string]: number } = { S: 0, A: 99 };
 let prev: { [key: string]: string | null } = { S: null };

 simulationSteps.push({
 iteration: 0, checkingEdge: null,
 distances: { ...dist }, previous: { ...prev },
 });
 });

```

```

 if (simulationSteps[simulationSteps.length - 1].iteration === vCount) {
 let cycleFound = false;
 for (const edge of edges) {
 if (dist[edge.u] !== 999 && dist[edge.u] + edge.weight < dist[edge.v]) {
 cycleFound = true;
 simulationSteps.push({
 iteration: vCount, checkingEdge: { u: edge.u, v: edge.v },
 distances: { ...dist }, previous: { ...prevDist },
 log: `    ВНИМАНИЕ! Проверка цикла: Ребро ${edge.u} -> ${edge.v}`,
 activeCodeLine: 9,
 });
 break;
 }
 }
 }

 if (!cycleFound) {
 simulationSteps.push({
 iteration: vCount, checkingEdge: null,
 distances: { ...dist }, previous: { ...prevDist },
 log: `    Проверка завершена. Ни одно ребро не попало в цикл`,
 activeCodeLine: 10,
 });
 }
 }

 setSteps(simulationSteps);
 setCurrentStepIndex(0);
 setIsPlaying(false);
}, [hasCycle]);

useEffect(() => {
 let timer: any = null;
 if (isPlaying && currentStepIndex < steps.length - 1) {
 timer = setTimeout(() => setCurrentStepIndex((p) => p + 1), 1000);
 } else if (currentStepIndex >= steps.length - 1) {
 setIsPlaying(false);
 }
}, [isPlaying, currentStepIndex, steps.length]);

```



```

 {nodes.map((node) => {
 const dist = currentStep.distances[node.id]
 const isCheckingNode = currentStep.checkingNode ===
 node.id);

 return (
 <g key={node.id} className="transition-
 <circle cx={node.x} cy={node.y} r={is
 ckingNode ? '#7dd3fc' : '#64748b'} strokeWid
 <text x={node.x} y={node.y + 5} fill=
 <text x={node.x} y={node.y - 28} fill=
 ight="bold" textAnchor="middle" className="
 <text x={node.x} y={node.y + 35} fill=
 split(' ')[0]}</text>
 </g>
);
 })}
 </svg>

 <div className="w-full bg-slate-950/80 p-4 ro
 <p className="font-semibold text-amber-400
 <p className="min-h-[40px] flex items-cente
 </div>
 </div>

 {/* Список смежности */}
 <div className="w-full lg:w-1/3 bg-emerald-950/
 <h4 className="text-lg font-bold text-emerald
 <div className="space-y-2 text-sm font-mono t
 {Object.keys(adjList).map(u => {
 <div key={u} className="flex flex-wrap g
 -colors">
 <span className="font-bold text-white
 {adjList[u].map((edge, idx) => {
 const isChk = currentStep.checkingE
 return (
 <span key={idx} className={`px-2

```

```
 {codeLines.map(item => {
 <div key={item.line} className={`py-1.5
e === item.line ? 'bg-blue-900/80 text-amber
 <span className="text-slate-600 w-5 s
 <span className="whitespace-pre flex-
 </div>
 }}}
 </div>
 </div>
</div>
</div>
</div>
);
}
```

---

ФАЙЛ 33/111: src/components/BfsViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

---

```
import { useState, useEffect, useCallback } from 'react'
import { Play, Pause, RotateCcw, StepForward } from 'lucide-react'
```

```
const NODES = [
 { id: 0, label: '0', x: 300, y: 50 },
 { id: 1, label: '1', x: 150, y: 150 },
 { id: 2, label: '2', x: 450, y: 150 },
 { id: 3, label: '3', x: 75, y: 250 },
 { id: 4, label: '4', x: 225, y: 250 },
 { id: 5, label: '5', x: 375, y: 250 },
 { id: 6, label: '6', x: 525, y: 250 },
];
```

```
const EDGES = [
 { from: 0, to: 1 }, { from: 0, to: 2 },
 { from: 1, to: 3 }, { from: 1, to: 4 },
 { from: 2, to: 5 }, { from: 2, to: 6 }
];
```

```
while (queue.length > 0) {
 steps.push({
 activeLine: 4,
 queue: [...queue],
 visited: Array.from(visited),
 currentNode: null,
 logs: `Очередь не пуста, элементов: ${queue.length}
 });
```

```
const curr = queue.shift()!;
steps.push({
 activeLine: 5,
 queue: [...queue],
 visited: Array.from(visited),
 currentNode: curr,
 logs: `Достаем из очереди (${curr}).`
});
```

```
const neighbors = ADJ[curr];
if (neighbors.length > 0) {
 for (const n of neighbors) {
 if (!visited.has(n)) {
 steps.push({
 activeLine: 7,
 queue: [...queue],
 visited: Array.from(visited),
 currentNode: curr,
 logs: `Рассматриваем соседа: ${n}. Он еще не
 });
```

```
visited.add(n);
queue.push(n);
```

```
steps.push({
 activeLine: 9,
 queue: [...queue],
 visited: Array.from(visited),
 currentNode: curr,
```

```

// @ts-expect-error зарезервировано под кнопку «назад»
const handlePrev = useCallback(() => {
 setStepIdx(prev => Math.max(prev - 1, 0));
}, []);

useEffect(() => {
 if (!isPlaying) return;
 if (stepIdx >= STEPS.length - 1) {
 setIsPlaying(false);
 return;
 }
 const timer = setTimeout(handleNext, 1200);
 return () => clearTimeout(timer);
}, [isPlaying, stepIdx, handleNext]);

return (
 <div className="bg-slate-900 border-2 border-slate-100" >
 <div className="flex flex-col md:flex-row items-center justify-center" >
 <div>
 <h3 className="text-2xl font-bold text-white" >
 Ал
 </h3>
 <p className="text-slate-400 text-sm mt-1">Ви
 </div>
 <div className="flex bg-slate-800 p-1.5 rounded" >
 <button
 onClick={() => { setIsPlaying(false); setSt
 className="p-2 text-slate-400 hover:text-wh
 title="Сброс"
 >
 <RotateCcw className="w-5 h-5" />
 </button>
 <button
 onClick={() => setIsPlaying(p => !p)}
 className="p-2 text-blue-400 hover:text-bl
 title={isPlaying ? 'Пауза' : 'Воспроизведен
 >
 {isPlaying ? <Pause className="w-5 h-5" />

```

```

let fill = '#1e293b'; // slate-800
let stroke = '#475569'; // slate-600
let scale = 1;
void scale;

if (isCurrent) {
 fill = '#3b82f6'; // blue-500
 stroke = '#60a5fa'; // blue-400
 scale = 1.3;
} else if (isInQueue) {
 fill = '#059669'; // emerald-600
 stroke = '#34d399'; // emerald-400
 scale = 1.1;
} else if (isVisited) {
 fill = '#0f172a'; // slate-900
 stroke = '#3b82f6'; // blue-500
}

return (
 <g key={n.id} className="transition-tran
 {isCurrent && {
 <circle cx={n.x} cy={n.y} r="30" fi
 }}
 <circle
 cx={n.x} cy={n.y}
 r="20"
 fill={fill}
 stroke={stroke}
 strokeWidth="3"
 className="transition-colors duration
 />
 <text
 x={n.x} y={n.y}
 textAnchor="middle"
 dy=".3em"
 className="fill-white font-bold tex
 >

```



```

 </div>
 </div>

 <div className="bg-slate-800/80 border border
 <div className="text-sm font-bold text-eme
 Состояние Очереди (Queue):
 <span className="text-xs text-slate-500
 </div>
 <div className="flex items-center gap-2 mi
 {step.queue.length === 0 ? (
 <span className="text-slate-500 text-x
) : (
 step.queue.map(({qId, i}) => {
 <span key={`_${qId}-${i}`} className=
enter justify-center rounded-md shadow-lg s
 {qId}

))
)}
 </div>
 <div className="mt-2 text-sm text-amber-30
ms-center justify-center">
 "{step.logs}"
 </div>
 </div>
 </div>
 </div>
</div>
);
}

```

ФАЙЛ 34/111: src/components/ChapterImage.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРО

```
import dijkstraImg from '../assets/dijkstra-freakazoid.
```

```
 });
 }
}
```

---

ФАЙЛ 35/111: src/components/ChapterNav.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

---

```
import React from "react";
import { ChevronLeft, ChevronRight } from "lucide-react";
import type { Chapter } from "../types";
```

```
interface Props {
 prev?: Chapter;
 next?: Chapter;
 index: number;
 total: number;
 onGo: (id: string) => void;
 compact?: boolean;
}
```

```
/**
```

```
 * Переходы «предыдущая / следующая тема» – как на CodeSandbox
```

```
 * Компактный вариант живёт в шапке главы, крупный – по умолчанию
```

```
 */
```

```
export const ChapterNav: React.FC<Props> = ({ prev, next, onGo, compact }) => {
 if (compact) {
 return (
 <div className="flex items-center gap-2 text-xs">
 <button
 disabled={!prev}
 onClick={() => prev && onGo(prev.id)}
 title={prev ? prev.title : "Это первая тема"}
 className="flex items-center gap-1 px-2.5 py-1.5
 :bg-slate-700 enabled:hover:text-white disabled:opacity-50">
 <ChevronLeft className="w-4 h-4" />
 <button
 disabled={!next}
 onClick={() => next && onGo(next.id)}
 title={next ? next.title : "Это последняя тема"}
 className="flex items-center gap-1 px-2.5 py-1.5
 :bg-slate-700 enabled:hover:text-white disabled:opacity-50">
 <ChevronRight className="w-4 h-4" />
 </div>
 </div>
);
 }
 return (
 <div className="flex items-center gap-2 text-sm">
 <button
 disabled={!prev}
 onClick={() => prev && onGo(prev.id)}
 title={prev ? prev.title : "Это первая тема"}
 className="flex items-center gap-1 px-3 py-2
 :bg-slate-700 enabled:hover:text-white disabled:opacity-50">
 <ChevronLeft className="w-5 h-5" />
 <button
 disabled={!next}
 onClick={() => next && onGo(next.id)}
 title={next ? next.title : "Это последняя тема"}
 className="flex items-center gap-1 px-3 py-2
 :bg-slate-700 enabled:hover:text-white disabled:opacity-50">
 <ChevronRight className="w-5 h-5" />
 </div>
);
 }
};
```

```
 <button
 onClick={() => onGo(next.id)}
 className="group text-right p-4 rounded-xl bg-
 transition-all sm:col-start-2"
 >
 <div className="flex items-center justify-end
 text-indigo-400 mb-1">
 Следующая тема <ChevronRight className="w-3
 </div>
 <div className="text-sm font-bold text-slate-1
 </button>
 })
 </nav>
);
};
```

---

ФАЙЛ 36/111: src/components/ChapterQuiz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРО

---

```
import React, { useEffect, useState } from "react";
import { ArrowRight, CheckCircle2, RotateCcw, XCircle,
import { quizzes } from "../data/quizzes";
```

```
interface Props {
 chapterId: string;
}
```

```
/**
```

```
 * Блиц по прочитанной теме: 3–5 коротких вопросов с не
 *
```

```
 * Вопросы живут в `src/data/quizzes.ts` (по id главы).
```

```
 * компонент-викторина существовали, но не были подклю
```

```
 * контент был, а добраться до него было нельзя. Теперь
```

```
 * демонстрации: прочитал → посмотрел, как работает → пр
```

```
 *
```



```

 {correct ? "Верно" : "Неверно"}
 </p>
 <p className="text-[13px] leading-relaxed">
 </div>
)}

 <div className="mt-3 flex justify-end">
 <button
 type="button"
 onClick={next}
 disabled={!answered}
 className="inline-flex items-center gap-2 font-medium text-indigo-200 transition-colors hover:text-indigo-300"
 >
 {idx + 1 < list.length ? "Следующий вопрос" : "Завершить"}
 <ArrowRight className="w-3.5 h-3.5" />
 </button>
 </div>
</>
)}
</div>
</section>
);
};

export default ChapterQuiz;

```

ФАЙЛ 37/111: src/components/ChapterView.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 10

```

import React, { useCallback, useEffect, useRef, useState } from "react";
import { Check, Download, FileText, Loader2, Maximize2, Minimize2 } from "lucide-react";
import type { Chapter } from "../../types";
import { useTermHints } from "../../hooks/useTermHints";

```

```

 window.clearTimeout(hideTimer.current);
 hideTimer.current = null;
 }
}, []));

const scheduleHide = useCallback(() => {
 cancelHide();
 hideTimer.current = window.setTimeout(() => setAnchor(
}, [cancelHide]);

useEffect(() => {
 setAnchor(null);
 setSaving("idle");
}, [chapter.id]);

/**
 * Печать/«сохранить в PDF» этой темы: раскрываем все
 * чтобы на бумагу попал весь текст (доказательства,
 * а после печати возвращаем их в исходное состояние.
 */
useEffect(() => {
 const each = () => Array.from(contentRef.current?.q
 const openAll = () => {
 each().forEach((d) => {
 d.dataset.wasOpen = d.open ? "1" : "0";
 d.open = true;
 });
 };
 const restore = () => {
 each().forEach((d) => {
 if (d.dataset.wasOpen === "0") d.open = false;
 delete d.dataset.wasOpen;
 });
 };
 window.addEventListener("beforeprint", openAll);
 window.addEventListener("afterprint", restore);
 return () => {
 window.removeEventListener("beforeprint", openAll

```

```

// ссылка на другой билет: переходим внутри приложения
const link = (e.target as HTMLElement).closest<HTMLAnchor>('a');
if (link) {
 e.preventDefault();
 onGo(link.dataset.goto ?? "");
 return;
}
const el = (e.target as HTMLElement).closest<HTMLAnchor>('a');
if (!el) return;
e.preventDefault();
if (anchor && anchor.termId === el.dataset.termId)
 else open(el);
};

const handleFocus = (e: React.FocusEvent) => {
 const el = (e.target as HTMLElement).closest<HTMLAnchor>('a');
 if (el) open(el);
};

const viz = getViz(chapter.id);

return (
 <>
 <article className="chapter-body">
 {/* панель загрузки: сохранить именно эту страницу */}
 <div className="flex flex-wrap items-center gap-2">
 <button
 onClick={saveHtml}
 disabled={saving === "work"}
 title="Сохранить эту тему одним автономным файлом"
 className="flex items-center gap-1.5 text-xs font-medium border border-gray-200 rounded px-2.5 py-1.5"
 300 hover:text-white hover:border-emerald-500
 >
 {saving === "work" ? (
 <Loader2 className="w-3.5 h-3.5 animate-spin" />
) : saving === "done" ? (
 <Check className="w-3.5 h-3.5 text-emerald-500" />
) : (

```

```

ed-lg px-3 py-2 print:hidden">
<MousePointerClick className="w-4 h-4 shrink-0" />

 Подчёркнутые термины – интерактивные: наведи
 объяснение на пальцах, мини-анимацию и кноп

</p>

{viz && {
 <section id="chapter-viz" className="mt-8 scroll-mt-8" />
 <div className="flex items-center justify-between" />
 <h3 className="flex items-center gap-2 text-slate-600" />
 <Sparkles className="w-4 h-4 text-indigo-500" />
 Демонстрация: {viz.title}
 </h3>
 <div className="flex items-center gap-2" />
 <button
 onClick={() => onOpenSimulator(chapter) }
 className="flex items-center gap-1.5 text-slate-300
 hover:text-white hover:border-indigo-500" />
 <Maximize2 className="w-3.5 h-3.5" />
 </button>
 {onOpenCompiler && {
 <button
 onClick={onOpenCompiler}
 title="Открыть Python: код выполняе
рации"
 className="flex items-center gap-1.5 text-emerald-400
 hover:text-white hover:border-emerald-500" />
 <Terminal className="w-3.5 h-3.5" />
 </button>
 }}
 </div>
 </div>
 {viz.hint && <p className="text-xs text-slate-500" />
 <VizChapterContext.Provider value={chapter} />

```



```

*
* Четыре вкладки:
* 1. «Иерархия» — $P \subseteq NP \subseteq PSPACE \subseteq EXP$ и NP-полные
* 2. «Сведение» — $A \leq_p B$ по шагам: преобразовал вход
* 3. «Верификатор» — проверка сертификата для 3-SAT по
* 4. «Рост» — живой график по переменным n и ops
* сколько операций насчитал твой алгоритм
* похоже (полином или экспонента).
*
* Синхронизация с компилятором (PAGE_SYNC["complexity-
* cls — имя класса (подсветит нужный овал), step/i —
* k — номер проверяемого дизъюнкта, cert/assign — сер
* n и ops — точки на графике роста, ok/ans — вердикт
*/

```

```

type Tab = "hierarchy" | "reduce" | "verify" | "growth"

```

```

const TABS: { id: Tab; label: string; icon: React.ReactElement } = [
 { id: "hierarchy", label: "1 · Иерархия классов", icon: },
 { id: "reduce", label: "2 · Сведение $A \leq_p B$ ", icon: },
 { id: "verify", label: "3 · Верификатор (NP)", icon: },
 { id: "growth", label: "4 · Рост по n и ops ", icon: },
];

```

```

/* ----- Иерархия -----

```

```

const CLASSES = [
 { id: "EXP", label: "EXP", sub: "2^(n^k)", color: "#0000FF", note: "(кванторные формулы)" },
 { id: "PSPACE", label: "PSPACE", sub: "полиномиальная с полной информацией" },
 { id: "NP", label: "NP", sub: "сертификат проверяется за полиномиальное время", note: "AT", "гамильтонов цикл", "раскраска в 3 цвета", "клика"},
 { id: "P", label: "P", sub: "решается за $O(n^k)$ ", color: "#00FF00", note: "ид", "MST", "2-SAT, двудольность" },
];

```

```

const NPHARD NOTE = "NP-трудные — всё, к чему сводится

```

```

];
/** Сертификат, который проверяем: $x = (1, 0, 1, 0)$ — ok
const CERT = [1, 0, 1, 0];

const VERIFY_STEPS = CLAUSES.map((cl, i) => ({
 title: `Дизъюнкт $\{i + 1\}$: $\{\text{\${cl.map((l) => (l > 0 ?$
 note: `Подставляем сертификат $x = (\text{\${CERT.join(", ")}\}$
 .map((l) => $(l > 0 ? \text{\`x\${l}}=\text{\${CERT[l - 1]}}\`}$: $\text{\`¬x\${$
 .join(", ")}\}. $\{\text{\${cl.some((l) => (l > 0 ? CERT[l - 1]$
 : "Все литералы ложны — дизъюнкт НЕ выполнен.")\`
 ok: cl.some((l) => $(l > 0 ? CERT[l - 1] === 1 : CERT[l - 1] === 0)$
})));

/* ----- Пост -----

interface Sample { n: number; ops: number }

const GROWTH_LINES = [
 { id: "n", label: "n", fn: (n: number) => n, color: "blue" },
 { id: "n log n", label: " $n \cdot \log_2 n$ ", fn: (n: number) => n * Math.log2(n), color: "green" },
 { id: "n²", label: " n^2 ", fn: (n: number) => n * n, color: "red" },
 { id: "n³", label: " n^3 ", fn: (n: number) => n * n * n, color: "purple" },
 { id: "2ⁿ", label: " 2^n ", fn: (n: number) => Math.pow(2, n), color: "brown" }
];

export const ComplexityViz: React.FC = () => {
 const chapterId = useContext(VizChapterContext);
 const runtime = useVizRuntime();
 const vars = runtime?.variables;
 const [tab, setTab] = useState<Tab>("hierarchy");
 const [step, setStep] = useState(0);
 const [playing, setPlaying] = useState(false);
 const [samples, setSamples] = useState<Sample[]>([]);

 useEffect(() => {
 if (chapterId) emitVizDemo(chapterId, tab);
 }, [chapterId, tab]);
}

```

```

 setSamples((prev) => {
 const last = prev[prev.length - 1];
 if (last && last.n === liveN && last.ops === liveOps) return prev;
 const withoutN = prev.filter((s) => s.n !== liveN);
 return [...withoutN, { n: liveN, ops: liveOps }];
 });
 }, [liveN, liveOps]);

/** Оценка показателя роста по последним точкам (лог-лог) */
const verdict = useMemo(() => {
 const pts = samples.filter((s) => s.ops > 0 && s.n > 0);
 if (pts.length < 3) return null;
 const xs = pts.map((p) => Math.log(p.n));
 const ys = pts.map((p) => Math.log(p.ops));
 const mx = xs.reduce((a, b) => a + b, 0) / xs.length;
 const my = ys.reduce((a, b) => a + b, 0) / ys.length;
 let num = 0;
 let den = 0;
 for (let i = 0; i < xs.length; i++) {
 num += (xs[i] - mx) * (ys[i] - my);
 den += (xs[i] - mx) ** 2;
 }
 const slope = den === 0 ? 0 : num / den;
 // экспонента на лог-лог растёт неограниченно: сравнение
 const half = Math.floor(pts.length / 2);
 const slopeTail = (() => {
 const q = pts.slice(half);
 if (q.length < 2) return slope;
 const lx = q.map((p) => Math.log(p.n));
 const ly = q.map((p) => Math.log(p.ops));
 const ax = lx.reduce((a, b) => a + b, 0) / lx.length;
 const ay = ly.reduce((a, b) => a + b, 0) / ly.length;
 let nn = 0;
 let dd = 0;
 for (let i = 0; i < lx.length; i++) {
 nn += (lx[i] - ax) * (ly[i] - ay);
 dd += (lx[i] - ax) ** 2;
 }
 })

```

```

 {TABS.map((t) => {
 <button
 key={t.id}
 onClick={() => setTab(t.id)}
 className={"flex items-center gap-1.5 px-3 p-1.5 rounded-lg bg-slate-900 text-white"}
 >
 {t.icon} {t.label}
 </button>
 })}
 </div>

```

```

 {(tab === "reduce" || tab === "verify") && (
 <div className="flex flex-wrap items-center gap-2">
 <button onClick={() => { setStep(0); setPlaying(false); }}>
 <RotateCcw className="w-4 h-4" />
 </button>
 <button onClick={() => { setPlaying(false); setStep(1); }}>
 <RotateCcw className="w-4 h-4" />
 </button>
 <button onClick={() => setPlaying((p) => !p)}>
 {playing ? <Pause className="w-4 h-4" /> : <Play className="w-4 h-4" />}
 </button>
 <button onClick={() => { setPlaying(false); setStep(2); }}>
 <RotateCcw className="w-4 h-4" />
 </button>

 {tab === "reduce" ? "Reduce" : "Verify"}

 </div>
)}

```

```

 {/* ----- ИЕРАРХИЯ ----- */
 {tab === "hierarchy" && (
 <div className="space-y-4">
 <div className="bg-slate-950 rounded-xl p-3 border border-slate-800">
 <svg viewBox="0 0 500 380" className="w-full h-380">
 {CLASSES.map((c) => {
 const on = liveCls ? liveCls.toUpperCase() : c.name.toUpperCase()
 return (

```

```

 return (
 <React.Fragment key={b + i}>
 <div
 className="rounded-lg border px-3"
 style={{
 borderColor: active ? meta.c :
 background: active ? meta.c + "
 opacity: active ? 1 : 0.45,
 boxShadow: active ? `0 0 18px $
 minWidth: 96,
 }}
 >
 <span className="text-[11px] font
 </div>
 {i < reduceFrame_boxes.length - 1 &
 </React.Fragment>
);
 }}}
 </div>
 </div>
 <div className="bg-slate-900/70 border border
 <p className="text-sm font-bold text-white
 text-purple-400" />{reduceFrame.title}</p>
 <p className="text-xs text-slate-400 leadin
 </div>
</div>
)}

{/* _____ ВЕРИФИКАТОР _____
{tab === "verify" && (
 <div className="space-y-3">
 <div className="bg-slate-950 rounded-xl p-4 b
 <p className="text-[10px] uppercase trackin
 <div className="flex flex-wrap gap-2 mb-3">
 {CERT.map((v, i) => (
 <span key={i} className={"font-mono tex
 ld-600/50 text-emerald-300" : "bg-slate-900
 x{i + 1} = {liveCert && typeof liveCe

```

```

 <div className="bg-slate-900/70 border border-
 <p className="text-sm font-bold text-white
 <p className="text-xs text-slate-400 leading-
 определение NP: короткий сертификат + быстрый
 </div>
 </div>
)}

{/* ----- ПОСТ ----- */
{tab === "growth" && (
 <div className="space-y-3">
 <div className="bg-slate-950 rounded-xl p-4 b
 <div className="flex flex-wrap items-center
 <span className="text-[10px] uppercase tr
 <span className="font-mono text-xs bg-sla
 ?? "-">
 <span className="font-mono text-xs bg-sla
e0ps ?? "-">
 <button onClick={() => setSamples([])} cl
0 rounded-lg px-2 py-1">очистить график</bu
 </div>
 <svg viewBox="0 0 500 240" className="w-full
 {/* оси в лог-лог */}
 <line x1={44} y1={200} x2={486} y2={200}
 <line x1={44} y1={14} x2={44} y2={200} st
 <text x={265} y={222} textAnchor="middle"
 <text x={16} y={110} fill="#64748b" fontS
 {GROWTH_LINES.map((g) => {
 const pts: string[] = [];
 for (let i = 0; i <= 20; i++) {
 const n = 1 + i * 0.8;
 const v = g.fn(n);
 const x = 44 + (Math.log(n) / Math.lo
 const y = 200 - (Math.log(Math.max(1,
 pts.push(`${x.toFixed(1)}${Math.max(
 }
 return <polyline key={g.id} points={pts
 acity={0.55} />;

```

```

 <div className="mt-3 flex flex-wrap items-center" >
 <span className="text-[10px] uppercase tracking
 {vars ? (
 ["n", "ops", "cls", "k", "cert", "ok"].map((n) => {
 vars[name] === undefined ? null : (
 <span key={name} className="text-[11px] font-mono" style={{
 display: inline-block, width: 100px, text-align: center, vertical-align: middle;
 }}>
 {name} = {JSON.stringify(vars[name]).slice(0, 100)}

)
 })
) : (

 открой панель Python: переменные <code clas
 e-300">k</code>, <code className="font-mono
 e-300">n</code> управляют этой демонстрацией

)}
 </div>
 </div>
);
};

export default ComplexityViz;

```

ФАЙЛ 39/111: src/components/DfsBridgesSimulator.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```

import { useState, useEffect, useRef } from "react";
import { useVizStepSync, vizNumber, vizString } from ".";
import { Undo, Play, Pause, SkipForward, RefreshCw } from "react-icons/ri";

```

```

interface CodeLine {
 line: string;
 highlight: string;
}

```

```

 { id: 0, label: "A", x: 100, y: 120 },
 { id: 1, label: "B", x: 260, y: 120 },
 { id: 2, label: "C", x: 180, y: 200 },
 { id: 3, label: "D", x: 180, y: 300 },
 { id: 4, label: "E", x: 100, y: 370 },
 { id: 5, label: "F", x: 260, y: 370 },
 { id: 6, label: "G", x: 340, y: 200 }
];

const GRAPH_EDGES = [
 { u: 0, v: 1, key: "0-1" },
 { u: 1, v: 2, key: "1-2" },
 { u: 2, v: 0, key: "0-2" },
 { u: 2, v: 3, key: "2-3" },
 { u: 3, v: 4, key: "3-4" },
 { u: 4, v: 5, key: "4-5" },
 { u: 5, v: 3, key: "3-5" },
 { u: 1, v: 6, key: "1-6" }
];

const DFS_STEPS: DfsStep[] = [
 {
 currentNode: 0, visitedIndices: [0],
 tin: {0:1}, low: {0:1}, stack: [0], bridges: [],
 activeEdge: null, backEdge: null,
 log: "Входим в A. tin[A]=1, low[A]=1.",
 activeCodeLine: [0,1,2]
 },
 {
 currentNode: 1, visitedIndices: [0,1],
 tin: {0:1,1:2}, low: {0:1,1:2}, stack: [0,1], bridges: [],
 activeEdge: [0,1], backEdge: null,
 log: "Идём (A,B). B впервые. tin[B]=2, low[B]=2.",
 activeCodeLine: [4,5,7,10,11]
 },
 {
 currentNode: 6, visitedIndices: [0,1,6],
 tin: {0:1,1:2,6:3}, low: {0:1,1:2,6:3}, stack: [0,1,6]
 }
];

```



```

 },
 {
 currentNode: 5, visitedIndices: [0,1,6,2,3,4,5],
 tin: {0:1,1:2,6:3,2:4,3:5,4:6,5:7}, low: {0:1,1:2,6:3,2:4,3:5,4:6,5:7},
 activeEdge: [4,5], backEdge: null,
 log: "E→F. tin[F]=7, low[F]=7.",
 activeCodeLine: [4,5,7,10,11]
 },
 {
 currentNode: 5, visitedIndices: [0,1,6,2,3,4,5],
 tin: {0:1,1:2,6:3,2:4,3:5,4:6,5:7}, low: {0:1,1:2,6:3,2:4,3:5,4:6,5:7},
 activeEdge: null, backEdge: [5,3],
 log: "Из F видим D (посещён). Обратное ребро (F,D).",
 activeCodeLine: [7,8,9]
 },
 {
 currentNode: 4, visitedIndices: [0,1,6,2,3,4,5],
 tin: {0:1,1:2,6:3,2:4,3:5,4:6,5:7}, low: {0:1,1:2,6:3,2:4,3:5,4:6,5:7},
 activeEdge: null, backEdge: null,
 log: "Возврат F→E. low[E]=min(6,5)=5. (E,F) – не мо",
 activeCodeLine: [11,13,16,17]
 },
 {
 currentNode: 3, visitedIndices: [0,1,6,2,3,4,5],
 tin: {0:1,1:2,6:3,2:4,3:5,4:6,5:7}, low: {0:1,1:2,6:3,2:4,3:5,4:6,5:7},
 activeEdge: null, backEdge: null,
 log: "Возврат E→D. low[D]=min(6,5)=5. (D,E) – не мо",
 activeCodeLine: [11,13,16,17]
 },
 {
 currentNode: 2, visitedIndices: [0,1,6,2,3,4,5],
 tin: {0:1,1:2,6:3,2:4,3:5,4:6,5:7}, low: {0:1,1:2,6:3,2:4,3:5,4:6,5:7},
 activeEdge: null, backEdge: null,
 log: "Возврат D→C. low[5] = 5 > tin[2] = 4! (C,D) – не мо",
 activeCodeLine: [13,14,15,16,17]
 },
 {
 currentNode: 1, visitedIndices: [0,1,6,2,3,4,5],

```



```

 {step.currentNode !== null && step.currentNode !== null}
 const n = GRAPH_NODES.find(x => x.id === n);
 if (!n) return null;
 return <circle cx={n.x} cy={n.y} r={18} fill={n.fill} />
 >;
 }}())}
 {GRAPH_NODES.map(n => {
 const cur = step.currentNode === n.id;
 const vis = step.visitedIndices.includes(n.id);
 const st = step.stack.includes(n.id);
 return <g key={n.id}>
 <circle cx={n.x} cy={n.y} r={12}
 fill={cur ? "#10b981" : st ? "#064e3b" : "#fff"}
 stroke={cur ? "#fff" : st ? "#34d399" : "#000"}
 strokeWidth={2.5}
 className="transition-all duration-300" />
 <text x={n.x} y={n.y+3} textAnchor="middle">{n.label}</text>
 {vis && (
 <rect x={n.x-18} y={n.y-28} width={36} height={10}
 className="transition-all duration-300" />
 <text x={n.x} y={n.y-20} textAnchor="center">
 {step.low[n.id] || "?"}</text>
 </>
)}
 </g>
)}}
</svg>
<div className="flex items-center justify-between" >
 Шаг {dfsIdx+1}/{DFS_STEPS.length}
 <div className="flex-1 mx-2 bg-slate-950 h-10" >
 <div className="bg-indigo-600 h-full transition-all" style={style} />
 </div>
</div>
</div>

```

```

 </div>
))
 }
 </div>
</div>

 { /* Log */ }
 <div className="bg-indigo-950/10 border borde
 <p className="text-[11px] text-slate-300 le
 <div className="mt-2 pt-2 border-t border-s
 Мосты:</
 <span className="font-mono font-bold text
 {step.bridges.length > 0 ? step.bridges

 </div>
 </div>
 </div>
</div>
</div>
 </div>
);
}

```

---

ФАЙЛ 40/111: src/components/DijkstraViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

---

```

import { useState, useEffect } from 'react';
import { useVizRuntime, vizArray, vizNumber, vizRecord,
import { LiveDistTable } from './LiveDistTable';

interface Node { id: string; x: number; y: number; name
interface Edge { u: string; v: string; w: number; }
interface Step {
 currentNode: string | null;
 checkingEdge: { u: string; v: string } | null;
 distances: { [key: string]: number };

```

```

 adjList[e.u].push({ v: e.v, w: e.w });
 });

const codeLines = [
 { line: 1, text: 'function dijkstra(graph, start):' },
 { line: 2, text: ' dist = {v: ∞}; dist[start] = 0' },
 { line: 3, text: ' pq = PriorityQueue(); pq.push(start, 0)' },
 { line: 4, text: ' while not pq.empty():' },
 { line: 5, text: ' d, u = pq.pop() // Извлекаем минимальный элемент' },
 { line: 6, text: ' if d > dist[u]: continue' },
 { line: 7, text: ' for v, weight in graph.neighbours(u):' },
 { line: 8, text: ' if dist[u] + weight < dist[v]:' },
 { line: 9, text: ' dist[v] = dist[u] + weight' },
 { line: 10, text: ' pq.push(v, dist[v])' },
 { line: 11, text: ' return dist // Все кратчайшие пути найдены' },
];

const [steps, setSteps] = useState<Step[]>([]);
const [currentStepIndex, setCurrentStepIndex] = useState(0);
const runtime = useVizRuntime();
const [isPlaying, setIsPlaying] = useState(false);

useEffect(() => {
 const simulationSteps: Step[] = [];
 const dist: Record<string, number> = { A: 0, B: 999 };
 const visited: Record<string, boolean> = { A: false, B: false };
 const prev: Record<string, string | null> = { A: null, B: null };

 simulationSteps.push({
 currentNode: null, checkingEdge: null,
 distances: { ...dist }, visited: { ...visited },
 log: ' Дейкстра готов к доставке. Начинаем из Пикет-Пост-Оффиса',
 activeCodeLine: 2,
 });

 for (let iter = 0; iter < nodes.length; iter++) {
 let minD = 999;
 let u = null;

```

```

 });
 }
}

simulationSteps.push({
 currentNode: null, checkingEdge: null,
 distances: { ...dist }, visited: { ...visited },
 log: ' Дейкстра всё доставил! Все возможные крат
 activeCodeLine: 11,
});

setSteps(simulationSteps);
}, []));

useEffect(() => {
 let timer: any = null;
 if (isPlaying && currentStepIndex < steps.length -
 timer = setTimeout(() => {
 setCurrentStepIndex((prev) => prev + 1);
 }, 1500);
 } else if (currentStepIndex >= steps.length - 1) {
 setIsPlaying(false);
 }
 return () => clearTimeout(timer);
}, [isPlaying, currentStepIndex, steps.length]);

const baseStep = steps[currentStepIndex] || null;

if (!baseStep) return <div className="text-white">3arg
const vars = runtime?.variables;
const liveU = vizString(vars?.u);
const liveV = vizString(vars?.v);
const liveDist = vizRecord(vars?.dist);
const livePrev = vizRecord(vars?.prev);
const liveDone = vizArray(vars?.done) ?? vizArray(var
const compilerLinked = liveU !== null || liveV !== nu
const currentStep: Step = compilerLinked

```



```

 r="12" fill="#1e293b"
 stroke={isChecked ? '#f59e0b' : '#f87192'}
 />
 <text
 x={({uNode.x + vNode.x} / 2} y={({uNode.y + vNode.y} / 2})
 fill={isChecked ? '#f59e0b' : '#f87192'}
 fontSize="11" fontWeight="bold" text={label}
 >
 {edge.w}
 </text>
 </g>
);
}
}}}

{/* Вершины */}
{nodes.map((node) => {
 const isCurrent = currentStep.currentNode === node.id;
 const isVisited = currentStep.visited[node.id];
 const dist = currentStep.distances[node.id];

 return (
 <g key={node.id} className="transition-all duration-300">
 <circle
 cx={node.x} cy={node.y}
 r={isCurrent ? 22 : 18}
 fill={isCurrent ? '#4f46e5' : isVisited ? '#f87192' : '#d1d7db'}
 stroke={isCurrent ? '#a5b4fc' : isVisited ? '#f59e0b' : '#d1d7db'}
 strokeWidth={isCurrent ? 4 : 2}
 className="transition-all duration-300"
 />
 <text x={node.x} y={node.y + 5} fill="black">
 {node.id}
 </text>
 <text x={node.x} y={node.y - 28} fill="white">
 {dist === 999 ? '∞' : `d=${dist}`}
 </text>
 <text x={node.x} y={node.y + 35} fill="white">
 {node.w}
 </text>
 </g>
);
})}

```





```
 </div>
 </div>
 </div>
 </div>
</div>
);
}
```

---

ФАЙЛ 41/111: src/components/DPVisualizer.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

---

```
import { useState, useEffect } from 'react';
import { useVizRuntime, vizNumber, vizRecord } from '...';
import { Play, Pause, RotateCcw, ChevronLeft, ChevronRight } from '...';

interface DPStep {
 id: number;
 n: number;
 action: 'call' | 'memo_hit' | 'base_case' | 'calc' | 'end';
 desc: string;
 codeLine: number;
 memoState: { [key: number]: number };
}

const DP_CODE_LINES = [
 /* 1 */ "function fibMemo(n, memo = {}) {",
 /* 2 */ " if (n in memo) return memo[n];",
 /* 3 */ " if (n <= 2) return 1;",
 /* 4 */ " memo[n] = fibMemo(n - 1, memo) + fibMemo(n - 2, memo);",
 /* 5 */ " return memo[n];",
 /* 6 */ "}";
];

export function DPVisualizer() {
 const [targetN, setTargetN] = useState<number>(5);
```

```

 action: 'base_case',
 desc: `Базовый случай: n <= 2 (n=${n}). Возвр
 codeLine: 3,
 memoState: { ...memoMap }
 });
 return 1;
}

generatedSteps.push({
 id: stepId++,
 n,
 action: 'call',
 desc: `Вычисляем рекурсивно: fibМемо(${n - 1})
 codeLine: 4,
 memoState: { ...memoMap }
});

const res1 = simulateFib(n - 1);
const res2 = simulateFib(n - 2);
memoMap[n] = res1 + res2;

generatedSteps.push({
 id: stepId++,
 n,
 action: 'calc',
 desc: `Сохраняем в кэш: мемо[${n}] = ${res1} +
 codeLine: 5,
 memoState: { ...memoMap }
});

return memoMap[n];
}

const finalRes = simulateFib(targetN);

generatedSteps.push({
 id: stepId++,
 n: targetN,
```

```

 memoState,
 desc: `Компилятор: k=${liveK ?? "-"}; таблица m
 }
 : baseStep;

return (
 <div className="bg-slate-900 rounded-2xl border border-
 {/* Header & Controls */}
 <div className="flex flex-col lg:flex-row lg:items-
 <div>
 <div className="flex items-center gap-3 mb-2">
 <div className="p-2 bg-emerald-500/10 border
 <RefreshCw className="w-6 h-6" />
 </div>
 <h3 className="text-2xl font-extrabold text-
 </div>
 <p className="text-sm text-slate-400">
 Смотри, как таблица DP кэширует результаты
 </p>
 </div>

 <div className="flex flex-wrap items-center gap-
 <div className="flex items-center gap-2 bg-sl
 N:
 <select
 value={targetN}
 onChange={(e) => setTargetN(Number(e.targ
 disabled={isPlaying}
 className="bg-transparent text-white font-
 >
 <option value={4}>N = 4</option>
 <option value={5}>N = 5</option>
 <option value={6}>N = 6</option>
 <option value={7}>N = 7</option>
 </select>
 </div>

 <div className="flex items-center bg-slate-950

```

```

 {/* DP Table View */}
 <div className="mb-8 bg-slate-950 p-4 md:p-6 rounded"
 <h4 className="text-xs font-bold uppercase tracking-wider"
 <div className="flex flex-wrap items-center gap-2"
 {Array.from({ length: targetN }, (_, i) => i + 1).map((num) => {
 const isCached = num in memoState || num <= 2;
 const val = num <= 2 ? 1 : memoState[num];
 const isCurrent = currentStep?.n === num;

 return (
 <div
 key={num}
 className={`flex flex-col items-center justify-center gap-2
 ${isCurrent ? 'bg-emerald-950 border-emerald-500' : ''}
 ${isCached ? 'bg-slate-900 border-emerald-500/50' : ''}
 ${!isCurrent & !isCached ? 'bg-slate-900/50 border-slate-800' : ''}
 `}
 >
 N
 <span className={`text-xl md:text-2xl font-medium text-slate-100`}
 {isCached ? val : '0'}>

 {num <= 2 && 1}
 </div>
);
 })}
 </div>
 </div>
 </div>
);
}
}
}
</div>

```

```

{currentStep && (
 <div className="mt-6 pt-4 border-t border-slate-800"
 <div className="text-sm font-medium text-slate-100"


```

```

 <p className="text-sm text-slate-400">
 В классической рекурсии для $N=\{targetN\}$ по
 вычисленное значение в таблице, мы момента.
 </p>
 <div className="p-4 bg-slate-900 rounded-xl t-slate-300">
 Всего шагов симуляции: {steps.length}
 <span className="text-emerald-400 font-bo
 </div>
 </div>
})

{activeTab === 'code' && (
 <div className="bg-slate-950 rounded-2xl border
 <div className="bg-slate-900 px-6 py-3 border
 <span className="text-xs font-bold font-m
 <span className="text-xs text-emerald-400
 </div>
 <pre className="p-6 font-mono text-sm space
 {DP_CODE_LINES.map((line, idx) => {
 const lineNum = idx + 1;
 const isActive = currentStep?.codeLine
 return (
 <div
 key={lineNum}
 className={`flex items-center px-4
 isActive
 ? 'bg-emerald-600/20 border-l-4
 : 'text-slate-300 hover:bg-slate
 `}>
 </div>
)
 })}
 </pre>
 <div className="p-6 bg-slate-900/50 border-

```

```

 { id: 3, label: "D", x: 280, y: 250 },
 { id: 4, label: "E", x: 100, y: 250 }
];

const C1_EDGES = [
 { u: 0, v: 1 }, { u: 0, v: 2 },
 { u: 1, v: 3 }, { u: 2, v: 4 },
 { u: 1, v: 2 },
 { u: 3, v: 4 }
];

const ekey = (a: number, b: number) => `${Math.min(a, b)}-${Math.max(a, b)}`;
const DEG = C1_NODES.map((n) => C1_EDGES.filter((e) => ekey(e.u, e.v) === n.id));
const ODD = C1_NODES.map((n) => n.id).filter((id) => DEG[id] % 2 !== 0);

/**
 * Билет 13 – «Графы. Эйлеров цикл».
 *
 * Две части:
 * 1. игра «пройди все рёбра» кликами (Эйлер не прощает ошибок);
 * 2. критерий: степени вершин и число нечётных (0 → цикл).
 *
 * Синхронизация с Python-компилятором (PAGE_SYNC["euler"]);
 * path – маршрут (массив вершин) рисуется поверх графа;
 * пройдены, deg/odd – степени и нечётные вершины, lastStep – номер шага.
 */
export function EulerSimulator() {
 const chapterId = useContext(VizChapterContext);
 const runtime = useVizRuntime();
 const vars = runtime?.variables;

 const [c1Path, setC1Path] = useState<number[]>([]);
 const [c1VisitedEdges, setC1VisitedEdges] = useState<Set<number>>();
 const [c1Msg, setC1Msg] = useState("Клики на вершину");
 const [c1Done, setC1Done] = useState(false);

 useEffect(() => {

```

```

 } else {
 setClMsg(`Пройдено рёбер: ${nextEdges.length} / $
 }
 };

/* — что говорит компилятор — */
const live = useMemo(() => {
 if (!vars) return null;
 const rawPath = vizArray(vars.path) ?? vizArray(var
 const path = (rawPath ?? [])
 .map((x) => (typeof x === "number" ? x : vizNumbe
 .filter((x): x is number => x !== null && Number.
 const usedRaw = vizArray(vars.used) ?? vizArray(var
 const usedFlags = (usedRaw ?? []).map((x) => Boolean
 const degRec = vizRecord(vars.deg) ?? vizRecord(var
 const deg = degRec
 ? C1_NODES.map((n) => {
 const val = degRec[String(n.id)] ?? degRec[n.
 const num = typeof val === "number" ? val : v
 return num ?? DEG[n.id];
 })
 : null;
 const oddRaw = vizArray(vars.odd);
 const odd = oddRaw ? oddRaw.map((x) => (typeof x ===
 const last = vizNumber(vars.last) ?? vizNumber(vars
 const step = vizNumber(vars.step) ?? vizNumber(vars
 if (path.length === 0 && usedFlags.length === 0 &&
 // рёбра маршрута: пары соседних вершин
 const keys: string[] = [];
 for (let i = 0; i + 1 < path.length; i++) keys.push
 usedFlags.forEach((f, i) => {
 if (f && C1_EDGES[i]) keys.push(ekey(C1_EDGES[i]).
));
 return {
 path,
 keys: Array.from(new Set(keys)),
 deg,
 odd,
 };

```



```
</div>
```

```
<div className="bg-slate-950 rounded-xl p-4 border-1 border-gray-800 -hidden">
```

```
<div className="absolute inset-0 bg-[radial-gradient(circle_1px_1px_1px_1px)]">
```

```
<svg className="w-full h-[260px] z-10 relative">
```

```
 {C1_EDGES.map((e, i) => {
```

```
 const u = C1_NODES.find(n => n.id === e.u)!
```

```
 const v = C1_NODES.find(n => n.id === e.v)!
```

```
 const key = ekey(e.u, e.v);
```

```
 const used = visitedKeys.includes(key);
```

```
 // номер этого ребра в маршруте – подписываем
```

```
 const order = path.length > 1 ? (() => {
```

```
 for (let s = 0; s + 1 < path.length; s++)
```

```
 return null;
```

```
 })() : null;
```

```
 const isLastEdge = order !== null && order === path.length - 1;
```

```
 return <g key={i}>
```

```
 <line x1={u.x} y1={u.y} x2={v.x} y2={v.y}
```

```
 stroke={isLastEdge ? "#a5b4fc" : used ? "#6366f1" : "#fff"}
```

```
 strokeWidth={isLastEdge ? 5 : used ? 4 : 2}
```

```
 className="transition-all duration-300"/>
```

```
 {order !== null && (
```

```
 <text x={({u.x + v.x} / 2) y={({u.y + v.y} / 2)}
```

```
)}</text>
```

```
)}
```

```
 </g>;
```

```
)})
```

```
{C1_NODES.map(n => {
```

```
 const isLast = lastId === n.id;
```

```
 const visited = path.includes(n.id);
```

```
 const isOdd = odd.includes(n.id);
```

```
 return <g key={n.id} className="cursor-pointer">
```

```
 {isLast && <circle cx={n.x} cy={n.y} r={11}
```

```
 <circle cx={n.x} cy={n.y} r={11}
```

```
 fill={isLast ? "#6366f1" : visited ? "#6366f1" : "#fff"}
```

```
 stroke={isLast ? "#fff" : visited ? "#6366f1" : "#fff"}
```

```
 strokeWidth={2.5}
```

```

 {live.path.map((id) => C1_NODES[id].label).

 ойдено: {live.keys.length}/{C1_EDGES.length}

 {odd.length}
 {live.step !== null &&
 x-2 py-1">step = {live.step}}
 </>
) : (

 открой панель Python и запусти код: массив
 том графе, а <code className="font-mono text-slate-500">
 есчитают критерий

)}
 </div>
</div>
);
}

```

ФАЙЛ 43/111: src/components/FloydViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

```

import { useState, useEffect } from 'react';
import { liveMatrix, pickLiveVar, useVizRuntime, vizNum

```

```

interface Step {
 k: number; i: number; j: number;
 matrix: (number | null)[][]; updated: boolean;
 log: string; activeCodeLine: number;
}

```

```

/** Подсказка для кнопок, когда шаги ведёт компилятор.
const HINT_TEXT = "Шаги сейчас ведёт панель Python: ← →
 ва листать вручную.";

```

```
const codeLines = [
 { line: 1, text: 'function floyd_warshall(matrix, V' },
 { line: 2, text: ' dist = copy(matrix)' },
 { line: 3, text: ' for k from 0 to V - 1: // Пос' },
 { line: 4, text: ' for i from 0 to V - 1:' },
 { line: 5, text: ' for j from 0 to V - 1' },
 { line: 6, text: ' if dist[i][k] + d' },
 { line: 7, text: ' dist[i][j] = ' },
 { line: 8, text: ' return dist' },
];
```

```
useEffect(() => {
 const simulationSteps: Step[] = [];
 const dist = initialMatrix.map(row => [...row]);

 simulationSteps.push({
 k: -1, i: -1, j: -1, matrix: dist.map(r => [...r]),
 log: ' Инициализация матрицы прямых путей, ∞ (999)',
 activeCodeLine: 2,
 });
});
```

```
const N = 7;
for (let k = 0; k < N; k++) {
 simulationSteps.push({
 k, i: -1, j: -1, matrix: dist.map(r => [...r]),
 log: `+ Внешний цикл. Разрешаем пути через пром'`,
 activeCodeLine: 3,
 });
};
```

```
for (let i = 0; i < N; i++) {
 for (let j = 0; j < N; j++) {
 if (i === j || i === k || j === k) continue;

 const oldVal = dist[i][j];
 const viaVal = dist[i][k] + dist[k][j];
 if (dist[i][k] !== 999 && dist[k][j] !== 999 && viaVal < oldVal)
 dist[i][j] = viaVal;
 }
}
```

```

const liveJ = vizNumber(vars?.j);
/**
 * Матрица из кода пользователя – СВОЕЙ размерности.
 * протискивались в демо-сетку 7×7: при n < 7 появлял
 * при n > 7 компонент падал на nodeNames[i].split(..
 */
const liveSource = pickLiveVar(vars, ["d", "dist", "d
const liveMat = liveMatrix(liveSource?.value);
const compilerLinked = liveK !== null || liveI !== nu

const rows = liveMat?.rows ?? baseStep.matrix.length;
const cols = liveMat?.cols ?? (baseStep.matrix[0]?.le
const shownMatrix: (number | null)[][] = liveMat
 ? Array.from({ length: rows }, (_, i) => Array.from
 : baseStep.matrix;

const demoSize = baseStep.matrix.length;
const sizeMismatch = compilerLinked && liveMat && (li
 ? `Демо-схема графа нарисована для ${demoSize} вершн
 ерности; схема графа остаётся демонстрационной.`
 : null;
const labelOf = (idx: number) => nodeNames[idx]?.spli

const currentStep: Step = compilerLinked
 ? {
 ...baseStep,
 k: liveK ?? -1,
 i: liveI ?? -1,
 j: liveJ ?? -1,
 matrix: shownMatrix,
 updated: !!runtime?.changed?.some(({name}) => nam
 log: `Компилятор: k=${liveK ?? "-"}, i=${liveI
 ame})` : ""}. Подсветка взята из реальных пер
 }
 : baseStep;

return (
 <div className="bg-slate-800/90 p-6 rounded-2xl bor

```

```

<path d="M0,0 L6,3 L0,6 z" fill="#10b981" />
 <marker id="arrow-fw-via" markerWidth="6"
th d="M0,0 L6,3 L0,6 z" fill="#818cf8" /></
</defs>
{Object.keys(adjList).flatMap((uStr) => {
 const u = parseInt(uStr);
 return adjList[u].map((edge) => {
 const uNode = nodesSvg.find((n) => n.id
 const vNode = nodesSvg.find((n) => n.id
 const isTarget = currentStep.i === u &&
 const isVia = (currentStep.i === u && c
 let strokeColor = '#475569'; let marker
 if (isTarget) { strokeColor = '#10b981'
 else if (isVia) { strokeColor = '#818cf8
 return (
 <g key={`${u}-${edge.v}`}>
 <line x1={uNode.x} y1={uNode.y} x2=
markerEnd={marker} strokeDasharray={isTarg
 <circle cx={({uNode.x + vNode.x) / 2
isVia ? '#818cf8' : '#64748b'} strokeWidth
 <text x={({uNode.x + vNode.x) / 2} y=
8fafc'} fontSize="10" fontWeight="bold" tex
 </g>
);
 });
});
})}
{nodesSvg.map((node) => {
 const isK = currentStep.k === node.id;
 const isI = currentStep.i === node.id;
 const isJ = currentStep.j === node.id;
 let fill = '#1e293b'; let stroke = '#64748b';
 if (isK) { fill = '#4f46e5'; stroke = '#a
 else if (isI) { fill = '#b45309'; stroke
 else if (isJ) { fill = '#047857'; stroke
 return (
 <g key={node.id} className="transition-
 <circle cx={node.x} cy={node.y} r={is
sJ ? 4 : 2} className="transition-colors du

```

```

)
 }}}
 </div>
);
 }}}
</div>
</div>
</div>

<div className="flex flex-col lg:flex-row gap-6">
 {/* Матрица расстояний */}
 <div className="w-full lg:w-1/2 bg-rose-950/10">
 <div className="flex justify-between items-center">
 <h4 className="text-lg font-bold text-rose-950">
 </div>
 {sizeMismatch ? (
 <p className="mb-3 rounded-lg border border-rose-950/40">
 {sizeMismatch}
 </p>
) : null}
 <div className="overflow-x-auto">
 <table className="w-full text-center border border-rose-950/40">
 <thead><tr className="bg-rose-950/40 text-rose-950">
 <th className="p-2 border-r border-rose-950/40">
 {Array.from({ length: cols }, (_, idx) => {
 <th key={idx} className={`p-2 border-rose-950/40 ${
 labelOf(idx)} {idx}</th>
 })}
 </tr></thead>
 <tbody className="text-xs font-mono">
 {currentStep.matrix.map((row, i) => (
 <tr key={i} className="border-b border-rose-950/40">
 <td className={`p-2 bg-rose-950/40 ${
 tStep.k === i ? 'text-amber-400' : ''}`}>
 {labelOf(i)} {i}
 </td>
 {row.map((val, j) => {
 const isUpd = currentStep.i === i

```

---

ФАЙЛ 44/111: src/components/GraphComponentsViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

---

```
import React, { useCallback, useContext, useEffect, useLayoutEffect } from 'react';
import { ChevronLeft, ChevronRight, Play, Pause, RotateCcw } from 'lucide-react';
import { VizChapterContext, emitVizDemo, useVizRuntime, useVizState } from './viz-context';
```

```
/**
 * Билет 8 – «Графы. Компоненты связности».
 *
 * Две вкладки:
 * 1. «Обход» – DFS/BFS-покраска: внешний цикл по непокрашенным вершинам,
 * каждый новый запуск = +1 к счётчику компонент.
 * 2. «DSU» – рёбра приходят по одному, union склеивает компоненты,
 * comps уменьшается.
 *
 * Синхронизация с Python-компилятором (PAGE_SYNC["graph-components-viz"]):
 * - comp / color / used – массив разметки: вершины покрашены / нет
 * - parent / p – массив DSU: рисуем множества
 * - v – текущая вершина (пульсирует)
 * - stack / dq / q – фронт обхода (подсвечивает)
 * - comps / c – счётчик компонент (большая цифра)
 * - i / e / step – номер шага: двигает покадровую анимацию
 */
```

```
type Mode = "dfs" | "dsu";
```

```
interface Node {
 id: number;
 label: string;
 x: number;
 y: number;
}
```

```
/** Граф из 10 вершин и трёх компонент – ровно как в те...
```

```

/** Покадровая раскладка DFS-обхода: ровно тот порядок,
function buildDfsFrames(): DfsFrame[] {
 const frames: DfsFrame[] = [];
 const comp = new Array(NODES.length).fill(0);
 const push = (cur: number | null, frontier: number[],
 => {
 frames.push({ title, note, comp: [...comp], cur, fr
 });
 let c = 0;
 push(null, [], c, "Старт: все вершины серые", "comp =
 дой непокрашенной.");
 for (let start = 0; start < NODES.length; start++) {
 if (comp[start] !== 0) continue;
 c += 1;
 const stack = [start];
 comp[start] = c;
 push(start, stack, c, `Новый запуск: c = ${c}, стар
 цвет ${c} и кладём в стек.`);
 while (stack.length) {
 const v = stack[stack.length - 1];
 const next = ADJ[v].find((to) => comp[to] === 0);
 if (next === undefined) {
 stack.pop();
 push(v, stack, c, `Из ${v} идти некуда – возвра
 ;
 continue;
 }
 comp[next] = c;
 stack.push(next);
 push(next, stack, c, `${v} → ${next}: красим в цв
 }, кладём ${next} в стек.`, [v, next]);
 }
 push(null, [], c, `Компонента ${c} готова`, `Стек п
 } вершин. Ищем следующую серую вершину.`);
 }
 push(null, [], c, "Готово: компонент = " + c, `Внешни
 вязности.`);

```



```

const DFS_FRAMES = buildDfsFrames();
const DSU_FRAMES = buildDsuFrames();

/** Корень DSU-множества с полным сжатием пути. */
const dsuRoot = (parent: number[], v: number): number => {
 let cur = v;
 const seen = new Set<number>();
 while (parent[cur] !== cur && !seen.has(cur)) {
 seen.add(cur);
 cur = parent[cur];
 }
 return cur;
};

export const GraphComponentsViz: React.FC = () => {
 const chapterId = useContext(VizChapterContext);
 const runtime = useVizRuntime();
 const [mode, setMode] = useState<Mode>("dfs");
 const [step, setStep] = useState(0);
 const [playing, setPlaying] = useState(false);
 const timer = useRef<number | null>(null);

 const frames = mode === "dfs" ? DFS_FRAMES : DSU_FRAMES;
 const maxStep = frames.length - 1;
 const frame = frames[Math.min(step, maxStep)] as DfsFrame;

 /** — вкладка сообщает компилятору, какой код подстав. */
 useEffect(() => {
 if (chapterId) emitVizDemo(chapterId, mode);
 }, [chapterId, mode]);

 /** — автопроигрывание — */
 useEffect(() => {
 if (!playing) return;
 timer.current = window.setInterval(() => {
 setStep((s) => {
 if (s >= maxStep) {

```

```

useEffect(() => {
 if (liveStep === null || !Number.isFinite(liveStep))
 goTo(liveStep);
}, [liveStep, goTo]);

/* — какая разметка сейчас в силе: код пользователя */
/** Массив parent, который сейчас рисуем: из кода пользователя */
const parentNow = liveParent ?? (mode === "dsu" ? frame.parent : null);
/** Номер множества по parent[]: корни нумеруются в parentNow */
const setOf = useCallback(
 (parent: number[], id: number): number => {
 const roots = Array.from(new Set(parent.map((_, i) => i)));
 return roots.indexOf(dsuRoot(parent, id)) + 1;
 },
 []
);

const compOf = (id: number): number => {
 if (liveComp && liveComp[id] !== undefined && liveComp[id] !== null)
 if (parentNow) return setOf(parentNow, id);
 return frame.comp?.[id] ?? 0;
};

const frontier = liveFrontier ?? (mode === "dfs" ? frame.frontier : null);
const cur = liveCur ?? frame.cur;
const comps = liveComps ?? frame.comps;
const edge = frame.edge;
const title = runtime && vars ? `${frame.title} • Контур` : frame.title;

const switchMode = (m: Mode) => {
 setMode(m);
 setStep(0);
 setPlaying(false);
};

return (
 <div className="bg-slate-800/90 p-5 sm:p-6 rounded-2xl" >
 <div className="flex flex-wrap items-center justify-center" >

```



```

 });
 }}}
 </g>
 </svg>
</div>

```

```

{/* подпись кадра */}
<div className="mt-3 bg-slate-900/70 border border-
 <p className="text-sm font-bold text-white mb-1
 {runtime && vars && <Terminal className="w-3.
 {title}
 </p>
 <p className="text-xs text-slate-400 leading-re
</div>

```

```

{/* живые переменные из компилятора */}
<div className="mt-3 flex flex-wrap items-center
 <span className="text-[10px] uppercase tracking
 {vars ? (
 ["v", "comps", "c", "i", "e", "comp", "parent
 vars[name] === undefined ? null : (
 <span key={name} className="text-[11px] f
 py-1">
 {name} = {JSON.stringify(vars[name]).sl

)
)
) : (

 открой панель Python – переменные <code cla
 erald-400">parent</code>, <code className="
 comps</code> покрасят этот граф сами

)}
 </div>
 </div>
);
};

```

```

 'F': ['C'],
 'G': ['C'],
};

type Action = {
 type: 'visit' | 'process' | 'backtrack';
 node: string;
 desc: string;
 queueOrStack?: string[];
 visitedNodes?: string[];
};

function generateDFSSteps(start: string) {
 const steps: Action[] = [];
 const vis = new Set<string>();
 const stack: string[] = []; // for visualization, j

 function dfs(v: string) {
 vis.add(v);
 stack.push(v);
 steps.push({ type: 'visit', node: v, desc: `Замечание: ${v} посещено`,
 visitedNodes: Array.from(vis) });

 for (const u of adj[v]) {
 if (!vis.has(u)) {
 steps.push({ type: 'process', node: v, desc: `Процесс: ${v} -> ${u}`,
 queueOrStack: [...stack], visitedNodes: Array.from(vis) });
 dfs(u);
 }
 }

 stack.pop();
 steps.push({ type: 'backtrack', node: v, desc: `Обратный шаг: ${v}`,
 queueOrStack: [...stack], visitedNodes: Array.from(vis) });
 }

 dfs(start);
 return steps;
}

```

```

const [stepIdx, setStepIdx] = useState(0);

const steps = useMemo(() => mode === 'dfs' ? genera
useVizStepSync(stepIdx, setStepIdx, steps.length -
 const v = vizString(vars.v);
 const structure = vizArray(vars.stack) ?? vizArra
 const visited = vizArray(vars.visited);
 if (v === null && !structure && !visited) return
 const candidates = steps
 .map(({step, index}) => ({ step, index }))
 .filter(({ step }) =>
 (v === null || step.node === v) &&
 (!structure || step.queueOrStack?.length ===
 (!visited || step.visitedNodes?.length === vi
);
 return candidates.at(-1)?.index ?? null;
});

useEffect(() => {
 setStepIdx(0);
 setAutoPlay(false);
}, [mode]);

useEffect(() => {
 if (autoPlay) {
 const t = setInterval(() => {
 setStepIdx(p => {
 if (p >= steps.length - 1) {
 setAutoPlay(false);
 return p;
 }
 return p + 1;
 });
 }, 1200);
 return () => clearInterval(t);
 }
}, [autoPlay, steps.length]);

```

```

 { /* Nodes */
 {nodes.map(n => {
 const visited = isVisited(n)
 const current = isCurrent(n)

 let fillColor = "fill-slate"
 let strokeColor = "stroke-slate"
 let textColor = "fill-slate"
 let radius = 18;

 if (visited) {
 fillColor = mode === 'd' ? "fill-slate" : "fill-white"
 strokeColor = mode === 'd' ? "stroke-slate" : "stroke-white"
 textColor = "fill-white"
 }

 if (current && step.type !== 'd') {
 strokeColor = "stroke-amber"
 textColor = "fill-amber"
 radius = 22;
 }

 return (
 <g key={n.id} className={mode === 'd' ? "node-d" : "node-v"}>
 <circle cx={n.x} cy={n.y} radius={radius}
 className={mode === 'd' ? "stroke-slate" : "stroke-white"} />
 <text x={n.x} y={n.y} text={n.label}
 className={mode === 'd' ? "fill-slate" : "fill-white"} />
 { /* Ripple for BFS */
 {current && mode === 'd' ? (
 <circle cx={n.x} cy={n.y} radius={22}
 className="stroke-amber" />
) : null}
)}
 </g>
)
 })}
)}

```

```

 <button onClick={() => {setAutoPlay
="Шаг назад" title="Шаг назад" className="p
0 disabled: hover: bg-transparent">
 <Undo strokeWidth={3} size={16} />
 </button>
 <button onClick={() => setAutoPlay(
justify-center min-w-[80px] ${mode === 'df
>
 {autoPlay ? <><Pause size={16} />
 </button>
 <button onClick={() => {setAutoPlay
= steps.length - 1} aria-label="Шаг вперед"
00 rounded-lg disabled:opacity-30 disabled:
 <SkipForward strokeWidth={3} si
 </button>
 <button onClick={() => {setAutoPlay
ame="p-2 text-slate-500 hover:text-rose-400
 <RefreshCw strokeWidth={3} size
 </button>
 </div>

 {/* Description Log */}
 <div className="flex-1 bg-slate-900/50
ow-hidden">
 <div className="text-[10px] text-sl
{steps.length}]</div>
 <div className="text-sm text-slate-
 {step.desc}
 </div>
 </div>
</div>
</div>
);
}

```



```

 if (l < n && a[l].priority > a[largest].priority) l = l;
 if (r < n && a[r].priority > a[largest].priority) l = r;
 if (largest !== idx) {
 [a[largest], a[idx]] = [a[idx], a[largest]];
 idx = largest;
 } else break;
 }
 return a;
}

function heapInsert(arr: Patient[], item: Patient): Patient[] {
 return siftUp([...arr, { ...item }]);
}

// Position in tree layout
function nodePos(index: number): { x: number; y: number } {
 const level = Math.floor(Math.log2(index + 1));
 const posInLevel = index - (Math.pow(2, level) - 1);
 const count = Math.pow(2, level);
 const x = ((posInLevel + 0.5) / count) * 100;
 const y = 10 + level * 24;
 return { x, y };
}

// Priority presets for realistic ER scenarios
const PATIENT_PRESETS = [
 { name: 'Иван (Инфаркт)', symptom: ' Болиит сердце', emoji: '👤🔴' },
 { name: 'Маша (Перелом)', symptom: ' Открытый перелом', emoji: '👤🔴' },
 { name: 'Кот Барсик (Укус)', symptom: ' Укусил шмеля', emoji: '🐈🔴' },
 { name: 'Семён (Температура)', symptom: ' Жар 39.5', emoji: '👤🔴' },
 { name: 'Оля (Порез)', symptom: ' Глубокий порез', emoji: '👤🔴' },
 { name: 'Пётр (Кашель)', symptom: ' Простуда', emoji: '👤🔴' },
];

const INITIAL_PATIENTS: Patient[] = (() => {
 let arr: Patient[] = [];
 const start = [
 { name: 'Иван (Инфаркт)', symptom: ' Болиит сердце', emoji: '👤🔴' },
];
 arr.push(...start);
 return arr;
})();

```

```

const [healingPatient, setHealing] = useState<Patient>({
 name: 'Пациент',
 emoji: '👤',
 symptom: 'Пациент',
 priority: 1,
 animState: 'in',
})

const addLog = (msg: string) => setLog(prev => [msg, ...prev])

// --- INSERT: Новый пациент поступает в приемный покой
const insertPatient = useCallback((name: string, priority: number) => {
 if (busy || heap.length >= 15) {
 addLog('⚠ Все палаты переполнены! Дождитесь выписки')
 setShake(true)
 setTimeout(() => setShake(false), 400)
 return
 }
 setBusy(true)

 const preset = PATIENT_PRESETS[Math.floor(Math.random() * PATIENT_PRESETS.length)]
 const finalName = name || preset.name.split(' ')[0]
 const finalEmoji = preset.emoji
 const finalSymptom = preset.symptom

 const newPatient: Patient = {
 id: `p${uid++}`,
 name: finalName,
 emoji: finalEmoji,
 symptom: finalSymptom,
 priority,
 animState: 'in',
 }

 const nextHeap = heapInsert(heap, newPatient).map(x => x)
 // Mark specifically this new patient as 'in' in the heap
 const target = nextHeap.find(x => x.priority === priority)
 const marked = nextHeap.map(x => x.id === (target ? target.id : ''))

 setHeap(marked)
 addLog(` Поступил пациент: ${finalName} с тяжестью ${priority}`)

 setTimeout(() => {
 setHeap(nextHeap)
 }, 1000)
}, [heap, busy, uid])

```



```
</div>
```

```
{/* УПРАВЛЕНИЕ */}
```

```
<div className="flex flex-col lg:flex-row items-s
```

```
 {/* Добавить случайного */}
```

```
 <button
```

```
 onClick={handleRandomInsert}
```

```
 disabled={busy || heap.length >= 15}
```

```
 className="flex-1 min-w-[150px] bg-gradient-to
```

```
 opacity-40 disabled:cursor-not-allowed text
```

```
 ve:scale-95 transition-all cursor-pointer f
```

```
 >
```

```
 Привезти по Скорой (INSERT)
```

```
 </button>
```

```
 {/* Добавить кастомного */}
```

```
 <form onSubmit={handleCustomInsert} className="
```

```
 <input
```

```
 type="text"
```

```
 required
```

```
 value={customName}
```

```
 onChange={e => setCustomName(e.target.value
```

```
 placeholder="Имя пациента"
```

```
 className="flex-1 bg-slate-800 border borde
```

```
 er-emerald-500 transition-colors placeholde
```

```
 />
```

```
 <div className="flex flex-col items-center mi
```

```
 <span className="text-[9px] font-mono text-
```

```
 <input
```

```
 type="range"
```

```
 min="10"
```

```
 max="99"
```

```
 value={customPriority}
```

```
 onChange={e => setCustomPriority(Number(e
```

```
 className="w-20 accent-emerald-500 cursor
```

```
 />
```

```
 </div>
```

```
</div>
```

```
<div className="relative w-full h-full min-h-
 <svg className="absolute inset-0 w-full h-f
 {edges}
 </svg>
```

```
{visualHeap.map((patient, idx) => {
 const pos = nodePos(idx);
 const isRoot = idx === 0;
 const isCompilerNode = liveI === idx;
 return (
```

```
 <div
 key={patient.id}
 className={`
 absolute flex flex-col items-center
 -translate-x-1/2 -translate-y-1/2 t
 ${patient.animState === 'in' ? 'anim
 ${patient.animState === 'winner' ?
 ${patient.animState === 'out' ? 'an
 ${isCompilerNode ? 'ring-4 ring-amb
 ${isRoot ? 'bg-rose-950/80 border-r
 `}
 >
```

```
 style={{
 left: `${pos.x}%`,
 top: `${pos.y}%`,
 width: isRoot ? 60 : 52,
 height: isRoot ? 60 : 52,
 }}
 >
```

```
 <span className="text-2xl leading-non
 <span className="text-[10px] font-mon
 {patient.priority}%

```

```
 {/* Tooltip */}
 <div className="absolute bottom-full
 <div className="bg-slate-950 border
```

```

 </div>
 </div>
) : (
 <div className="text-center text-slate-600">

 <p className="text-xs font-bold font-mono">
 <p className="text-[10px] mt-1">Ждём са
 </p>
 </div>
 </div>
)}
 </div>

 <div className="w-full h-3 bg-gradient-to-r f
</div>

</div>

{/* ЛОГ ДЕЙСТВИЙ */}
<div className="bg-slate-900 border border-slate-
 <div className="text-[10px] font-mono text-slate
 {log.map((entry, idx) => {
 <div
 key={idx}
 className={`text-xs font-mono flex items-ce
 >
 {idx === 0 ? <span className="text-emerald-5
 {entry}
 </div>
 })}
 </div>
</div>
);
};

```

```

 text: "#e2e8f0",
 dim: "#94a3b8",
 active: "#6366f1",
 done: "#10b981",
 warn: "#f59e0b",
 bad: "#f43f5e",
 edge: "#475569",
 };

export const Svg: React.FC<{ children: React.ReactNode;
 const paused = useContext(DemoPauseContext);
 return (
 <div className="w-full">
 <svg viewBox={`0 0 ${W} ${H}`} className="w-full h-full">
 {children}
 </svg>
 {caption && (
 <div className="text-[10px] text-slate-400 text-center">
 {caption}
 </div>
)}
 <div className="text-[9px] text-slate-600 text-center">
 {paused ? "пауза – курсор на карточке" : "наведи"}
 </div>
 </div>
);
};

export const Node: React.FC<{
 x: number;
 y: number;
 r?: number;
 fill: string;
 label?: string | number;
 stroke?: string;
}> = ({ x, y, r = 13, fill, label, stroke }) => (
 <g style={{ transition: MOVE }}>
 <circle cx={x} cy={y} r={r} fill={fill} stroke={stroke} />
 </g>
);

```

```

const step = useLoop(3, 1200);
// i -> j напрямую 9; через k=1 получаем 3+4=7
const pos: Record<string, [number, number]> = { i: [4, 1], k: [1, 1], j: [1, 4] };
const viaOpen = step >= 1;
const relaxed = step >= 2;
return (
 <Svg
 caption={
 step === 0
 ? "k ещё запрещена: путь i → j = 9"
 : step === 1
 ? "Разрешаем пересадку через k: 3 + 4 = 7"
 : "7 < 9 → D[i][j] = 7"
 }
 >
 <Edge a={pos.i} b={pos.j} color={relaxed ? C.bad : C.good} />
 <Edge a={pos.i} b={pos.k} color={viaOpen ? C.done : C.pending} />
 <Edge a={pos.k} b={pos.j} color={viaOpen ? C.done : C.pending} />

 <text x={130} y={112} textAnchor="middle" fontSize=
 9
 </text>
 <text x={72} y={54} textAnchor="middle" fontSize=
 3
 </text>
 <text x={188} y={54} textAnchor="middle" fontSize=
 4
 </text>

 <Node x={pos.i[0]} y={pos.i[1]} label="i" fill={C.i} />
 <Node x={pos.k[0]} y={pos.k[1]} label="k" fill={C.k} />
 <Node x={pos.j[0]} y={pos.j[1]} label="j" fill={C.j} />
 </Svg>
);
};

```

/\*\* Компоненты связности: несколько «островов», которые

```

export const ComponentsDemo: React.FC = () => {

```



```

 ? "Вершины – объекты {города, слова, состояни
 : step === 1
 ? "Рёбра – связи между ними"
 : "Вес ребра – цена перехода"
 }
 >
 {edges.map(([u, v, w], i) => {
 const a = pos[u];
 const b = pos[v];
 return (
 <g key={i}>
 <Edge a={a} b={b} color={step >= 1 ? C.acti
 {step >= 2 && (
 <text
 x={({a[0] + b[0]) / 2}
 y={({a[1] + b[1]) / 2 - 4}
 textAnchor="middle"
 fontSize={10}
 fontWeight="700"
 fill={C.warn}
 >
 {w}
 </text>
)}
 </g>
);
 })}
 </Svg>
);
};

```

/\*\* Сжатие координат: разрежённые числа → плотные индексы

```

export const CoordCompressDemo: React.FC = () => {
 const raw = [1000, 5, 74000, 5, 300];
 const sorted = [5, 300, 1000, 74000];

```

```

 {sorted.indexOf(v)}
 </text>
 </g>
)))
</Svg>
);
};

/* ----- Splay: три случая поворота ---

type Pt = [number, number];

const SplayFrames: React.FC<{
 frames: { pos: Record<string, Pt>; edges: [string, st
 captions: string[];
 ms?: number;
}> = ({ frames, captions, ms = 1500 }) => {
 const step = useLoop(frames.length, ms);
 const f = frames[step];
 const color = (id: string) =>
 id === "x" ? "#6366f1" : id === "p" ? "#0ea5e9" : i

 return (
 <Svg caption={captions[step]}>
 {f.edges.map(([a, b], i) => (
 <line
 key={i}
 x1={f.pos[a][0]}
 y1={f.pos[a][1]}
 x2={f.pos[b][0]}
 y2={f.pos[b][1]}
 stroke={C.edge}
 strokeWidth={1.8}
 style={{ transition: MOVE }}
 />
))}
 {Object.entries(f.pos).map(([id, [x, y]]) => (
 <g key={id} style={{ transform: `translate(${x},

```

```

 { pos: { g: [176, 24], p: [110, 70], x: [152, 112]
 { pos: { g: [176, 24], x: [110, 70], p: [64, 112]
 { pos: { x: [130, 30], p: [80, 100], g: [180, 100]
]]
 />
};

```

ФАЙЛ 49/111: src/components/hints/demosGraph.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

```

import React from "react";
import { C, Edge, Node, Svg, useLoop } from "../demoKit"

const GRAPH_POS: Record<string, [number, number]> = {
 A: [32, 65], B: [92, 30], C: [92, 100], D: [152, 65],
};
const GRAPH_EDGES: [string, string][] = [
 ["A", "B"], ["A", "C"], ["B", "D"], ["C", "D"], ["D",
];

export const BfsDemo: React.FC = () => {
 const order = [["A"], ["B", "C"], ["D"], ["E", "F"]];
 const step = useLoop(order.length + 1, 850);
 const visited = new Set(order.slice(0, step).flat());
 const wave = step > 0 && step <= order.length ? order
 return (
 <Svg caption={`Уровень ${Math.max(0, Math.min(step,
 {GRAPH_EDGES.map(([u, v], i) => {
 <Edge key={i} a={GRAPH_POS[u]} b={GRAPH_POS[v]}
 }}})
 {Object.entries(GRAPH_POS).map(([k, [x, y]]) => {
 <Node key={k} x={x} y={y} label={k}
 fill={wave.includes(k) ? C.active : visited.h
 stroke={wave.includes(k) ? "#a5b4fc" : undefi
 }}})

```

```

)))
 {nodes.map((n) => {
 const idx = placed.indexOf(n.id);
 return (
 <g key={n.id}>
 <rect x={n.x - 34} y={n.y - 12} width={68} h={24}
 " }} />
 <text x={n.x} y={n.y} textAnchor="middle" d={n.d}
 {idx >= 0 && <circle cx={n.x + 30} cy={n.y - 12} r={12}
 {idx >= 0 && {
 <text x={n.x + 30} y={n.y - 12} textAnchor="middle"
 x + 1}</text>
 }}
 </g>
);
)))
 <text x={224} y={62} textAnchor="middle" fontSize={14}>
 <text x={224} y={74} textAnchor="middle" fontSize={14}>
 </Svg>
);
};

```

```

export const RelaxDemo: React.FC = () => {
 const step = useLoop(3, 1100);
 const dV = [12, 12, 7][step];
 const improved = step === 2;
 return (
 <Svg caption={improved ? "12 > 3 + 4 → пишем 7. Пусть

 <Edge a={[60, 70]} b={[200, 70]} color={improved ? "red" : "blue"} />
 <text x={130} y={54} textAnchor="middle" fontSize={14}>12 > 3 + 4 = 7</text>
 <Node x={60} y={70} r={20} fill={C.active} label="12" />
 <Node x={200} y={70} r={20} fill={improved ? C.done : C.active} label="3 + 4" />
 <text x={60} y={106} textAnchor="middle" fontSize={14}>12</text>
 <text x={200} y={106} textAnchor="middle" fontSize={14}>3 + 4</text>
 <text x={130} y={22} textAnchor="middle" fontSize={14}>7</text>
 </Svg>
);
};

```

```

 <Node key={k} x={x} y={y} label={k} fill={k} stroke={k} />
)
 })
 </Svg>
);
};

export const MstDemo: React.FC = () => {
 const pos: Record<string, [number, number]> = {
 A: [40, 35], B: [130, 22], C: [220, 45], D: [70, 100], E: [180, 100]
 };
 const edges = [
 { u: "A", v: "B", w: 2 }, { u: "B", v: "C", w: 3 },
 { u: "D", v: "E", w: 4 }, { u: "B", v: "E", w: 6 },
];
 const chosen = ["A-D", "A-B", "B-C", "D-E"];
 const step = useLoop(chosen.length + 1, 800);
 const taken = new Set(chosen.slice(0, step));
 return (
 <Svg caption={`Жадно берём самые лёгкие рёбра без циклов`} >
 {edges.map((e, i) => {
 const on = taken.has(`${e.u}-${e.v}`);
 return (
 <g key={i}>
 <Edge a={pos[e.u]} b={pos[e.v]} color={on ? "red" : "black"} />
 <text x={(pos[e.u][0] + pos[e.v][0]) / 2} y={pos[e.v][1]}
 textAnchor="middle" fontSize={9} fontWeight="bold">{e.u}-{e.v}</text>
 </g>
);
 })}
 {Object.entries(pos).map(([k, [x, y]]) => (<Node key={k} x={x} y={y} label={k} fill={k} stroke={k} />))}
 </Svg>
);
};

export const SccDemo: React.FC = () => {
 const pos: Record<string, [number, number]> = {
 A: [40, 35], B: [130, 22], C: [220, 45], D: [70, 100], E: [180, 100]
 };
 const edges = [
 { u: "A", v: "B", w: 2 }, { u: "B", v: "C", w: 3 },
 { u: "D", v: "E", w: 4 }, { u: "B", v: "E", w: 6 },
];
 const chosen = ["A-D", "A-B", "B-C", "D-E"];
 const step = useLoop(chosen.length + 1, 800);
 const taken = new Set(chosen.slice(0, step));
 return (
 <Svg caption={`Жадно берём самые лёгкие рёбра без циклов`} >
 {edges.map((e, i) => {
 const on = taken.has(`${e.u}-${e.v}`);
 return (
 <g key={i}>
 <Edge a={pos[e.u]} b={pos[e.v]} color={on ? "red" : "black"} />
 <text x={(pos[e.u][0] + pos[e.v][0]) / 2} y={pos[e.v][1]}
 textAnchor="middle" fontSize={9} fontWeight="bold">{e.u}-{e.v}</text>
 </g>
);
 })}
 {Object.entries(pos).map(([k, [x, y]]) => (<Node key={k} x={x} y={y} label={k} fill={k} stroke={k} />))}
 </Svg>
);
};

```

```

 <Node key={i} x={pos[i][0]} y={pos[i][1]} label={label[i]} />
)}}
 </Svg>
);
};

```

```

export const StackDemo: React.FC = () => {
 const states = [["A"], ["A", "B"], ["A", "B", "C"], ["A", "B", "C", "D"]];
 const step = useLoop(states.length, 800);
 const items = states[step];
 const popping = step >= 3;
 return (
 <Svg caption={popping ? "pop: забираем ВЕРХНИЮЮ тарелку" : ""}>
 <rect x={90} y={22} width={80} height={96} rx={6} />
 {items.map((v, i) => (
 <g key={v} style={{ transition: "all .3s" }}>
 <rect x={94} y={100 - i * 26} width={72} height={26} />
 <rect x={94} y={100 - i * 26} width={72} height={26} fill={i === items.length - 1 ? (popping ? "red" : "blue")} />
 <text x={130} y={111 - i * 26} textAnchor="middle">{v}</text>
 </g>
))}
 <text x={130} y={16} textAnchor="middle" fontSize={14}>{popping ? "pop" : "push"}</text>
 </Svg>
);
};

```

```

export const QueueDemo: React.FC = () => {
 const states = [["A", "B"], ["A", "B", "C"], ["B", "C", "D"], ["C", "D", "E"]];
 const step = useLoop(states.length, 800);
 const items = states[step];
 return (
 <Svg caption="pop_front слева, push_back справа – к задней тарелке" />
 <defs>
 <marker id="mdArrow" markerWidth="6" markerHeight="6" />
 <path d="M0,0 L6,3 L0,6 Z" fill={C.done} />
 </defs>
 <rect x={90} y={22} width={80} height={96} rx={6} />
 {items.map((v, i) => (
 <g key={v} style={{ transition: "all .3s" }}>
 <rect x={94} y={100 - i * 26} width={72} height={26} />
 <rect x={94} y={100 - i * 26} width={72} height={26} fill={i === items.length - 1 ? "red" : "blue"} />
 <text x={130} y={111 - i * 26} textAnchor="middle">{v}</text>
 </g>
))}
 <text x={130} y={16} textAnchor="middle" fontSize={14}>{popping ? "pop" : "push"}</text>
 </Svg>
);
};

```

```

 { id: 4, x: 102, y: 104, ch: "с", parent: 1 },
 { id: 5, x: 188, y: 104, ch: "с", parent: 2 },
];

export const TrieDemo: React.FC = () => {
 const step = useLoop(TRIE_NODES.length + 1, 700);
 return (
 <Svg caption="Путь от корня по буквам = слово; общи
 {TRIE_NODES.filter((n) => n.parent !== null).map(
 const p = TRIE_NODES[n.parent as number];
 return <Edge key={n.id} a={[p.x, p.y]} b={[n.x,
]}]
 {TRIE_NODES.map((n) => (<Node key={n.id} x={n.x}
 </Svg>
);
};

export const TrieBfsDemo: React.FC = () => {
 const levels = [[0], [1, 2], [3, 4, 5]];
 const step = useLoop(levels.length + 1, 950);
 const doneIds = new Set(levels.slice(0, step).flat());
 const wave = step > 0 && step <= levels.length ? leve
 return (
 <Svg caption={`BFS по бору: уровень ${Math.max(0, M
 {TRIE_NODES.filter((n) => n.parent !== null).map(
 const p = TRIE_NODES[n.parent as number];
 return <Edge key={n.id} a={[p.x, p.y]} b={[n.x,
]}]
 {TRIE_NODES.map((n) => (
 <Node key={n.id} x={n.x} y={n.y} label={n.ch}
 fill={wave.includes(n.id) ? C.active : doneId
)})
 <text x={6} y={12} fontSize={9} fill={C.dim}>очер
 </Svg>
);
};

export const SuffixLinkDemo: React.FC = () => {

```

```

 <rect x={n.x - n.w / 2} y={n.y - 10} width={n.w} height={n.h}
 fill={n.lvl === step ? C.active : C.idle} stroke={C.idleStroke} style={{ transition: 'fill 0.2s' }}
 <text x={n.x} y={n.y} textAnchor="middle" domClass={n.domClass}>{n.val}</text>
 </g>
)}}
 </Svg>
);
};

```

```

export const PrefixSumDemo: React.FC = () => {
 const a = [3, 1, 4, 1, 5, 9];
 const pref = a.reduce<number[]>((acc, v) => [...acc, acc.length + v], []);
 const step = useLoop(4, 1200);
 const l = 1, r = 3;
 const show = step >= 1;
 return (
 <Svg caption={show ? `a[${l}..${r}] = P[${r + 1}] -` : `о префиксные суммы P`}
 <text x={4} y={18} fontSize={8} fill={C.dim}>a</text>
 {a.map((v, i) => (
 <g key={i}>
 <rect x={20 + i * 39} y={24} width={34} height={12}
 fill={show && i >= l && i <= r ? C.active : C.idle} stroke={C.idleStroke} style={{ transition: 'fill 0.2s' }}
 <text x={37 + i * 39} y={36} textAnchor="middle" domClass={C.dim}>{v}</text>
 </g>
))}
 <text x={4} y={72} fontSize={8} fill={C.dim}>P</text>
 {pref.map((v, i) => (
 <g key={i}>
 <rect x={20 + i * 33} y={78} width={29} height={12}
 fill={show && i === l ? C.bad : show && i === r ? C.good : C.idle}
 stroke={C.idleStroke} style={{ transition: 'fill 0.2s' }}
 <text x={34 + i * 33} y={90} textAnchor="middle" domClass={C.dim}>{v}</text>
 </g>
))}
 </Svg>
);
};

```



```
};
```

```
export const NpDemo: React.FC = () => {
 const step = useLoop(2, 1400);
 return (
 <Svg caption={step === 0 ? "Найти решение – долго ("}
 <ellipse cx={130} cy={64} rx={112} ry={48} fill="
 <ellipse cx={92} cy={64} rx={52} ry={32} fill="#3
 <text x={92} y={64} textAnchor="middle" dominantB
 <text x={206} y={34} textAnchor="middle" fontSize
 <circle cx={196} cy={84} r={13} fill={step === 1
 <text x={196} y={84} textAnchor="middle" dominant
 ?"}</text>
 <text x={130} y={124} textAnchor="middle" fontSize
 </Svg>
);
};
```

```
export const PrefixFunctionDemo: React.FC = () => {
 const s = "abacaba";
 const pi = [0, 0, 1, 0, 1, 2, 3];
 const step = useLoop(s.length + 1, 700);
 const i = Math.max(0, step - 1);
 const len = step > 0 ? pi[i] : 0;
 return (
 <Svg caption={step > 0 ? `π[${i}] = ${len}: префикс
 {s.split("").map((ch, idx) => (
 <g key={idx}>
 <rect x={18 + idx * 33} y={26} width={28} hei
 fill={step > 0 && (idx < len || (idx > i -
 style={{ transition: "fill .25s" }} />
 <text x={32 + idx * 33} y={40} textAnchor="mi
 ext>
 <text x={32 + idx * 33} y={70} textAnchor="mi
 fill={idx <= i && step > 0 ? C.warn : C.idle
 </g>
)}}
 <text x={4} y={40} fontSize={8} fill={C.dim}>s</t
```

ФАЙЛ 51/111: src/components/hints/MiniDemo.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

```
import React, { useState } from "react";
import { DemoPauseContext } from "../demoKit";
import { ArticulationDemo, BfsDemo, BridgeDemo, DfsDemo,
import {
 AmortizedDemo, DsuDemo, HeapDemo, NpDemo, PrefixSumDemo,
 SparseTableDemo, StackDemo, SuffixLinkDemo, TrieBfsDemo,
} from "../demosStruct";
import {
 ComponentsDemo, CoordCompressDemo, FloydDemo, GraphBfsDemo,
 ZigDemo, ZigZagDemo, ZigZigDemo,
} from "../demosExtra";

export type DemoKind =
 | "bfs" | "dfs" | "heap" | "stack" | "queue" | "dsu"
 | "suffix-link" | "toposort" | "relax" | "prefix-sum"
 | "segment-tree" | "bridge" | "articulation" | "mst"
 | "prefix-function" | "dp" | "floyd" | "components" |
 | "zig" | "zig-zig" | "zig-zag";

const REGISTRY: Record<DemoKind, React.FC> = {
 bfs: BfsDemo,
 dfs: DfsDemo,
 heap: HeapDemo,
 stack: StackDemo,
 queue: QueueDemo,
 dsu: DsuDemo,
 trie: TrieDemo,
 "trie-bfs": TrieBfsDemo,
 "suffix-link": SuffixLinkDemo,
 toposort: ToposortDemo,
 relax: RelaxDemo,
 "prefix-sum": PrefixSumDemo,
 "sparse-table": SparseTableDemo,
 "segment-tree": SegmentTreeDemo,
```

ФАЙЛ 52/111: src/components/hints/SimulatorModal.tsx  
КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 10

```
import React, { useEffect } from "react";
import { createPortal } from "react-dom";
import { X } from "lucide-react";
import { getViz } from "../vizRegistry";
import { VizChapterContext } from "../../data/vizStepBuilder";
import { VizLiveStatus } from "../VizLiveStatus";
```

```
interface Props {
 vizId: string | null;
 onClose: () => void;
}
```

```
/** Модальное окно с полноценным симулятором – вызывает
export const SimulatorModal: React.FC<Props> = ({ vizId, onClose }) => {
 useEffect(() => {
 if (!vizId) return;
 const onKey = (e: KeyboardEvent) => {
 if (e.key === "Escape") onClose();
 };
 window.addEventListener("keydown", onKey);
 document.body.style.overflow = "hidden";
 return () => {
 window.removeEventListener("keydown", onKey);
 document.body.style.overflow = "";
 };
 }, [vizId, onClose]);

 const viz = getViz(vizId ?? undefined);
 if (!vizId || !viz) return null;
 const Component = viz.Component;

 return createPortal(
 <div className="fixed inset-0 z-[110] flex items-end
```

```

import { GLOSSARY_BY_ID } from "../../data/glossary";
import { MiniDemo } from "../MiniDemo";
import { getViz } from "../vizRegistry";

export interface HintAnchor {
 termId: string;
 rect: DOMRect;
}

interface Props {
 anchor: HintAnchor | null;
 onClose: () => void;
 onOpenSimulator: (vizId: string) => void;
 onCardEnter: () => void;
 onCardLeave: () => void;
}

const CARD_W = 320;
const GAP = 10;

export const TermPopover: React.FC<Props> = ({ anchor,
 const cardRef = useRef<HTMLDivElement>(null);
 const [pos, setPos] = useState<{ top: number; left: number}>({
 top: 0, left: 0,

 useLayoutEffect(() => {
 if (!anchor) {
 setPos(null);
 return;
 }
 const vw = window.innerWidth;
 const vh = window.innerHeight;
 const width = Math.min(CARD_W, vw - 16);
 const height = cardRef.current?.offsetHeight ?? 300;

 let left = anchor.rect.left + anchor.rect.width / 2;
 left = Math.max(8, Math.min(left, vw - width - 8));

 const below = anchor.rect.top < height + GAP + 8;

```

```

<div className="bg-slate-900 border border-indigo-500" >
 <div className="flex items-start gap-2 px-3 pt-3" >
 <h4 className="text-sm font-bold text-indigo-500" >
 <button
 onClick={onClose}
 className="text-slate-500 hover:text-white"
 aria-label="Заккрыть подсказку"
 >
 <X className="w-4 h-4" />
 </button>
 </div>

 <div className="p-3 space-y-2.5 max-h-[60vh] overflow-y-auto" >
 <p className="text-[13px] leading-relaxed text-slate-300" >
 {entry.demo && <MiniDemo kind={entry.demo} />}

 {entry.formal && (
 <p className="text-[11.5px] leading-relaxed text-slate-300" >
 {entry.formal}
 </p>
)}

 {entry.complexity && (
 <div className="text-[11px] font-mono text-slate-300" >
 Сложность: {entry.complexity}
 </div>
)}

 {viz && simulatorId && (
 <button
 onClick={() => onOpenSimulator(simulatorId)}
 className="w-full flex items-center justify-center text-slate-300 rounded-lg py-2 transition-colors"
 >
 <Play className="w-3.5 h-3.5" /> Показать
 </button>
)}
 </div>
 </div>
 </div>

```

```

 { id: 'CA', u: 'C', v: 'A', weight: 2, newWeight: 1 };

```

```

const isNodeVisible = (id: string) => {
 if (id === 'S' && (step === 0 || step === 7)) return true;
 return false;
};

```

```

const getPotential = (id: string) => {
 if (step >= 2 && step < 7) {
 if (id === 'S') return 0;
 if (id === 'A') return 0;
 if (id === 'B') return -2;
 if (id === 'C') return -1;
 }
 return null;
};

```

```

const getEdgeColorClass = (e: any) => {
 if (e.u === 'S') {
 if (step === 0 || step === 7) return "opacity-0";
 if (step === 1) return "stroke-amber-500 stroke-dasharray-4";
 if (step === 2) return "stroke-pink-500 stroke-dasharray-4";
 return "stroke-slate-700 stroke-dasharray-4";
 }

 if (e.id === 'AB') {
 if (step === 4) return "stroke-amber-400 stroke-dasharray-4";
 if (step > 4) return "stroke-emerald-500";
 }
 if (e.id === 'BC') {
 if (step === 5) return "stroke-amber-400 stroke-dasharray-4";
 if (step > 5) return "stroke-emerald-500";
 }
 if (e.id === 'CA') {
 if (step === 6) return "stroke-amber-400 stroke-dasharray-4";
 if (step > 6) return "stroke-emerald-500";
 }
};

```

```

 if (step < 6) return e.weight;
 if (step === 6) return `${e.weight} + (${-1
 return e.newWeight;
 }
 return e.weight;
};

const isEdgeTextHighlighted = (e: any) => {
 if (e.id === 'AB' && step === 4) return true;
 if (e.id === 'BC' && step === 5) return true;
 if (e.id === 'CA' && step === 6) return true;
 return false;
};

const isEdgeTextEmerald = (e: any) => {
 if (e.id === 'AB' && step > 4) return true;
 if (e.id === 'BC' && step > 5) return true;
 if (e.id === 'CA' && step > 6) return true;
 return false;
};

const getDesc = () => {
 switch (step) {
 case 0: return (
 <div>
 Исходный граф. Имеется ребро
 Если мы запустим быстрый алгоритм Д
 </div>
);
 case 1: return (
 <div>
 Шаг 1: Точка отсчета.

 Добавляем фиктивную вершину S. Соед
 ми.
 Это наша база для расчета потенциал
 </div>
);
 case 2: return (

```

```

 Готово! Фиктивную вершину S
 Все новые веса ребер в графе стали
 При этом старые кратчайшие пути оста
шин!
 </div>
);
 default: return "";
}
};

return (
 <div className="bg-slate-800 p-4 rounded-xl border
 <h4 className="text-sm md:text-base font-bo
 <span className="w-2.5 h-2.5 bg-amber-4
 Визуализация: Перевзвешивание алгоритмо
 </h4>

 <div className="flex flex-col md:flex-row g
 <div className="bg-[#0f172a] flex-1 min
 nter p-4">
 <svg width="400" height="300" class
 <defs>
 <marker id="arrow-slate" vie
 start-reverse">
 <path d="M 0 0 L 10 5 L
 </marker>
 <marker id="arrow-slate-dim
 uto-start-reverse">
 <path d="M 0 0 L 10 5 L
 </marker>
 <marker id="arrow-amber" vie
 start-reverse">
 <path d="M 0 0 L 10 5 L
 </marker>
 <marker id="arrow-pink" vie
 tart-reverse">
 <path d="M 0 0 L 10 5 L
 </marker>

```



```

 {getEdgeContent
 </text>
 </g>
)
 }}}

 {/* Nodes */}
 {nodes.map(n => {
 if (!isNodeVisible(n.id)) r
 const pot = getPotential(n.)

 return (
 <g key={n.id} className=
 <circle
 cx={n.x} cy={n.y}
 className={`stroke-
 ${n.id === 'S'
 ? 'fill-[#202020]
 : 'fill-slate-500'
 }
 />
 <text x={n.x} y={n.y}
 -white">
 {n.label}
 </text>

 {pot !== null && (
 <g transform={`
fade-in zoom-in duration-500">
 <rect x="-100" y="-100"
k-500 stroke-1" />
 <text x="0" y="0"
font-bold fill-pink-400">
 {pot}
 </text>
 </g>
)}
 </g>
)
)
 })
 }
)

```

```
 }}}
 </div>
 </div>
 </div>
</div>
);
}
```

---

ФАЙЛ 55/111: src/components/KosarajuViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

---

```
import React, { useState, useEffect } from "react";
import { useVizStepSync, vizArray, vizNumber } from "...";
import {
 Play,
 Pause,
 RotateCcw,
 Layers,
} from "lucide-react";

interface GNode {
 id: number;
 x: number;
 y: number;
 label: string;
}

interface GEdge {
 u: number;
 v: number;
}

const initialNodes: GNode[] = [
 { id: 0, x: 150, y: 100, label: "0" },
 { id: 1, x: 300, y: 100, label: "1" },
```

```

const [isPlaying, setIsPlaying] = useState(false);

useEffect(() => {
 // Generate step-by-step Kosaraju steps
 const generatedSteps: AlgoStep[] = [];
 const visited = new Set<number>();
 const order: number[] = [];
 const sccMap: Record<number, number> = {};

 const recordStep = (
 phase: AlgoStep["phase"],
 desc: string,
 currentNode: number | null,
 transposed: boolean,
 currentSccId: number,
) => {
 generatedSteps.push({
 phase,
 desc,
 visited: new Set(visited),
 currentNode,
 order: [...order],
 transposed,
 sccMap: { ...sccMap },
 currentSccId,
 activeEdges: transposed
 ? initialEdges.map((e) => ({ u: e.v, v: e.u })
 : [...initialEdges],
));
 };

 recordStep(
 "init",
 "Инициализация алгоритма Косарайю. Начинаем первый",
 null,
 false,
 -1,
);

```

```

const topoOrder = [...order].reverse();
recordStep(
 "transpose",
 `Первый проход завершен! Стек выхода в топологиче
 ебра.`,
 null,
 true,
 -1,
);

// Transpose adj
const adjT: number[][] = Array.from({ length: 5 },
initialEdges.forEach((e) => adjT[e.v].push(e.u));

// Reset visited for phase 2
visited.clear();
let sccId = 0;

const dfs2 = (u: number, currentScc: number) => {
 visited.add(u);
 sccMap[u] = currentScc;
 recordStep(
 "dfs2",
 `DFS 2: заходим в ${u} на транспонированном гра
 u,
 true,
 currentScc,
);

 for (const to of adjT[u]) {
 if (!visited.has(to)) {
 dfs2(to, currentScc);
 }
 }
};

// Run DFS2 using the order (reversed from back to

```

```

 visited: new Set(),
 currentNode: null,
 order: [],
 transposed: false,
 sccMap: {},
 currentSccId: -1,
 activeEdges: initialEdges,
 });

 useEffect(() => {
 let timer: NodeJS.Timeout;
 if (isPlaying && currentStepIdx < steps.length - 1) {
 timer = setInterval(() => {
 setCurrentStepIdx((prev) => prev + 1);
 }, 1500);
 } else {
 setIsPlaying(false);
 }
 return () => clearInterval(timer);
 }, [isPlaying, currentStepIdx, steps.length]);

 const togglePlay = () => setIsPlaying(!isPlaying);
 const reset = () => {
 setIsPlaying(false);
 setCurrentStepIdx(0);
 };

 const getSccColor = (id: number) => {
 if (id === 1) return "fill-emerald-600 stroke-emera
 if (id === 2) return "fill-indigo-600 stroke-indigo
 return "fill-slate-800 stroke-slate-500";
 };

 return (
 <div className="bg-slate-900 rounded-xl border bord
 <div className="bg-slate-800 p-4 border-b border-
 <h3 className="font-bold text-white flex items-
 <Layers className="w-5 h-5 text-emerald-400"

```

```

 refX="22"
 refY="5"
 markerWidth="6"
 markerHeight="6"
 orient="auto-start-reverse"
 >
 <path d="M 0 1 L 10 5 L 0 9 z" fill="#6
</marker>
<marker
 id="arrow-active"
 viewBox="0 0 10 10"
 refX="22"
 refY="5"
 markerWidth="6"
 markerHeight="6"
 orient="auto-start-reverse"
>
 <path d="M 0 1 L 10 5 L 0 9 z" fill="#1
</marker>
<marker
 id="arrow-transposed"
 viewBox="0 0 10 10"
 refX="22"
 refY="5"
 markerWidth="6"
 markerHeight="6"
 orient="auto-start-reverse"
>
 <path d="M 0 1 L 10 5 L 0 9 z" fill="#6
</marker>
</defs>

```

```

{ /* Edges */ }
{initialEdges.map((edge, idx) => {
 const u = initialNodes.find((n) => n.id === edge.u)
 const v = initialNodes.find((n) => n.id === edge.v)

 // If transposed, draw reversed coordinates

```

```

 stroke={stroke}
 strokeWidth="2.5"
 markerEnd={`url(#${markerId})`}
 className="transition-all duration-300"
 />
);
 }}}

{/* Nodes */}
{initialNodes.map((node) => {
 const isCurrent = step.currentNode === node.id;
 const isVisited =
 step.visited.has(node.id) ||
 (step.phase === "dfs1" && step.currentNode === node.id);
 const sccId = step.sccMap[node.id];

 let classes = "fill-slate-800 stroke-slate-800";
 if (sccId) {
 classes = getSccColor(sccId);
 } else if (isCurrent) {
 classes = "fill-amber-600 stroke-amber-600";
 } else if (isVisited && step.phase === "dfs2") {
 classes = "fill-emerald-700 stroke-emerald-700";
 }

 return (
 <g key={node.id}>
 <circle
 cx={node.x}
 cy={node.y}
 r="20"
 className={classes}
 strokeWidth="3"
 />
 <text
 x={node.x}
 y={node.y}
 dy=".3em"

```

#### Порядок обхода и стек

</h4>

```
<div className="bg-slate-900 border border-slate-800" style={step.desc}
 <p className="text-xs text-white leading-relaxed" style={step.desc}
 {step.desc}
 </p>
</div>
```

```
<div className="flex-1 space-y-4 overflow-y-auto" style={{
 /* Top-Sort output / Order stack representation */
 <div>
 <span className="text-[10px] text-slate-500" style={{
 CТЕК ВЫХОДА (DFS 1 ORDER):

 <div className="flex flex-wrap gap-1 bg-slate-900" style={{
 {step.order.length === 0 ? (
 <span className="text-slate-600 text-sm" style={{
) : (
 [...step.order].reverse().map(({val, idx}) => {
 <div
 key={idx}
 className="bg-indigo-950 border border-indigo-800" style={{
 ex items-center gap-1"
 >
 {val}
 </div>
 })
)}
 </div>
 </div>
```

```
{/* List of actions log */}
```

```
<div>
 <span className="text-[10px] text-slate-500" style={{
 ЛОГ ОПЕРАЦИЙ:

 <div className="space-y-1.5 max-h-[150px]" style={{
```



```
 </div>
 </div>
);
 };
};
```

---

ФАЙЛ 56/111: src/components/KruskalSimulator.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

---

```
import React, { useCallback, useContext, useEffect, useId, useLayoutEffect, useReducer, useState } from 'react';
import { Play, RotateCcw, SkipForward, HelpCircle, Check } from 'lucide-react';
import { VizChapterContext, emitVizDemo, useVizRuntime, useVizStepBus } from '../data/vizStepBus';
```

```
interface Vertex {
 id: number;
 label: string;
 x: number;
 y: number;
 color: string; // 'none', 'red', 'blue', 'purple', 'magenta'
}
```

```
interface Edge {
 id: string;
 u: number;
 v: number;
 weight: number;
 status: 'pending' | 'inspecting' | 'included' | 'rejected';
 color: string;
}
```

```
interface StepLog {
 title: string;
 description: string;
 type: 'info' | 'success' | 'warning' | 'error';
}
```

```

const [logs, setLogs] = useState<StepLog[]>([
 {
 title: 'Введение в раскраску (Краскал)',
 description: '«Краскал – это раскраска?». Предста
 овы начинать!',
 type: 'info',
 },
]);

// --- KRUSKAL STEPS ---
const kruskalSteps = [
 // Step 1: Inspect A-B (2)
 () => {
 setEdges(prev => prev.map(e => e.id === '0-1' ? {
 setLogs(prev => [{
 title: 'Шаг 1: Берем самое дешевое ребро A-B (в
 description: 'Оно соединяет две бесцветные верш
 type: 'info'
 }, ...prev]));
 },
 // Step 2: Include A-B
 () => {
 setVertices(prev => prev.map(v => v.id === 0 || v
 setEdges(prev => prev.map(e => e.id === '0-1' ? {
 setLogs(prev => [{
 title: 'Успех: Ребро A-B в осто
 description: 'Вершины A and B окрашены в красны
 type: 'success'
 }, ...prev]));
 },
 // Step 3: Inspect D-E (3)
 () => {
 setEdges(prev => prev.map(e => e.id === '3-4' ? {
 setLogs(prev => [{
 title: 'Шаг 2: Берем второе ребро D-E (вес 3)',
 description: 'Оно соединяет две другие бесцветн
 type: 'info'
 }, ...prev]));
 },

```

```

 type: 'warning'
 }, ...prev]);
},
// Step 8: Include A-D (Merge to purple)
() => {
 setVertices(prev => prev.map(v => v.color === 'blue' ? 'purple' : v.color));
 setEdges(prev => prev.map(e => e.status === 'included' ? 'excluded' : e.status));
 setLogs(prev => [{
 title: 'Слияние кланов: Ребро A-D добавлено',
 description: 'Мы объединили красный и синий кланы',
 type: 'success'
 }, ...prev]);
},
// Step 9: Inspect B-E (6) - Cycle!
() => {
 setEdges(prev => prev.map(e => e.id === '1-4' ? {status: 'excluded'} : e));
 setLogs(prev => [{
 title: 'Шаг 5: Берем ребро B-E (вес 6)',
 description: 'НО: это ребро соединяет вершины B и E, которые уже соединены!',
 type: 'warning'
 }, ...prev]);
},
// Step 10: Reject B-E
() => {
 setEdges(prev => prev.map(e => e.id === '1-4' ? {status: 'excluded'} : e));
 setLogs(prev => [{
 title: 'Отказ: Цикл обнаружен!',
 description: 'Мы выбрасываем ребро B-E. Иначе получится цикл!',
 type: 'error'
 }, ...prev]);
},
// Step 11: Inspect E-F (7)
() => {
 setEdges(prev => prev.map(e => e.id === '4-5' ? {status: 'excluded'} : e));
 setLogs(prev => [{
 title: 'Шаг 6: Берем ребро E-F (вес 7)',
 description: 'Оно соединяет фиолетовую вершину E и синюю вершину F',
 type: 'info'
 }, ...prev]);
},

```

```

 }, ...prev]));
 },
 // Step 16: Reject C-F
 () => {
 setEdges(prev => prev.map(e => e.id === '2-5' ? {
 setLogs(prev => [{
 title: 'Отказ: Цикл обнаружен!',
 description: 'Выбрасываем ребро C-F.',
 type: 'error'
 }, ...prev]));
 },
 // Step 17: Inspect G-H (10)
 () => {
 setEdges(prev => prev.map(e => e.id === '6-7' ? {
 setLogs(prev => [{
 title: 'Шаг 9: Берем ребро G-H (вес 10)',
 description: 'Соединяет фиолетовую G и бесцветную H.',
 type: 'info'
 }, ...prev]));
 },
 // Step 18: Include G-H
 () => {
 setVertices(prev => prev.map(v => v.id === 7 ? {
 setEdges(prev => prev.map(e => e.id === '6-7' ? {
 setLogs(prev => [{
 title: 'Успех: Ребро G-H в остове',
 description: 'Вершина H перекрашена в фиолетовую.',
 type: 'success'
 }, ...prev]));
 },
 // Step 19: Inspect E-H (11) - Cycle!
 () => {
 setEdges(prev => prev.map(e => e.id === '4-7' ? {
 setLogs(prev => [{
 title: 'Шаг 10: Берем ребро E-H (вес 11)',
 description: 'Оба конца E и H уже фиолетовые. Соединение создаст цикл!',
 type: 'warning'
 }, ...prev]));
 },

```

```

 setEdges(prev => prev.map(e => ['3-4', '1-4', '4-5']));
 setLogs(prev => [{
 title: 'Шаг 1: Плесень зарождается в Токио (Вершина E)',
 description: 'Мы выбрали вершину E как стартовую вершину. Добавляем в очередь (Heap): D-E(3), B-E(6), E-F(7)',
 type: 'info'
 }, ...prev]);
 },
 // Step 2: Pop D-E (3)
 () => {
 setEdges(prev => prev.map(e => e.id === '3-4' ? {
 id: '3-4', status: 'in_heap'
 } : e));
 setLogs(prev => [{
 title: 'Шаг 2: Куча выдает самое дешевое ребро D-E (3)',
 description: 'Плесень тянется по ребру D-E к вершине D',
 type: 'info'
 }, ...prev]);
 },
 // Step 3: Include D-E, add D's edges to heap
 () => {
 setVertices(prev => prev.map(v => v.id === 3 ? {
 id: 3, status: 'in_heap'
 } : v));
 setEdges(prev => prev.map(e => e.id === '3-4' ? {
 id: '3-4', status: 'in_heap'
 } : e));
 setHeapQueue(['A-D (5)', 'B-E (6)', 'E-F (7)', 'D-E (3)']);
 setLogs(prev => [{
 title: 'Успех: Плесень захватила вершину D',
 description: 'В кучу добавлены новые пути из D: A-D (5), B-E (6), E-F (7)',
 type: 'success'
 }, ...prev]);
 },
 // Step 4: Pop A-D (5)
 () => {
 setEdges(prev => prev.map(e => e.id === '0-3' ? {
 id: '0-3', status: 'in_heap'
 } : e));
 setLogs(prev => [{
 title: 'Шаг 3: Куча выдает ребро A-D (вес 5)',
 description: 'Плесень расширяется в сторону вершины A',
 type: 'info'
 }, ...prev]);
 },

```

```

 description: 'Плесень тянется к вершине C.',
 type: 'info'
 }, ...prev]);
},
// Step 9: Include B-C, add C's edges
() => {
 setVertices(prev => prev.map(v => v.id === 2 ? {
 setEdges(prev => prev.map(e => e.id === '1-2' ? {
 eap' } : e));
 setHeapQueue(['B-E (6 - цикл)', 'E-F (7)', 'D-G (8)']);
 setLogs(prev => [{
 title: 'Успех: Вершина C захвачена',
 description: 'Ребро C-F(9) добавлено в кучу.',
 type: 'success'
 }, ...prev]);
},
// Step 10: Pop B-E (6) - Cycle!
() => {
 setEdges(prev => prev.map(e => e.id === '1-4' ? {
 setLogs(prev => [{
 title: 'Шаг 6: Куча выдает ребро B-E (вес 6)',
 description: 'ВНИМАНИЕ! Плесень проверяет вершину E',
 type: 'warning'
 }, ...prev]);
},
// Step 11: Reject B-E
() => {
 setEdges(prev => prev.map(e => e.id === '1-4' ? {
 setHeapQueue(['E-F (7)', 'D-G (8)', 'C-F (9)', 'B-E (6)']);
 setLogs(prev => [{
 title: 'Отказ: Игнорируем внутреннее ребро',
 description: 'Ребро B-E отброшено. Плесень не р
 type: 'error'
 }, ...prev]);
},
// Step 12: Pop E-F (7)
() => {
 setEdges(prev => prev.map(e => e.id === '4-5' ? {

```

```

 },
 // Step 16: Pop C-F (9) - Cycle!
 () => {
 setEdges(prev => prev.map(e => e.id === '2-5' ? {
 setLogs(prev => [{
 title: 'Шаг 9: Куча выдает ребро C-F (вес 9)',
 description: 'Вершины C и F уже заражены плесенью',
 type: 'warning'
 }, ...prev]));
 },
 // Step 17: Reject C-F
 () => {
 setEdges(prev => prev.map(e => e.id === '2-5' ? {
 setHeapQueue(['G-H (10)', 'E-H (11)', 'F-I (13)'])
 setLogs(prev => [{
 title: 'Отказ: Игнорируем ребро C-F',
 description: 'Выбрасываем из кучи.',
 type: 'error'
 }, ...prev]));
 },
 // Step 18: Pop G-H (10)
 () => {
 setEdges(prev => prev.map(e => e.id === '6-7' ? {
 setLogs(prev => [{
 title: 'Шаг 10: Куча выдает ребро G-H (вес 10)',
 description: 'Плесень ползет к вершине H.',
 type: 'info'
 }, ...prev]));
 },
 // Step 19: Include G-H, add H's edges
 () => {
 setVertices(prev => prev.map(v => v.id === 7 ? {
 setEdges(prev => prev.map(e => e.id === '6-7' ? {
 eap' } : e));
 setHeapQueue(['E-H (11 - цикл)', 'H-I (12)', 'F-I (13)'])
 setLogs(prev => [{
 title: 'Успех: Вершина H захвачена',
 description: 'В кучу добавлено H-I(12).',
 }

```

```

 description: 'Все 9 вершин поглощены. Итоговый
 иной — мы разрастались из одной точки с помо
 type: 'success'
 }, ...prev]);
},
];

// --- BORUVKA STEPS ---
const boruvkaSteps = [
 // Step 1: Five-Year Plan 1 (Inspecting outgoing)
 () => {
 // Highlight the cheapest outgoing edges for EACH
 // Node 0(A) -> A-B(2)
 // Node 1(B) -> A-B(2)
 // Node 2(C) -> B-C(4)
 // Node 3(D) -> D-E(3)
 // Node 4(E) -> D-E(3)
 // Node 5(F) -> E-F(7)
 // Node 6(G) -> D-G(8)
 // Node 7(H) -> G-H(10)
 // Node 8(I) -> H-I(12)
 setEdges(prev => prev.map(e =>
 ['0-1', '3-4', '1-2', '4-5', '3-6', '6-7', '7-8
));
 setLogs(prev => [{
 title: 'Первая пятилетка: Каждая деревня выбира
 description: 'Каждая из 9 деревень одновременно
 А выбирает A-B(2), D выбирает D-E(3), а G вы
 !',
 type: 'info'
 }, ...prev]);
 },
 // Step 2: Concurrently merge into Kolkhozes (Compo
 () => {
 // Concurrently build chosen roads. Merge into:
 // Component 1: {A, B, C} (colored red)
 // Component 2: {D, E, F, G, H, I} (colored gold/
 setVertices(prev => prev.map(v =>

```



```

 setLogs(prev => [{
 title: 'Объединение завершено: Один гигантский ',
 description: 'Оба колхоза объединились по мосту',
 type: 'success'
 }, ...prev]));
 },
 // Step 5: Check remaining edges & mark as rejected
 () => {
 setEdges(prev => prev.map(e => e.status === 'pending' ? 'rejected' : e.status));
 setLogs(prev => [{
 title: 'Успех: MST построен Борувкой за 2 шага',
 description: 'Все лишние ребра (В-Е, Е-Н, С-Ф, ...), удалены. Вес: 51. Благодаря параллельности, мы потратили 2 секунды.',
 type: 'success'
 }, ...prev]));
 }
];

const currentSteps =
 activeAlgo === 'kruskal' ? kruskalSteps :
 activeAlgo === 'prim' ? primSteps :
 boruvkaSteps;

const handleNextStep = () => {
 if (stepIndex < currentSteps.length) {
 currentSteps[stepIndex]();
 setStepIndex(stepIndex + 1);
 }
};

const introLog = (algo: 'kruskal' | 'prim' | 'boruvka') => {
 if (algo === 'kruskal') {
 return [{
 title: 'Введение в раскраску (Краскал)',
 description: 'Представь, что все 9 вершин графа соединены по кругу. Мы хотим построить минимальное остовное дерево (MST).',
 type: 'info',
 }];
 }
};

```

```

 setVertices(initialVertices);
 setEdges(initialEdges);
 setHeapQueue([]);
 setLogs(introLog(activeAlgo));
 for (let i = 0; i < target; i++) currentSteps[i]();
 setStepIndex(target);
 // eslint-disable-next-line react-hooks/exhaustive-deps
 }, [activeAlgo, currentSteps]);

```

```

/** Номер кадра из переменных кода: i / step / число
const selectByVariables = useCallback<VizStepSelector>(
 const i = vizNumber(v.i) ?? vizNumber(v.step);
 if (i !== null) return i + 1;
 const mst = vizArray(v.mst) ?? vizArray(v.answer) ?? vizArray(v.mst);
 if (mst) return mst.length;
 const taken = vizArray(v.taken) ?? vizArray(v.used);
 if (taken) return Math.max(0, taken.length - 1);
 return null;
}, []);

```

```

useVizStepSync(stepIndex, goToStep, currentSteps.length);

```

```

/** A..I ↔ id 0..8 */
const idOf = (name: string | number | null | undefined): number | null {
 if (name === null || name === undefined) return null;
 if (typeof name === 'number') return Number.isInteger(name) ? name : null;
 const c = name.trim().toUpperCase().replace(/[^A-I]/g, '');
 return c ? c.charCodeAt(0) - 65 : null;
};

```

```

/** Разбор одного ребра из кода: (2, 'A', 'B') / [2, 'A', 'B']
const parseEdge = (item: unknown): { u: number; v: number; label: string } | null {
 if (item === null || item === undefined) return null;
 if (typeof item === 'string') {
 const nums = item.match(/-?\d+(?:\.\d+)?/g);
 const letters = item.match(/[A-Ia-i]/g);
 let u: number | null = null;
 let v: number | null = null;
 let label = '';
 for (let i = 0; i < nums.length; i++) {
 const num = parseInt(nums[i], 10);
 if (num < 0) {
 label = '-';
 } else {
 label = '';
 }
 if (i > 0) label += ',';
 u = num;
 }
 for (let i = 0; i < letters.length; i++) {
 const letter = letters[i];
 if (i > 0) label += ',';
 v = letter.charCodeAt(0) - 65;
 }
 return { u, v, label };
 }
 if (typeof item === 'object') {
 const { u, v, label } = item;
 if (u === null || v === null || label === null || label === '') return null;
 return { u, v, label };
 }
 return null;
};

```

```

 /** То же, но только если метка – вершина графа: по
 const parentIdx = new Map<number, number>();
 const feed = (id: number, val: unknown) => {
 const label = String(typeof val === 'string' ? va
 rawComp.set(id, label);
 const inner = typeof val === 'string' ? val : typ
 const pid = idOf(inner);
 if (pid !== null) parentIdx.set(id, pid);
 };
 if (parentRec) {
 Object.entries(parentRec).forEach(([k, val]) => {
 const id = idOf(k);
 if (id === null) return;
 feed(id, val);
 });
 } else if (parentArr) {
 parentArr.forEach((val, id) => {
 if (id > 8) return;
 feed(id, val);
 });
 }
 /**
 * Ключ множества вершины: если код дал DSU-массив
 * идём по цепочке до корня – иначе вершины одной к
 * разные цвета. Для произвольных меток («comp1») б
 */
 const compOfVertex = new Map<number, string>();
 rawComp.forEach((label, id) => {
 if (parentIdx.size === 0) {
 compOfVertex.set(id, label);
 return;
 }
 let cur = id;
 const seen = new Set<number>([id]);
 for (let hop = 0; hop < 16; hop++) {
 const nxt = parentIdx.get(cur);
 if (nxt === undefined || nxt === cur || seen.has(nxt))
 seen.add(nxt);

```

```

 return { ...e, status: 'pending' as const };
 });
 // eslint-disable-next-line react-hooks/exhaustive-deps
 }, [edges, live]);

const effVertices: Vertex[] = useMemo(() => {
 if (!live) return vertices;
 return vertices.map((v) => {
 const root = live.compOfVertex.get(v.id);
 if (root !== undefined) return { ...v, color: colorOf(root) };
 if (live.takenVertices.includes(v.id)) return { ...v, color: 'none' };
 if (live.curU === v.id || live.curV === v.id) return { ...v, color: 'none' };
 return { ...v, color: 'none' };
 });
 // eslint-disable-next-line react-hooks/exhaustive-deps
}, [vertices, live]);

const getColorClass = (color: string, id: number) => {
 switch (color) {
 case 'red': return 'bg-red-500 border-red-300 text-white';
 case 'blue': return 'bg-blue-500 border-blue-300 text-white';
 case 'purple': return 'bg-purple-600 border-purple-300 text-white';
 case 'mold': return 'bg-emerald-500 border-emerald-300 text-white';
 case 'gold': return 'bg-amber-400 border-amber-200 text-white';
 default:
 if (activeAlgo === 'prim' && id === 4 && stepIn === 1)
 return 'bg-slate-700 border-emerald-500 text-white';
 return 'bg-slate-700 border-slate-500 text-slate-200';
 }
};

const getEdgeStroke = (status: string, color: string) => {
 if (status === 'inspecting') return '#f59e0b';
 if (status === 'rejected') return '#ef4444';
 if (status === 'in_heap') return '#0284c7';
 if (status === 'included') {
 if (color === 'red') return '#ef4444';
 if (color === 'blue') return '#0284c7';
 if (color === 'purple') return '#8b57a2';
 if (color === 'mold') return '#2e8b57';
 if (color === 'gold') return '#d4ac3d';
 }
 return 'black';
};

```

```

 >
 <GitGraph className="w-3.5 h-3.5" />
 Краскал (Билет 18)
 </button>
 <button
 onClick={() => handleReset('prim')}
 className={
 "flex items-center gap-1.5 px-3 py-2
+
 (activeAlgo === 'prim'
 ? 'bg-emerald-600 text-white shadow
 : 'text-slate-400 hover:text-slate-:
)
 >
 <Layers className="w-3.5 h-3.5" />
 Прима (Билет 19)
 </button>
 <button
 onClick={() => handleReset('boruvka')}
 className={
 "flex items-center gap-1.5 px-3 py-2
+
 (activeAlgo === 'boruvka'
 ? 'bg-rose-600 text-white shadow-md
 : 'text-slate-400 hover:text-slate-:
)
 >
 <Users className="w-3.5 h-3.5" />
 Борувка (Билет 20)
 </button>
 </div>

 <button
 onClick={() => handleReset(activeAlgo)}
 className="flex items-center justify-cent
-200 font-semibold text-xs rounded-xl borde
 >
 <RotateCcw className="w-3.5 h-3.5" />

```

```


 в осто́ве: {live.takenEdges.map((e) =>

))
 {live.compOfVertex.size > 0 && (

 компонент: {new Set(Array.from(live.c

))
 {live.takenVertices.length > 0 && (

 дерево вы́росло: {live.takenVertices.m

))
 {live.heapEdges.length > 0 && (

 в куче: {live.heapEdges.length}

))
 {live.cheapestEdges.length > 0 && (

 cheapest: {live.cheapestEdges.length}

))
 {live.total !== null && (

 вес осто́ва = {live.total}

))
</>
) : (

 открой панель Python и запусти код вкладки
 ка'}»: переменные
 {' '}<code className="font-mono text-purp
 e className="font-mono text-purple-300">u</
 mono text-purple-300">w</code> перекрасят э


```



```

 </div>

 {stepIndex >= currentSteps.length && (
 <div className="absolute inset-x-6 bottom-6
 shadow-xl z-20">
 <p className={"font-bold text-base " +
 'ld-300' : 'text-rose-300'}>
 MST построено с помощью {activeAlgo}
 </p>
 <p className="text-slate-300 text-xs mt-1">
 Общий вес минимального остова: 51. Всего
 </p>
 </div>
)}
 </div>

 {/* Logs / Heap */}
 <div className="bg-slate-950 rounded-2xl border-b
 <div className="p-4 bg-slate-900 border-b
 <h4 className="font-bold text-slate-200">
 Ход выполнения
 </h4>
 </div>

 {activeAlgo === 'prim' && (
 <div className="p-3 bg-slate-900/60 border
 <p className="text-[11px] font-bold text-slate-200">
 ✂ Приоритетная очередь (Min-Heap):
 </p>
 <div className="flex flex-wrap gap-1.5">
 {heapQueue.length === 0 ? (

 Пусто

) : (
 heapQueue.map(({item, idx}) => (
 <span key={idx} className={"px-2 py-1
 0 shadow-md animate-pulse" : "bg-slate-800
 {item}

)
)
)
 </div>
)}
)}
)}

```



```

<div className="flex flex-col sm:flex-row items
 <div className="flex items-center gap-2.5">
 <BookOpen className="w-5 h-5 text-indigo-400">
 <h4 className="text-xl font-bold text-white">
 </div>

 <div className="flex bg-slate-950 p-1 rounded
 <button
 onClick={() => setActiveCheatTab('kruskal')}
 className={"px-3 py-1.5 rounded text-xs font-medium"}
 == 'kruskal' ? "bg-indigo-600 text-white" : "bg-slate-950 text-white"
 >
 Краскал (Билет 18)
 </button>
 <button
 onClick={() => setActiveCheatTab('prim')}
 className={"px-3 py-1.5 rounded text-xs font-medium"}
 == 'prim' ? "bg-emerald-600 text-white" : "bg-slate-950 text-white"
 >
 Прима (Билет 19)
 </button>
 <button
 onClick={() => setActiveCheatTab('boruvka')}
 className={"px-3 py-1.5 rounded text-xs font-medium"}
 == 'boruvka' ? "bg-rose-600 text-white" : "bg-slate-950 text-white"
 >
 Борувка (Билет 20)
 </button>
 </div>
</div>

{activeCheatTab === 'kruskal' && (
 <div className="grid grid-cols-1 lg:grid-cols-2">
 <div className="lg:col-span-1 space-y-4">
 <div className="bg-slate-950 p-4 rounded-lg">
 <p className="text-slate-400 text-xs font-medium">
 <Clock className="w-3.5 h-3.5 text-indigo-400">
 </p>

```

```
edges.sort() # O(E log E)
```

```
mst = []
```

```
for w, u, v in edges:
```

```
 if union(u, v):
```

```
 mst.append((u, v, w))`}
```

```
 </pre>
```

```
 </div>
```

```
 </div>
```

```
 </div>
```

```
}}
```

```
{activeCheatTab === 'prim' && {
```

```
 <div className="grid grid-cols-1 lg:grid-cols-
```

```
 <div className="lg:col-span-1 space-y-4">
```

```
 <div className="bg-slate-950 p-4 rounded-
```

```
 <p className="text-slate-400 text-xs for
```

```
 <Clock className="w-3.5 h-3.5 text-em
```

```
 </p>
```

```
 <p className="text-2xl font-black text-v
```

```
 <p className="text-slate-400 text-xs mt
```

```
 </div>
```

```
 <div className="bg-slate-950 p-4 rounded-
```

```
 <p className="text-slate-400 text-xs for
```

```
 <Award className="w-3.5 h-3.5 text-em
```

```
 </p>
```

```
 <p className="text-2xl font-black text-v
```

```
 <p className="text-slate-400 text-xs mt
```

```
 </div>
```

```
 <div className="bg-slate-950 p-4 rounded-
```

```
 <p className="text-slate-400 text-xs for
```

```
 ложении:</p>
```

```
 <p className="text-xs text-slate-300 le
```

```
 ие чистые вершины через самое дешевое досту
```

```
 </div>
```

```
 </div>
```

```
 <div className="lg:col-span-2">
```

```
 <div className="bg-slate-950 p-4 rounded-
```

```
 <p className="text-slate-400 text-xs for
```

```

<div className="bg-slate-950 p-4 rounded-1
 <p className="text-slate-400 text-xs fo
 <Award className="w-3.5 h-3.5 text-ro
 </p>
 <p className="text-2xl font-black text-r
 <p className="text-slate-400 text-xs mt
</div>
<div className="bg-slate-950 p-4 rounded-1
 <p className="text-slate-400 text-xs fo
ении:</p>
 <p className="text-xs text-slate-300 le
, организуя массовое распараллеленное слиян
</div>
</div>
<div className="lg:col-span-2">
 <div className="bg-slate-950 p-4 rounded-1
 <p className="text-slate-400 text-xs fo
 <Code className="w-3.5 h-3.5 text-ros
 </p>
 <pre className="text-xs text-emerald-40
80 flex-1">
{`def boruvka(n, edges):
 parent = list(range(n))
 mst = []
 while len(mst) < n - 1:
 # Для каждого колхоза ищем самый дешёвый мост н
 cheapest = [-1] * n
 for u, v, w in edges:
 ru, rv = find(u), find(v)
 if ru != rv:
 if cheapest[ru] == -1 or cheapest[ru][2]
 if cheapest[rv] == -1 or cheapest[rv][2]
 # Объединяем все колхозы по выбранным мостам
 for bridge in cheapest:
 if bridge != -1 and union(bridge[0], bridge
 mst.append(bridge)`}
 </pre>
 </div>

```

```

body { background: #ffffff !important; color: #111827; }
.lite-wrap { max-width: 46rem; margin: 0 auto; padding: 1.6; }
.lite-wrap h1 { font-size: 1.75rem; font-weight: 700; }
.lite-wrap h2 { font-size: 1.35rem; font-weight: 700; color: #111827; }
.lite-wrap h3 { font-size: 1.1rem; font-weight: 700; }
.lite-wrap h4 { font-size: 1rem; font-weight: 700; margin: 0 0 1.2rem 0; }
.lite-wrap p, .lite-wrap li { color: #1f2937; font-size: 1rem; }
.lite-wrap a { color: #1d4ed8; }
.lite-meta { color: #4b5563; font-size: .85rem; margin: 0 0 1.2rem 0; }
.lite-topics { font-size: .8rem; color: #374151; margin: 0 0 1.2rem 0; }
.lite-viz { margin: 1.25rem 0; padding: .8rem 1rem; border: 1px solid #f3f4f6; }
.lite-viz h4 { margin: 0 0 .35rem; font-size: .95rem; }
.lite-viz ul { margin: .35rem 0 0 1.1rem; padding: 0; }
.lite-viz code, .lite-wrap code { font-family: ui-monospace; padding: 0 .25rem; }
.lite-wrap pre { background: #f3f4f6; border: 1px solid #f3f4f6; padding: 1.45; }
.lite-wrap table { border-collapse: collapse; width: 100%; }
.lite-wrap th, .lite-wrap td { border: 1px solid #d1d3d4; padding: .5rem; }
.lite-wrap th { background: #f3f4f6; }
.lite-wrap details { margin: .6rem 0; border: 1px solid #d1d3d4; }
.lite-wrap summary { cursor: default; font-weight: 700; }
.lite-toc { font-size: .92rem; }
.lite-toc ol { margin: .4rem 0 0 1.2rem; }
@media print {
 .lite-wrap { max-width: none; padding: 0; }
 h2 { page-break-before: always; }
 h2:first-of-type { page-break-before: avoid; }
 pre, table, details { page-break-inside: avoid; }
}

```

```

/** Раскрываем все спойлеры и вычищаем служебную разметку
function toPlainTextHtml(html: string): string {
 return html
}

```

```


 {list.map((c) => {
 <li key={c.id}>
 <a href={`?lite=1&topic=${encodeURIComponent(
 chapterTopics[c.id]?.length ? ` – биле
 ""}

)}}

 </nav>
)}

```

```

{list.map((chapter) => {
 const viz = getViz(chapter.id);
 const topics = chapterTopics[chapter.id] ?? [];
 const syncKeys = Object.keys(PAGE_SYNC).filter(
 return (
 <article key={chapter.id} id={chapter.id}>
 <h2>{chapter.title}</h2>
 <p className="lite-topics">
 {topics.length > 0 && <>Билет{topics.leng
 {chapter.category ? <>Раздел: {chapter.ca
 id страницы: <code>{chapter.id}</code>
 </p>

 <div
 dangerouslySetInnerHTML={{
 __html: linkifyTicketsHtml(toPlainTextH
 }}
 />

```

```

{viz && (
 <div className="lite-viz">
 <h4>Визуализация: {viz.title}</h4>
 {viz.hint && <p>{viz.hint}</p>}
 {syncKeys.map((k) => {
 const sync = PAGE_SYNC[k];
 const demo = k.includes("#") ? k.spli

```

```

 <li key={j}>
 {j === q.correctIndex ?

)}}

 <p className="lite-meta">{q.explan

)}}

</div>
 })
</article>
);
 })}
</div>
);
};

export default LiteGuide;

```

ФАЙЛ 58/111: src/components/LiveDistTable.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРО

```

import type { DebugValue } from "../data/debugRunner";

interface LiveDistTableProps {
 /** dist из кода пользователя: имя вершины → расстояние */
 dist: Record<string, DebugValue> | null;
 /** prev/p – предок на кратчайшем пути (необязательно) */
 prev?: Record<string, DebugValue> | null;
 /** done/visited – зафиксированные вершины (необязательно) */
 done?: DebugValue[] | null;
 /** Идентификаторы вершин, нарисованных на схеме демо */
 demoIds: string[];
 /** Подпись таблицы (по умолчанию «Ваши расстояния»).

```

```

const doneSet = new Set((done ?? []).map((value) => S
const shown = keys.slice(0, 64);

return (
 <div className="mb-6 rounded-xl border border-amber
 <p className="text-[11px] leading-relaxed text-am
 {title} В вашем коде вершины{" "}

 {unknown.slice(0, 8).join(", ")}
 {unknown.length > 8 ? " ..." : ""}

 , а на схеме демо-графа нарисованы{" "}
 {demoIds.join(", ")}
 показаны отдельной таблицей. Чтобы они управлял
 (кнопка-«глаз» в панели Python подставляет его
 </p>
 <div className="mt-2.5 flex flex-wrap gap-1.5">
 {shown.map((key) => {
 const parent = prev ? fmt(prev[key]) : null;
 const fixed = doneSet.has(key);
 return (
 <span
 key={key}
 className={`inline-flex items-baseline gap-
 fixed
 ? "border-emerald-500/40 bg-emerald-5
 : "border-slate-700 bg-slate-900/70 t
 }`}
 title={fixed ? "вершина зафиксирована (don
 >
 <b className={fixed ? "text-emerald-300"
 =
 {fmt(dist[key])}
 {parent && parent !== "-" ? <span classNa
 {fixed ? <span className="text-emerald-40

);

```

```

 alt: 'Бэтмен на машине Форда (Форд-Беллман)',
 title: ' Бэтмен на машине Форда',
 desc: 'Патрулирует n-1 ночей в кабине машины Генри',
 highlight: 'отрицательные циклы',
 highlightColor: 'text-rose-400',
 rest: '(бесконечную кормушку Джокера).',
 complexity: 'O(V · E)',
 border: 'border-rose-500/30 hover:border-rose-400/60',
 titleColor: 'text-rose-400',
 badge: 'text-rose-400/70 bg-rose-950/40',
 },
 {
 img: floydImg,
 alt: 'Fortnite Все против Всех (Флойд-Уоршелл)',
 title: ' Все против Всех / Fortnite (Флойд)',
 desc: 'Матрица N×N + три цикла (',
 highlight: 'k, i, j',
 highlightColor: 'text-emerald-400',
 rest: ')). Все против всех – ищет кратчайшие пути от',
 complexity: 'O(V³)',
 border: 'border-emerald-500/30 hover:border-emerald-400/60',
 titleColor: 'text-emerald-400',
 badge: 'text-emerald-400/70 bg-emerald-950/40',
 },
];

```

```

export default function MnemonicCards() {
 return (
 <div className="grid grid-cols-1 md:grid-cols-3 gap-4">
 {cards.map((c, i) => (
 <div key={i} className={`bg-slate-800 rounded-2xl p-4`}>
 <div className="h-48 overflow-hidden">
 <img src={c.img} alt={c.alt}
 className="w-full h-full object-cover group-hover:scale-110" />
 </div>
 <div className="p-4">
 <h3 className={`text-lg font-bold mb-1 ${c.titleColor}`}>{c.title}</h3>
 <p className="text-slate-400 text-sm">{c.desc}</p>
 <p className="text-slate-400 text-sm">{c.rest}</p>
 <p className="text-slate-400 text-sm">{c.complexity}</p>
 <p className="text-slate-400 text-sm">{c.badge}</p>
 </div>
 </div>
))}
 </div>
);
}

```



```
return () => {
 document.body.style.overflow = "";
};
}, [open]));

useEffect(() => {
 const onKey = (e: KeyboardEvent) => e.key === "Escape";
 window.addEventListener("keydown", onKey);
 return () => window.removeEventListener("keydown", onKey);
}, []);

return (
 <>
 <div className="lg:hidden sticky top-[57px] z-40">
 <button
 onClick={() => setOpen(true)}
 className="w-full flex items-center gap-2.5 p-2"
 aria-label="Открыть содержание"
 >

 <List className="w-4 h-4" />

 Содержание · {index + 1} из {total}

 {currentSection}

 </button>
 </div>

 {open && (
 <div className="lg:hidden fixed inset-0 z-[90]">
 <div className="absolute inset-0 bg-slate-950">
 <div className="absolute inset-y-0 left-0 w-[80%]>
 te-[tocIn_.18s_ease-out]">
 <div className="flex items-center justify-between">
 Содержание
 {index + 1} из {total}
 </div>
 <div>
 {currentSection}
 </div>
 </div>
 </div>
 </div>
)}
 </>
)
```

```
/**
 * Шапка приложения.
 *
 * Кнопка «Скачать PDF» всегда на самом видном месте во
 */
export const Navbar: React.FC<NavbarProps> = ({ compile
 const [busy, setBusy] = useState(false);

 const saveBook = async () => {
 setBusy(true);
 try {
 await downloadBookHtml(chapters);
 } finally {
 setBusy(false);
 }
 };

 return (
 <header className="bg-slate-900 border-b border-sla
 <div className="max-w-7xl mx-auto px-4 sm:px-6 lg
 gap-2.5 lg:gap-0">
 <div className="flex items-center gap-3 w-full
 <div className="bg-gradient-to-tr from-indigo
 0">
 <Sparkles className="w-5 h-5 lg:w-6 lg:h-6"
 </div>
 <div className="min-w-0 flex-1">
 <h1 className="text-lg lg:text-xl font-extr
 Универсальное пособие
 </h1>
 <p className="text-xs text-indigo-400 font-
 Подготовка к экзаменам: Алгоритмы, Структ
 </p>
 </div>
 </div>

 <div className="flex items-center flex-wrap sm:
```

```

<summary
 className="flex items-center justify-center
bg-slate-700/60 border border-slate-600 cu
 title="Ещё форматы: PDF с кодом, TXT и
>
 Ещё
</summary>
<div className="absolute right-0 top-full
5 shadow-xl shadow-black/50 min-w-[170px]">
 <a
 id="download-pdf-btn"
 href={` ${import.meta.env.BASE_URL}exp
 download="full_code_all_pages.pdf"
 target="_blank"
 rel="noreferrer"
 className="flex items-center gap-2 px
rald-600/20 border border-emerald-500/30 tr
 title="Полный PDF-дамп исходного кода
 >
 <FileDown className="w-3.5 h-3.5" />

 <a
 id="download-txt-btn"
 href={` ${import.meta.env.BASE_URL}exp
 download="full_code_all_pages.txt"
 target="_blank"
 rel="noreferrer"
 className="flex items-center gap-2 px
/20 border border-sky-500/30 transition-all
 title="Скачать полный TXT файл со все
 >
 <FileText className="w-3.5 h-3.5" />

 <button
 id="download-html-btn"
 onClick={saveBook}
 disabled={busy}
 className="flex items-center gap-2 w-

```

```

* Два графа на выбор: «основной» (8 вершин, $\Delta = 3$, есть
* и «двудольный» (6 вершин, $\Delta = 3$, $\chi = 2$, $\chi' = 3$).
*
* Синхронизация с Python-компилятором (PAGE_SYNC["planar"]);
* color/col – массив цветов вершин (красит кружки на панели);
* ecolord/ec – массив цветов рёбер, v – текущая вершина;
* u – сосед, chi/k – число цветов, delta/D – максимальная степень;
* i/step – номер кадра.
*/

type Tab = "planar" | "greedy" | "bipartite" | "edges";
type GraphKey = "main" | "bip";

interface Graph {
 key: GraphKey;
 label: string;
 n: number;
 edges: [number, number][];
 adj: number[][];
 pos: { x: number; y: number }[];
 names: string[];
}

const PALETTE = ["#6366f1", "#10b981", "#f43f5e", "#f59e0b"];
const colorOf = (c: number) => (c >= 1 && c <= PALETTE.length) ? PALETTE[c - 1] : "#000000";

function makeGraph(key: GraphKey): Graph {
 if (key === "bip") {
 const edges: [number, number][] = [[0, 3], [0, 4], [1, 3], [1, 4], [2, 3], [2, 4], [3, 4]];
 const pos = [
 { x: 80, y: 60 }, { x: 80, y: 150 }, { x: 80, y: 240 },
 { x: 380, y: 60 }, { x: 380, y: 150 }, { x: 380, y: 240 }
];
 const g: Graph = { key, label: "Двудольный (6 вершин)", pos, edges };
 g.adj = buildAdj(g);
 return g;
 }
 const edges: [number, number][] = [[0, 1], [0, 2], [1, 2], [1, 3], [1, 4], [1, 5], [2, 3], [2, 4], [2, 5], [3, 4], [3, 5], [4, 5]];
 const pos = [
 { x: 80, y: 60 }, { x: 200, y: 60 }, { x: 320, y: 60 },
 { x: 80, y: 150 }, { x: 200, y: 150 }, { x: 320, y: 150 },
 { x: 80, y: 240 }, { x: 200, y: 240 }, { x: 320, y: 240 }
];
 const g: Graph = { key, label: "Основной (8 вершин)", pos, edges };
 g.adj = buildAdj(g);
 return g;
}

```

```

 title: "Старт: ни одна вершина не покрашена",
 note: `color = [0]·n. Порядок обхода – просто по номерам. Гарантированно уложится в $\Delta + 1 = \{\delta + 1\}$ цветов: у вершины v
 color: [...color], v: null, u: null, e: null,
 });
 for (let v = 0; v < g.n; v++) {
 const used = Array.from(new Set(g.adj[v].filter((x) => x !== v)));
 let c = 1;
 while (used.includes(c)) c += 1;
 color[v] = c;
 const deg = g.adj[v].length;
 frames.push({
 title: `Вершина ${g.names[v]}: берём цвет ${c}`,
 note: used.length
 ? `Соседи уже заняли цвета ${used.join(", ")}
 всегда хватит ($\chi \leq \Delta + 1$).`
 : `У вершины ${g.names[v]} ещё нет покрашенных соседей`,
 color: [...color], v, u: null, e: null, used,
 });
 }
 const chi = Math.max(...color);
 frames.push({
 title: `Готово: жадная покраска использовала ${chi} цветов`,
 note: ` $\chi_{\text{жадн}} = \{\chi\}$. Истинное хроматическое число χ
 , а теорема Брукса ($\chi \leq \Delta$, если граф не полный и не двудольный)`,
 color: [...color], v: null, u: null, e: null,
 });
 return frames;
}

```

```

/** Проверка двудольности BFS-ом: 2 краски, нет нечётных циклов
function bipartiteFrames(g: Graph): Frame[] {
 const frames: Frame[] = [];
 const color = new Array(g.n).fill(0);
 const queue: number[] = [];
 let conflict: [number, number] | null = null;
 frames.push({
 title: "Старт: пробуем покрасить в 2 цвета",

```

```

frames.push({
 title: conflict ? "Граф НЕ двудольный" : "Граф двудольный",
 note: conflict
 ? `BFS уперся в нечётный цикл (${g.names[conflict]}):
 краски — их и даёт жадный алгоритм на вкладке «Жадная окраска»
 : "Все вершины разбиты на две доли, рёбер внутри долей нет,
 справедлива теорема Кёнига: $\chi' = \Delta$ (рёберная окраска)",
 color: [...color], v: null, u: null, e: null, bad: null,
});
return frames;
}

```

```

/** Жадная покраска рёбер: $\Delta \leq \chi' \leq \Delta + 1$ (Визинг), для
function edgeFrames(g: Graph): Frame[] {
 const frames: Frame[] = [];
 const ecol = new Array(g.edges.length).fill(0);
 const inc: Map<number, number>[] = Array.from({ length: g.n }, () => new Map());
 const delta = Math.max(...g.adj.map(a => a.length));
 frames.push({
 title: "Старт: рёбра не покрашены",
 note: `Правильная рёберная покраска: у каждой вершины
 рёбер одной вершины различны, а по теореме Визинга
 $\chi' \leq \Delta + 1$ `,
 color: new Array(g.n).fill(0), ecol: [...ecol],
 });
 g.edges.forEach(([a, b], i) => {
 const usedA = Array.from(inc[a].values());
 const usedB = Array.from(inc[b].values());
 const used = Array.from(new Set([...usedA, ...usedB]));
 let c = 1;
 while (used.includes(c)) c += 1;
 ecol[i] = c;
 inc[a].set(i, c);
 inc[b].set(i, c);
 frames.push({
 title: `Рёбро (${g.names[a]}, ${g.names[b]}): цвет ${c}`,
 note: `В вершине ${g.names[a]} заняты цвета ${usedA}
 (использованы цвета {${usedA.map(x => x - y).join(", ") || "—"}}). Наименьший свободный цвет — ${c}`,
 });
 });
 return frames;
}

```

```

const frames = useMemo(() => {
 if (tab === "greedy") return greedyFrames(graph);
 if (tab === "bipartite") return bipartiteFrames(graph);
 if (tab === "edges") return edgeFrames(graph);
 return [] as Frame[];
}, [tab, graph]);
const maxStep = Math.max(0, frames.length - 1);

useEffect(() => {
 setStep(0);
 setPlaying(false);
}, [tab, graphKey]);

useEffect(() => {
 if (!playing) return;
 const t = window.setInterval(() => {
 setStep((s) => {
 if (s >= maxStep) {
 setPlaying(false);
 return s;
 }
 return s + 1;
 });
 }, 1300);
 return () => window.clearInterval(t);
}, [playing, maxStep]);

/* живые переменные компилятора */
const liveStep = vars ? vizNumber(vars.step) ?? vizNumber(0) : null;
useEffect(() => {
 if (liveStep === null || !Number.isFinite(liveStep))
 setStep(Math.max(0, Math.min(maxStep, liveStep)));
}, [liveStep, maxStep]);

const liveColor = vars ? vizArray(vars.color) ?? vizArray([0, 0, 0]) : null;
const liveEcolor = vars ? vizArray(vars.ecolor) ?? vizArray([0, 0, 0]) : null;
const liveV = vars ? vizNumber(vars.v) : null;
const liveU = vars ? vizNumber(vars.u) : null;

```

```

return (
 <div className="bg-slate-800/90 p-5 sm:p-6 rounded-
 <Header />
 <Tabs tab={tab} setTab={setTab} />

 <div className="flex flex-wrap items-center gap-2
 <span className="text-[10px] uppercase tracking
 {[["main", "bip"] as GraphKey[]}.map((k) => (
 <button
 key={k}
 onClick={() => setGraphKey(k)}
 className={"px-3 py-1.5 rounded-lg text-[11px]
 "bg-slate-900/60 text-slate-400 hover:text-
 >
 {makeGraph(k).label}
 </button>
 </div>
 </div>

 <div className="flex flex-wrap items-center gap-2
 <button onClick={() => { setStep(0); setPlaying
 rounded-lg bg-slate-900 border border-slate-
 <button onClick={() => go(-1)} disabled={step =
 00 border border-slate-700 text-slate-300 hove
 <button onClick={() => setPlaying((p) => !p)} c
 ext-xs font-bold hover:bg-indigo-500">
 {playing ? <Pause className="w-4 h-4" /> : <P
 </button>
 <button onClick={() => go(1)} disabled={step >=
 late-900 border border-slate-700 text-slate-3
 on>
 <span className="text-[11px] text-slate-500 ml-
 <span className="ml-auto flex flex-wrap items-c
 <span className="bg-slate-950 border border-s
 {tab === "edges"
 ? <span className="bg-slate-950 border bord
 rs}
 : <span className="bg-slate-950 border bord

```



```

 ames[id]]</text>
 {c > 0 && tab !== "edges" && <text x=
old">цвет {c}</text>}
 {tab === "edges" && <text x={p.x} y={
}</text>}
 </g>
 });
 }}}
</svg>
</div>

<div className="lg:col-span-2 space-y-3">
 <div className="bg-slate-900/70 border border
 <p className="text-sm font-bold text-white
 <p className="text-xs text-slate-400 leadin
 </div>
 <div className="bg-slate-950 border border-sl
 <p className="text-[10px] uppercase trackin
 <div className="flex flex-wrap gap-1.5 font
 {(tab === "edges" ? graph.edges.map((_, i
 <span key={i} className="px-1.5 py-0.5
 ? colorOf(c) : "#475569", background: c > 0
 {tab === "edges" ? `e${i}` : graph.na

)})}
 </div>
 </div>
 <div className="bg-slate-900/70 border border
 {tab === "greedy" && <>Жадная покраска: <b
 : <b className="text-slate-200"> $\chi \leq \Delta$,
 в $\chi \leq 5$ (доказывается через $E \leq 3V - 6$ и це
 ром).</>}
 {tab === "bipartite" && <>Двудольный граф
 /DFS за $O(V + E)$. Распознавание 3-раскрашив
 {tab === "edges" && <>Рёберная покраска: <b
 Δ (Кёниг). Задача « $\chi' = \Delta$?» NP-полна даже
 </div>
</div>

```

```
const Tabs: React.FC<{ tab: Tab; setTab: (t: Tab) => void }> = ({ tab, setTab }) => {
 <div className="flex flex-wrap gap-2 mb-4">
 {TABS.map((t) => (
 <button
 key={t.id}
 onClick={() => setTab(t.id)}
 className={"flex items-center gap-1.5 px-3 py-2 text-white" : "bg-slate-900/60 text-slate-900"}
 >
 {t.icon} {t.label}
 </button>
))}
 </div>
);
```

```
export default PlanarColoringViz;
```

---

ФАЙЛ 63/111: src/components/PlanarityDemo.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 10

---

```
export function PlanarityDemo() {
 return (
 <div className="space-y-4">
 <h4 className="text-base font-bold text-white">K5
 <p className="text-xs text-slate-400">Просто смотрим на K5
 <div className="grid grid-cols-1 md:grid-cols-2 gap-4">
 { /* K5 */ }
 <div className="bg-slate-950 rounded-xl p-4 border border-slate-900">
 <div className="absolute top-2 left-2 bg-slate-900/30 w-auto z-20">K5 – НЕПЛАНАРЕН</div>
 <svg className="w-full h-full absolute inset-0">
 {(() => {
 const pts = [
```

```

 const labels = ["A", "B", "C", "D", "E"];
 for(const p of pts) {
 circles.push(<g key={`c${p.id}`}>
 <circle cx={p.x} cy={p.y} r={8} fill=
 <text x={p.x} y={p.y+3} textAnchor="m
 </g>);
 }
 return <>{lines}{circles}</>;
 })()}
</svg>
<div className="absolute bottom-2 left-2 right
 z-20">
 V=5, E=10 {'>'} 3V-6 = 9 {'-->'} {'\u274C'}
</div>
</div>

{/* K3,3 */}
<div className="bg-slate-950 rounded-xl p-4 bor
 <div className="absolute top-2 left-2 bg-slate
 00/30 w-auto z-20">K3,3 – НЕПЛАНАРЕН</div>
 <svg className="w-full h-full absolute inset-0
 {(() => {
 const pts = [
 {id:0,x:60,y:60},{id:1,x:160,y:60},{id:2,x:260,y:60},
 {id:3,x:60,y:220},{id:4,x:160,y:220},{id:5,x:260,y:220},
];
 const edges = [
 [0,3],[0,4],[0,5],
 [1,3],[1,4],[1,5],
 [2,3],[2,4],[2,5]
];
 function findPt(id:number) { return pts.f
 function intersect(a:number,b:number,c:nu
 if (a===c||a===d||b===c||b===d) return
 const p1=findPt(a),p2=findPt(b),p3=find
 if (!p1||!p2||!p3||!p4) return false;
 function ccw(ax:number,ay:number,bx:num
 return (cy-ay)*(bx-ax)>(by-ay)*(cx-ax)

```

```
 </div>
 </div>
);
}
```

---

ФАЙЛ 64/111: src/components/PythonCompiler.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

---

```
import { useCallback, useEffect, useMemo, useRef, useState } from 'react';
import {
 ArrowRightLeft,
 Check,
 CheckCircle2,
 ChevronDown,
 ChevronLeft,
 ChevronRight,
 ChevronUp,
 CircleStop,
 Copy,
 Eye,
 EyeOff,
 HelpCircle,
 Link2,
 Loader2,
 Pause,
 Pencil,
 Play,
 Plus,
 RotateCcw,
 SkipBack,
 SkipForward,
 Terminal,
 Unlink,
 Variable,
 X,
}
```

```

 setStderr: (options: { batched: (message: string) => Promise<string> }) => Promise<string>
 setStdin: (options: { stdin: () => string | number | null }) => Promise<string>
};
```

```
type PyodideModule = {
 loadPyodide: (options: { indexURL: string }) => Promise<PyodideRuntime>
};
```

```
let runtimePromise: Promise<PyodideRuntime> | null = null;
```

```
/**
 * Подмена интерпретатора для проверок: DOM-хакер (jsdom) и CI-скриншоты кладут сюда готовый
 * на CDN. В браузере у пользователя поле не определено
 */
```

```
type PythonRuntimeProvider = PyodideRuntime | (() => Promise<PyodideRuntime>);
```

```
declare global {
 interface Window {
 __algv0PythonRuntime?: PythonRuntimeProvider;
 }
}
```

```
function getPythonRuntime(): Promise<PyodideRuntime> {
 const injected = typeof window !== "undefined" ? window.__algv0PythonRuntime : undefined;
 if (injected) return typeof injected === "function" ? injected : Promise.resolve(injected);
 if (!runtimePromise) {
 runtimePromise = import(/* @vite-ignore */ PYODIDE_MODULE_URL)
 .then((module as PyodideModule).loadPyodide({ indexURL: indexURL }));
 }
 return runtimePromise;
}
```

```
function errorText(error: unknown) {
 if (error instanceof Error) return error.message;
 return String(error);
}
```

```

 items: [
 {
 label: "arr.sort() / sorted()",
 tip: "In-place сортировка по возрастанию и новая",
 code: 'arr = [5, 2, 8, 1]\narr.sort()
) # копия по убыванию\n',
 },
 {
 label: "sorted(key=...)",
 tip: "Сортировка по ключу: рёбра по весу, пары",
 code: 'edges = [(3, "A", "B"), (1, "B", "C"), (1, "A',
 es)\n',
 },
],
},
{
 group: "Циклы",
 items: [
 {
 label: "for i in range(n)",
 tip: "Вложенные циклы по i и j – как обход матрицы",
 code: "n, m = 3, 4\nfor i in range(n):",
 },
 {
 label: "while ...",
 tip: "Классический while-счётчик – как n-1 итерация",
 code: "i = 1\nwhile i < 5:\n print(f\"итерация {i}\")\n",
 },
 {
 label: "for i, x in enumerate(...)",
 tip: "Индекс и элемент одновременно: i – позиция, x – элемент",
 code: "for i, x in enumerate(['a', 'b', 'c']):",
 },
],
},
{
 group: "Структуры",
 items: [

```

```

 },
];

/**
 * Переживает сккрытие панели: код пользователя не теряе
 *
 * Хранилище — ПО СТРАНИЦАМ. Раньше слот был один, и ко
 * восстанавливался на новой странице: редактор показыва
 * и визуализация жили значениями чужого кода.
 */
const savedEditors = new Map<string, { code: string; ba

export function PythonCompiler({ chapterId, chapterTitle
 ilderProps) {
 /** Активная вкладка демонстрации страницы (у sparse-
 const [demoId, setDemoId] = useState<string | null>(n
 const sync = useMemo(() => getPageSync(chapterId, dem
 const syncVarBases = useMemo(
 () =>
 [...new Set(
 sync.variables.flatMap((v) =>
 v.name
 .split(/\s*(?:,|\s\/)\s*/)
 .map(baseVarName)
 .filter((name) => /^[A-Za-z_][A-Za-z0-9_]*$/
)
)],
 [sync]
);
 const syncVarSet = useMemo(() => new Set(syncVarBases
 /** Полный эталон: глаз сразу вставляет его в обычный
 const template = useMemo(() => buildSyncTemplate(char
 /** Дефолт редактора: только инициализация демо; она
 const initTemplate = useMemo(() => buildInitTemplate(

 const [code, setCode] = useState(() => {
 const saved = savedEditors.get(chapterId);
 if (!saved) return initTemplate;
 });

```

```

/** Активная трасса. input нужен, чтобы изменение std
const [debug, setDebug] = useState<{ result: DebugRes
/** Esc оставляет неизменённый текст в редакторе, пока
const [editHold, setEditHold] = useState<string | null>
/** Автопроход включается пользователем; первый шаг по
const [playing, setPlaying] = useState(false);
const [playbackSpeed, setPlaybackSpeed] = useState<num
 if (typeof window === "undefined") return 1;
 const saved = Number(window.localStorage.getItem(PL
 return PLAYBACK_SPEEDS.includes(saved as (typeof PL
));
const listRef = useRef<HTMLDivElement>(null);

useEffect(() => {
 window.localStorage.setItem(PLAYBACK_SPEED_KEY, Str
}, [playbackSpeed]);

useEffect(() => {
 savedEditors.set(chapterId, { code, base: initTempl
}, [code, initTemplate, chapterId]);

// демонстрация сообщает, какая вкладка открыта – код
useEffect(() => setDemoId(null), [chapterId]);
useEffect(() => onVizDemo(chapterId, (d) => setDemoId
useEffect(() => () => clearVizState(chapterId), [chap

// Смена страницы (или вкладки демо): нетронутый реда
// инициализацию; свой код не затираем – предлагаем с
useEffect(() => {
 const prev = prevTplRef.current;
 if (initTemplate === prev.base && template === prev
 prevTplRef.current = { base: initTemplate, full: te
 setPlaying(false);
 setEditHold(null);
 setDebug(null); // трасса ссылается на строки старо
 setRefOn(false);
 if (code === prev.base || code === prev.full || cod

```



```

* события: после st[i][j]= Python уже мог увеличить
*/
const snapshotAt = useCallback((result: DebugResult,
 const step = result.steps[idx];
 const next = result.steps[idx + 1];
 const afterLocals = next?.locals ?? result.finalLocals;
 const afterValues = next?.values ?? result.finalValues;
 const beforeValues = step?.values ?? {};
 const line = step?.line ? source.split("\n")[step.line - 1] : null;
 // Нас интересуют только простые скалярные цели (`i` или `i +=`). Контейнеры и так всегда берутся из afterValues
 // внутри print(f"i={i}") не будет ошибочно принят `i`
 const assignment = line?.match(
 /^\\s*([A-Za-z_][A-Za-z0-9_]*)\\s*(?:\\s*=|\\s*[A-Za-z_][A-Za-z0-9_]*\\s*=)\\s*$/;
);
 const assigned = new Set(assignment?.[1].split(",") || []);

 const variables = { ...afterValues };
 const locals = { ...afterLocals };
 for (const [name, value] of Object.entries(beforeValues)) {
 const scalar = value === null || typeof value !== "object";
 if (scalar && !assigned.has(name) && name in afterValues) {
 variables[name] = value;
 if (step?.locals[name] !== undefined) locals[name] = value;
 }
 }
 const changed = changedVars(step?.locals, locals);
 return {
 line: step?.line,
 func: step?.func,
 locals,
 variables,
 changed: Object.keys(changed).filter((name) => changed[name]),
 };
}, []);

const publishDebug = useCallback(
 (result: DebugResult, idx: number, source: string) => {

```

```

 []
);

 const stepTo = useCallback(({idx: number}) => goDebug(idx));

 const stopDebug = useCallback(() => {
 setPlaying(false);
 setEditHold(code);
 setDebug(null);
 clearVizState(chapterId);
 }, [chapterId, code]);

 // Базовый интервал 650 мс; множитель скорости хранится в state
 useEffect(() => {
 if (!playing || !debug) return;
 if (debug.idx >= debug.result.steps.length - 1) {
 setPlaying(false);
 return;
 }
 const t = window.setTimeout(() => stepBy(1), 650 / playbackSpeed);
 return () => window.clearTimeout(t);
 }, [playing, debug, stepBy, playbackSpeed]);

 const debugRun = useCallback(async (autoPlay = false) => {
 if (running) return;

 const source = code;
 const input = stdin;
 const request = traceRequest;
 setRunning(true);
 setPlaying(false);
 setEditHold(null);
 setDebug(null);
 clearVizState(chapterId);
 setOutputOpen(false);

 // Пустой редактор: Pyodide не гоняем, панель в режиме редактирования
 // связь с визуализацией снимаем — демо показывает

```

```

 ? `Ошибка:\n${result.error}`
 : printed || "В коде нет исполняемых строк
);
 setRuntimeMessage(
 result.error
 ? "Код не выполнен — редактор остался открыт"
 : "Нет шагов трассы: связь с визуализацией
);
} else {
 setDebug({ result, idx: 0, src: source, input,
 setPlaying(autoPlay && result.steps.length > 1)
 // Программа что-то напечатала — значит, вывод
 // результат своего же print() панель не имеет
 if (printed.trim() !== "") setOutputOpen(true);
 setOutput(
 (printed || "Программа ничего не вывела — тра
 (result.error ? `\\n\\nОшибка (выполнение пре
);
 if (result.error) setOutputOpen(true);
 setRuntimeMessage(
 result.truncated
 ? `Выполнение остановлено после ${DEBUG_STE
 : "Трасса готова автоматически. ← → — шаги,
);
}
} catch (error) {
 if (!aliveRef.current) return;
 const message = errorText(error);
 setDebug({ result: { steps: [], error: message, t
 clearVizState(chapterId);
 setOutputOpen(true);
 setOutput(`Ошибка:\n${message}`);
 setRuntimeMessage("Не удалось выполнить код — про
} finally {
 setRunning(false);
}
}, [chapterId, code, stdin, traceRequest, running, run

```

```

 }, [debug, stepBy, stopDebug]);

// держим текущую строку листинга в видимой области
/**
 * Панель показывает листинг трассы, только если шаги
 * Пустой код или синтаксическая ошибка больше не пряч
 * раньше на их месте возникал пустой листинг без возм
 * «исчезала» и кнопка, и само поле ввода.
 */
const showTrace = !!debug && debug.result.steps.length
/** Трасса построена по старому тексту: пользователь
const stale = !!debug && debug.src !== code;
/** Листинг трассы можно свернуть, но редактор от это
const [traceOpen, setTraceOpen] = useState(true);
const curDebug = debug ? { step: debug.result.steps[d
const curDebugLine = curDebug?.step?.line;
useEffect(() => {
 if (curDebugLine === undefined || !listRef.current)
 listRef.current
 .querySelector(`[data-line="${curDebugLine}"]`)
 ?.scrollIntoView({ block: "nearest" });
}, [curDebugLine]);

// производные данные шага: что изменилось, в каком п
// Строка, которую назначает текущий шаг, подсвечивае
// локали берём со следующего шага трассы (для послед
// трейсером на возврате из фрейма). Значит на строке
const shownLocals = useMemo(() => {
 if (!debug || !curDebug?.step) return {};
 return snapshotAt(debug.result, debug.idx, debug.src
}, [debug, curDebug, snapshotAt]);
const debugChanged = curDebug?.step ? changedVars(curi
const debugOrdered = curDebug?.step ? orderVars(shown

/** Одна и та же кнопка запуска всегда видна: из реда
 * проигрывает её, в отладчике переключает play/pause
const runOrTogglePlayback = useCallback(() => {
 if (running) return;

```

```

>
 { /* Верхний край меняет высоту на телефоне и десктопе. */
 {onHeightChange && (
 <div
 role="separator"
 aria-orientation="horizontal"
 aria-label="Потяните, чтобы изменить высоту терминала"
 title="Потяните, чтобы изменить высоту терминала"
 className="absolute -top-1.5 left-0 right-0 h-10"
 onPointerDown={(event) => {
 const el = event.currentTarget;
 el.setPointerCapture(event.pointerId);
 const startY = event.clientY;
 const startH = el.parentElement?.getBoundingClientRect().height;
 const onMove = (moveEvent: PointerEvent) => {
 const maxHeight = window.innerHeight - (window.topBarHeight || 0);
 const minHeight = Math.min(260, maxHeight);
 const next = Math.round(startH + startY - moveEvent.clientY);
 onHeightChange(Math.max(minHeight, Math.min(maxHeight, next)));
 };
 const onUp = () => {
 el.removeEventListener("pointermove", onMove);
 el.removeEventListener("pointerup", onUp);
 el.removeEventListener("pointercancel", onUp);
 };
 el.addEventListener("pointermove", onMove);
 el.addEventListener("pointerup", onUp);
 el.addEventListener("pointercancel", onUp);
 }}
 >
 <div className="absolute left-1/2 top-1/2 h-10 w-10
 -translate-x-1/2 -translate-y-1/2 rounded-digo-400 transition-colors" />
 </div>
)}
 }

 { /* Левый край меняет ширину на десктопе. */
 {onWidthChange && (
 <div

```

```

>
 <button
 type="button"
 onClick={runOrTogglePlayback}
 disabled={running}
 className={`inline-flex items-center gap-
0 ${
 playing
 ? "bg-amber-500/15 text-amber-300 hover:"
 : "bg-emerald-500/15 text-emerald-300
 }`}
 aria-label={running ? "Python выполняется"
 >
 {running ? <Loader2 className="h-4 w-4 an
 " />}
 {playbackSpeed}x
 </button>
</Tooltip>
<Tooltip side="bottom" content="Скопировать к
 <button
 type="button"
 onClick={() => void copyCode()}
 className="p-1.5 rounded-lg text-slate-400"
 aria-label="Скопировать код"
 >
 {copied ? <Check className="w-4 h-4 text-
 </button>
</Tooltip>
<Tooltip side="bottom" content="Вернуть коммен
 <button
 type="button"
 onClick={resync}
 className="p-1.5 rounded-lg text-slate-400"
 aria-label="Сбросить к шаблону страницы"
 >
 <RotateCcw className="w-4 h-4" />
 </button>
</Tooltip>

```

```

 refOn
 ? "bg-indigo-500/30 text-white ring-1
 : "text-indigo-300 hover:text-white h
 }` }
 aria-label={refOn ? "Убрать эталонный код
ктор"}
 aria-pressed={refOn}
 >
 {refOn ? <Eye className="w-4 h-4" /> : <E
 </button>
</Tooltip>
<label
 className="ml-1 inline-flex items-center gap
 der border-slate-700 hover:border-emerald-5
 title="Скорость автоматического проигрывани
 >
 {running ? <Loader2 className="w-3.5 h-3.5
 ld-400" />}
 {running
 <select
 aria-label="Скорость автопрохода"
 value={playbackSpeed}
 onChange={(event) => setPlaybackSpeed(Num
 className="cursor-pointer rounded bg-slate
 >
 {PLAYBACK_SPEEDS.map(({speed}) => (
 <option key={speed} value={speed}>{speed
)))}
 </select>
 </label>
</div>
</div>

/* — Мини-вкладки: переменные страницы / шпаргал
<div className="shrink-0 border-b border-slate-800
 <div className="flex items-center gap-1 px-2 pt
 <button
 type="button"

```

```

{stripTab === "vars" ? {
 <div className="px-3 pb-2.5 space-y-2 max-h-300">
 <div className="flex items-center gap-2 min-w-100">
 {onOpenGuide ? {
 <Tooltip side="bottom" content="Открыть руководство" >
 <button
 type="button"
 onClick={onOpenGuide}
 className="inline-flex items-center justify-center gap-2 text-slate-300 hover:text-white hover:bg-slate-700" >
 {chapterName}
 </button>
 </Tooltip>
 } : {

 {chapterTitle}

 }}
 </div>
 {desyncedFrom !== null && {
 <Tooltip
 side="bottom"
 content="В редакторе ваш код, а страница — чужой. Если вы не уверены, что не допустили ошибок, нажмите на кнопку ниже, чтобы заменить код на оригинальный." >
 <button
 type="button"
 onClick={resync}
 className="flex items-center gap-1.5 text-amber-400 hover:bg-amber-500/20 transition-colors" >
 <Unlink className="w-3.5 h-3.5 shrink-0" />
 Страница сменилась — подставить её пересинхронизировать
 </button>
 </Tooltip>
 }
 }
}

```



```

 </div>
)}
 </div>
) : (
 <div className="px-3 pb-2.5 max-h-36 overflow
 {SNIPPETS.map(({gItem}) => (
 <div key={gItem.group}>
 <div className="text-[9px] font-bold up
 <div className="flex flex-wrap gap-1.5"
 {gItem.items.map(({s}) => (
 <Tooltip key={s.label} side="bottom
 <button
 type="button"
 onClick={() => insertSnippet(s.
 className="inline-flex items-ce
 -[10.5px] text-indigo-300 hover:text-white
 title=""
 >
 <Plus className="w-3 h-3 text-s
 {s.label}
 </button>
 </Tooltip>
)}}
 </div>
 </div>
)}}
 <p className="text-[10px] text-slate-600">H
 </div>
)}
 </div>
)}
)}
 </div>

```

```

/* — Редактор (всегда на месте) + трасса + ввод
<div className="flex-1 min-h-0 flex flex-col">
 /* Заголовок редактора: сразу видно, что выполн
 <div className="shrink-0 flex items-center gap-1
 <Pencil className="w-3.5 h-3.5 text-emerald-4
 <span className="text-[10px] font-bold upperc
 Редактор • выполняется этот текст

```

```

<div className={`flex flex-col border-t borde
 /* панель управления шагами */
 <div className="shrink-0 flex flex-wrap ite
 <Tooltip content={playing ? `Пауза автопро
 роиграть трассу заново со скоростью ${playb
 л)`}>
 <button
 type="button"
 onClick={runOrTogglePlayback}
 className={`mx-0.5 inline-flex items-
 r-500/15 hover:bg-amber-500/25" : "text-еме
 aria-label={playing ? `Пауза автопрох
 >
 {playing ? <Pause className="w-4 h-4"
 <span className="text-[9px] font-bold
 </button>
 </Tooltip>
 <Tooltip content="В начало трассы">
 <button type="button" aria-label="В нач
 }} disabled={!curDebug?.step} className="p-
 transition-colors">
 <SkipBack className="w-4 h-4" />
 </button>
 </Tooltip>
 <Tooltip content="Шаг назад (←) – автопро
 <button type="button" aria-label="Шаг на
 bled={!curDebug?.prev} className="p-1 round
 tion-colors">
 <ChevronLeft className="w-4 h-4" />
 </button>
 </Tooltip>
 <Tooltip content="Шаг вперёд (→) – автопро
 <button type="button" aria-label="Шаг в
 abled={!curDebug?.step || debug.idx >= debu
 e hover:bg-slate-800 disabled:opacity-40 tr
 <ChevronRight className="w-4 h-4" />
 </button>
 </Tooltip>

```

```

 и переменные текущего шага."}>
 <button
 type="button"
 aria-label={traceOpen ? "Свернуть лис
 aria-expanded={traceOpen}
 onClick={() => setTraceOpen((v) => !v
 className="ml-auto inline-flex shrink
ext-[10px] font-bold text-slate-300 transit
 >
 {traceOpen ? <ChevronDown className="
 ЛИСТИНГ
 </button>
 </Tooltip>
 <Tooltip content="Остановить прогон (Esc)
на показывает встроенное демо. Автозапуск н
 <button
 type="button"
 aria-label="Остановить прогон трассы"
 onClick={stopDebug}
 className="inline-flex shrink-0 items
ext-[10px] font-bold text-rose-300 transit
 >
 <CircleStop className="w-3.5 h-3.5" />
 стоп
 <kbd className="rounded border border
 </button>
 </Tooltip>
 </div>

```

```

{traceOpen && (
 <>
 /* листинг с подсветкой текущей строки */
 <div ref={listRef} className="flex-1 min-h-1
 {debug.src.split("\n").map((line, i) => {
 const ln = i + 1;
 const active = ln === curDebug?.step?.l
 return (
 <div

```

```

 <span
 key={name}
 className={`inline-flex items-base
 changed
 ? "border-amber-500/50 bg-amber-500/50"
 : synced
 ? "border-emerald-500/40 bg-emerald-500/40"
 : "border-slate-700 bg-slate-700"}
 >

 {name}

 {value}

);
 }
 }
</div>
</div>
</>
)}
</div>
)}
{debug && !showTrace && debug.result.error ? (
 <pre className="shrink-0 max-h-28 overflow-auto py-2 font-mono text-[11px] leading-snug text-slate-500">
 {debug.result.error}
 </pre>
) : null}
<div className="shrink-0 border-t border-slate-700">
 <label htmlFor="python-stdin" className="text-slate-500">
 stdin для input()
 </label>
 <input
 id="python-stdin"
 value={stdin}
 onChange={(event) => {

```

```
 });
 }
}
```

---

ФАЙЛ 65/111: src/components/QueueViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

---

```
import React, { useState, useCallback } from 'react';
import { Sparkles } from 'lucide-react';
import { useVizRuntime, vizArray, vizString } from '../viz';
```

```
interface Customer {
 id: string;
 name: string;
 emoji: string;
 color: string;
 bubbleText: string;
 animState: 'in' | 'idle' | 'eating' | 'out';
}
```

```
const PRESETS = [
 { name: 'Студент Вася', emoji: '👨‍🎓', color: 'from-blue',
 bubbleText: 'Матана!' },
 { name: 'Кодер Семён', emoji: '👨‍💻', color: 'from-violet',
 bubbleText: 'Порцию борща!' },
 { name: 'Кот Борис', emoji: '🐈', color: 'from-amber',
 bubbleText: 'Плиз.' },
 { name: 'Бабушка Люба', emoji: '👵', color: 'from-rose',
 bubbleText: '' },
 { name: 'Инопланетянин', emoji: '👽', color: 'from-emerald',
 bubbleText: 'Пиво для нашего НЛО!' },
 { name: 'Бизнесмен', emoji: '👔', color: 'from-slate',
 bubbleText: 'Тартапы.' },
];
```

```
let counter = 3;
```

```

 setTimeout(() => setShake(false), 400);
 return;
 }

 const preset = PRESETS[counter % PRESETS.length];
 counter++;

 const newGuest: Customer = {
 id: Date.now().toString(),
 ...preset,
 animState: 'in',
 };

 setQueue(prev => [...prev, newGuest]);
 addLog(`- ENQUEUE: ${newGuest.name} ${newGuest.emoji}`);

 setTimeout(() => {
 setQueue(prev => prev.map(c => c.id === newGuest.id ? newGuest : c));
 }, 450);
}, [queue, isServing]);

// DEQUEUE: Выдать суп первому (голова / Head, FIFO)
const dequeue = useCallback(() => {
 if (isServing) return;
 if (queue.length === 0) {
 setShake(true);
 addLog(`⚠ DEQUEUE: В очереди никого нет! Повар скучает`);
 setTimeout(() => setShake(false), 400);
 return;
 }

 setIsServing(true);
 const headGuest = queue[0];

 addLog(`- DEQUEUE: Повар наливает суп первому – ${headGuest.name} ${headGuest.emoji}`);

 // Step 1: Head guest gets eating animation
 setQueue(prev => prev.map((c, idx) => idx === 0 ? { ...c, animState: 'eating' } : c));

```

```

 <div>
 <h3 className="font-black text-indigo-400 text-2xl">
 В очереди за супом всё честно. Кто первый встал, тот и ест.
 <p className="text-xs text-slate-300 mt-1 leading-4">
 Новые гости встают строго в конец очереди.
 Здесь нет обхода и блата – только строгий порядок.
 </p>
 </div>
 </div>

 { /* УПРАВЛЕНИЕ */ }
 <div className="flex items-center gap-3 flex-wrap">
 <button
 onClick={enqueue}
 disabled={isServing}
 className="flex-1 min-w-[150px] bg-gradient-to-r from-indigo-500 to-indigo-600
 text-white opacity-40 disabled:cursor-not-allowed text-sm font-medium
 active:scale-95 transition-all cursor-pointer"
 >
 Встать в очередь (ENQUEUE)
 </button>

 <button
 onClick={dequeue}
 disabled={isServing || queue.length === 0}
 className="flex-1 min-w-[150px] bg-gradient-to-r from-indigo-500 to-indigo-600
 text-white opacity-40 disabled:cursor-not-allowed text-sm font-medium
 active:scale-95 transition-all cursor-pointer flex-shrink-0"
 >
 Налить суп первому (DEQUEUE)
 </button>

 <button
 onClick={reset}
 className="px-5 py-3.5 rounded-2xl bg-slate-800 text-white font-medium
 cursor-pointer border border-slate-700"
 >
 Сброс
 </button>
 </div>

```

```

 <span className="text-[8px] font-mono t
 </div>

 {/* СПИСОК ОЧЕРЕДИ */}
 <div className="flex-1 flex items-end gap
 {displayQueue.length === 0 ? {
 <div className="flex-1 flex flex-col
 </
 <p className="text-xs font-bold fon
 <p className="text-[10px]">Все сыты
 </div>
 } : {
 displayQueue.map(({customer, idx}) => {
 const isHead = idx === 0;
 const isTail = idx === displayQueue
 return (
 <div
 key={customer.id}
 className={`
 flex flex-col items-center ga
 ${customer.animState === 'in'
 ${customer.animState === 'eat
 ${customer.animState === 'out
 `}
 >
 {/* Облачко мыслей */}
 <div className={`
 absolute -top-16 left-1/2 -tr
 8 text-center text-slate-200 leading-tight
 ${isHead ? 'opacity-100 borde
 `}>
 <div className="font-bold tex
 {customer.bubbleText}
 <div className="absolute -bot
 it rotate-45"></div>
 </div>

 {/* Персонаж */}

```



```

 order-emerald-800/80 text-[9px] font-mono">
 <Sparkles className="w-3.5 h-3.5 animate-pu
 СЫТЫ
 </div>

 <div className="flex-1 flex flex-col justify-c
 {servedHistory.length === 0 ? (
 <div className="flex-1 flex flex-col item

 <p className="text-xs font-bold font-mo
 <p className="text-[10px] mt-1">Здесь 6
 </div>
) : (
 <div className="flex flex-col gap-1.5 w-f
 {servedHistory.map((item, idx) => (
 <div
 key={idx}
 className="w-full py-1.5 px-3 round
 0 flex items-center gap-2 text-[11px] text-r
 >
 {item}
 </div>
))}
 </div>
)}
 </div>

 {/* Пол */}
 <div className="w-full h-3 bg-gradient-to-r f
 </div>
</div>

{/* ЛОГ ДЕЙСТВИЙ */}
<div className="bg-slate-900 border border-slate-
 <div className="text-[10px] font-mono text-slate
 {log.map((entry, idx) => (
 <div
 key={idx}

```

```

const [newWord, setNewWord] = useState('');
const [nodes, setNodes] = useState<{ [key: number]: T >({});
const [simulationStep, setSimulationStep] = useState<number>(0);
const [bfsQueue, setBfsQueue] = useState<number[]>([]);
const [currentNode, setCurrentNode] = useState<number>(0);
const [speechText, setSpeechText] = useState<string>('Привет! Я Эхо-Лосось Сеня! Введи слова в словарь и поиграем в эхо-буль!');
const [isTalking, setIsTalking] = useState<boolean>(false);
const [isBlinking, setIsBlinking] = useState<boolean>(false);

/* — связь с Python-компилятором (PAGE_SYNC["aho-compilers"])
 words/patterns — набор образцов: бор ПЕРЕСТРАИВАЕТСЯ
 state — текущая вершина автомата, step — шаг BFS, queue — очередь BFS
const chapterId = useContext(VizChapterContext);
const runtime = useVizRuntime();
const vars = runtime?.variables;

useEffect(() => {
 if (chapterId) emitVizDemo(chapterId, "aho");
}, [chapterId]);

const liveWordsRaw = vars ? vizArray(vars.words) ?? vizArray('') : vizArray('');
const liveWordsSig = useMemo(() => (liveWordsRaw ? JSON.stringify(liveWordsRaw) : ''));
const liveState = vars ? vizNumber(vars.state) ?? vizNumber(0) : vizNumber(0);
const liveStep = vars ? vizNumber(vars.step) ?? vizNumber(0) : vizNumber(0);
const liveQueue = vars ? vizArray(vars.queue) : null;
const liveNodes = vars ? vizNumber(vars.nodes) : null;
const liveFail = vars ? vizRecord(vars.fail) : null;

// набор образцов из кода перестраивает бор целиком
useEffect(() => {
 if (!liveWordsSig) return;
 const parsed = (JSON.parse(liveWordsSig) as unknown[]).filter((w) => Boolean(w)).map((w) => w.toUpperCase().replace(/[^A-ZА-Я]/g, ''));

```

```

useEffect(() => {
 setIsTalking(true);
 const timer = setTimeout(() => setIsTalking(false),
 return () => clearTimeout(timer);
}, [speechText]);

// Построение начального бора (Trie) без fail-ссылок
const buildInitialTrie = (wordList: string[]) => {
 const newNodes: { [key: number]: TrieNode } = {
 0: { id: 0, char: 'ROOT', parent: 0, children: {} }
 };
 let count = 0;

 wordList.forEach((word) => {
 let curr = 0;
 for (let i = 0; i < word.length; i++) {
 const c = word[i].toUpperCase();
 if (newNodes[curr].children[c] === undefined) {
 count++;
 newNodes[curr].children[c] = count;
 newNodes[count] = {
 id: count,
 char: c,
 parent: curr,
 children: {},
 fail: 0,
 term_link: 0,
 is_terminal: false,
 words: [],
 depth: newNodes[curr].depth + 1
 };
 }
 curr = newNodes[curr].children[c];
 }
 if (!newNodes[curr].words.includes(word)) {
 newNodes[curr].words.push(word);
 newNodes[curr].is_terminal = true;
 }
 });
}

```

```

useEffect(() => {
 buildInitialTrie(words);
}, []);

const handleAddWord = (e: React.FormEvent) => {
 e.preventDefault();
 const cleanWord = newWord.trim().toUpperCase().replace(' ', '');
 if (!cleanWord) return;
 if (words.includes(cleanWord)) {
 setSpeechText(`Слово «${cleanWord}» уже есть в слове`);
 return;
 }
 const updated = [...words, cleanWord];
 setWords(updated);
 setNewWord('');
 buildInitialTrie(updated);
};

const removeWord = (wordToRemove: string) => {
 if (words.length <= 1) {
 setSpeechText('Оставь хотя бы одно слово, иначе мы не сможем продолжить');
 return;
 }
 const updated = words.filter((w) => w !== wordToRemove);
 setWords(updated);
 buildInitialTrie(updated);
};

// Занульск BFS
const startBFS = () => {
 const rootChildren = Object.values(nodes[0].children);
 const queue = [...rootChildren];
 const updatedNodes = { ...nodes };
 rootChildren.forEach((childId) => {
 updatedNodes[childId].fail = 0;
 });

 setNodes(updatedNodes);
};

```

```

 // Рассчитываем терминальную ссылку (term_link)
 if (updatedNodes[failTarget].is_terminal) {
 updatedNodes[u].term_link = failTarget;
 } else {
 updatedNodes[u].term_link = updatedNodes[failTa
 }

 logs += `Для #${u} ('${char}') fail = #${failTarg
 }

 if (childrenIds.length === 0) {
 logs += 'У этой вершины нет детей, просто переход
 }

 setNodes(updatedNodes);
 setBfsQueue(newQueue);
 setCurrentNode(r);
 setSpeechText(logs);
};

return (
 <div className="bg-slate-800/90 rounded-2xl p-6 bor
 {/* Шапка виджета */}
 <div className="flex flex-wrap items-center justifi
 <div className="flex items-center space-x-3">

 <div>
 <h4 className="text-2xl font-bold text-white
 Интерактивный конструктор: Говорящий Эхо-
 </h4>
 <p className="text-sm text-slate-400">
 Построение дерева и расчет суффиксных ссы
 </p>
 </div>
 </div>
 <div className="flex items-center gap-2">
 <button
 onClick={() => buildInitialTrie(words)}

```

```

 fill="#f87171"
 className="origin-[120px_115px] animate
 />

 {/* Глаз (с морганием) */}
 <circle cx="145" cy="85" r="12" fill="#ff
 {isBlinking ? (
 <line x1="133" y1="85" x2="157" y2="85"
) : (
 <>
 <circle cx="148" cy="85" r="6" fill="
 <circle cx="151" cy="82" r="2" fill="
 </>
)}

 {/* Рот (анимация разговора) */}
 {isTalking ? (
 <path d="M 165 105 Q 155 120 165 125 Q
) : (
 <path d="M 165 110 Q 155 115 168 118" s
)}

 {/* Улыбка / жабры */}
 <path d="M 130 80 Q 125 100 130 120" stro
 <path d="M 120 85 Q 115 100 120 115" stro
</svg>

 {/* Декоративные пузыри */}
 <div className="absolute -top-1 right-4 tex
 <div className="absolute top-8 right-1 text
</div>
 <span className="mt-3 bg-rose-600/30 border b
 Эхо-Лосось Сеня

</div>

 {/* Пузырь с текстом (Баббл) */}
 <div className="md:col-span-2 bg-slate-800 bord

```

```


 </button>
 })
 {simulationStep === 2 && (
 <button
 onClick={() => buildInitialTrie(words)}
 className="bg-indigo-600 hover:bg-indigo-500"
 transition-transform active:scale-95"
 >
 <RotateCcw className="w-4 h-4" /> Перезапустить
 </button>
)}
</div>
</div>
</div>
</div>

```

```

{/* SVG Canvas for Automaton Tree */}
<div className="mb-8 bg-slate-900 border border-slate-800" >
 <h5 className="text-white font-bold mb-3 flex items-center">
 Дерево и Суффиксные (fail) ссылки
 </h5>

```

```

 (() => {
 let maxX = 0;
 let maxY = 0;
 Object.values(nodes).forEach(n => {
 if (n.x && n.x > maxX) maxX = n.x;
 if (n.y && n.y > maxY) maxY = n.y;
 });
 const svgWidth = Math.max(maxX + 80, 400);
 const svgHeight = Math.max(maxY + 60, 200);

 return (
 <svg viewBox={`0 0 ${svgWidth} ${svgHeight}`} >
 <defs>
 <marker id="arrowhead" markerWidth="6" markerHeight="6"
 <polygone points="0 0, 6 3, 0 6" fill="white" />

```

```

 { /* Draw Fail Links */
 {simulationStep > 0 && Object.values(nodes)
 if (node.id === 0 || node.fail === 0) return
 }
 // Wait, all nodes get fail link. Let's draw it
 // To avoid too many lines, let's draw it at step
 // To avoid too many lines, let's draw it at step
 const target = nodes[node.fail];
 if (!target || !node.x || !node.y || !target.x || !target.y) return

 // Simple bezier curve for fail link
 const isCurrent = currentNode === node.id

 return (
 <path
 key={`fail-${node.id}`}
 d={`M ${node.x} ${node.y} Q ${node.x + 10} ${node.y + 10} ${node.x + 16} ${node.y + 16}`}
 fill="none"
 stroke={isCurrent ? '#f43f5e' : 'rgb(128, 128, 128)'}
 strokeWidth={isCurrent ? 2 : 1.5}
 strokeDasharray="4 4"
 markerEnd="url(#fail-arrow)"
 />
);
 })}

 { /* Draw Nodes */
 Object.values(nodes).map(node => {
 if (!node.x || !node.y) return null;
 const isRoot = node.id === 0;
 const isCurrent = currentNode === node.id;
 const isInQueue = bfsQueue.includes(node.id);

 let fill = '#1e293b';
 let stroke = '#475569';

 if (isCurrent) {

```



```

 </text>
 })
 </g>
);
 }}}
</svg>
);
}}({})
</div>

{/* Управление словарем */}
<div className="grid grid-cols-1 lg:grid-cols-3 gap-4">
 <div className="bg-slate-900/50 p-5 rounded-xl">
 <div>
 <h5 className="text-white font-bold mb-3">
 Словарь (Паттерны)
 </h5>
 <div className="flex flex-wrap gap-2 mb-4">
 {words.map((w) => {
 <span
 key={w}
 className="bg-slate-800 border border-slate-700
 px-2 py-1 gap-1 group"
 >
 {w}
 <button
 onClick={() => removeWord(w)}
 className="text-slate-500 hover:text-white"
 title="Удалить"
 >
 ✕
 </button>

)}}
 </div>
 </div>

 <form onSubmit={handleAddWord} className="flex

```

```

 <th className="py-2 px-3">term_link</th>
 <th className="py-2 px-3">Дети</th>
 <th className="py-2 px-3">Слова (dict)
 </tr>
 </thead>
 <tbody className="divide-y divide-slate-800">
 {Object.values(nodes).map((node) => {
 const isCurrent = currentNode === node
 const isInQueue = bfsQueue.includes(node)

 return (
 <tr
 key={node.id}
 className={`transition-colors ${
 isCurrent
 ? 'bg-blue-900/40 font-bold text-white'
 : isInQueue
 ? 'bg-slate-800/80'
 : 'hover:bg-slate-800/40'
 }`}
 >
 <td className="py-2.5 px-3 flex items-center">
 <span
 className={`w-2 h-2 rounded-full ${
 node.id === 0
 ? 'bg-purple-500'
 : isCurrent
 ? 'bg-blue-400 animate-pulse'
 : isInQueue
 ? 'bg-amber-400'
 : 'bg-slate-600'
 }`}
 >
 #{node.id}
 {node.id === 0 && root}
 </td>
 <td className="py-2.5 px-3">


```

```


 }}
 </td>
 </tr>
);
 }}}
</tbody>
</table>
</div>
</div>
</div>

```

```

/* что сейчас сообщил компилятор */
<div className="mt-4 flex flex-wrap items-center" >
 <span className="text-[10px] uppercase tracking
 <Terminal className="w-3.5 h-3.5" /> из компи

 {compilerLinked ? (
 <>
 {liveWordsSig && <span className="text-[11px]
 y-1">words = {words.join(", ")}}
 {liveNodes !== null && <span className="tex
 x-2 py-1">вершин в бопе = {liveNodes}}
 {liveState !== null && <span className="tex
 -md px-2 py-1">state = {liveState}}
 {liveStep !== null && <span className="text
 -2 py-1">шаг BFS = {liveStep}}
 {liveQueue && <span className="text-[11px]
 -1">queue = {JSON.stringify(liveQueue).slice
 </>
) : (

 открой панель Python: массив <code classNam
 nt-mono text-cyan-300">state</code>/<code c

)}
</div>
</div>

```

```
}
```

```
// ===== STEP GENERATORS =====
```

```
// 1. Build steps (recursive top-down)
```

```
function genBuildSteps(arr: number[]): Step[] {
```

```
 const n = arr.length;
```

```
 const steps: Step[] = [];
```

```
 let id = 0;
```

```
 const tree: Record<number, number> = {};
```

```
 const CODE = [
```

```
 "build(v, l, r) {", // 1
```

```
 " if (l === r) {", // 2
```

```
 " tree[v] = A[l]; return", // 3
```

```
 " }", // 4
```

```
 " mid = (l + r) >> 1", // 5
```

```
 " build(2v, l, mid)", // 6
```

```
 " build(2v+1, mid+1, r)", // 7
```

```
 " tree[v] = tree[2v]+tree[2v+1]", // 8
```

```
 "}", // 9
```

```
];
```

```
void CODE;
```

```
function go(v: number, l: number, r: number) {
```

```
 if (l === r) {
```

```
 steps.push({ id: id++, desc: `build(${v}, ${l}, $
```

```
 , highlightNodes: [v], arrayHighlight: [l, r] } }
```

```
 tree[v] = arr[l];
```

```
 steps.push({ id: id++, desc: `tree[${v}] = ${arr[
```

```
 l, r] } });
```

```
 return;
```

```
 }
```

```
 const m = (l + r) >> 1;
```

```
 steps.push({ id: id++, desc: `build(${v}, ${l}, ${r
```

```
 { ...tree }, highlightNodes: [v], arrayHighlight:
```

```
 go(2 * v, l, m);
```

```
 steps.push({ id: id++, desc: `Вернулись в build(${v
```

```

 return res;
 }
 const ans = go(1, 0, n - 1, qL, qR);
 steps.push({ id: id++, desc: `Запрос sum([${qL}..${qR}])`,
 highlightNodes: [1], arrayHighlight: [qL, qR], result: ans });
 return steps;
}

```

// 3. Bottom-Up Query steps (iterative on implicit tree)

function genQueryBottomUpSteps(arr: number[], qL: number, qR: number): Step[] {

```

 const n = arr.length;
 // Build iterative tree: leaves at [n..2n-1]
 const tree: Record<number, number> = {};
 for (let i = 0; i < n; i++) tree[n + i] = arr[i];
 for (let i = n - 1; i > 0; --i) tree[i] = (tree[2 * i] + tree[2 * i + 1]) / 2;

```

```

 const steps: Step[] = [];
 let id = 0;
 let l = qL + n;
 let r = qR + n + 1; // half-open [l, r)
 let res = 0;

```

```

 steps.push({ id: id++, desc: `Старт: l = qL + n = ${qL + n}, r = qR + n + 1 = ${qR + n + 1}`,
 codeLine: 3, treeState: tree, highlightNodes: [l, r] });

```

```

 while (l < r) {
 const highlights: number[] = [];
 if (l & 1) {
 res += tree[l] ?? 0;
 steps.push({ id: id++, desc: `l=${l} нечётный → l`,
 codeLine: 4, treeState: tree, highlightNodes: [l], result: res });
 highlights.push(l);
 l++;
 }
 if (r & 1) {
 r--;
 res += tree[r] ?? 0;
 steps.push({ id: id++, desc: `r=${r + 1} нечётный → r`,
 codeLine: 5, treeState: tree, highlightNodes: [r + 1], result: res });
 highlights.push(r + 1);
 }
 l = (l + 1) / 2;
 r = (r - 1) / 2 + 1;
 treeState = tree;
 highlights.push(l, r);
 steps.push({ id: id++, desc: `Сумма = ${res}`, codeLine: 5, treeState: tree, highlightNodes: [1], result: res });
 }
 return steps;
}

```

```

"query(v, l, r, ql, qr) {",
" if (ql > r || qr < l) return 0;",
" // -----",
" if (ql <= l && r <= qr) return tree[v];",
" // -----",
" mid = (l + r) >> 1;",
" // спускаемся в обоих детей",
" return query(left) + query(right);",
"}"
];

const QUERY_BU_CODE = [
 "queryBU(ql, qr) {",
 " res = 0;",
 " l = ql + n; r = qr + n + 1;",
 " // -----",
 " while (l < r) {",
 " if (l & 1) res += tree[l++];",
 " // -----",
 " if (r & 1) res += tree[--r];",
 " l >>= 1; r >>= 1;",
 " }",
 " return res; }"
];

// ===== PLAYER COMPONENT =====
function Player({ steps, color }: { steps: Step[]; color: string }) {
 const [idx, setIdx] = useState(0);
 const [playing, setPlaying] = useState(false);
 const [speed, setSpeed] = useState(700);

 useEffect(() => { setIdx(0); setPlaying(false); }, [steps]);

 useEffect(() => {
 if (!playing || idx >= steps.length - 1) { if (idx < steps.length - 1) {
 const t = setTimeout(() => setIdx(i => i + 1), speed);
 return () => clearTimeout(t);
 }
 }, [playing, idx, steps, speed]);

 const step = steps[idx] || steps[0];

```

```

 }

 let cls = 'bg-slate-800 border-slate-700 text-slate
 if (isHigh) cls = `${clr.ring} text-white ring-2 sc
 else if (isChild) cls = `${clr.childRing} text-ambe
 else cls = `${clr.filled} text-slate-200`;

 return (
 <div key={nd.v} className="flex flex-col items-ce
 <div className={`flex flex-col items-center jus
] ${cls}`}>
 <div className="flex items-center gap-1 text-
 v{nd.v}
 {isCompilerActive && <span className="bg-am
 </div>
 <span className="text-[9px] font-bold text-am
 <span className="text-xs font-black font-mono
 {nd.l === nd.r ? `A[${nd.l}] = ${displayVal

 </div>
 {(nd.left || nd.right) && (
 <div className="flex flex-col items-center mt
 <div className="w-px h-2 bg-slate-700" />
 <div className="flex justify-around w-full l
 {nd.left && renderNode(nd.left)}
 {nd.right && renderNode(nd.right)}
 </div>
 </div>
 </div>
)}
 </div>
);
 }, [step, clr, fullTree, liveV, liveTree]);

 return (
 <div className={`rounded-2xl border overflow-hidden
 ? 'border-emerald-500/30' : 'border-amber-500/30'
 /* Header */
 <div className={`px-4 py-2.5 flex items-center ju

```





```

 const parsed = customInput.split(',').map(s => pars
 if (parsed.length >= 2 && parsed.length <= 16) {
 setArr(parsed);
 }
 };

const randomize = () => {
 const size = arr.length;
 const a = Array.from({ length: size }, () => Math.f
 setArr(a);
 setCustomInput(a.join(', '));
};

const buildSteps = useMemo(() => genBuildSteps(arr),
const queryTDSteps = useMemo(() => genQueryTopDownSte
const queryBUSteps = useMemo(() => genQueryBottomUpSt

const totalSum = arr.reduce((a, b) => a + b, 0);

return (
 <div className="bg-slate-900 rounded-2xl border bor
 /* ---- TOP BAR: array editor + query range ----
 <div className="mb-6 space-y-4">
 <div className="flex flex-col lg:flex-row gap-4
 /* Array input */
 <form onSubmit={handleApply} className="flex-
 <div className="flex items-center justify-b
 <label className="text-[10px] font-bold up
 label>
 <button type="button" onClick={randomize}
 "><RefreshCw className="w-3.5 h-3.5" /></bu
 </div>
 <div className="flex gap-2">
 <input
 value={customInput}
 onChange={e => setCustomInput(e.target.
 className="flex-1 bg-slate-900 border b
 one focus:border-indigo-500"

```

```

 value={qR}
 onChange={e => setQR(Math.min(Number(
 className="w-14 bg-slate-900 border b
 ocus:border-indigo-500"
 />
 </div>
 </div>
 <div className="text-[10px] text-slate-500 p
 Запрос: sum(A[{qL}..{qR}]) = {arr.slice(q
 </div>
</div>
</div>
</div>

{/* ---- TABS ---- */}
<div className="flex flex-wrap items-center gap-1
 <button onClick={() => setView('build')} classN
 -colors ${view === 'build' ? 'border-indigo-5
 -slate-200'}`}>
 <Hammer className="w-3.5 h-3.5" /> Построить
 </button>
 <button onClick={() => setView('queries')} clas
 on-colors ${view === 'queries' ? 'border-emerg
 er:text-slate-200'}`}>
 <Search className="w-3.5 h-3.5" /> Запрос: св
 </button>
 <button onClick={() => setView('theory')} class
 n-colors ${view === 'theory' ? 'border-blue-5
 te-200'}`}>
 <Info className="w-3.5 h-3.5" /> Почему / Спра
 </button>
</div>

{/* ---- TAB CONTENT ---- */}
{view === 'build' && (
 <SimPanel
 key={`build-${arr.join('-')}`}
 title="Построение дерева (рекурсия)"

```

```

 Каждый узел берёт <code className="bg-slate-900 p-4 rounded-lg">
 к получает <code className="bg-slate-900 px-1 rounded font-mono text-emerald-500">
 -slate-900 px-1 rounded font-mono text-emerald-500">
 евая часть забирает на 1 больше (округление)
 </p>
 <div className="bg-slate-900 p-4 rounded-lg">
 <div className="text-slate-400">Корень: L
 <div className="text-slate-300">mid = l(0
 <div className="text-indigo-300">→ Левый:
 <div className="text-emerald-300">→ Правый:
 h - l) >> l)} эл.)</div>
 <div className="text-slate-500 mt-2">Всего:
 } внутренних.</div>
 </div>
</div>

{/* Comparison cards */}
<div className="grid grid-cols-1 xl:grid-cols-2">
 <div className="bg-slate-950 p-5 rounded-xl">
 <div className="absolute -top-3 left-4 bg-emerald-500 border border-emerald-500 shadow-md">
 Запрос Сверху-вниз (Рекурсия)
 </div>
 <p className="text-xs text-slate-300 mb-3">
 Начинаем с корня. Если отрезок узла полн
 наче спускаемся в обоих детей.
 </p>
 <div className="space-y-1.5 text-xs">
 <div className="flex justify-between border-b border-slate-700">
 0(
 <div className="flex justify-between border-b border-slate-700">

 <div className="flex justify-between">✓ легко</div>
 </div>
 </div>
 <div className="bg-slate-950 p-5 rounded-xl">
 <div className="absolute -top-3 left-4 bg-emerald-500 border border-emerald-500 shadow-md">

```

```
import React, { useMemo, useState } from "react";
import { ChevronRight, Compass, Search, Sparkles, X } from "react-bootstrap-icons";
import { chapters, SECTIONS, sectionOf } from "../data/chapters";
import { VIZ_REGISTRY } from "../vizRegistry";

interface SidebarProps {
 selectedChapterId: string;
 setSelectedChapterId: (id: string) => void;
 /** Мобильный режим: список живёт в выдвижной панели */
 onNavigate?: () => void;
 /** Узкий режим: номер билета и название в одну строку */
 compact?: boolean;
}

export const Sidebar: React.FC<SidebarProps> = ({ selectedChapterId, setSelectedChapterId, onNavigate, compact }) => {
 const [query, setQuery] = useState("");

 /**
 * Группы – это разделы пособия (SECTIONS), а не случайные
 * связанные темы стоят рядом, порядок разделов не зависит от
 * страница встретилась первой.
 */
 const groups = useMemo(() => {
 const q = query.trim().toLowerCase();
 const filtered = q ? chapters.filter((c) => c.title.toLowerCase().includes(q)) : chapters;
 return SECTIONS.map((section) => ({
 section,
 items: filtered.filter((c) => sectionOf(c.id).id === section.id).filter((g) => g.items.length > 0);
 })), [query]);

 return (
 <aside className="w-full bg-slate-900/80 lg:bg-transparent lg:rounded-r-2xl" >
 <div className={compact ? "p-2 pb-1.5 shrink-0" : "p-2 pb-1.5"} >
 {compact && (
 <h3 className="text-[10px] font-bold text-slate-100" >
 <Compass className="w-3.5 h-3.5 text-indigo-400" />
 <div className="flex items-center gap-1" >
 Группы
 —>
 </div>
 </h3>
)}
 </div>
 </aside>
);
};
```

```

 })
 </div>
 <div className="space-y-1">
 {items.map((chapter) => {
 const isSelected = chapter.id === selectedChapterId
 const hasViz = Boolean(VIZ_REGISTRY[chapter.id])
 // «18–20. Графы. Основное дерево...» → «18–20. Графы»
 const num = chapter.title.match(/^(\d+)/)[1]
 const label = num ? chapter.title.slice(0, num) : chapter.title
 return (
 <button
 key={chapter.id}
 onClick={() => {
 setSelectedChapterId(chapter.id)
 onNavigate?.()
 window.scrollTo({ top: 0, behavior: 'smooth' })
 }}
 title={chapter.title}
 aria-current={isSelected ? "page" : undefined}
 className={`w-full text-left rounded-md px-2 py-1.5`} ${
 isSelected
 ? "bg-indigo-600/15 border-indigo-600/20"
 : "bg-slate-800/40 border-transparent"
 }`}
 >

 {num ? num[1] : ""}

 <span className={`font-semibold leading-tight`} ${
 hasViz && (

 «визуализация»

)
 }`}

 {!compact && (

```

```
import React, { useState } from 'react';
import { Layers, AlignLeft, Award, Plus, ArrowRight, Ar

interface Plate {
 id: string;
 name: string;
 icon: string;
}

interface Person {
 id: string;
 name: string;
 icon: string;
}

interface Hypothesis {
 id: string;
 text: string;
 probability: number;
}

export const Simulator: React.FC = () => {
 const [activeTab, setActiveTab] = useState<'stack' |

 // Stack state
 const [stack, setStack] = useState<Plate[]>([
 { id: '1', name: 'Тарелка из-под пасты', icon: ' '
 { id: '2', name: 'Тарелка из-под пиццы', icon: ' '
 { id: '3', name: 'Блюдец от торта', icon: ' ' },
]));
 const [popMessage, setPopMessage] = useState<string |

 // Queue state
 const [queue, setQueue] = useState<Person[]>([
 { id: '1', name: 'Студент Вася', icon: ' ' },
 { id: '2', name: 'Голодный кодер', icon: ' ' },
]),
```

```

 const topPlate = stack[stack.length - 1];
 setStack(prev => prev.slice(0, prev.length - 1));
 setPopMessage(`  Вымыта верхняя тарелка (LIFO): ${topPlate}`);
 });

const resetStack = () => {
 setStack([
 { id: '1', name: 'Тарелка из-под пасты', icon: '🍝' },
 { id: '2', name: 'Тарелка из-под пиццы', icon: '🍕' },
 { id: '3', name: 'Блюдецце от торта', icon: '🍰' },
]);
 setPopMessage(" Стопка тарелок восстановлена.");
};

// Queue handlers
const addPerson = () => {
 const presets = [
 { name: 'Любитель борща', icon: '🥘' },
 { name: 'Курьер доставки', icon: '📦' },
 { name: 'Усталый админ', icon: '🤖' },
 { name: 'Турист с рюкзаком', icon: '🎒' },
];
 const randomPreset = presets[Math.floor(Math.random() * presets.length)];
 const newPerson = {
 id: Date.now().toString(),
 name: randomPreset.name,
 icon: randomPreset.icon
 };
 setQueue(prev => [...prev, newPerson]);
 setDequeueMessage(`  Встал в конец очереди: ${newPerson.name} ${newPerson.icon}`);
};

const servePerson = () => {
 if (queue.length === 0) {
 setDequeueMessage("⚠ Очередь пуста! Суп ждет новыа");
 return;
 }
 const firstPerson = queue[0];

```

```

 }
 const best = heap[maxIdx];
 setHeap(prev => prev.filter((_, idx) => idx !== maxIdx));
 setHeapMessage(`    Выбран самый "тяжелый" кандидат: ${best}`);
 };

const resetHeap = () => {
 setHeap([
 { id: '1', text: 'Кандидат: "Привет, я искусственный интеллект"' },
 { id: '2', text: 'Кандидат: "Привет, я испек вкусный хлеб"' },
 { id: '3', text: 'Кандидат: "Привет, я испанский язык"' },
]);
 setHeapMessage(" Гипотезы восстановлены.");
};

// Sorted heap for display (Max-Heap property view)
const sortedHeap = [...heap].sort((a, b) => b.probability - a.probability);

return (
 <div className="bg-slate-900/90 border border-slate-800 p-6">
 <div className="border-b border-slate-800 pb-6 mb-6">
 <h2 className="text-2xl font-bold text-transparent bg-clip-text bg-gradient-to-r from-blue-400 to-blue-600">
 Интерактивный симулятор структур данных
 </h2>
 <p className="text-slate-400 text-sm mt-2">
 Проверь работу принципов LIFO, FIFO и Приоритетной очереди
 </p>
 </div>

 { /* Tabs */ }
 <div className="grid grid-cols-1 md:grid-cols-3 gap-4">
 <button
 onClick={() => setActiveTab('stack')}
 className={`flex items-center justify-between gap-2 px-4 py-2 rounded ${
 activeTab === 'stack'
 ? 'bg-blue-600/20 border-blue-500 text-blue-400'
 : 'bg-slate-800/60 border-slate-700/60 text-slate-400'
 }`}
 >
 Стек (LIFO)
 </button>
 <button
 onClick={() => setActiveTab('queue')}
 className={`flex items-center justify-between gap-2 px-4 py-2 rounded ${
 activeTab === 'queue'
 ? 'bg-blue-600/20 border-blue-500 text-blue-400'
 : 'bg-slate-800/60 border-slate-700/60 text-slate-400'
 }`}
 >
 Очередь (FIFO)
 </button>
 <button
 onClick={() => setActiveTab('priority')}
 className={`flex items-center justify-between gap-2 px-4 py-2 rounded ${
 activeTab === 'priority'
 ? 'bg-blue-600/20 border-blue-500 text-blue-400'
 : 'bg-slate-800/60 border-slate-700/60 text-slate-400'
 }`}
 >
 Приоритетная очередь
 </button>
 </div>
 </div>
);

```



```

 ? 'bg-emerald-600/20 border-emerald-500 te
 : 'bg-slate-800/60 border-slate-700/60 te
 }`}
 >
 <div className="flex items-center gap-3">
 <Award className={`w-5 h-5 ${activeTab ===
 <div className="text-left">
 <div className="font-bold text-sm text-wh
 <div className="text-xs opacity-80">Приор
 </div>
 </div>
 <span className="text-xs bg-emerald-500/20 te
 Beam Search

</button>
</div>

{/* STACK TAB */}
{activeTab === 'stack' && {
 <div className="space-y-6">
 <div className="bg-slate-800/80 p-5 rounded-x
 tify-between gap-4">
 <div>
 <h3 className="text-lg font-bold text-whi
 Симуляция стопки тарелок
 <span className="text-xs font-mono bg-b
 </h3>
 <p className="text-slate-400 text-xs mt-1
 Новые тарелки ставятся наверх. Мыть мож
 </p>
 </div>
 <div className="flex flex-wrap items-center
 <button
 onClick={addPlate}
 className="flex-1 md:flex-none flex ite
 rounded-xl text-sm font-bold shadow-lg tra
 >
 <Plus className="w-4 h-4" /> Поставить

```

```

 <div className="absolute top-0 text-xs
full flex items-center gap-1">
 ВЕРШИНА СТЕКА (TOP)
 <ArrowDown className="w-3.5 h-3.5 ani
 </div>

 {/* Reversed map so top of stack is vis
 {stack.slice().reverse().map((plate, in
 const isTop = index === 0;
 return (
 <div
 key={plate.id}
 className={`w-full flex items-cen
 isTop
 ? 'bg-gradient-to-r from-blue
le-102'
 : 'bg-slate-900/90 border-sla
 }`}
 >
 <div className="flex items-center
 {pla
 <span className={`font-bold tex
 {plate.name}

 </div>
 <span className={`text-xs font-mo
 isTop ? 'bg-blue-500 text-white
 }`}>
 {isTop ? 'Готово к помывке' : `

 </div>
);
 }}}
</div>
))
<div className="mt-6 text-xs text-slate-500
 ПОЛОЖИЛ ПОСЛЕДНЕЙ → ПОМЫЛ ПЕРВОЙ (LIFO) •
</div>

```

```

 tion-colors cursor-pointer"
 >
 <RotateCcw className="w-5 h-5" />
 </button>
 </div>
 </div>

 {dequeueMessage && {
 <div className="bg-indigo-950/60 border border-
 p-3 animate-fade-in">
 <span className="inline-block w-2 h-2 round
 {dequeueMessage}
 </div>
 }}

 {/* Visualization */}
 <div className="bg-slate-950/60 p-6 rounded-2
 overflow-x-auto">
 {queue.length === 0 ? {
 <div className="text-center py-12 text-sl
 <div className="text-5xl mb-3"> </div>
 <p className="text-base font-bold">Очер
 <p className="text-xs mt-1">Добавьте ро
 </div>
 } : {
 <div className="w-full flex items-center
 <div className="flex flex-col items-cen
 300">

 <span className="text-xs font-bold font
 </div>

 <div className="text-indigo-500 flex it
 <ArrowRight className="w-6 h-6 animat
 </div>

 {queue.map((person, index) => {
 const isFirst = index === 0;

```

```

 </div>
 </div>
 </div>
 })

 { /* HEAP TAB */ }
 { activeTab === 'heap' && {
 <div className="space-y-6">
 <div className="bg-slate-800/80 p-5 rounded-x
 tify-between gap-4">
 <div>
 <h3 className="text-lg font-bold text-whi
 Приоритетная очередь / Куча</sp
 <span className="text-xs font-mono bg-e
 x-Heap
 </h3>
 <p className="text-slate-400 text-xs mt-1
 Здесь не важно, кто пришел первым. Важн
 </p>
 </div>
 <div className="flex items-center gap-3 w-f
 <button
 onClick={popHeap}
 className="flex-1 md:flex-none flex ite
 py-2.5 rounded-xl text-sm font-bold shadow-
 >
 Взять кандидата с макс. вероятностью
 </button>
 <button
 onClick={resetHeap}
 title="Сбросить"
 className="p-2.5 bg-slate-800 hover:bg-
 tion-colors cursor-pointer"
 >
 <RotateCcw className="w-5 h-5" />
 </button>
 </div>
 </div>
 </div>
 </div>
 }
}

```

```

 </button>
 </div>
 </form>

 {heapMessage && {
 <div className="bg-emerald-950/60 border border-1
 gap-3 animate-fade-in">
 <span className="inline-block w-2 h-2 rounded
 {heapMessage}
 </div>
 }}

 {/* Visualization */}
 <div className="bg-slate-950/60 p-6 rounded-2
 >
 {sortedHeap.length === 0 ? {
 <div className="text-center py-12 text-slate-400">
 <div className="text-5xl mb-3"> </div>
 <p className="text-base font-bold">Куча
 <p className="text-xs mt-1">Добавьте ги
 </div>
) : {
 <div className="w-full max-w-2xl space-y-3">
 <div className="flex items-center justify-between">
 КАНДИДАТ / ГИПОТЕЗА
 ВЕРОЯТНОСТЬ (ПРИОРИТЕТ)
 </div>

 {sortedHeap.map(({item, index}) => {
 const isTop = index === 0;
 const percent = (item.probability * 100).toFixed(
 return {
 <div
 key={item.id}
 className={`flex items-center justify-between
 isTop
 ? 'bg-gradient-to-r from-emerald-500 to-emerald-950/60
00/10 scale-[1.01]`

```

```

 }}}
 </div>
)}
 <div className="mt-6 text-xs text-slate-500">
 НЕ ВАЖНО, КТО ПРИШЕЛ ПЕРВЫМ • ВАЖЕН САМЫЙ
 </div>
 </div>
</div>
)}
</div>
);
};

```

ФАЙЛ 70/111: src/components/SplayPlayground.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

```

import React, { useContext, useEffect, useState } from "react";
import { VizChapterContext, emitVizDemo } from "../data";
import { SplayRotationsViz } from "../SplayRotationsViz";
import SplayWalkViz from "../SplayWalkViz";
import { SplayRotationSandbox } from "../SplayRotationSandbox";

```

```
/**
```

```

 * Страница splay – три вкладки по нарастанию сложности
 * 1. «Механика» – покадровый разбор Zig / Zig-Zig / Zig-Zag
 * (прицел → поворот, призраки старых позиций, обход)
 * 2. «Большое дерево» – те же случаи внутри настоящего дерева
 * поворот – кадры «прицел → отстёгиваем среднее поделённое»
 * 3. «Собери сам» – песочница: подними узел кликами, разбей
 */

```

```
type Tab = "cases" | "big" | "play";
```

```

export const SplayPlayground: React.FC<{ defaultTab?: Tab }> = ({
 defaultTab,
}) => {
 const [tab, setTab] = useState<Tab>(defaultTab);
 const chapterId = useContext(VizChapterContext);

```

```

 {tab === "cases" && <SplayRotationsViz />}
 {tab === "big" && <SplayWalkViz />}
 {tab === "play" && <SplayRotationSandbox />}

 {tab === "play" && {
 <p className="text-xs text-slate-500 mt-3">
 Порядок поворотов в песочнице сверяется с пра
 </p>
 }}
 </div>
);
};

export default SplayPlayground;

```

ФАЙЛ 71/111: src/components/SplayRotationSandbox.tsx  
 КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```

import React, { useCallback, useMemo, useState } from "
import { CheckCircle2, Dices, Eye, EyeOff, RotateCcw, U

/**
 * Песочница поворотов.
 *
 * Здесь ничего не подсказывается: дано настоящее дерев
 * поднять отмеченный узел в корень. Клик по узлу = оди
 * Порядок поворотов выбираешь сам; правильный ли он по
 * только после того, как узел окажется наверху (или по
 */

export interface TNode {
 key: number;
 left: TNode | null;
 right: TNode | null;
}

```

```

 hintCase: "Zig-Zag, затем Zig-Zig",
 target: 5,
 build: () => {
 const n5 = leaf(5);
 const n6: TNode = { key: 6, left: n5, right: leaf(7) };
 const n4: TNode = { key: 4, left: leaf(3), right: n6 };
 return { key: 8, left: n4, right: leaf(9) };
 },
},
];

/* ----- операции над деревом -----

export const findPath = (root: TNode | null, key: number): TNode[] => {
 const path: TNode[] = [];
 let cur = root;
 while (cur) {
 path.push(cur);
 if (key === cur.key) return path;
 cur = key < cur.key ? cur.left : cur.right;
 }
 return [];
};

/** Поднять узел key на один уровень (поворот с родителем)
export function rotateUp(root: TNode, key: number): TNode {
 const path = findPath(root, key);
 if (path.length < 2) return root;

 const x = path[path.length - 1];
 const p = path[path.length - 2];
 const gg = path.length >= 3 ? path[path.length - 3] : null;

 if (p.left === x) {
 p.left = x.right;
 x.right = p;
 } else {
 p.right = x.left;
 x.left = p;
 }

 if (gg) {
 gg.left = x;
 gg.right = p;
 }

 return root;
}

```



```

const walk = (n: TNode | null, depth: number) => {
 if (!n) return;
 walk(n.left, depth + 1);
 nodes.push({ key: n.key, x: order++, y: depth, depth });
 if (n.left) edges.push([n.key, n.left.key]);
 if (n.right) edges.push([n.key, n.right.key]);
 walk(n.right, depth + 1);
};
walk(root, 0);

const cols = Math.max(1, order);
const rows = Math.max(1, Math.max(...nodes.map((n) => n.depth)));
const colW = 46;
const rowH = 62;
return {
 nodes: nodes.map((n) => ({ ...n, x: n.x * colW + colW / 2, y: n.y * rowH + rowH / 2 })),
 edges,
 width: cols * colW,
 height: rows * rowH + 20,
};
}

/* ----- КОМПОНЕНТ ----- */

export const SplayRotationSandbox: React.FC = () => {
 const [presetIdx, setPresetIdx] = useState(0);
 const preset = PRESETS[presetIdx];

 const [tree, setTree] = useState<TNode>(() => preset.root);
 const [history, setHistory] = useState<TNode[]>([]);
 const [moves, setMoves] = useState<{ node: number; count: number }>({ node: 0, count: 0 });
 const [showAnswer, setShowAnswer] = useState(false);

 const target = preset.target;
 const solved = tree.key === target;

 const reset = useCallback(

```

```

 <div>
 <h4 className="text-sm sm:text-base font-bold">
 <p className="text-[12px] text-slate-400 lead">
 Клик по узлу = один поворот с его родителем

 поворотов выбираешь сам, разбор покажем в к

 </p>
 </div>
 <button
 onClick={() => reset((presetIdx + 1) % PRESETS.length)}
 className="flex items-center gap-1.5 px-3 py-1 text-sm font-medium transition-colors shrink-0"
 >
 <Dices className="w-3.5 h-3.5" /> Другое дерево
 </button>
 </div>

 <div className="flex gap-2 mb-3 overflow-x-auto no-scrollbar">
 {PRESETS.map((p, i) => {
 <button
 key={p.name}
 onClick={() => reset(i)}
 className={`px-3 py-1.5 rounded-lg text-[11px] font-medium transition-colors
 ${i === presetIdx ? "bg-indigo-600 text-white" : "bg-white text-slate-600"}
 `}
 >
 {p.name}
 </button>
 })}
 </div>

 <div className="bg-slate-900 rounded-xl border border-slate-800 p-4">
 <svg
 viewBox={`0 0 ${width} ${height}`}
 className="h-auto block mx-auto"
 style={{ minWidth: Math.min(width * 1.5, 420) }}
 role="img"
 >

```

```

 stroke={isTarget ? "#a5b4fc" : "#475568"}
 strokeWidth={2}
 />
 <text textAnchor="middle" dominantBaseline="middle">
 {n.key}
 </text>
 </g>
);
}
</svg>
</div>

<div className="flex flex-wrap items-center gap-2">
 <button
 onClick={undo}
 disabled={history.length === 0}
 className="flex items-center gap-1.5 px-3 py-1.5 text-white disabled:opacity-40 text-xs font-bold">
 <Undo2 className="w-3.5 h-3.5" /> ОТМЕНИТЬ
 </button>
 <button
 onClick={() => reset()}
 className="flex items-center gap-1.5 px-3 py-1.5 text-white font-bold transition-colors">
 <RotateCcw className="w-3.5 h-3.5" /> Заново
 </button>
 <button
 onClick={() => setShowAnswer((s) => !s)}
 className={`flex items-center gap-1.5 px-3 py-1.5 text-white ${
 showAnswer
 ? "bg-amber-600/20 border-amber-500 text-amber-500"
 : "bg-slate-800 border-slate-700 text-slate-200"
 }`}
 >
 {showAnswer ? <EyeOff className="w-3.5 h-3.5" /> : "Скрыть разбор"}
 {showAnswer ? "Скрыть разбор" : "Сдаться / показать"}
 </button>
</div>

```

```

 }}

 }}
 {wrong === 0 && moves.length > minimal && (
 <div className="opacity-80">Порядок верный,
 }}
 </div>
 }}
</div>
);
};

```

ФАЙЛ 72/111: src/components/SplayRotationsViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```

import React, { useEffect, useMemo, useRef, useState } from 'react';
import { Gauge, Pause, Play, RotateCcw, Terminal } from 'lucide-react';
import { useVizRuntime, vizNumber, vizString } from 'viz-runtime';

```

```

/**
 * Разбор трёх случаев операции Splay: Zig, Zig-Zig, Zig-Zag.
 *
 * Что сделано ради понятности:
 * * у узлов настоящие числа (20 / 40 / 60), а не абстрактные;
 * * сразу видно, что дерево остаётся деревом поиска;
 * * под деревом печатается обход слева направо: он ОДНОУРОВНЕВНЫЙ;
 * * это и есть доказательство, что поворот ничего не двигает;
 * * кадр «прицел» ничего не двигает, только подсвечивает;
 * * на кадре «результат» остаются призраки – пунктирные узлы в
 * местах и стрелки «откуда → куда»;
 * * по умолчанию анимация НЕ крутится сама: пользователь
 * идёт в своём темпе. Автопрокрутка включается кнопкой.
 */

```

```

type NodeId = "x" | "p" | "g" | "A" | "B" | "C" | "D";

```

```
} as unknown as Pick<Frame, "pos" | "edges">;
```

```
const ZIG: Case = {
 key: "zig",
 label: "Zig",
 when: "родитель x — уже корень, деда нет",
 rotations: "1 поворот",
 keys: { x: 20, p: 40 },
 inorder: ["A", "20", "B", "40", "C"],
 ranges: "A < 20 · B между 20 и 40 · C > 40",
 frames: [
 {
 ...ZIG_BEFORE,
 kind: "start",
 title: "Было: 20 висит под корнем 40",
 text: "Нам нужен ключ 20, а он на глубине 1. Деду",
 ms: RESULT_MS,
 },
 {
 ...ZIG_BEFORE,
 kind: "aim",
 title: "Прицел: крутим ребро 40 — 20",
 text: "Ничего пока не двигается. Смотри на подсве",
 focus: ["p", "x"],
 ms: AIM_MS,
 },
 {
 pos: { x: [160, 50], A: [80, 130], p: [245, 130],
 edges: [["x", "A"], ["x", "p"], ["p", "B"], ["p",
 kind: "result",
 title: "Стало: 20 — корень",
 text: "40 стало правым сыном двадцатки. Поддерев",
 ева от 40 — это одно и то же место.",
 moved: ["x", "p", "B"],
 ms: RESULT_MS,
 },
],
};
```

```

 ...ZZ_S1,
 kind: "result",
 title: "Поворот 1: 40 поднялось над 60",
 text: "40 встало в корень, 60 опустилось к нему по
 убине 1, а не 2.",
 moved: ["p", "g", "C"],
 ms: RESULT_MS,
 },
 {
 ...ZZ_S1,
 kind: "aim",
 title: "Прицел 2: ребро 40 – 20",
 text: "А теперь обычный одиночный поворот – тот же
 focus: ["p", "x"],
 ms: AIM_MS,
 },
 {
 pos: { x: [100, 45], A: [45, 118], p: [188, 118],
 edges: [["x", "A"], ["x", "p"], ["p", "B"], ["p",
 kind: "result",
 title: "Стало: 20 в корне, ветка x подтянулась",
 text: "Сравни с первым кадром: А было на глубине
 ась. Splay не «уменьшает высоту вообще», он под
 сумме запросов (амортизация).",
 moved: ["x", "p", "A", "B"],
 ms: RESULT_MS,
 },
],
};

```

/\* ----- Zig-Zag -----

```

const ZG_S0 = {
 pos: { g: [258, 40], D: [328, 112], p: [148, 112], A:
 edges: [["g", "p"], ["g", "D"], ["p", "A"], ["p", "x"]
} as unknown as Pick<Frame, "pos" | "edges">;

```

```

const ZG_S1 = {
 pos: { g: [258, 40], D: [328, 112], x: [148, 112], p:

```

```

 kind: "aim",
 title: "Прицел 2: ребро 60 – 40",
 text: "Остался одиночный поворот вокруг деда – и",
 focus: ["g", "x"],
 ms: AIM_MS,
 },
 {
 pos: { x: [180, 50], p: [90, 130], g: [275, 130],
 edges: [{"x", "p"}, {"x", "g"}, {"p", "A"}, {"p",
 kind: "result",
 title: "Стало: 20 и 60 – два сына сорока",
 text: "Дерево стало идеально симметричным: все че
 очку». ",
 moved: ["x", "p", "g", "C"],
 ms: RESULT_MS,
 },
],
};

```

```
const CASES: Case[] = [ZIG, ZIGZIG, ZIGZAG];
```

```

const COLOR: Record<string, { fill: string; stroke: str
 x: { fill: "#6366f1", stroke: "#a5b4fc" },
 p: { fill: "#0ea5e9", stroke: "#7dd3fc" },
 g: { fill: "#f59e0b", stroke: "#fcd34d" },
 sub: { fill: "#1e293b", stroke: "#475569" },
};

```

```

const ROLE_RU: Partial<Record<NodeId, string>> = {
 x: "x – ищем его",
 p: "p – родитель",
 g: "g – дед",
};

```

```
const isSubtree = (id: NodeId) => id === "A" || id ===
```

```

const Tree: React.FC<{ frame: Frame; prev: Pos; keys: C
 const { pos, edges, focus, moved } = frame;

```

```
{edges.map([a, b], i) => {
 const pa = pos[a];
 const pb = pos[b];
 if (!pa || !pb) return null;
 const hot = Boolean(focus && focus.includes(a));
 return (
 <line
 key={` ${a} - ${b} - ${i} `}
 x1={pa[0]}
 y1={pa[1]}
 x2={pb[0]}
 y2={pb[1]}
 stroke={hot ? "#fbbf24" : "#475569"}
 strokeWidth={hot ? 5 : 2}
 opacity={focus && !hot ? 0.25 : 1}
 style={{ transition: "all 1200ms cubic-bezier(0.4, 0, 0.2, 1)" }}
 />
);
}}}
```

```
{{Object.keys(pos) as NodeId[]}.map(id) => {
 const p = pos[id];
 if (!p) return null;
 const sub = isSubtree(id);
 const c = COLOR[sub ? "sub" : id];
 const r = sub ? 15 : 20;
 const hot = Boolean(focus && focus.includes(id));
 return (
 <g
 key={id}
 style={{
 transform: `translate(${p[0]}px, ${p[1]}px)`,
 transition: MOVE,
 opacity: dim(id) ? 0.28 : 1,
 }}
 >
 {hot && <circle r={r + 7} fill="none" stroke="black" style={{ stroke-width: 2, stroke-dasharray: [5, 5] }} />}
 </g>
);
}}
```



```

const liveX = vars ? vizNumber(vars.xk) ?? vizNumber(0) : 0;
const liveP = vars ? vizNumber(vars.pk) ?? vizNumber(0) : 0;
const liveG = vars ? vizNumber(vars.gk) ?? vizNumber(0) : 0;
const lastSig = useRef<string | null>(null);

useEffect(() => {
 if (!liveCase && liveStep === null) return;
 const norm = (liveCase ?? "").trim().toLowerCase();
 const idx = CASE_BY_NAME[norm] ?? caseIdx;
 const sig = `${idx}:${liveStep ?? ""}`;
 if (sig === lastSig.current) return;
 const first = lastSig.current === null;
 lastSig.current = sig;
 setPlaying(false);
 if (first || idx !== caseIdx) {
 setCaseIdx(idx); // новый случай – начинаем его
 setFrame(0);
 } else {
 // тот же случай, следующий шаг кода – пролистываем
 setFrame((f) => Math.min(CASES[idx].frames.length, f + 1));
 }
 // eslint-disable-next-line react-hooks/exhaustive-deps
}, [liveCase, liveStep]);

const active = CASES[caseIdx];
const total = active.frames.length;
const idx = Math.min(frame, total - 1);
const current = active.frames[idx];
const prevPos = active.frames[Math.max(idx - 1, 0)].pos;
const duration = useMemo(() => Math.round(current.ms - prevPos), [current]);
const last = idx === total - 1;

useEffect(() => {
 setFrame(0);
}, [caseIdx]);

useEffect(() => {
 if (!playing) return;

```

```


 Шаг {idx + 1} из {total}. {current.title}

 </div>
 <p className="text-[13px] text-slate-400 leading-5" style={{fontStyle: 'italic'}}>
 </div>

 <div className="bg-slate-900 rounded-xl border border-slate-800" style={{fontStyle: 'italic'}}>
 <Tree frame={current} prev={prevPos} keys={activeKeys} />

 {/* обход слева направо – одинаковый на всех кадрах */}
 <div className="mt-2 pt-2 border-t border-slate-800" style={{fontStyle: 'italic'}}>
 <div className="flex items-center gap-1.5 flex-wrap" style={{fontStyle: 'italic'}}>
 Обход
 {active.inorder.map((t, i) => {
 <span
 key={i}
 className={`px-1.5 py-0.5 rounded font-mono text-[10px]
 /\d/.test(t) ? "bg-emerald-500/15 text-emerald-400" : "text-slate-500"
 }`}
 >{t}
 })}
 Итого: {active.inorder.length}
 </div>
 <p className="text-[11px] text-slate-500 mt-1" style={{fontStyle: 'italic'}}>
 </div>
 </div>

 {/* индикатор автопрокрутки */}
 <div className="h-1 w-full bg-slate-800 rounded-f" style={{fontStyle: 'italic'}}>
 <div
 key={` ${caseIdx} - ${frame} - ${playing} - ${slow} `}
 className="h-full bg-indigo-500"
 style={{
 playing
 }}
 </div>
 </div>

```

```

 title="Автопрокрутка ещё медленнее"
 className={`flex items-center gap-1.5 px-3 py-3
 slow
 ? "bg-emerald-600/20 border-emerald-500 text-emerald-500"
 : "bg-slate-800 border-slate-700 text-slate-200"
 }`}
 >
 <Gauge className="w-3.5 h-3.5" /> {slow ? "0.5" : "1"}
 </button>
 <button
 onClick={() => {
 setFrame(0);
 setPlaying(false);
 }}
 className="flex items-center gap-1.5 px-3 py-3 text-xs font-bold transition-colors"
 >
 <RotateCcw className="w-3.5 h-3.5" /> Сброс
 </button>

 <div className="flex items-center gap-3 ml-auto"
 {(["x", "p", "g"] as const).map((id) =>
 active.keys[id] === undefined ? null : (
 <span key={id} className="flex items-center gap-1"
 <span className="w-3 h-3 rounded-full"
 style={{ background: active[id], border: 1px solid black }}

 {active[id]}

)
)}
 </div>
 </div>

 /* что сейчас сообщил компилятор */
 <div className="mt-3 pt-3 border-t border-slate-800"
 <span className="text-[10px] uppercase tracking-wider"
 <Terminal className="w-3.5 h-3.5" /> из компилятора

 {liveCase || liveX !== null || liveStep !== null}
 </div>

```

```

/**
 * Splay на БОЛЬШОМ дереве – по урокам treap-прохода.
 *
 * Главный принцип: НИКАКИХ «хоп». Каждый одиночный поворот.
 * aim – прицел на тройку g–p–x, назван случай;
 * cut – отстёгиваем среднее поддереву β (пунктир, поворот);
 * rot – поворот: зелёным подсвечены рёбра, которых не было.
 * Рядом с логом всегда висит врезка «Механика одного поворота».
 * На каждом кадре – живая автопроверка in-order (сортировка).
 *
 * Чистые функции (splaySteps, rotateUp, bstInsert, cloneWithParents).
 */

```

```

export interface SN {
 key: number;
 left: SN | null;
 right: SN | null;
 parent: SN | null;
}

```

```

export function cloneWithParents(t: SN | null): SN | null {
 if (!t) return null;
 const n: SN = { key: t.key, left: null, right: null, parent: null };
 const l = t.left ? cloneWithParents(t.left) : null;
 const r = t.right ? cloneWithParents(t.right) : null;
 if (l) {
 n.left = l;
 l.parent = n;
 }
 if (r) {
 n.right = r;
 r.parent = n;
 }
 return n;
}

```

```

export function bstInsert(root: SN | null, key: number): SN | null {
 if (!root) return { key, left: null, right: null, parent: null };

```

```

 else g.right = child;
}

/** Рёбра в виде "p→c" — для diff между кадрами. */
export function collectEdges(t: SN | null): string[] {
 const es: string[] = [];
 const walk = (n: SN | null) => {
 if (!n) return;
 if (n.left) es.push(`${n.key}→${n.left.key}`);
 if (n.right) es.push(`${n.key}→${n.right.key}`);
 walk(n.left);
 walk(n.right);
 };
 walk(t);
 return es.sort();
}

const diffEdges = (before: string[], after: string[]):
 const rm = before.filter((e) => !after.includes(e));
 const ad = after.filter((e) => !before.includes(e));
 const parts: string[] = [];
 for (const a of ad) {
 const [p, c] = a.split("→");
 if (rm.includes(`${c}→${p}`)) parts.push(`ребро ${p}`);
 else parts.push(`поддерево ${c} переехало к ${p}`);
 }
 return parts.join("; ");
};

export type SplayStepKind = "search" | "aim" | "cut" |
export interface SplayStep {
 kind: SplayStepKind;
 root: SN | null;
 aimEdge?: [number, number];
 /** Кадр cut: ребро, по которому отстёгивается среднее */
 cutEdge?: [number, number];
 cutLabel?: string;
 /** Куда β переедет: сторона у верхнего узла. */

```

```

 ? `Вставили ${key} обычным BST-спуском. Теперь по
 : found
 ? `Спуск по ключу ${key}: путь ${pathKeys.join(
 : `Ключа ${key} нет. Спуск уперся в ${last.key}
 rotCount: 0,
 });

// --- подъём ---
let rotCount = 0;
let target: SN = x as SN;
const fixRoot = () => {
 while (root && root.parent) root = root.parent;
};
/** Один поворот = три кадра: aim → cut → rot. */
const emitRotation = (child: SN, label: string, aimEdge: string) => {
 steps.push({
 kind: "aim",
 root: cloneWithParents(root)!,
 aimEdge,
 caseLabel: label,
 note: aimNote,
 rotCount,
 });
 const p = child.parent!;
 const mid = p.left === child ? child.right : child.left;
 steps.push({
 kind: "cut",
 root: cloneWithParents(root)!,
 cutEdge: [child.key, mid ? mid.key : child.key],
 cutLabel: mid ? `β = поддереву ${mid.key}` : "β пусто",
 cutDest: mid ? (p.left === child ? "справа у p" : "слева у p") : "",
 caseLabel: label,
 note: mid
 ? `Отстёгиваем среднее поддерево β = ${mid.key}
 й у них общий – после поворота переедет ${p.left}
 : `Среднего поддерева нет (β пусто): ${child.key}
 rotCount,
 });

```

```

 kind: "aim",
 root: cloneWithParents(root)!,
 aimEdge: [target.parent.key, target.key],
 caseLabel: "Zig-Zig",
 note: `Теперь обычный зиг: ребро ${target.parent.key} над ${target.key}`,
 rotCount,
));
 emitRotation(
 target,
 "Zig-Zig",
 [target.parent.key, target.key],
 `Доводим: ${target.key} над ${target.parent.key}`,
);
} else {
 const side = `${g.key} → ${pIsLeft ? "влево" : "вправо"}`;
 emitRotation(
 target,
 "Zig-Zag",
 [p.key, target.key],
 `Zig-Zag (змейка: ${side}). Порядок ОБРАТНЫЙ: п. ${p.key} над ${target.key}`,
);
 steps.push({
 kind: "aim",
 root: cloneWithParents(root)!,
 aimEdge: [target.parent.key, target.key],
 caseLabel: "Zig-Zag",
 note: `Второе ребро: ${target.parent.key}—${target.key}`,
 rotCount,
));
 emitRotation(
 target,
 "Zig-Zag",
 [target.parent.key, target.key],
 `Доводим: ${target.key} над ${target.parent.key}`,
);
}
}
}

```

```

 { line: 6, text: " rotate(p over g) # B
 { line: 7, text: " rotate(x over p)" },
 { line: 8, text: " else: # Z
 { line: 9, text: " rotate(x over p) # H
 { line: 10, text: " rotate(x over g)" },
];

```

/\* ----- врезка «механика одного по

```

const MechanismInset: React.FC<{ step: SplayStep | null
 const isCut = step?.kind === "cut";
 const isRot = step?.kind === "rot";
 return (
 <div className="bg-slate-950/80 rounded-lg border b
 <h4 className="text-xs font-bold text-slate-300 m
 <p className="text-[10px] text-slate-500 mb-2">
 Как вправление вывиха: одна кость ходит вокруг
 </p>
 <svg viewBox="0 0 340 190" className="w-full">
 <g transform="translate(10,6)">
 <text y="10" fill="#94a3b8" fontSize="11" fon
 <line x1="55" y1="46" x2="30" y2="86" stroke=
 <line x1="55" y1="46" x2="80" y2="86" stroke=
 <line x1="30" y1="86" x2="12" y2="126" stroke=
 <line
 x1="30" y1="86" x2="48" y2="126"
 stroke={isCut ? "#f59e0b" : "#64748b"}
 strokeWidth={isCut ? 3 : 2}
 strokeDasharray={isCut ? "5,3" : "none"}
 />
 <circle cx="55" cy="46" r="15" fill="#1e293b"
 <text x="55" y="50" textAnchor="middle" fill=
 <circle cx="30" cy="86" r="15" fill="#78350f"
 <text x="30" y="90" textAnchor="middle" fill=
 <circle cx="12" cy="126" r="13" fill="#0f172a"
 <text x="12" y="130" textAnchor="middle" fill=
 <circle
 cx="48" cy="126" r="13"

```



```

 <text x="95" y="130" textAnchor="middle" fill="black">
 <text x="112" y="92" fill={isRot ? "#10b981" : "black"}>
 {isRot ? "↓ β теперь при p" : "β при p"}
 </text>
 <text x="30" y="160" fill="#64748b" fontSize=12>
 </g>
 </svg>
 <p className="text-[10px] text-slate-500 leading-5">
 Обход слева-направо α x β p γ не меняется никогд
 числа поворотов. Единственный, кто меняет хозяин
 </p>
</div>
);
};

```

/\* ----- рендер -----

```

const VIEW_W = 760;
const SLOT_W = 88;
const LEVEL_H = 76;

const layout = (root: SN | null) => {
 const pos = new Map<number, { px: number; py: number }>();
 let slot = 0;
 const walk = (n: SN | null, d: number) => {
 if (!n) return;
 walk(n.left, d + 1);
 pos.set(n.key, { px: 56 + slot * SLOT_W, py: 40 + d * LEVEL_H });
 slot += 1;
 walk(n.right, d + 1);
 };
 walk(root, 0);
 return pos;
};

```

```

const SplayWalkViz: React.FC = () => {
 const initial = useMemo(buildInitial, []);
 const [steps, setSteps] = useState<SplayStep[] | null>();

```

```

 walk(n.left);
 walk(n.right);
 };
 walk(viewRoot);
 return es;
}, [viewRoot, treeVers, idx]);

const autoW = Math.max(VIEW_W, ...nodes.map((n) => n.px));
const autoH = Math.max(300, ...nodes.map((n) => n.py));

const dBefore = useMemo(() => (steps ? depthOf(steps[steps.length - 1], viewRoot) : 0));
const dNow = steps ? depthOf(viewRoot, steps[steps.length - 1]) : 0;
const hBefore = useMemo(() => (steps ? heightOf(steps[steps.length - 1]) : 0));
const hNow = heightOf(viewRoot);

// переехавшие рёбра: разница с предыдущим кадром
const prevEdgeSet = useMemo(() => {
 if (!steps || !step || idx <= 0) return null;
 return new Set(collectEdges(steps[idx - 1].root));
}, [steps, step, idx]);
const movedCount = useMemo(() => {
 if (!prevEdgeSet) return 0;
 return edges.filter((e) => !prevEdgeSet.has(`${e.a}${e.b}`)).length;
}, [prevEdgeSet, edges]);

const ino0 = useMemo(() => (steps ? inorderKeys(steps[steps.length - 1].root) : ''));
const inoNow = inorderKeys(viewRoot).join(",");
const checkOk = !steps || ino0 === inoNow;

const doInsert = () => {
 const v = parseInt(input, 10);
 if (Number.isNaN(v)) return;
 const withNew = bstInsert(baseRef.current, v);
 baseRef.current = withNew;
 setTreeVers((t) => t + 1);
 runKey(v, true);
 setInput("");
};

```

```

 value={input}
 onChange={({e}) => setInput(e.target.value)}
 onKeyDown={({e}) => {
 if (e.key === "Enter") {
 const v = parseInt(input, 10);
 if (!Number.isNaN(v)) {
 const has = inorderKeys(baseRef.current);
 if (has) runKey(v);
 else doInsert();
 setInput("");
 }
 }
 }}
 placeholder="ключ + Enter"
 className="w-28 bg-slate-950 border border-
s:border-indigo-500"
 />
 <button
 onClick={() => {
 baseRef.current = buildInitial();
 setSteps(null);
 setIdx(0);
 setPlaying(false);
 setTreeVers((v) => v + 1);
 }}
 className="px-3 py-2 rounded-lg text-xs fon
 >
 Исходное
 </button>
 <button
 onClick={() => {
 const shuffled = [...INITIAL_KEYS].sort((
 let r: SN | null = null;
 for (const k of shuffled) r = bstInsert(r, k);
 baseRef.current = r!;
 setSteps(null);
 setIdx(0);
 setPlaying(false);

```

```

 })

 {/* Дерево */}
 <div className="bg-indigo-950/20 rounded-xl p-4 b
 {!steps && {
 <div className="absolute top-3 left-3 bg-indi
 ont-bold">
 Кликни по любому узлу – splay покадрово
 </div>
 }}
 <svg viewBox={`0 0 ${autoW} ${autoH}`} className
 {edges.map((e) => {
 const a = pos.get(e.a);
 const b = pos.get(e.b);
 if (!a || !b) return null;
 const isAim = step?.aimEdge && step.aimEdge
 const isCutEdge =
 step?.kind === "cut" && step.cutEdge && s
 .cutEdge[1];
 const isMoved = !isAim && !isCutEdge && pre
 return {
 <g key={`e-${e.a}-${e.b}`}>
 <line
 x1={a.px} y1={a.py} x2={b.px} y2={b.p
 stroke={isAim ? "#f59e0b" : isCutEdge
 strokeWidth={isAim ? 4 : isCutEdge ?
 strokeDasharray={isAim ? "6,4" : isCu
 className="transition-all duration-500
 />
 {isCutEdge && {
 <text x={({a.px + b.px} / 2 + 14} y={{
 nospace" className="transition-all duration
 {step?.cutLabel} → {step?.cutDest
 </text>
 }}
 </g>
);
 }}}

```

```

 : step?.kind === "rot"
 ? " Поворот:"
 : step?.kind === "search"
 ? " Спуск:"
 : "Статус:"}
 </p>
 <p className="min-h-[40px] flex items-center"
 {step ? step.note : "Кликни по узлу или в
 ке справа – механика одного поворота."}
 </p>
 {steps && step && step.kind !== "search" &&
 <p className={`text-xs mt-2 ${checkOk ? "
 {checkOk ? " " : " "} проверка кадра: o
 {inorderKeys(viewRoot).length}
 {movedCount > 0 ? ` · переехало рёбер:
 </p>
 }}
 </div>
 <div className="w-full lg:w-[300px] shrink-0"
 <MechanismInset step={step} />
 </div>
</div>
</div>

{/* Счётчики + код */}
<div className="flex flex-col lg:flex-row gap-6 m
 <div className="w-full lg:w-1/2 bg-rose-950/10
 <h4 className="text-sm font-bold text-rose-300
 {steps ? (
 <table className="w-full text-sm">
 <tbody className="font-mono">
 <tr className="border-b border-slate-800
 <td className="py-2 text-slate-400">r
 <td className="py-2 text-right text-s
 </tr>
 <tr className="border-b border-slate-800
 <td className="py-2 text-slate-400">v
 <td className="py-2 text-right text-s

```

```

export default SplayWalkViz;

```

ФАЙЛ 74/111: src/components/StackViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```

import React, { useState, useCallback } from 'react';
import { ArrowUp, Sparkles } from 'lucide-react';
import { useVizRuntime, vizArray } from '../data/vizSteps';
```

```
interface Plate {
 id: string;
 name: string;
 emoji: string;
 color: string;
 isDirty: boolean;
 animState: 'in' | 'idle' | 'washing' | 'clean';
}
```

```
const PLATE_PRESETS = [
 { name: 'Тарелка из-под спагетти', emoji: '🍝', color: 'from-pasta' },
 { name: 'Тарелка из-под пиццы', emoji: '🍕', color: 'from-pizza' },
 { name: 'Блюдец от торта', emoji: '🍰', color: 'from-pie' },
 { name: 'Тарелка от тако', emoji: '🌮', color: 'from-yaki' },
 { name: 'Тарелка из-под суши', emoji: '🍣', color: 'from-sushi' },
 { name: 'Сковорода от глазуньи', emoji: '🍳', color: 'from-pan-fry' },
];
```

```
let counter = 0;
```

```
export const StackViz: React.FC = () => {
 const [dirtyStack, setDirtyStack] = useState<Plate[]>([
 { id: 'p1', name: 'Тарелка из-под спагетти', emoji: '🍝',
 animState: 'idle' },
 { id: 'p2', name: 'Тарелка из-под пиццы', emoji: '🍕',
 animState: 'idle' },
]);
};
```

```

 const newPlate: Plate = {
 id: Date.now().toString(),
 ...preset,
 isDirty: true,
 animState: 'in',
 };

 setDirtyStack(prev => [...prev, newPlate]);
 addLog(` PUSH: Положили ${newPlate.emoji} сверху с

 setTimeout(() => {
 setDirtyStack(prev => prev.map(p => p.id === newP
 }, 500);
 }, [dirtyStack, isWashing]);

// POP: Помыть верхнюю тарелку (LIFO)
const popPlate = useCallback(() => {
 if (isWashing) return;
 if (dirtyStack.length === 0) {
 setShake(true);
 addLog('⚠ POP: Нечего мыть! Все тарелки чистые.')
 setTimeout(() => setShake(false), 400);
 return;
 }

 setIsWashing(true);
 const topPlate = dirtyStack[dirtyStack.length - 1];

 addLog(` POP: Начинаем мыть верхнюю тарелку с ${to

// Step 1: Start washing animation (sponge and soap
setDirtyStack(prev => prev.map(p => p.id === topPla
setShowSoap(true);

setTimeout(() => {
 // Step 2: Wash complete. Plate becomes clean and
 const cleanPlate: Plate = {
 ...topPlate,

```

Ты складываешь грязные тарелки друг на друга.  
А когда приходит время мыть, ты берёшь именно  
Попытаешься вытащить нижнюю – всё рухнет!

```
 </p>
 </div>
</div>

{/* УПРАВЛЕНИЕ */}
<div className="flex items-center gap-3 flex-wrap" >
 <button
 onClick={pushPlate}
 disabled={isWashing}
 className="flex-1 min-w-[150px] bg-gradient-to-r
 from-slate-500 to-slate-600 opacity-40 disabled:cursor-not-allowed text-white
 font-medium rounded-xl p-3 transition-all cursor-pointer flex items-center justify-center"
 >
 Положить грязную (PUSH)
 </button>

 <button
 onClick={popPlate}
 disabled={isWashing || dirtyStack.length === 0}
 className="flex-1 min-w-[150px] bg-gradient-to-r
 from-slate-500 to-slate-600 opacity-40 disabled:cursor-not-allowed text-white
 font-medium rounded-xl p-3 transition-all cursor-pointer flex items-center justify-center"
 >
 Помыть верхнюю (POP)
 </button>

 <button
 onClick={reset}
 className="px-5 py-3.5 rounded-2xl bg-slate-800 text-white font-medium
 cursor-pointer border border-slate-700"
 >
 Сброс
 </button>
</div>
```



```

 { /* Указатель TOP */ }
 <div
 className="absolute right-8 flex items-center gap-1"
 style={{ bottom: `${24 + topIndex * 44}px` }}
 >

 <ArrowUp className="w-3.5 h-3.5" />
 ВЕРШИНА (TOP)

 </div>

 { /* Тарелки (отрисовываем в обратном порядке) */ }
 {displayStack.slice().reverse().map((plate, idx) => {
 const idx = displayStack.length - 1 - idx;
 const isTop = idx === topIndex;
 return (
 <div
 key={plate.id}
 className={`
 w-full max-w-[280px] h-10 rounded-md
 shadow-md select-none transition-all duration-200
 ${plate.color}
 ${plate.animState === 'in' ? 'animate-in' : ''}
 ${plate.animState === 'washing' ? 'animate-washing' : ''}
 ${isTop ? 'ring-4 ring-blue-500/50' : ''}
 `}
 >
 {plate.name}

 {isTop && (

 LIFO

)}

 </div>
)
 })}
 </div>

```

```

 </div>
)}}
 </div>
)}
</div>

 { /* Столешница сушилки */ }
 <div className="w-full h-4 bg-gradient-to-r f
</div>
</div>

{ /* ЛОГ ДЕЙСТВИЙ */ }
<div className="bg-slate-900 border border-slate-
 <div className="text-[10px] font-mono text-slate
 {log.map((entry, idx) => {
 <div
 key={idx}
 className={`text-xs font-mono flex items-ce
 >
 {idx === 0 ? <span className="text-blue-500
 {entry}
 </div>
)}}
 </div>
</div>
);
};

```

---

ФАЙЛ 75/111: src/components/StringAlgorithmsViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

---

```

import { useState, useEffect, useRef, useMemo } from "r
import { useVizRuntime, vizArray, vizNumber, vizString
import { Play, Pause, SkipForward, Undo, RefreshCw } fr

```

```

}

function generateZSteps(pattern: string, text: string)
 if (!pattern) pattern = " ";
 const s = pattern + "#" + text;
 const n = s.length;
 const z = new Array(n).fill(0);
 const steps: any[] = [];

 let l = 0, r = 0;
 steps.push({ i: 0, l, r, z: [...z], desc: `Начало.` });

 for (let i = 1; i < n; i++) {
 steps.push({ i, l, r, z: [...z], desc: `Шаг i=$` });

 if (i <= r) {
 steps.push({ i, l, r, z: [...z], desc: `Вни
 !` });
 z[i] = Math.min(r - i + 1, z[i - l]);
 steps.push({ i, l, r, z: [...z], desc: `Пре
 }. Копипаста сработала.` });
 }

 steps.push({ i, l, r, zTarget: z[i], zSource: i
 while (i + z[i] < n && s[z[i]] === s[i + z[i]])
 steps.push({ i, l, r, zTarget: z[i], zSource
 падают! Окно растёт.` });
 z[i]++;
 }

 if (i + z[i] < n) {
 steps.push({ i, l, r, zTarget: z[i], zSource
 [i]}}' разные. Стоп.` });
 } else {
 steps.push({ i, l, r, z: [...z], desc: `Упё
 }

 steps.push({ i, l, r, z: [...z], desc: `Фиксир

```

```

 setStepIdx(0);
 setAutoPlay(false);
 }, [s, mode]);

// Timer loop for autoPlay
useEffect(() => {
 if (autoPlay) {
 timer.current = setInterval(() => {
 setStepIdx(prev => {
 if (prev >= steps.length - 1) { set
 return prev + 1;
 });
 }, 1200);
 }
 return () => { if (timer.current) clearInterval
}, [autoPlay, steps.length]);

const playPause = () => setAutoPlay(!autoPlay);
const nextStep = () => { setAutoPlay(false); setSte
const prevStep = () => { setAutoPlay(false); setSte
const reset = () => { setAutoPlay(false); setStepId

const baseStep = steps[stepIdx] || steps[0];
const vars = runtime?.variables;
const liveValues = vizArray(mode === "kmp" ? vars?.l
const hasLiveValues = !!vars && Object.prototype.ha
const liveI = vizNumber(vars?.i);
const liveJ = vizNumber(vars?.j);
const liveL = vizNumber(vars?.l);
const liveR = vizNumber(vars?.r);
const compilerLinked = liveI !== null || hasLiveVal
const step = compilerLinked
 ? {
 ...baseStep,
 i: liveI ?? baseStep.i,
 j: liveJ ?? baseStep.j,
 l: liveL ?? baseStep.l,
 r: liveR ?? baseStep.r,
 }
 : baseStep;

```

```

{s.split("").map(({char, index}) => {
 const isPattern = index < patte
 const isSeparator = index === p

 let borderColor = "border-slate
 let bgColor = "bg-slate-900";
 let textColor = isSeparator ? "

// KMP Logic
if (mode === "kmp") {
 if (step.highlight?.include
 borderColor = "border-a
 bgColor = step.match ===
-900/40");
 }
}

// Z Logic
if (mode === "z") {
 if (index >= step.l && index
 bgColor = "bg-emerald-9
 borderColor = "border-e
 }
 if (step.zTarget === index
 borderColor = "border-a
 bgColor = step.match ===
-900/40");
 }
}

return (
 <div key={index} className=
 {/* L/R markers for Z */
 {mode === "z" && step.l
 <div className="abs
 }}
 {mode === "z" && step.r
 <div className="abs

```

```

 <div className="abs
e z-20">
 <span className
 </div>
 }}
 {mode === "z" && step.z}
 <div className="abs
 <span className
 </div>
 }}
 {mode === "z" && step.z}
 <div className="abs
 <span className
 </div>
 }}

 {/* Found flash effect
 {step.found && mode ===
 <div className="abs
ulse z-0 bg-emerald-500/20 shadow-[0_0_15px_
 }}
 {step.found && mode ===
 <div className="abs
ulse z-0 bg-emerald-500/20 shadow-[0_0_15px_
 }}
 </div>
)
 }}}
 </div>
</div>

<div className="flex flex-col sm:flex-row g
 {/* Controls */}
 <div className="flex bg-slate-900 borde
 <button onClick={prevStep} disabled
-400 hover:text-white hover:bg-slate-800 ro
 <Undo strokeWidth={3} size={16}
 </button>

```

```

 </p>
 </div>

 { /* Code Sneak Peek */
 <div className="bg-slate-900 border border-
 <div className="bg-slate-800 px-4 py-2 b
late-400 uppercase">
 Код C++ {mode === 'kmp' ? 'КМП
 </div>
 <pre className="p-4 text-xs font-mono te
{mode === 'kmp' ? `vector<int> pi(s.length());
for (int i = 1; i < s.length(); i++) {
 int j = pi[i-1];
 while (j > 0 && s[i] != s[j]) j = pi[j-1];
 if (s[i] == s[j]) j++;
 pi[i] = j;
}` : `int l = 0, r = 0;
for (int i = 1; i < n; i++) {
 if (i <= r) z[i] = min(r - i + 1, z[i - l]);
 while (i + z[i] < n && s[z[i]] == s[i + z[i]]) z[i]
 if (i + z[i] - 1 > r) {
 l = i;
 r = i + z[i] - 1;
 }
}` }
 </pre>
 </div>
 </div>

);
}

```

---

ФАЙЛ 76/111: src/components/Tooltip.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

---

```
import React, { useState } from "react";
```

```
 <span
 className={`absolute left-1/2 h-2 w-2 -transl
 side === "top" ? "-bottom-[5px] border-b bo
 }`}
 />

);
};
```

---

ФАЙЛ 77/111: src/components/TopologicalSortViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРО

---

```
import React, { useState, useEffect } from "react";
import { useVizStepSync, vizArray, vizNumber } from "...
import {
 Play,
 Pause,
 RotateCcw,
 Clock,
} from "lucide-react";

interface TNode {
 id: number;
 x: number;
 y: number;
 label: string;
 name: string;
}

interface TEdge {
 u: number;
 v: number;
}
```



```

const [isPlaying, setIsPlaying] = useState(false);

useEffect(() => {
 const listSteps: TStep[] = [];
 const visited = new Set<number>();
 const fullyProcessed = new Set<number>();
 const topoList: number[] = [];
 const dfsStack: number[] = [];

 const recordStep = (desc: string, currentNode: number) => {
 listSteps.push({
 desc,
 visited: new Set(visited),
 fullyProcessed: new Set(fullyProcessed),
 currentNode,
 topoList: [...topoList],
 dfsStack: [...dfsStack],
 });
 };

 recordStep(
 "Начало топологической сортировки. Используем кла",
 null,
);

 // Adj list
 const adj: number[][] = Array.from({ length: 5 }, () => []);
 dagEdges.forEach((e) => adj[e.u].push(e.v));

 const dfs = (u: number) => {
 visited.add(u);
 dfsStack.push(u);
 recordStep(
 `DFS: зашли в вершину ${u} ("${dagNodes[u].name}`,
 u,
);

 for (const to of adj[u]) {

```

```

 }, []));

 const step = steps[currentStepIdx] || {
 desc: "Заныск...",
 visited: new Set(),
 fullyProcessed: new Set(),
 currentNode: null,
 topoList: [],
 dfsStack: [],
 };

 useEffect(() => {
 let timer: NodeJS.Timeout;
 if (isPlaying && currentStepIdx < steps.length - 1) {
 timer = setInterval(() => {
 setCurrentStepIdx((prev) => prev + 1);
 }, 1600);
 } else {
 setIsPlaying(false);
 }
 return () => clearInterval(timer);
 }, [isPlaying, currentStepIdx, steps.length]);

 const togglePlay = () => setIsPlaying(!isPlaying);
 const reset = () => {
 setIsPlaying(false);
 setCurrentStepIdx(0);
 };

 const getTopoListString = () => {
 return [...step.topoList]
 .reverse()
 .map((id) => dagNodes[id].name)
 .join(" → ");
 };

 return (
 <div className="bg-slate-900 rounded-xl border bord

```

```

 id="dag-arrow"
 viewBox="0 0 10 10"
 refX="24"
 refY="5"
 markerWidth="6"
 markerHeight="6"
 orient="auto-start-reverse"
 >
 <path d="M 0 1 L 10 5 L 0 9 z" fill="#4
 </marker>
 <marker
 id="dag-arrow-active"
 viewBox="0 0 10 10"
 refX="24"
 refY="5"
 markerWidth="6"
 markerHeight="6"
 orient="auto-start-reverse"
 >
 <path d="M 0 1 L 10 5 L 0 9 z" fill="#8
 </marker>
</defs>

{/* Edges */}
{dagEdges.map(({edge, idx}) => {
 const u = dagNodes.find((n) => n.id === e
 const v = dagNodes.find((n) => n.id === e

 const isActive =
 (step.currentNode === edge.u || step.cu
 step.visited.has(edge.v));

 return (
 <line
 key={`edge-${idx}`}
 x1={u.x}
 y1={u.y}
 x2={v.x}

```

```

 y={node.y - 22}
 width="110"
 height="44"
 rx="8"
 fill={fill}
 stroke={stroke}
 strokeWidth="2"
 className="transition-all duration-100"
 />
 <text
 x={node.x}
 y={node.y - 4}
 textAnchor="middle"
 fill="white"
 fontSize="11px"
 fontWeight="bold"
 className="pointer-events-none"
 >
 ({node.label}) {node.name.split(" ").join(" ")}
 </text>
 <text
 x={node.x}
 y={node.y + 12}
 textAnchor="middle"
 fill="#94a3b8"
 fontSize="9px"
 className="pointer-events-none"
 >
 {node.name.split(" ").slice(1).join(" ")}
 </text>
 </g>
);
}
}
</svg>
</div>

```

```

{/* Console column */}
<div className="lg:col-span-4 bg-slate-950 p-5"

```

```


 </div>
 <div className="flex items-center gap-2">

 Черные (готово)

 </div>
 </div>
 </div>
</div>

<div className="mt-4 pt-3 border-t border-slate-200">

 Шаг {currentStepIdx + 1} из {steps.length}

 <div className="flex gap-1">
 <button
 disabled={currentStepIdx === 0}
 onClick={() => setCurrentStepIdx((prev) => prev - 1)}
 className="bg-slate-900 border border-slate-700 text-white"
 >
 Назад
 </button>
 <button
 disabled={currentStepIdx >= steps.length - 1}
 onClick={() => setCurrentStepIdx((prev) => prev + 1)}
 className="bg-slate-900 border border-slate-700 text-white"
 >
 Вперед
 </button>
 </div>
</div>
</div>
</div>
</div>
);

```

```

 grade: number;
 left: TNode | null;
 right: TNode | null;
}

export const POINTS: Pair[] = [
 { key: 45, grade: 8 },
 { key: 3, grade: 6 },
 { key: 6, grade: 5 },
 { key: 2, grade: 4 },
 { key: 9, grade: 3 },
 { key: 7, grade: 2 },
 { key: 12, grade: 0 },
 { key: 1, grade: -4 },
];

/** Порядок ввода (как в черновике) — намеренно НЕ отсортирован */
export const INPUT_ORDER: Pair[] = [
 { key: 3, grade: 5 },
 { key: 45, grade: 8 },
 { key: 3, grade: 6 },
 { key: 1, grade: -4 },
 { key: 7, grade: 2 },
 { key: 6, grade: 5 },
 { key: 2, grade: 4 },
 { key: 9, grade: 3 },
 { key: 12, grade: 0 },
];

// (3,5) дубль ключа/приоритета — уберём: ввод без повторов
export const INPUT_CLEAN: Pair[] = INPUT_ORDER.filter((p, i) => !INPUT_ORDER.some((p2, j) => j < i && p2.key === p.key));

const sortedByY = (ps: Pair[]): Pair[] => [...ps].sort((a, b) => a.grade - b.grade);

/* ----- чистые алгоритмы (экспортируются для тестов) ----- */

export function insert(root: TNode | null, p: Pair): TNode {
 if (!root) return { id: pairId(p.key, p.grade), key: p.key, grade: p.grade, left: null, right: null };
 if (p.key <= root.key) root.left = insert(root.left, p);
 else root.right = insert(root.right, p);
 return root;
}

```

```

 trace: { aId: NodeId | null; bId: NodeId | null; chosenId: NodeId | null; }
}
export function mergeTree(a: TNode | null, b: TNode | null): TNode {
 const trace: MergeResult["trace"] = [];
 const go = (a: TNode | null, b: TNode | null): TNode | null => {
 if (!a || !b) {
 trace.push({ aId: a?.id ?? null, bId: b?.id ?? null, chosenId: null, note: "Обе ветви пустые, возвращаемое целиком." });
 return a ?? b;
 }
 if (a.grade > b.grade) {
 trace.push({ aId: a.id, bId: b.id, chosenId: a.id, note: "Выбор a, так как его grade больше, чем его правое поддереве с (${b.key}, ${b.grade})." });
 a.right = go(a.right, b);
 return a;
 }
 trace.push({ aId: a.id, bId: b.id, chosenId: b.id, note: "Выбор b, так как его grade больше, чем его левое поддереве с (${a.key}, ${a.grade})." });
 b.left = go(a, b.left);
 return b;
 };
 return { root: go(a, b), trace };
}

```

```

export function eraseNode(root: TNode | null, key: number): TNode | null {
 const trace: { atId: NodeId | null; note: string }[] = [];
 const go = (t: TNode | null): TNode | null => {
 if (!t) return null;
 if (key < t.key) {
 trace.push({ atId: t.id, note: `${key} < ${t.key}` });
 t.left = go(t.left);
 return t;
 }
 if (key > t.key) {
 trace.push({ atId: t.id, note: `${key} > ${t.key}` });
 t.right = go(t.right);
 return t;
 }
 // key === t.key
 trace.push({ atId: t.id, note: "Удаление узла" });
 if (!t.left) return t.right;
 if (!t.right) return t.left;
 // both left and right exist
 let left = t.left;
 let right = t.right;
 while (right) {
 left = right;
 right = right.left;
 }
 t.key = left.key;
 t.left = left.left;
 t.right = go(right, key);
 return t;
 };
 return { root: go(root, key), trace };
}

```

```

 walk(n.left, depth + 1);
 pos.set(n.id, { px: 64 + slot * 92, py: 44 + depth * 20 });
 slot += 1;
 walk(n.right, depth + 1);
 };
 walk(root, 0);
 return pos;
};

```

```

const treeEdges = (root: TNode | null): { from: NodeId; to: NodeId }[] => {
 const es: { from: NodeId; to: NodeId }[] = [];
 const walk = (n: TNode | null) => {
 if (!n) return;
 if (n.left) { es.push({ from: n.id, to: n.left.id }); }
 if (n.right) { es.push({ from: n.id, to: n.right.id }); }
 };
 walk(root);
 return es;
};

```

```

interface TreeViewProps {
 root: TNode | null;
 width?: number;
 height?: number;
 nodeColor?: (id: NodeId) => string;
 nodeStroke?: (id: NodeId) => string;
 edgeColor?: (from: NodeId, to: NodeId) => string;
 edgeDash?: (from: NodeId, to: NodeId) => boolean;
 pulseId?: NodeId | null;
 subtitle?: string;
}

```

```

const TreeView: React.FC<TreeViewProps> = ({ root, width, height }) => {
 const pos = useMemo(() => layoutTree(root), [root]);
 const nodes = useMemo(() => [...pos.entries()].map(([id, pos]) => ({ id, px: pos.px, py: pos.py })), [pos]);
 const edges = useMemo(() => treeEdges(root), [root]);
 const autoW = Math.max(420, ...nodes.map((n) => n.px));
 const autoH = Math.max(260, ...nodes.map((n) => n.py));

```



```

const pairParts = (id: NodeId): [number, number] => {
 const [a, b] = id.split(":").map(Number);
 return [a, b];
};

/* ----- плоскость (клетчатая бумага) -----

const PL_W = 780;
const PL_H = 300;
const PX = (key: number) => 42 + (key - 1) * ((PL_W - 60) / 14);
const PY = (grade: number) => 24 + (8 - grade) * ((PL_H - 24) / 8);

interface PlaneProps {
 points: Pair[];
 drawnEdges: { a: Pair; b: Pair }[];
 pending: Pair | null;
 interactive?: boolean;
 selected?: Pair | null;
 cursor?: { x: number; y: number } | null;
 badEdge?: { a: Pair; b: Pair } | null;
 onPointDown?: (p: Pair) => void;
 onPointUp?: (p: Pair) => void;
 onMove?: (e: React.PointerEvent<SVGSVGElement>) => void;
}

const Plane: React.FC<PlaneProps> = ({ points, drawnEdges, pending,
 }) => {
 const byPair = (p: Pair) => ({ x: PX(p.key), y: PY(p.grade) });
 return (
 <svg
 viewBox={`0 0 ${PL_W} ${PL_H}`}
 className={`w-full ${interactive ? "touch-none select-none" : ""}`}
 onPointerMove={onMove}
 style={interactive ? { cursor: "crosshair" } : undefined}
 >
 <defs>
 <pattern id="pl-grid" width="14" height="14" patternUnits="userSpaceOnUse">
 <path d="M 14 0 L 0 0 0 14" fill="none" stroke="black" stroke-width="1"/>
 </pattern>
 </defs>
 <rect width="100%" height="100%" fill="url(#pl-grid)"/>
 {points.map(p => <circle key={p.key} cx={PX(p.key)} cy={PY(p.grade)} r="4" fill="black"/>)}
 {drawnEdges.map(e => <line key={e.a.key} x1={PX(e.a.key)} y1={PY(e.a.grade)} x2={PX(e.b.key)} y2={PY(e.b.grade)}/>)}
 {pending ? <circle key={pending.key} cx={PX(pending.key)} cy={PY(pending.grade)} r="8" fill="black" stroke="black" stroke-width="2"/> : null}
 </svg>
);
};

```

```

 className="transition-all duration-500"
 onPointerDown={interactive && onPointDown ?
 onPointerUp={interactive && onPointUp ? () :
 style={interactive ? { cursor: "pointer" } :
 >
 <circle cx={PX(p.key)} cy={PY(p.grade)} r={
 stroke={isSel ? "#a5b4fc" : done ? "#38bdf8" :
 <text x={PX(p.key)} y={PY(p.grade) + 3.5} f
 ">{p.key}</text>
 </g>
 });
}}
 {pending && (
 <g className="transition-all duration-500">
 <circle cx={PX(pending.key)} cy={PY(pending.g
 >
 <text x={PX(pending.key)} y={PY(pending.grade
 ly="monospace">{pending.key}</text>
 </g>
)}
</svg>
);
};

```

/\* ===== ИГРА: соединение мышкой =====

export type GEdge = { a: Pair; b: Pair }; // a – родите

```

export const CANON_GAME: GEdge[] = (() => {
 const es: GEdge[] = [];
 const walk = (n: TNode | null) => {
 if (!n) return;
 if (n.left) { es.push({ a: { key: n.key, grade: n.g
 if (n.right) { es.push({ a: { key: n.key, grade: n.
 };
 walk(buildTree(POINTS));

```

```
/** Полный вердикт по текущим линиям. */
export function gameVerdict(edges: GEdge[], all: Pair[]) {
 const heap = edges.every((e) => e.a.grade > e.b.grade)
 const oneParent = new Set(edges.map((e) => gpId(e.b)))
 const acyclic = !edges.some((e) => reaches(edges, e.a, e.b))
 const bst = forestIsBst(edges, all);
 const roots = all.filter((p) => !edges.some((e) => gpId(e.a) === p.id))
 const complete = edges.length === all.length - 1 && roots.length === 1
 return { heap, oneParent, acyclic, bst, complete, win: true }
}
```

```
/** Попытка хода: почему нельзя – или ОК. */
export function tryConnect(edges: GEdge[], a: Pair, b: Pair) {
 if (sameP(a, b)) return { ok: false, reason: "Это одна и та же точка" }
 if (a.grade === b.grade)
 return { ok: false, reason: "«!» равны – «кто выше?»" }
 const parent = a.grade > b.grade ? a : b;
 const child = a.grade > b.grade ? b : a;
 if (parentOf(edges, gpId(child)))
 return { ok: false, reason: `У точки (${child.key}) уже есть родительская` };
 if (reaches(edges, parent, child))
 return { ok: false, reason: "Цикл: линия замкнёт саму себя" };
 const next = [...edges, { a: parent, b: child }];
 if (!forestIsBst(next, POINTS))
 return { ok: false, reason: "Линия вниз по y допустима только если не образует цикла" };
 return { ok: true };
}
```

```
/* ===== РЕЖИМ BUILD ===== */
```

```
type BuildPhase = "sort" | "connect" | "game";
type SortAlgo = "none" | "quicksort" | "counting";
type ConnectOrder = "y-desc" | "left-right";
```

```
const parsePairs = (raw: string): { pairs: Pair[]; error?: string } => {
 const pairs: Pair[] = [];
 const error: string = [];
```

```

 if (x.key === y.key && x.grade === y.grade) return {
 for (const e of edges) {
 if ((pid(e.a) === pid(x) && pid(e.b) === pid(y)) ||
 return { ok: false, reason: "такая линия уже пров
 }
 if (x.grade === y.grade) return { ok: false, reason:
 " };
 const [parent, child] = x.grade > y.grade ? [x, y] :
 if (edges.some((e) => pid(e.b) === pid(child)))
 return { ok: false, reason: `у точки (${child.key});
 // цикл: родитель уже сидит в компоненте ребёнка (реб
 const childComp = new Set<string>([pid(child)]);
 let grew = true;
 while (grew) {
 grew = false;
 for (const e of edges) {
 if (childComp.has(pid(e.b)) && !childComp.has(pid
 childComp.add(pid(e.a)));
 grew = true;
 }
 }
}
if (childComp.has(pid(parent))) return { ok: false, r
return {
 ok: true,
 edge: { a: parent, b: child },
 reason:
 x.grade > y.grade ? undefined : "линия в treap вс
 };
}

```

```

export function checkGameState(
 points: Pair[],
 edges: GameEdge[],
): { heapOk: boolean; singleParent: boolean; connected:
 const n = points.length;
 if (n < 2) return { heapOk: true, singleParent: true,
 const heapOk = edges.every((e) => e.a.grade > e.b.gra

```

```

 }
 const sorted = [...kids].sort((u, v) => u.key - v.key);
 let left: Pair | null = null;
 let right: Pair | null = null;
 if (sorted.length === 2) {
 if (sorted[0].key === sorted[1].key || sorted[0].key === sorted[1].key) {
 bstOk = false; // двое детей с равным x или с равным y
 return;
 }
 left = sorted[0];
 right = sorted[1];
 } else if (sorted.length === 1) {
 // одиночный ребёнок: сторона важна – определяем, где вставить
 if (sorted[0].key <= p.key) left = sorted[0];
 else right = sorted[0];
 }
 if (left) walk(left);
 inorder.push(p.key);
 if (right) walk(right);
 };
 walk(roots[0]);
 if (seen.size !== n) bstOk = false;
 for (let i = 1; i < inorder.length; i++)
 if (inorder[i] < inorder[i - 1]) bstOk = false;
}
return { heapOk, singleParent, connected, countOk, bstOk };
}

```

```

/** Дерево из проведённых линий – для показа результата
export function edgesToTree(edges: GameEdge[]): TNode | null {
 const parentOf = new Map<string, GameEdge>();
 for (const e of edges) parentOf.set(pid(e.b), e);
 const roots = edges.filter((e) => !parentOf.has(pid(e.b)));
 if (!roots.length && edges.length) return null;
 const nodes = new Map<string, TNode>();
 const make = (p: Pair): TNode => ({
 id: pid(p),
 key: p.key,
 left: null,
 right: null,
 });
 for (const r of roots) {
 const n = make(r);
 nodes.set(n.id, n);
 if (parentOf.has(n.id)) {
 const p = parentOf.get(n.id)!;
 if (p.left === null) p.left = n;
 else p.right = n;
 }
 }
 return nodes.get(roots[0].id);
}

```

```

const [algo, setAlgo] = useState<SortAlgo>("none");
const [order, setOrder] = useState<ConnectOrder>("y-desc");
const [sortIdx, setSortIdx] = useState(0);
const [connIdx, setConnIdx] = useState(0);
const [playing, setPlaying] = useState(false);
const [sortCost, setSortCost] = useState<string | null>(null);

const parsed = useMemo(() => parsePairs(raw), [raw]);
const userPairs = parsed.errors.length ? [] : parsed.pairs;

const n = userPairs.length;

const userTree = useMemo(() => (userPairs.length >= 1 ?
const canonEdges = useMemo(() => {
 const es: { a: Pair; b: Pair }[] = [];
 const walk = (node: TNode | null) => {
 if (!node) return;
 if (node.left) { es.push({ a: { key: node.key, group: node.group, left: node.left }, b: node }); }
 if (node.right) { es.push({ a: { key: node.key, group: node.group, right: node.right }, b: node }); }
 };
 walk(userTree);
 return es;
}, [userTree]);
const leftRightEdges = useMemo(() => [...canonEdges], [canonEdges]);

useEffect(() => {
 if (!playing || phase === "game") return;
 const t = setTimeout(() => {
 if (phase === "sort") {
 if (algo === "none") { setPlaying(false); return; }
 if (sortIdx < 3) setSortIdx(sortIdx + 1);
 else setPlaying(false);
 } else {
 const total = order === "y-desc" ? canonEdges.length : canonEdges.length / 2;
 if (connIdx < total) setConnIdx(connIdx + 1);
 else setPlaying(false);
 }
 }, 1000);
}, [playing, phase, algo, sortIdx, connIdx, order]);

```

```

const verdict = useMemo(() => checkGameState(userPair)
const gameWon = phase === "game" && verdict.won;
const gameTree = useMemo(() => (gameWon ? edgesToTree

const attemptConnect = (x: Pair, y: Pair) => {
 const r = canConnect(gameEdges, x, y);
 if (r.ok && r.edge) {
 setGameEdges((prev) => [...prev, r.edge!]);
 setGameMsg(r.reason ?? null);
 setBadEdge(null);
 } else {
 setGameMsg(r.reason ?? "так нельзя");
 setBadEdge({ a: x, b: y });
 window.setTimeout(() => setBadEdge(null), 900);
 }
};

const pointDown = (p: Pair) => {
 if (phase !== "game") return;
 if (!sel) {
 setSel(p);
 setGameMsg(`Выбрана (${p.key}; ${p.grade}) – теперь
 } else if (sel.key === p.key && sel.grade === p.grade) {
 setSel(null);
 setGameMsg(null);
 } else {
 attemptConnect(sel, p);
 setSel(null);
 }
};

const pointUp = (p: Pair) => {
 if (phase !== "game" || !sel) return;
 if (sel.key === p.key && sel.grade === p.grade) return;
 attemptConnect(sel, p);
 setSel(null);
 setCursor(null);
};

const planeMove = (e: React.PointerEvent<SVGSVGElement>) => {
 if (phase !== "game" || !sel) return;
 if (sel.key === p.key && sel.grade === p.grade) return;
 attemptConnect(sel, p);
 setSel(null);
 setCursor(null);
};

```

```

 </div>
)}
 {parsed.errors.length === 0 && (parsed.equalX |
 <div className="mt-2 text-xs text-amber-300 b
 {parsed.equalY && <div>✗ Есть равные y
 кто раньше в списке (сортировка ста
 {parsed.equalX && <div>✗ Есть равные x
 de.key → left).</div>}
 </div>
)}
 </div>

 { /* Координатная плоскость – НАВЕРХУ */ }
 <div className="bg-emerald-950/20 rounded-xl p-4
 <div className="flex flex-wrap items-center jus
 <h4 className="text-base font-bold text-emera
 <div className="flex items-center gap-2">
 {phase === "sort" ? {
 <>
 <span className="text-xs text-slate-400
 {["quicksort", "counting"] as const }.m
 <button
 key={a}
 onClick={() => { setAlgo(a); setSor
 Math.round(n * Math.log2(n)) : 0} сравнений
 disabled={n < 2}
 className={`px-3 py-1.5 rounded-lg
 600 text-white" : "bg-slate-700 text-slate-
 >
 {a === "quicksort" ? "quicksort (сп
 </button>
)}}
 </>
) : (
 <>
 <span className="text-xs text-slate-400
 <button
 onClick={() => startConnect("y-desc")

```



```

 algo === "none" ? (
 <>Ввод приходит как попало ({n} па
 ть.</>
) : sortIdx < 3 ? (
 algo === "quicksort" ? (
 <> quicksort: счётчик сравнений: <b cl
 {comparisons} (n·log2n).</>
) : (
 <> counting: точки в корзины по значени
 , потому что у – маленькие целые.</>
)
) : (
 <>Отсортировано ({sortCost ?? "способ не
 Игру </>
)
) : phase === "game" ? (
 gameWon ? (
 <> Готово! {hintUsed ? "Решение подсказа
 ево: линии вниз по у, обход по х отсортиров
) : gameMsg ? (
 <>△ {gameMsg}</>
) : (
 <>Клики точку-родителя (выше по «!»), за
)
) : connDone ? (
 <> Проведено {connEdges.length} линий в по
 : точки прибиты, дерево одно. А теперь
) : (
 <>Линий проведено: {connIdx} из {connEdgesA
)}
</div>
</div>

```

```

{ /* Чек-лист инвариантов */ }
<div className={`rounded-xl p-4 border transition
 er-slate-700 bg-slate-900/40`} `>
 <h4 className="text-sm font-bold text-slate-300">
 <ul className="space-y-1.5 text-sm">

```

```

 {order === "y-desc" && userTree && {
 <div className="bg-slate-900/40 rounded-xl p-4">
 <h4 className="text-sm font-bold text-slate-300">
 <TreeView root={userTree} subtitle="Слева-направо" />
 </h4>
 </div>
 }}
 </div>
);
};

/* ===== РЕЖИМ SPLIT ===== */

const SPLIT_KEYS = [2, 3, 6, 7, 9, 12, 45];

const SplitMode: React.FC = () => {
 const [x0, setX0] = useState(7);
 const [stepIdx, setStepIdx] = useState(0);
 const live = useTreapLive();
 const fullTree = useMemo(() => buildTree(POINTS), [POINTS]);
 const split = useMemo(() => splitTree(buildTree(POINTS), x0), [fullTree, x0]);
 const steps = split.trace;
 const idx = Math.min(stepIdx, steps.length - 1);

 // mid из кода выбирает разрез, step листает кадры сплит-анимации
 useEffect(() => {
 if (live.mid === null || !SPLIT_KEYS.includes(live.mid)) {
 setX0(live.mid);
 setStepIdx(0);
 // eslint-disable-next-line react-hooks/exhaustive-deps
 }, [live.mid]);
 }, [live]);
 useEffect(() => {
 if (live.step === null) return;
 setStepIdx(Math.max(0, Math.min(steps.length - 1, live.step)));
 // eslint-disable-next-line react-hooks/exhaustive-deps
 }, [live.step, steps.length]);
 const done = idx >= steps.length - 1;

 const inL = new Set<NodeId>();

```

```

 <div className="bg-sky-950/20 rounded-xl p-3" style={lStyle}>
 <h5 className="text-sm font-bold text-sky-300">Левое дерево</h5>
 <TreeView root={lTree} width={480} height={300}></TreeView>
 </div>
 <div className="bg-rose-950/20 rounded-xl p-3" style={rStyle}>
 <h5 className="text-sm font-bold text-rose-300">Правое дерево</h5>
 <TreeView root={rTree} width={480} height={300}></TreeView>
 </div>
 </div>
 <div className="text-xs text-slate-400 bg-slate-900 p-3" style={infoStyle}>
 Как это работает на самом деле: идём с корнем (или с веткой
 дерева), отдаётся вместе с веткой – делаем копию,
 </div>
</div>
);
};

/* ===== РЕЖИМ MERGE ===== */

const MergeMode: React.FC = () => {
 const [stepIdx, setStepIdx] = useState(0);
 const live = useTreapLive();
 const { l, r } = useMemo(() => splitTree(buildTree(P0, P1)), [P0, P1]);
 const merged = useMemo(() => mergeTree(cloneTree(l), cloneTree(r)), [l, r]);
 const steps = merged.trace;

 // step из кода листает кадры слияния
 useEffect(() => {
 if (live.step === null) return;
 setStepIdx(Math.max(0, Math.min(steps.length - 1, live.step)));
 // eslint-disable-next-line react-hooks/exhaustive-deps
 }, [live.step, steps.length]);
 const idx = Math.min(stepIdx, steps.length - 1);
 const done = idx >= steps.length - 1;
 const cur = steps[idx];

```

```

 Грамотно про «слияние»: у корней сравни
 в (то, что сохраняет порядок x) отправляется ,
 </div>
</div>
);
};

/* ===== РЕЖИМ ERASE ===== */

const EraseMode: React.FC = () => {
 const [key, setKey] = useState(9);
 const [stepIdx, setStepIdx] = useState(0);
 const live = useTreapLive();
 const fullTree = useMemo(() => buildTree(POINTS), [])
 const res = useMemo(() => eraseNode(buildTree(POINTS)
 const steps = res.trace;

 // key из кода выбирает удаляемую точку, step листает
 useEffect(() => {
 if (live.key === null || !POINTS.some((p) => p.key === key))
 setKey(live.key);
 setStepIdx(0);
 // eslint-disable-next-line react-hooks/exhaustive-deps
 }, [live.key]);
 useEffect(() => {
 if (live.step === null) return;
 setStepIdx(Math.max(0, Math.min(steps.length - 1, live.step)));
 // eslint-disable-next-line react-hooks/exhaustive-deps
 }, [live.step, steps.length]);
 const idx = Math.min(stepIdx, steps.length - 1);
 const done = idx >= steps.length - 1;

 const pathIds = useMemo(() => {
 const ids: NodeId[] = [];
 for (const s of steps.slice(0, idx + 1)) if (s.atId !== null)
 return ids;
 }, [steps, idx]);
 const pathSet = new Set(pathIds);

```

```

 </div>

 <div className="text-xs text-slate-400 bg-slate-900" style={
 Ответ на «убрать можно только самый верхний
 ягивает merge двух детей. Удаляется любая<
 </div>
 </div>
);
};

/* ===== РЕЖИМ LAYERS (2k/2k+1) =====

const LayersMode: React.FC = () => {
 const [pick, setPick] = useState<"heap" | "treap">("h
 // Полное дерево из 7 узлов (куча): индексы 1..7
 const heapArr = [0, 98, 45, 90, 32, 21, 76, 12];
 const full = useMemo(() => buildTree(POINTS), []);
 // Честная слотовая разметка treap: индексы как в куче
 const slots = useMemo(() => {
 const arr: (string | null)[] = Array(16).fill(null);
 const walk = (n: TNode | null, i: number) => {
 if (!n || i >= 16) return;
 arr[i] = n.id;
 walk(n.left, 2 * i);
 walk(n.right, 2 * i + 1);
 };
 walk(full, 1);
 return arr;
 }, [full]);
 const nodeCount = slots.filter(Boolean).length;

 return (
 <div className="space-y-4">
 <div className="flex gap-2">
 <button onClick={() => setPick("heap")} className="
 ext-white" : "bg-slate-700 text-slate-300"}`>
 <button onClick={() => setPick("treap")} className="
 text-white" : "bg-slate-700 text-slate-300">

```

```

 казателями (left/right), а не арифметикой.<
 </div>
 })

 <div className="text-xs text-slate-400 bg-slate-900" >
 Грабля «дети в массиве по 2k/2k+1»: фор
 ивым: слои дырявые, и «адрес» ребёнка посчита
 </div>
</div>
);
};

/* ===== РЕЖИМ SEARCH (поиск ≠ сортировка) =====

const SearchMode: React.FC = () => {
 // Подобрано так, что бинарный поиск ГАРАНТИРОВАННО пр
 // mid (индекс 3) = 9 > 7 → уходит влево, а цель лежи
 const shuffled: Pair[] = [
 { key: 12, grade: 0 },
 { key: 3, grade: 6 },
 { key: 45, grade: 8 },
 { key: 9, grade: 3 },
 { key: 6, grade: 5 },
 { key: 2, grade: 4 },
 { key: 7, grade: 2 },
 { key: 1, grade: -4 },
];
 const target = 7;
 const sortedByKey = [...shuffled].sort((a, b) => a.key
 const [stage, setStage] = useState<0 | 1 | 2>(0); //
 const [bidx, setBidx] = useState(0);

 // шаги бинарного поиска на отсортированном массиве д
 const bsSteps = useMemo(() => {
 const steps: { lo: number; hi: number; mid: number;
 let lo = 0, hi = sortedByKey.length - 1;
 while (lo <= hi) {
 const mid = Math.floor((lo + hi) / 2);

```

```

 return (
 <div key={` ${p.key}:${p.grade}-${i}`} className=
 position-all ${highlight ? "border-rose-500 b
 r-amber-500 bg-amber-950/30" : inBs ? "bord
 <div className="text-white font-bold">{p.
 <div className="text-slate-400 text-[10px
 </div>
 </div>
 </div>
 </div>

 <div className="text-xs text-slate-300 bg-slate-900
 {stage === 0 && <>Массив пар не отсортирован
 а беспорядок.</>}
 {stage === 1 && <> Бинарный поиск посмотрел на
 ю половину... в которой лежала цель. На
 реставил: массив как был в беспорядке, та
 {stage === 2 && bidx < bsSteps.length - 1 && <>
 {stage === 2 && bidx >= bsSteps.length - 1 && <>
 сравнения. Но чтобы поиск вообще заработал,
 сравнения на всех. «Отсортирую бинарни
 </div>

 <div className="text-xs text-slate-400 bg-slate-900
 Есть ли разница, чем сортировать? Сравн
 ня граница, каждое сравнение даёт 1 бит. Но
 ля маленьких целых у (вкладка «Собрать», кноп
 задать порядок соединения.
 </div>
 </div>
);
};

/* ===== РЕЖИМ GAME ===== */

const GameMode: React.FC = () => {
 const [edges, setEdges] = useState<GEdge[]>([]);
 const [sel, setSel] = useState<Pair | null>(null);

```

```

<button onClick={() => { setEdges(CANON_GAME.map
); }} className="px-3 py-1.5 rounded-lg text-
линий:
</div>

```

```

<div className="bg-emerald-950/20 rounded-xl p-4 l
 <svg viewBox={`0 0 ${PL_W} ${PL_H + 20}`} classl
 <defs>
 <pattern id="gm-grid" width="14" height="14"
 <path d="M 14 0 L 0 0 0 14" fill="none" s
 </pattern>
 </defs>
 <rect x="34" y="16" width={PL_W - 46} height=
 <line x1="34" y1={PY(0)} x2={PL_W - 12} y2={P
 <line x1={PX(1) - 8} y1="16" x2={PX(1) - 8} y
 {edges.map((e, i) => {
 <line
 key={`ge-${i}`}
 x1={PX(e.a.key)} y1={PY(e.a.grade)}
 x2={PX(e.b.key)} y2={PY(e.b.grade)}
 stroke={i === edges.length - 1 ? "#34d399"
 strokeWidth={i === edges.length - 1 ? 3.5
 className="transition-all duration-300"
 />
 })}
)}

```

```

{POINTS.map((p) => {
 const isSel = sel && sameP(sel, p);
 const isHover = hover && sameP(hover, p) &&
 return {
 <g key={`gp-${p.key}:${p.grade}`} onClick=
 l)} className="cursor-pointer">
 <circle cx={PX(p.key)} cy={PY(p.grade)}
 <circle cx={PX(p.key)} cy={PY(p.grade)}
 "#a5b4fc" : isHover ? "#94a3b8" : "#38bdf8"
 transition-all duration-200" />
 <text x={PX(p.key)} y={PY(p.grade) + 3.5

```



```

* step / i – номер шага (листаёт кадры Split / Merge)
* height, n – высота дерева и число узлов (показывает)
*/
const useTreapLive = () => {
 const runtime = useVizRuntime();
 const vars = runtime?.variables;

 const pairsText = useMemo(() => {
 if (!vars) return null;
 const rawPts = vizArray(vars.points) ?? vizArray(vars.points);
 if (!rawPts || rawPts.length === 0) return null;
 const parts: string[] = [];
 rawPts.forEach((it) => {
 if (Array.isArray(it) && it.length >= 2) {
 const x = vizNumber(it[0]);
 const y = vizNumber(it[1]);
 if (x !== null && y !== null) parts.push(`${x} ${y}`);
 } else if (typeof it === "string") {
 const m = it.match(/-?\d+(?:\.\d+)?/g);
 if (m && m.length >= 2) parts.push(`${m[0]} ${m[1]}`);
 }
 });
 return parts.length ? parts.join(", ") : null;
 }, [vars]);

 const mid = vars ? vizNumber(vars.mid) : null;
 const key = vars ? vizNumber(vars.key) : null;
 const step = vars ? vizNumber(vars.step) ?? vizNumber(0) : null;
 const height = vars ? vizNumber(vars.height) ?? vizNumber(0) : null;
 const n = vars ? vizNumber(vars.n) : null;
 return { vars, pairsText, mid, key, step, height, n };
};

/* ===== КОРПУС ===== */

type Tab = "build" | "split" | "merge" | "erase" | "layout";

const TABS: { id: Tab; label: string }[] = [

```

```

 className={`px-4 py-2 rounded-lg text-xs font-medium
 900/60 text-slate-400 hover:text-white border
 >
 {t.label}
 </button>
)})
 </div>

```

```

{ /* что сейчас сообщил компилятор */ }
<div className="mb-4 flex flex-wrap items-center" >
 <span className="text-[10px] uppercase tracking
 <Terminal className="w-3.5 h-3.5" /> из компилятора

 {live.vars ? (
 <>
 {live.pairsText && <span className="text-[10px] font-medium
 d px-2 py-1">points = {live.pairsText}
 {live.n !== null && <span className="text-[10px] font-medium
 py-1">n = {live.n}}
 {live.height !== null && <span className="text-[10px] font-medium
 px-2 py-1">height = {live.height}}
 {live.mid !== null && <span className="text-[10px] font-medium
 2 py-1">mid = {live.mid}}
 {live.key !== null && <span className="text-[10px] font-medium
 x-2 py-1">key = {live.key}}
 {live.step !== null && <span className="text-[10px] font-medium
 x-2 py-1">step = {live.step}}
 </>
) : (

 открой панель Python: переменные <code class="font-mono text-emerald-300">mid</code>, <code class="font-mono text-emerald-300">for</code>
 step</code> управляют этой демонстрацией

)}
</div>

```

```

{tab === "build" && <BuildMode />}

```

```
export function VizLiveStatus({ demoTitle, className =
 const runtime = useVizRuntime();
 const vars = runtime?.variables;
 const names = vars ? Object.keys(vars).filter((name) =>
 const linked = names.length > 0;

 const matrixSource = pickLiveVar(vars, MATRIX_NAMES);
 const matrix = liveMatrix(matrixSource?.value);

 const detail = matrix
 ? `${matrixSource?.name} ${matrix.sizeLabel}`
 : names.length
 ? names.slice(0, 6).join(", ") + (names.length > 6 ? "..." : "")
 : undefined;

 return (
 <VizSourceBadge
 linked={linked}
 awaiting={!linked && !!runtime}
 line={runtime?.line}
 detail={detail}
 demoTitle={demoTitle}
 className={className}
 />
);
}
```

---

ФАЙЛ 80/111: src/components/vizRegistry.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

---

```
import React from "react";

import { ArticulationPointsViz } from "../ArticulationPointsViz";
import BellmanFordViz from "../BellmanFordViz";
import BfsViz from "../BfsViz";
```

```

*/
export const VIZ_REGISTRY: Record<string, VizEntry> = {
 "stack-dfs": {
 title: "Стек и обход в глубину",
 hint: "Кладите и снимайте тарелки — это ровно то, что нужно",
 Component: StackViz,
 },
 "queue-bfs": {
 title: "Очередь и обход в ширину",
 hint: "Очередь слева, живой BFS по графу справа.",
 Component: () => (
 <div className="space-y-10">
 <QueueViz />
 <BfsViz />
 </div>
),
 },
 "heap-beam-search": {
 title: "Куча (priority queue)",
 hint: "Добавляйте гипотезы — куча сама держит лучшую",
 Component: HeapViz,
 },
 "segment-trees": {
 title: "Дерево отрезков",
 hint: "Запросы и обновления на отрезке за $O(\log n)$ ",
 Component: SegmentTreeVisualizer,
 },
 "sparse-table": {
 title: "Разрезанная таблица (1D и 2D)",
 hint: "Наведите курсор на любую ячейку — подсветится",
 Component: SparseTableViz,
 },
 treap: {
 title: "Treap: плоскость → дерево → Split/Merge/Eraser",
 hint: "7 вкладок: Собрать (сортировка + соединение) → Проверить → Обернуть → Обернуть → Проверить → Проверить → Проверить",
 Component: TreapBuildViz,
 },

```

```

 title: "Флойд–Уоршелл",
 Component: FloydViz,
 },
 "johnson-algo": { title: "Алгоритм Джонсона", Component: JohnsonAlgo,
 mst: {
 title: "Остовное дерево: Краскал, Прима, Борувка",
 hint: "Три алгоритма (билеты 18–20) на одном графе и переменные parent/mst/taken/u/v/w перекрашивают граф.",
 Component: KruskalSimulator,
 },
 "graph-components": {
 title: "Компоненты связности: обход и DSU",
 hint: "Две вкладки: покраска обходом (каждый новый компонент из вашего кода красят вершины.",
 Component: GraphComponentsViz,
 },
 "complexity-classes": {
 title: "Классы сложности и сведения",
 hint: "Иерархия $P \subseteq NP \subseteq PSPACE \subseteq EXP$, сведение $A \leq B$ ",
 Component: ComplexityViz,
 },
 "string-kmp": { title: "Префикс-функция (КМП)", Component: StringKmp,
 "string-z-func": { title: "Z-функция", Component: StringZFunc,
 "aho-corasick": {
 title: "Ахо–Корасик",
 hint: "Бор, суффиксные ссылки и BFS-обход по уровням",
 Component: () => (
 <div className="space-y-10">
 <WaterfallAnimationWidget />
 <SalmonAutomatonWidget />
 </div>
),
 },
 intro: {
 title: "Интерактивный симулятор базовых структур: стек, очередь",
 hint: "Проверь LIFO, FIFO и приоритетную очередь на практике.",
 Component: Simulator,
 },

```

```

 className?: string;
}

/**
 * Явный индикатор источника данных визуализации.
 *
 * Без него демонстрация молча проигрывала собственный
 * когда редактор Python пуст или в нём другие имена/ра
 * так, будто компилятор «исполняет какой-то свой код».
 * «из компилятора» или «встроенное демо».
 */
export function VizSourceBadge({ linked, awaiting = false,
 const tip = linked
 ? `Значения берутся из выполненного Python-кода${detail}
 : awaiting
 ? `Панель Python выполнила код и стоит на строке
 ажмите «шаг вперёд» (→) или «в конец трассы» —
 лизация их не увидит: список имён этой темы есть
 : "Сейчас показана встроенная демонстрация компон
 визуализация переключится на ваши переменные.";

 return (
 <div className={`flex flex-wrap items-center gap-1.5`}>
 <Tooltip side="bottom" content={tip}>
 <span
 tabIndex={0}
 className={`cursor-help inline-flex items-center`}
 linked
 ? "border-emerald-500/50 bg-emerald-500/10"
 : awaiting
 ? "border-sky-500/50 bg-sky-500/10 text-slate-600"
 : "border-slate-600 bg-slate-800/80 text-slate-400"
 `}>
 >
 {linked ? <Cpu className="h-3 w-3" /> : awaiting ? "CPU" : "Python"}
 {linked ? "данные из компилятора" : awaiting ? "встроенное демо"}
 {linked && detail ?

```



```

 setIsSearchDone(false);
 setJumpPath(null);
 setActiveMode("search");
 }
}
if (liveState !== null && liveState >= 0 && WATERFA
if (liveI !== null && liveI >= 0) setCurrentIndex(M
 ToScan).length));
if (liveMatchesSig) {
 const parsed = (JSON.parse(liveMatchesSig) as unk
 .map((it) => {
 if (Array.isArray(it) && it.length >= 2) {
 const idx = vizNumber(it[0]);
 const word = vizString(it[1]);
 if (idx !== null && word) return { index: i
 }
 if (typeof it === "string") {
 const m = it.match(/(\d+)\D+([A-Za-zA-Яa-я]
 if (m) return { index: Number(m[1]), word:
 }
 return null;
 })
 .filter((x): x is { index: number; word: string
 if (parsed.length) setFoundMatches(parsed);
}
// eslint-disable-next-line react-hooks/exhaustive-
}, [vars]));

const compilerLinked = Boolean(liveText || (liveState

// === СОСТОЯНИЯ РЕЖИМА ПОИСКА (SEARCH) ===
const [textToScan, setTextToScan] = useState<string>(
const [currentIndex, setCurrentIndex] = useState<numb
const [currentNode, setCurrentNode] = useState<number
const [isSearchDone, setIsSearchDone] = useState<bool
const [searchLog, setSearchLog] = useState<string>(
 'Режим поиска: Введите текст и жмите «Следующий шаг
);

```



```

 let originalCurr = curr;

 if (TRIE_TRANSITIONS[curr][char] !== undefined) {
 const nextNode = TRIE_TRANSITIONS[curr][char];
 logMsg += `У русла #${curr} (${WATERFALL_NODES[curr].label}). `;
 setJumpPath({ from: curr, to: nextNode, type: 'normal' });
 curr = nextNode;
 setActiveSearchCodeLine(3);
 } else {
 setActiveSearchCodeLine(2);
 let jumps = 0;
 while (curr !== 0 && TRIE_TRANSITIONS[curr][char] === undefined) {
 curr = WATERFALL_NODES[curr].fail;
 jumps++;
 }
 const nextNode = TRIE_TRANSITIONS[curr][char] ? TRIE_TRANSITIONS[curr][char] : 0;
 if (originalCurr === 0) {
 logMsg += `В корне нет перехода по '${char}'. Лист`;
 setJumpPath(null);
 } else {
 logMsg += `Поток по '${char}' у вершины #${originalCurr} не закончился. Переходим по fail-ссылке в #${curr} (${WATERFALL_NODES[curr].label}). `;
 setJumpPath({ from: originalCurr, to: nextNode, type: 'normal' });
 curr = nextNode;
 }
 }

 const currentMatches: { index: number; word: string }[] = [];
 const matchedWordsForLog: string[] = [];

 let temp = curr;
 while (temp !== 0) {
 if (WATERFALL_NODES[temp].is_terminal) {
 WATERFALL_NODES[temp].words.forEach((word) => {
 currentMatches.push({ index: currentIndex, word });
 matchedWordsForLog.push(word);
 });
 }
 temp = WATERFALL_NODES[temp].fail;
 }

```

```

 inspectedParentFail: 0,
 checkTargetChar: 'H',
 checkingBranchesFrom: null,
 },
 {
 targetNodeId: 7,
 title: 'Шаг 2 (Depth 1): Вершина #7 (S)',
 desc: 'Обрабатываем дочернюю вершину #7 (S) на Depth 1. Родитель: H. Разведчик плывет в ROOT и ищет ветку по "S".',
 codeLine: 5,
 scoutPos: 0,
 inspectedParentFail: 0,
 checkTargetChar: 'S',
 checkingBranchesFrom: null,
 },
 {
 targetNodeId: 2,
 title: 'Шаг 3 (Depth 2): Вершина #2 (HE)',
 desc: 'Переходим на Depth 2 к вершине #2 (HE). Родитель: H. Разведчик плывет в ROOT и ищет ветку по "E".',
 codeLine: 3,
 scoutPos: 0,
 inspectedParentFail: 0,
 checkTargetChar: 'E',
 checkingBranchesFrom: 0,
 },
 {
 targetNodeId: 5,
 title: 'Шаг 4 (Depth 2): Вершина #5 (HI)',
 desc: 'Обрабатываем #5 (HI) на Depth 2. Родитель: H. Разведчик плывет в ROOT и ищет ветку по "I". H и S не подходят. Ветки нет, fail[HI] = ROOT',
 codeLine: 3,
 scoutPos: 0,
 inspectedParentFail: 0,
 checkTargetChar: 'I',
 checkingBranchesFrom: 0,
 },
 {

```

```

 codeLine: 4,
 scoutPos: 1,
 inspectedParentFail: 1,
 checkTargetChar: 'E',
 checkingBranchesFrom: 1,
 },
 {
 targetNodeId: 4,
 title: 'Шаг 9 (Depth 4): Вершина #4 (HERS)',
 desc: 'Финальная вершина #4 (HERS) на Depth 4. По-
 #7 (S). fail[HERS] = #7 (S). Автомат полностью
 codeLine: 4,
 scoutPos: 0,
 inspectedParentFail: 0,
 checkTargetChar: 'S',
 checkingBranchesFrom: 0,
 },
];

```

```

const currentTrainInfo = TRAINING_STEPS_INFO[trainStep];
const activeFishPosNode = activeMode === 'training' ?
const targetTrainingNode = WATERFALL_NODES[currentTrainInfo.targetStep];

```

```

return (
 <div className="bg-slate-800/95 rounded-2xl p-6 border" >
 {/* Шапка и переключатель режимов */}
 <div className="flex flex-wrap items-center justify-between" >
 <div className="flex items-center space-x-3" >

 <div>
 <h4 className="text-2xl font-bold text-white" >
 Анимация водопада: Полное обучение автомата
 </h4>
 <p className="text-sm text-slate-400" >
 Визуализация пошагового построения всех ф
 </p>
 </div>
 </div>
 </div>
 </div>

```

```

 <p className="text-xs text-slate-300 mb-3">
 Показаны все шаги очереди BFS (начиная с шага 8 (SHE → HE), где наглядно видно, как происходит обучение)
 </p>
 <div className="space-y-1.5 mb-6 overflow-hidden">
 {TRAINING_STEPS_INFO.map((step, idx) => (
 <button
 key={idx}
 onClick={() => setTrainStep(idx)}
 className={`w-full text-left px-3 py-2 transition-colors duration-200
 ${
 idx === trainStep
 ? 'bg-indigo-600 text-white font-medium'
 : 'bg-slate-800 text-slate-400'
 }`}
 >
 {step}
 {idx === trainStep && Текущий шаг}
 </button>
))}
 </div>
 </div>

 <div className="flex gap-2">
 <button
 onClick={() => setTrainStep((prev) => Math.max(0, prev - 1))}
 disabled={trainStep === 0}
 className="flex-1 bg-slate-700 hover:bg-slate-600 transition-colors duration-200"
 >
 Назад
 </button>
 <button
 onClick={() => setTrainStep((prev) => Math.min(TRAINING_STEPS_INFO.length - 1, prev + 1))}
 disabled={trainStep === TRAINING_STEPS_INFO.length - 1}
 className="flex-1 bg-indigo-600 hover:bg-indigo-500 transition-colors duration-200 shadow-lg shadow-indigo-900/50"
 >
 Вперед
 </button>
 </div>

```

```

 r-l-4 border-emerald-500 text-emerald-300 font-bold
 4. if (trie[v][char] !== undefined) {
 /span>
 {currentTrainInfo.codeLine === 4 && <
 ал IF!}
 </div>
 <div className={`p-1.5 rounded flex items-center
 -4 border-blue-500 text-blue-300 font-bold'
 5. else fail[u] = root; // Бег
 {currentTrainInfo.codeLine === 5 && <
 root}
 </div>
 </div>
</div>

<div>
 <h5 className="text-slate-200 font-bold mb-2">
 <Lightbulb className="w-4 h-4 text-amber-400"/>
 </h5>
 <div className="bg-slate-800 p-4 rounded-lg min-h-[120px]
 w-inner min-h-[72px] flex items-center">
 {currentTrainInfo.desc}
 </div>
</div>
</div>
) : (
 /* ПАНЕЛЬ РЕЖИМА ПОИСКА */
 <div className="grid grid-cols-1 lg:grid-cols-3 gap-4">
 <div className="bg-slate-900/60 p-5 rounded-xl">
 <div>
 <h5 className="text-white font-bold mb-2">
 ❖ Текст для сканирования
 </h5>
 <p className="text-xs text-slate-400 mb-4">
 Введи текст, чтобы лосось просканировал
 </p>
 <input type="text" value={searchText} onChange={handleSearch} />
 </div>
 </div>
 </div>
)

```

```

 isSearchDone || textToScan.length === 0
 ? 'bg-slate-700 text-slate-500 cursor-text'
 : 'bg-gradient-to-r from-blue-600 to-blue-900/40 active:scale-[0.98]'
 }` }
 >
 {currentIndex === 0 ? 'Начать план' : ''}
 <ArrowRight className="w-4 h-4" />
 </button>
</div>
</div>
</div>
<div className="lg:col-span-2 bg-slate-900/60 p-4" >
 <div>
 <h5 className="text-white font-bold mb-3">Текущее действие в коде</h5>
 1. curr = state; char = text[i];
 {activeSearchCodeLine === 1 && инициализация}
 </div>
 <div className={`p-1.5 rounded flex items-center border-rose-500 text-rose-300 font-bold`} >
 2. while (curr !== root && !trie[curr].get(char)) {
 {activeSearchCodeLine === 2 && рыжок по fail!}
 </div>
 <div className={`p-1.5 rounded flex items-center border-blue-400 text-blue-200 font-bold`} >
 3. curr = trie[curr].get(char);
 {activeSearchCodeLine === 3 && ем по течению}
 </div>
 <div className={`p-1.5 rounded flex items-center border-emerald-500 text-emerald-300 font-

```

```

 <span className="w-3 h-0.5 bg-blue-500 in

 <span className="flex items-center gap-1 te
 <span className="w-3 h-0.5 border-t borde

 </div>
 </div>

```

```

 { /* Само SVG дерево водопада */ }
 <div className="w-full overflow-x-auto custom-s
 <svg viewBox="0 0 800 520" className="w-full

```

```

 { /* Горизонтальные линии и подписи уровней */ }
 {[
 { depth: 0, y: 60, label: 'Depth 0 (Корень'
 { depth: 1, y: 160, label: 'Depth 1 (H, S'
 { depth: 2, y: 260, label: 'Depth 2 (HE, I'
 { depth: 3, y: 360, label: 'Depth 3 (HER, '
 { depth: 4, y: 460, label: 'Depth 4 (HERS'
]}.map((level) => {
 <g key={`depth-line-${level.depth}`}>
 <line
 x1="20"
 y1={level.y}
 x2="780"
 y2={level.y}
 stroke="#334155"
 strokeWidth="1"
 strokeDasharray="4,4"
 opacity="0.4"
 />
 <rect x="25" y={level.y - 12} width="14
 />
 <text x="32" y={level.y + 4} fill="#94a
 {level.label}
 </text>
 </g>
)})

```

```

 {char}
 </text>

 {/* ВИЗУАЛЬНЫЕ МЕТКИ ПРОВЕРКИ IF В
 {isTrainingExamined && (
 <g transform={`translate(${(fromN
.y) / 2)}`}>
 <rect x="-15" y="-12" width="30
strokeWidth="2" />
 <text x="0" y="5" fontSize="14"
 {isMatchBranch ? ' ' : ' '}
 </text>
 </g>
)}
 </g>
);
 });
 }}}

{/* Рендер fail-ссылок (пунктирные водопадн
Object.values(WATERFALL_NODES).map((node) :
 if (node.id === 0) return null;
 const targetNode = WATERFALL_NODES[node.f

 // В режиме обучения показываем fail-ссыл
 // Найдем индекс шага, на котором обработ
 const stepIdxForNode = TRAINING_STEPS_INF
 if (activeMode === 'training' && trainStep

 let isHighlighted = activeMode === 'search
e === 'fail';
 let isJustEstablished = activeMode === 't
-ссылку

 // Вычисляем изогнутую кривую для красоты
 const dx = targetNode.x - node.x;
 const dy = targetNode.y - node.y;
 const cx = node.x + dx / 2 + (node.id % 2

```



```

 { /* Основной круг вершины */ }
 <circle
 cx={node.x}
 cy={node.y}
 r={isCurrent || isTargetChild ? '24' : '12'}
 fill={isCurrent ? '#2563eb' : isTargetChild ? '#f8d7da' : '#f8d7da'}
 stroke={isCurrent ? '#ffffff' : isTargetChild ? '#000000' : '#000000'}
 strokeWidth={isCurrent || isTargetChild ? '2' : '1'}
 className="transition-all duration-300"
 />

 { /* Текст внутри вершины */ }
 <text
 x={node.x}
 y={node.y + 5}
 fill={isCurrent || isTargetChild ? 'white' : 'black'}
 fontSize={isCurrent || isTargetChild ? '14' : '12'}
 fontWeight="bold"
 textAnchor="middle"
 >
 {node.label}
 </text>

 { /* Метка искомой вершины в режиме обхода */ }
 {isTargetChild && (
 <g transform={`translate(${node.x}, ${node.y})`}
 <rect x="-35" y="-12" width="70" height="24" fill="#f8d7da" stroke="#000000" strokeWidth="1" />
 <text x="0" y="2" fontSize="10" fill="white" font-weight="bold">
 ИЩЕМ FAIL
 </text>
 </g>
)}

 { /* Индикатор словарных слов */ }
 {hasWords && (
 <g transform={`translate(${node.x} + 10, ${node.y} + 10)`}
 <circle cx="0" cy="0" r="9" fill="#f8d7da" stroke="#000000" strokeWidth="1" />
 </g>
)}
 </div>
</pre>

```

```

 { /* Список найденных совпадений (только в режиме поиска) */
 {activeMode === 'search' && {
 <div className="mt-6 bg-slate-900/50 p-5 rounded" >
 <h5 className="text-white font-bold mb-3 text-sm" >
 Улов лосося (Найденные слова)
 </h5>
 {foundMatches.length === 0 ? {
 <p className="text-sm text-slate-400 italic" >
 Пока ничего не поймали. Запустите шаги симуляции
 </p>
 } : {
 <div className="flex flex-wrap gap-2" >
 {foundMatches.map((match, idx) => {
 <div
 key={idx}
 className="bg-emerald-950 border border-slate-800 text-sm font-bold shadow animate-[scaleUp_0.3s_ease-out]" >
 <CheckCircle2 className="w-4 h-4 text-white" />
 {match.word}

 Индекс {match.index}

 </div>
 })}
 </div>
 })}
 </div>
 })}
 }
}

```

```

 { /* что сейчас сообщил компилятор */ }
 <div className="mt-6 flex flex-wrap items-center" >

 <Terminal className="w-3.5 h-3.5" /> из компилятора

 {compilerLinked ? {
 <div className="flex flex-wrap items-center" >

```

```

 emitVizDemo,
 liveMatrix,
 pickLiveVar,
 useVizRuntime,
 vizArray,
 vizNumber,
 VizChapterContext,
 } from "../../data/vizStepBus";

/**
 * Префиксные суммы: 1D и 2D.
 *
 * Главная фишка – наведение на число:
 * * в 1D наводим на P[i] – подсвечивается кусок массива
 * * в 1D наводим на a[i] – видно, в какие префиксы он входит
 * * в 2D наводим на ячейку – подсвечивается прямоугольник
 * а при выбранном запросе показывается формула включения
 */

const A1 = [3, 1, 4, 1, 5, 9, 2, 6];

const A2 = [
 [1, 2, 3, 4],
 [5, 6, 7, 8],
 [9, 1, 2, 3],
 [4, 5, 6, 7],
];

const cellBase =
 "flex items-center justify-center rounded-md font-mono text-sm" +
 "w-[clamp(30px,9vw,46px)] h-[clamp(30px,9vw,46px)] text-center";

export const PrefixSumViz: React.FC = () => {
 const [mode, setMode] = useState<"1d" | "2d">("1d");
 const chapterId = useContext(VizChapterContext);

 // Вкладка 2D = своё демо (свой скелет): компилятор не знает, что такое 2D
 useEffect(() => {

```

```

/** Снимок i/P из автоматически выполненного Python-к
const runtime = useVizRuntime();
const vars = runtime?.variables;
const liveI = vizNumber(vars?.i);
const liveP = vizArray(vars?.P);
const hasLiveP = !!vars && Object.prototype.hasOwnProperty.call(vars, 'P');
const compilerLinked = liveI !== null || hasLiveP;
// i/P управляют ячейками напрямую; индекс Python-строки
const shownDriven = compilerLinked
 ? liveI !== null
 ? Math.max(0, Math.min(A1.length, Math.trunc(liveI)))
 : Math.max(0, Math.min(A1.length, (liveP?.length || 0)))
 : null;

const covered =
 hover?.kind === "pref"
 ? { from: 0, to: hover.i - 1 }
 : hover?.kind === "a"
 ? { from: hover.i, to: hover.i }
 : null;

const sum = pref[range.r + 1] - pref[range.l];

return (
 <div className="space-y-5">
 <div className="overflow-x-auto pb-1">
 <div className="inline-block min-w-full">
 {/* индексы */}
 <div className="flex gap-1.5 mb-1 pl-[52px]">
 {A1.map((_, i) => (
 <div key={i} className={` ${cellBase} !h-5`}>
 {i}
 </div>
))}
 </div>

 {/* массив a */}
 <div className="flex gap-1.5 items-center mb-1">

```

```

const isDrivenCur = shownDriven !== null && !hover ? (
const shownValue = hasLiveP ? liveValue : null
return (
 <div
 key={i}
 onMouseEnter={() => setHover({ kind: 'P', i })}
 onMouseLeave={() => setHover(null)}
 className={` ${cellBase} cursor-help ${
 isDrivenCur
 ? "bg-emerald-600 text-white ring-2 ring-emerald-600"
 : blank
 ? "bg-slate-950 text-slate-600"
 : masked
 ? "bg-slate-800/50 text-slate-700"
 : active
 ? "bg-emerald-500 text-white"
 : isR
 ? "bg-emerald-700 text-white"
 : isL
 ? "bg-rose-700 text-white"
 : "bg-slate-900 text-slate-300"
 }` }
 title={blank || masked ? `P[${i}] - ${shownValue}` : null}
 >
 {blank || masked ? "." : String(shownValue)}
 </div>
)
)
)
)
</div>
</div>
</div>

```

```

/* Пояснение под курсором */
<div className="min-h-[62px] bg-slate-900 border border-slate-800" >
 {shownDriven !== null && !hover ? (
 <p className="text-slate-300">
 <b className="text-emerald-400">Компилятор
 i =

```

```

 { /* Запрос */
 <div className="bg-slate-900 border border-indigo
 <div className="text-xs text-slate-400 mb-2">
 Запрос суммы на отрезке (кликайте по массиву
 </div>
 <p className="font-mono text-sm sm:text-base te
 sum(a[{range.l}..{range.r}]) = P[{range.r + 1
 {pref[range
 {pref[range.l
 {
 </p>
 <p className="text-[11px] text-slate-500 mt-1">
 Один вычитание вместо цикла: запрос за O(1) п
 </p>
 </div>
 </div>
);
};

/* ----- 2D -----
const Prefix2D: React.FC = () => {
 const n = A2.length;
 const m = A2[0].length;
 const runtime = useVizRuntime();
 const vars = runtime?.variables;
 const liveI = vizNumber(vars?.i);
 const liveJ = vizNumber(vars?.j);

 const P = useMemo(() => {
 const p = Array.from({ length: n + 1 }, () => Array
 for (let i = 1; i <= n; i++) {
 for (let j = 1; j <= m; j++) {
 p[i][j] = A2[i - 1][j - 1] + p[i - 1][j] + p[i
 }
 }
 return p;
 }, [n, m]);

```

```

 : null;

useEffect(() => {
 const r1 = vizNumber(vars?.r1);
 const c1 = vizNumber(vars?.c1);
 const r2 = vizNumber(vars?.r2);
 const c2 = vizNumber(vars?.c2);
 if (r1 === null || c1 === null || r2 === null || c2
 setRect({
 r1: Math.max(0, Math.min(srcRows - 1, Math.trunc(
 c1: Math.max(0, Math.min(srcCols - 1, Math.trunc(
 r2: Math.max(0, Math.min(srcRows - 1, Math.trunc(
 c2: Math.max(0, Math.min(srcCols - 1, Math.trunc(
 }));
}, [vars, srcRows, srcCols]));

// Границы запроса держим внутри реальной сетки (у по
const r1 = Math.max(0, Math.min(rect.r1, prefRows - 1
const c1 = Math.max(0, Math.min(rect.c1, prefCols - 1
const r2 = Math.max(r1, Math.min(rect.r2, prefRows -
const c2 = Math.max(c1, Math.min(rect.c2, prefCols -

const D = cellAt(r1, c1);
const B = cellAt(r1, c2 + 1);
const Cc = cellAt(r2 + 1, c1);
const Aa = cellAt(r2 + 1, c2 + 1);
const total = Aa !== null && B !== null && Cc !== nul
const fmt = (v: number | null) => (v === null ? "-" ;

const cornerOf = (i: number, j: number) => {
 if (i === r2 + 1 && j === c2 + 1) return "A";
 if (i === r1 && j === c2 + 1) return "B";
 if (i === r2 + 1 && j === c1) return "C";
 if (i === r1 && j === c1) return "D";
 return null;
};

return (

```

```

 </div>
 </div>

 {/* Матрица префиксных сумм */}
 <div className="overflow-x-auto">
 <div className="text-xs text-slate-400 mb-2">
 Префиксные суммы {liveS ? `${livePrefSource`
 </div>
 <div className="inline-block">
 {grid.map((row, i) => {
 <div key={i} className="flex gap-1.5 mb-1">
 {row.map((v, j) => {
 const blank = v === null || v === und
 const shownValue = v;
 const corner = cornerOf(i, j);
 const isHover = hover?.i === i && hov
 const isCompilerCell = liveI === i &&
 const cornerColor =
 corner === "A"
 ? "bg-emerald-600 text-white"
 : corner === "D"
 ? "bg-sky-600 text-white"
 : corner
 ? "bg-rose-600 text-white"
 : "bg-slate-900 border border
 return {
 <div
 key={j}
 onMouseEnter={() => setHover({ i,
 onMouseLeave={() => setHover(null
 className={`${cellBase} cursor-he
 isCompilerCell
 ? "bg-emerald-600 text-white
 : isHover
 ? "bg-amber-500 text-white
 : blank
 ? "bg-slate-950 text-slate
 : cornerColor

```



```

 </p>
 <p className="font-mono text-[11px] text-slate-500">
 прямоугольник ({r1}, {c1}) ... ({r2}, {c2}) · с
 </p>
 <p className="text-[11px] text-slate-500 mt-1">
 Вычли два «хвоста» и вернули дважды вычитенный
 </p>
 </div>
</div>
);
};

```

ФАЙЛ 84/111: src/components/visualizers/SparseTableViz.  
 КАТЕГОРИЯ: Визуализаторы (вложенные) | СТРОК: 899 | РАЗМ: 1000

```

import React, { useContext, useEffect, useRef, useState } from "react";
import {
 emitVizDemo,
 liveMatrix,
 useVizRuntime,
 vizArray,
 vizNumber,
 vizRecord,
 VizChapterContext,
} from "../../data/vizStepBus";
import { motion, AnimatePresence } from "motion/react";
import {
 Play,
 RotateCcw,
 ChevronRight,
 ChevronLeft,
 Lock,
 ChevronDown,
} from "lucide-react";

```

```

for (let kx = 0; kx <= 3; kx++) {
 for (let ky = 0; ky <= 3; ky++) {
 if (kx === 0 && ky === 0) continue;
 for (let r = 0; r + (1 << kx) <= 8; r++) {
 for (let c = 0; c + (1 << ky) <= 8; c++) {
 if (kx > 0) {
 ST2D[r][c][kx][ky] = Math.min(
 ST2D[r][c][kx-1][ky],
 ST2D[r + (1 << (kx-1))][c][kx-1][ky]
);
 } else {
 ST2D[r][c][kx][ky] = Math.min(
 ST2D[r][c][kx][ky-1],
 ST2D[r][c + (1 << (ky-1))][kx][ky-1]
);
 }
 }
 }
 }
}

```

```

export const SparseTableViz: React.FC = () => {
 // Эта страница посвящена именно двумерной таблице: 1D
 // вкладкой, но больше не подменяет 2D-визуализацию и
 const [tab, setTab] = useState<"1d" | "2d_build" | "2d_viz">("1d");
 const chapterId = useContext(VizChapterContext);

 // Каждая вкладка = своё демо с собственным скелетом:
 useEffect(() => {
 if (chapterId) emitVizDemo(chapterId, tab === "2d_build" ? "2d_build" : "2d_viz");
 }, [chapterId, tab]);

 return (
 <div className="w-full bg-slate-950 p-3 sm:p-4 md:p-6">
 <div className="flex gap-2 sm:gap-4 mb-5 border-b border-slate-800">
 <button
 onClick={() => setTab("1d")}
 />
 </div>
 </div>
);
}

```

```

const runtimeVars = runtime?.variables;
const liveI = vizNumber(runtimeVars?.i);
const liveJ = vizNumber(runtimeVars?.j);
const liveSource = vizArray(runtimeVars?.a);
const liveRows = vizArray(runtimeVars?.st);
const hasLiveSource = !!runtimeVars && Object.prototype.hasOwnProperty.call(runtimeVars, 'a');
const hasLiveTable = !!runtimeVars && Object.prototype.hasOwnProperty.call(runtimeVars, 'st');
const compilerLinked = liveI !== null || liveJ !== null;
const shownSource = hasLiveSource
 ? liveSource?.length
 ? liveSource
 : Array.from({ length: N }, () => null)
 : arr1D;
const liveTarget =
 liveI !== null && liveJ !== null && Number.isInteger(liveI) && Number.isInteger(liveJ)
 ? { i: liveI, j: liveJ }
 : null;

/** Размерность таблицы пользователя: сетка демо 8x4,
const liveTable = liveMatrix(runtimeVars?.st);
const dimMismatch = liveTable
 ? liveTable.rows !== N || liveTable.cols !== LOG
 ? `Демо-сетка построена для массива из ${N} элементов. Таблица ниже рисует ячейки демо-сетки: значения элементов в панели переменных компилятора.`
 : null
 : hasLiveSource && liveSource && liveSource.length
 ? `Демо-массив из ${N} элементов, а в вашем коде используется демо-размера ${N}x${LOG}.`
 : null;

// Total steps: we animate computing each element for
const computeSteps: { i: number; j: number }[] = [];
for (let j = 1; j < LOG; j++) {
 for (let i = 0; i + (1 << j) <= N; i++) {
 computeSteps.push({ i, j });
 }
}

```

```

 <button
 onClick={() => setStep(0)}
 className="p-2 bg-slate-800 hover:bg-slate-700"
 >
 <RotateCcw size={20} />
 </button>
 </div>
 </div>

 <div className="bg-slate-900 rounded-xl border border-slate-800" >
 <div className="text-[11px] uppercase tracking-wider" >
 <div className="flex gap-1 sm:gap-1.5 overflow-x-auto" >
 {shownSource.map((v, idx) => {
 const inCover = !!covered && idx >= covered
 const display = v === undefined || v === null
 ? "."
 : typeof v === "object"
 ? JSON.stringify(v)
 : String(v);
 return (
 <div key={idx} className="flex flex-col items-center justify-center gap-1" >
 <div
 className={`flex items-center justify-center gap-1 w-44px h-[clamp(26px,8vw,44px)] text-[clamp(9px,1.2em,14px)] font-mono font-normal ${
 inCover ? "bg-indigo-500 text-white" : "bg-slate-800 text-slate-300"
 }`}
 >
 {display}
 </div>
 {v}
 </div>
);
 })}
 </div>
 <div className="mt-2 text-xs min-h-[36px]" >
 {hover && covered ? (
 <p className="text-slate-300">
 <b className="text-sky-400">

```

```

</div>
<div className="text-center font-bold text-
rap">
 2^3 (L=8)
</div>

```

```

{/* Matrix */}
{Array.from({ length: N }).map((_, i) => {
 <React.Fragment key={i}>
 <div className="text-slate-500 font-mon
 {i}
 </div>
 {Array.from({ length: LOG }).map((_, j) => {
 const isValid = i + (1 << j) <= N;
 const liveRow = vizArray(liveRows?.[i]
 const liveValue = liveRow?.[j];
 // Уровень j=0 – это исходный a[i], о
 // очищаемого рабочего массива и не д
 const sourceValue = j === 0 ? liveSou
 const runtimeValue = liveValue ?? sou
 const runtimeHasValue = runtimeValue
 const manualVisible =
 isValid &&
 (j === 0 || computeSteps.findIndex(
 const isVisible = compilerLinked ? run

let isActive = !!liveTarget && liveTa
let isSrc1 = false;
let isSrc2 = false;

```

```

if (compilerLinked && liveTarget && l
 isSrc1 = i === liveTarget.i && j ===
 isSrc2 = i === liveTarget.i + (1 <<
} else if (!compilerLinked && step > 1
 const currentStep = computeSteps[ste
 isActive = currentStep.i === i && c
 if (currentStep.j - 1 === j) {
 if (currentStep.i === i) isSrc1 =

```

```

 : isSrc2
 ? "bg-amber-500 text-white"
 : isVisible
 ? "bg-slate-800 text-white"
 : compilerLinked
 ? "bg-slate-950 text-white"
 : "bg-transparent"
 `}
 title={compilerLinked ? `st[${value}]` : ""}
 >
 {displayValue}
 </motion.div>
)}

```

```

{/* Arrows visualization */}
{(isSrc1 || isSrc2) && (
 <motion.svg
 initial={{ opacity: 0 }}
 animate={{ opacity: 1 }}
 className="absolute pointer-events-none"
 style={{
 width: 100,
 height: isSrc2 ? 40 + (1 << isSrc1 ? 40 : 0) : 40,
 right: -50,
 top: 24,
 overflow: "visible",
 }}
 >
 <path
 d={`M 0 0 C 30 0, 30 ${isSrc1 ? 40 : 0} 0 0`}
 fill="none"
 stroke={isSrc1 ? "#10b981" : "#f59e00"}
 strokeWidth="3"
 />
 </motion.svg>
)}
</div>
);

```

```


) ={" "}
 <span className="text-indigo-400 font-b
 {st1D[c.i][c.j]}

 </p>
);
))()
) : (
 <p className="text-slate-500 italic">
 Нажмите далее, чтобы увидеть, как блоки сво
 </p>
 })
</div>
</div>
);
};

```

```

const Viz2DBuild: React.FC = () => {
 const [k, setK] = useState(1);
 const [hover, setHover] = useState<{ r: number; c: num
 const [locked, setLocked] = useState<{ r: number; c: n
 const runtime = useVizRuntime();
 const vars = runtime?.variables;
 const liveK = vizNumber(vars?.k);
 /**
 * Шаг, закреплённый кликом по вкладке. Без него live
 * перетирал выбор пользователя и вкладки «Шаг 1/2/3»
 * клик есть, а матрица не меняется. Закрепление живё
 */
 const [pinnedK, setPinnedK] = useState<number | null>
 const prevLiveK = useRef<number | null>(liveK);
 useEffect(() => {
 if (liveK !== prevLiveK.current) {
 prevLiveK.current = liveK;
 setPinnedK(null);
 }
 }, [liveK]);

```

```

<p className="text-sm text-slate-400 mb-6 text-
 Для наглядности показываем только <b className=

 <span className="text-rose-300 animate-pulse
 иц!
</p>
{dimMismatch2D ? {
 <p className="mb-4 max-w-[520px] rounded-lg b
 {dimMismatch2D}
 </p>
} : null}

<div className="flex bg-slate-950 p-1 rounded-l
 {[0, 1, 2, 3].map(step => {
 <button
 key={step}
 onClick={() => { setK(step); setPinnedK(s
 className={`px-4 py-2 rounded-md text-sm
 'text-slate-400 hover:text-white hover:bg-
 >
 {step === 0 ? "Оригинал (1x1)" : `Шаг ${s
 </button>
 }}}
</div>
{pinnedK !== null && compilerLinked && {
 <p className="-mt-6 mb-6 text-[11px] text-sla
 Шаг закреплён вручную: когда <span classNam
 демонстрация вернётся к шагу кода.
 </p>
}}

<div className="flex flex-col items-center gap-
 scrollbar">
 {[...Array(shownK + 1)].map((_, i) => shownK
 const stepL = 1 << step;
 const stepSize = 8 - stepL + 1;
 const isCurr = step === shownK;

```



```

// Значения в ячейке ещё не
// контур «сюда смотрит код
// залитая пустая клетка вы
highlightClass = isBlank
 ? `bg-indigo-500/10 text-
== c ? `ring-[3px] ring-white ring-offset-2
 : `bg-indigo-500 text-whi
&& locked.c === c ? `ring-[3px] ring-white
zIndex = `z-10`;
} else {
 const prevL = 1 << (shownK
 const isTop = r < activeCel
 const isLeft = c < activeCe

 if (isTop && isLeft) {
 highlightClass = `bg-ros
md:scale-[1.08]`;
 } else if (isTop && !isLeft
 highlightClass = `bg-blur
] md:scale-[1.08]`;
 } else if (!isTop && isLeft
 highlightClass = `bg-eme
9,0.3)] md:scale-[1.08]`;
 } else {
 highlightClass = `bg-amb
3)] md:scale-[1.08]`;
 }
 zIndex = `z-10`;
}
}

return (
 <div
 key={idx}
 onMouseEnter={() => { if(is
 onMouseLeave={() => { if(is
 onClick={() => {
 if(isCurr) {

```

```

 </div>
) : (
 <>
 <p className="mb-4">При <span classN
драты имеют размер <span className="font-bo
 <div className="bg-[#0a0fle] rounded
 /

 </div>
 <p className="mt-6 italic text-indig
 <Play size={14} className="fill-in
a.
 </p>
 </>
)}
</div>
) : (
 <div className="flex flex-col gap-4 animat
 <div className="bg-[#0a0fle] rounded-xl l
ate-300 shadow-inner overflow-x-auto">
 // Те
 int<
="text-amber-300">1}

 // Кв
 ST</
 <span className="text-rose
e="text-amber-300">1], <sp
 <span className="text-blue
ext-amber-300">1][k-<span className=
pan>

 <span className="text-emer
="text-amber-300">1][k-<span classNa
span>

 <span className="text-ambe
white">c + pL][k-<span className="te
ame="text-slate-500">// Правый Низ<b
 {''}}));
 </div>

```

```

 <div className="flex item
 <span className="flex
shadow-[0_0_8px_#f59e0b]"></div> ST[{active
 <span className="text
[activeCell.r + prevL][activeCell.c + prevL
 </div>
 </>
)
 }}({})
</div>

 <div className="pt-3 mt-3 border-t
) = <span className="text-white
0">{ST2D[activeCell.r][activeCell.c][shownK
 </div>
 <p className="mt-4 text-[12px] font
список матриц (слева) вниз, чтобы увидеть,
 </div>
) : {
 <div className="bg-slate-950/40 border
mt-2 flex items-center justify-center anim
 Наведите курсор на матрицу {L}x{L
 </div>
 }
 </div>
 }}
</div>
</div>
</div>
);
};

```

```

const Viz2DQuery: React.FC = () => {
 const [r1, setR1] = useState(1);
 const [c1, setC1] = useState(1);
 const [r2, setR2] = useState(5);
 const [c2, setC2] = useState(6);
 const runtime = useVizRuntime();

```

```

 <p className="mb-3 rounded-lg border b
 {queryMismatch}
 </p>
) : null}

 <div className="flex flex-col items-cent
 <div className="grid grid-cols-[repea
 active shadow-inner w-max max-w-full">
 {Array.from({length: 64}).map((_,
 const r = Math.floor(idx / 8);
 const c = idx % 8;
 const isInQuery = r >= r1 && r

 return (
 <div key={idx} className={`
 y-center rounded text-[clamp(9px,2.4vw,12px
 0 border-rose-500/50 border scale-105 z-10'
 {val2D[r][c]}
 </div>
)
)}}

 <div
 className="absolute border-[3px
 ow-[0_0_15px_rgba(244,63,94,0.3)] border-da
 style={{
 top: `${(r1 / 8) * 100}%`,
 left: `${(c1 / 8) * 100}%`,
 marginTop: '8px',
 marginLeft: '8px',
 height: 'calc(' + (H/8*100) +
 width: 'calc(' + (W/8*100) +
 }}
 />

 {/* Показываем 4 угла в самой матрице */}
 <div className="absolute pointer-e
 shadow-[0 0 10px #f43f5e]" style={{ top: `

```

```

 <p className="text-sm font-sans text-slate-300 rounded text-white">{Lx}. Аналогично
 te">{Ly}.
 </p>
 <p className="text-sm font-sans text-slate-300 rounded text-white">
 Чтобы покрыть весь прямоугольник, мы
 Name="font-mono bg-[#0a0fle] px-1 rounded b
 роса. (Цветные прямоугольники слева).
 </p>

 <div className="bg-[#0a0fle] rounded-xl text-slate-300 shadow-inner overflow-x-auto">
 <div className="absolute right-3 top-3">
 // 1.
 int<
 ext-amber-300">1}; <span className="
 int<
 ext-amber-300">1}; <span className="
 int<
 "text-slate-500">// Выс: {Lx}

 int<
 "text-slate-500">// Шир: {Ly}
<b

 // 2.
 int<
 В запроса)

 r1][<span className="text-white bg-r

 во, чтобы правый край уперся в c2)<b
 r1][<span className="text-white bg-b

 px, чтобы нижний край уперся в r2)<b

```

```

 return (
 <div key={idx} className={
tion-all duration-300 relative shrink-0 ${c
 {txt}
 {badge && <div classNam
r-slate-700 text-white px-1 font-bold round
 </div>
)
 }}}}
 </div>
 </div>
</div>

 <div className="mt-auto bg-slate-950 fon
)]">
 <div className="text-center font-bold
wrap bg-[#0a0f1e] py-3 px-2 rounded-lg bord
 <span className="text-slate-400 fon

 <span className="text-blue-400 m
n>
 <span className="text-emerald-40
span>
 <span className="text-amber-400 r
 <span className="text-slate-4
 <span className="ml-3 text-emerald-
+ 1][c1][kx][ky], ST2D[r2 - Lx + 1][c2 - Ly
 </div>
 </div>

 </div>
</div>
)
}

```

---

```

 } else {
 document.body.classList.remove('wtf-mode');
 if (aside) aside.style.display = '';
 if (headerMain) headerMain.style.display = '';
 if (questionsContainer) questionsContainer.style.display = '';
 if (wtfContainer) wtfContainer.classList.remove('hidden');
 }
}

function setTheme(theme) {
 const body = document.body;
 body.classList.remove('theme-dark', 'theme-light', 'theme-notebook');
 body.classList.add('theme-' + theme);

 // Just some visual feedback on buttons
 document.querySelectorAll('[id^="theme-"]')
 .forEach(btn => {
 btn.classList.remove('outline', 'outline-none');
 });

 if (theme === 'dark') document.getElementById('btn-theme-dark').classList.add('text-white transition-colors');
 if (theme === 'light') document.getElementById('btn-theme-light').classList.add('text-white transition-colors');
 if (theme === 'terminal') document.getElementById('btn-theme-terminal').classList.add('text-white transition-colors');

 document.getElementById('btn-theme-dark').classList.add('text-white transition-colors');
 document.getElementById('btn-theme-light').classList.add('text-white transition-colors');
 document.getElementById('btn-theme-notebook').classList.add('text-white transition-colors');
 document.getElementById('btn-theme-terminal').classList.add('text-white transition-colors');
}

// Initialize theme and mode
setTheme('dark');
```

```

document.querySelectorAll("#mobile-drawer a
 link.addEventListener("click", () => {
 if (mobileDrawer && !mobileDrawer.c
 toggleDrawer();
 }
 }));
});

const observerOptions = {
 root: null,
 rootMargin: '-10% 0px -70% 0px',
 threshold: 0
};

const observer = new IntersectionObserver((
 entries.forEach(entry => {
 if (entry.isIntersecting && !document
 // Highlight active TOC link lo
 const id = entry.target.id;
 document.querySelectorAll('aside
 if (link.getAttribute('href
 link.classList.add('bg-
 } else {
 link.classList.remove('l
 }
 }));
 }
 }));
}, observerOptions);

document.querySelectorAll('section[id^="que
 observer.observe(section);
});
});
</script>
</body>
</html>

```



```
}
.uno-card::before { top: 2px; left: 4px; }
.uno-card::after { bottom: 2px; right: 4px; tra
.uno-card.mini { width: 45px !important; height
.uno-card.mini::before, .uno-card.mini::after {

/* Global formula scroll settings */
.formula-scroll {
 overflow-x: auto;
 overflow-y: hidden;
 -webkit-overflow-scrolling: touch;
 scrollbar-width: none; /* Firefox */
 -ms-overflow-style: none; /* IE and Edge */
}
.formula-scroll::-webkit-scrollbar {
 display: none;
}

mjax-container {
 max-width: 100% !important;
}
mjax-container[display="true"] {
 overflow-x: auto;
 overflow-y: hidden;
 max-width: 100%;
 scrollbar-width: none;
 -ms-overflow-style: none;
 -webkit-overflow-scrolling: touch;
}
mjax-container[display="true"]::-webkit-scrollba
 display: none;
}

@media (max-width: 640px) {
 html { font-size: 13px; }

 .uno-card { width: 40px; height: 60px; font
 .uno-card::before, .uno-card::after { font-
```

```

/* THEME OVERRIDES */
body.theme-light {
 background-color: #f8fafc !important;
 color: #0f172a !important;
}
body.theme-light .bg-slate-900, body.theme-light .bg-slate-800 { background-color: #0f172a !important; }
body.theme-light .bg-slate-700, body.theme-light .bg-slate-600 { background-color: #1e293b !important; }
body.theme-light .text-slate-200, body.theme-light .text-slate-300 { color: #e2e3e5 !important; }
body.theme-light .text-slate-400 { color: #475569 !important; }
body.theme-light .border-slate-600, body.theme-light .border-slate-500 { border-color: #1e293b !important; }
body.theme-light #top-controls { background-color: #f8fafc !important; }

body.theme-notebook {
 background-color: #fcf8eb !important; /* light beige */
 color: #3b2818 !important; /* brown ink */
 background-image: repeating-linear-gradient(45deg, transparent, transparent 2px, #3b2818 2px, #3b2818 4px);
 background-attachment: local !important;
 font-family: "Comic Sans MS", "Caveat", "Serif";
}
body.theme-notebook .bg-slate-900, body.theme-notebook .bg-slate-800, body.theme-notebook .bg-slate-700 {
 background-color: rgba(255, 255, 255, 0.7) !important;
 border-color: #cbd5e1 !important;
 box-shadow: 2px 2px 5px rgba(0,0,0,0.05);
}
body.theme-notebook .text-slate-200, body.theme-notebook .text-slate-300 { color: #e2e3e5 !important; }
body.theme-notebook .text-slate-400 { color: #475569 !important; }
body.theme-notebook .border-slate-600, body.theme-notebook .border-slate-500 { border-color: #1e293b !important; }
body.theme-notebook .font-mono { font-family: "Monospace", "Courier New", "Serif"; }
body.theme-notebook #top-controls { background-color: #fcf8eb !important; }

/* CREEPY BLINKING */

```

```

 <button onclick="setTheme('dark')" id="
transition-colors">Темная</button>
 <button onclick="setTheme('light')" id=
ext-white hover:bg-slate-700 transition-col
 <button onclick="setTheme('notebook')"
over:text-white hover:bg-slate-700 transiti
 </div>
</div>
</div>

<!-- ОПРЕДЕЛЕНИЕ МАРКЕРОВ SVG -->
<svg width="0" height="0" class="absolute">
 <defs>
 <marker id="arrow-yellow" markerWidth="6" m
 <path d="M0,0 L6,3 L0,6 Z" fill="#fcd34d"
 </marker>
 <marker id="arrow-orange" markerWidth="6" m
 <path d="M0,0 L6,3 L0,6 Z" fill="#f9731d"
 </marker>
 <marker id="arrow-pink" markerWidth="6" mar
 <path d="M0,0 L6,3 L0,6 Z" fill="#f472b0"
 </marker>
 <marker id="arrow-cyan" markerWidth="6" mar
 <path d="M0,0 L6,3 L0,6 Z" fill="#2dd4bf"
 </marker>
 <marker id="arrow-indigo" markerWidth="6" m
 <path d="M0,0 L6,3 L0,6 Z" fill="#818cf8"
 </marker>
 <marker id="arrow-lime" markerWidth="6" mar
 <path d="M0,0 L6,3 L0,6 Z" fill="#a3e63d"
 </marker>
 <marker id="arrow-green" markerWidth="6" ma
 <path d="M0,0 L6,3 L0,6 Z" fill="#4ade80"
 </marker>

 <!-- ЛЕГО БЛОКИ -->
 <g id="lego-yellow">
 <rect x="0" y="0" width="40" height="20

```



```
</div>
</details>

<details class="group">
 <summary class="list-none cursor-pointer text-amber-500 pl-3 font-bold text-amber-300">
 4. Степени и корни

 </summary>
 <div class="pl-6 space-y-2 text-amber-300">

 </div>
</details>

<details class="group">
 <summary class="list-none cursor-pointer text-amber-500 pl-3 font-bold text-amber-300">
 5. Группа корней из единицы

 </summary>
 <div class="pl-6 space-y-2 text-amber-300">

 </div>
</details>

<details class="group">
 <summary class="list-none cursor-pointer text-amber-500 pl-3 font-bold text-amber-300">
 6. Степени и корни

 </summary>
 <div class="pl-6 space-y-2 text-amber-300">

 </div>
</details>
```

```


 </div>
</details>

<details class="group">
 <summary class="list-none cursor-pointer text-green-500 pl-3 font-bold text-green-400">
 9. Кольцо многочленов

</summary>
 <div class="pl-6 space-y-2 text-green-400">

 </div>
</details>

<details class="group">
 <summary class="list-none cursor-pointer text-rose-500 pl-3 font-bold text-rose-400">
 10. Делимость и Схема Гильберта

</summary>
 <div class="pl-6 space-y-2 text-rose-400">

 </div>
</details>

```

```
 <summary class="list-none cursor-pointer-emerald-500 pl-3 font-bold text-emerald-500">
 13. Корни многочленов.

 </summary>
 <div class="pl-6 space-y-2 text-emerald-500">

 </div>
</details>
```

```
<details class="group">
 <summary class="list-none cursor-pointer-fuchsia-500 pl-3 font-bold text-fuchsia-500">
 14. Кратность корня. Ко

 </summary>
 <div class="pl-6 mt-2 space-y-2 text-fuchsia-500">

 deg)
 К
 </div>
</details>
```

```
<details class="group">
 <summary class="list-none cursor-pointer-cyan-500 pl-3 font-bold text-cyan-400">
 15. Производная. Критер

 </summary>
 <div class="pl-6 mt-2 space-y-2 text-cyan-400">
```

```
 <a href="#question-21" class="block
go-500 pl-3 text-slate-400">
 <span class="font-bold text-ind

 <a href="#question-22" class="block
sia-500 pl-3 text-slate-400">
 <span class="font-bold text-fuch

```

```
 <a href="#proof-algorithms" class="l
-red-500 pl-3 mt-4 text-red-500 font-bold m
 ▲ Ликбез: Алгоритмы выживания

```

```
</nav>
```

```
<script>
```

```
 document.querySelectorAll('details.
 detail.addEventListener('toggle
 if (detail.open) {
 document.querySelector
 if (other !== detail
 other.removeAtt
 }
 });
 }
 });
});
```

```
// Scroll spy logic
document.addEventListener('DOMConten
 const sections = document.query
="q5-"], div[id^="q6-"], div[id^="q7-"], di
v[id^="q13-"], div[id^="q14-"], div[id^="q1
 let isScrolling = false;
```

```
 document.querySelectorAll('.grou
 link.addEventListener('clie
 isScrolling = true;
```



```

 threshold: [0, 0.5]
 });

 sections.forEach(section => {
 observer.observe(section);
 });
});

function fallbackModal(contentRaw, ...args) {
 const modal = document.createElement('div');
 modal.style.position = 'fixed';
 modal.style.top = '0';
 modal.style.left = '0';
 modal.style.width = '100vw';
 modal.style.height = '100vh';
 modal.style.backgroundColor = 'black';
 modal.style.zIndex = '99999';
 modal.style.display = 'flex';
 modal.style.flexDirection = 'column-reverse';
 modal.style.alignItems = 'center';
 modal.style.justifyContent = 'center';

 const box = document.createElement('div');
 box.style.width = '80%';
 box.style.height = '80%';
 box.style.backgroundColor = '#1a1a1a';
 box.style.padding = '20px';
 box.style.borderRadius = '10px';
 box.style.display = 'flex';
 box.style.flexDirection = 'column-reverse';
 box.style.color = '#fff';

 const header = document.createElement('div');
 header.innerText = 'Экспорт в PDF';
 header.style.marginBottom = '10px';

 const subHeader = document.createElement('div');
 subHeader.innerText = 'Из-за некорректной работы';

```

```

 const closeBtn = document.createElement('button');
 closeBtn.innerText = 'Заккрыть';
 closeBtn.style.padding = '10px';
 closeBtn.style.backgroundColor = 'white';
 closeBtn.style.color = '#fff';
 closeBtn.style.border = 'none';
 closeBtn.style.borderRadius = '10px';
 closeBtn.style.cursor = 'pointer';
 closeBtn.onclick = () => modal.close();

 box.appendChild(header);
 box.appendChild(subHeader);
 if (title !== 'PDF') {
 box.appendChild(textarea);
 btnContainer.appendChild(closeBtn);
 }
 btnContainer.appendChild(closeBtn);
 box.appendChild(btnContainer);
 modal.appendChild(box);
 document.body.appendChild(modal);
 }
}

```

```

function downloadMD() {
 let clone = document.querySelector('#md').cloneNode(true);

 // Smart markdown parsing based on the original text
 // Smart markdown parsing based on the original text
 clone.querySelectorAll('h2').forEach(el => {
 clone.querySelectorAll('h3').forEach(el => {
 clone.querySelectorAll('.analog').forEach(el => {
 clone.querySelectorAll('li').forEach(el => {
 clone.querySelectorAll('svg').forEach(el => {
 let caption = '';
 if (el.nextElementSibling && el.nextElementSibling.tagName === 'img') {
 caption = el.nextElementSibling.alt;
 }
 el.textContent = '\n\n[Иллюстрация: ' + caption + ']';
 });
 });
 });
 });
 });
}

```



```

 }
 },
 },
 document.head.appendChild(style)
}
</script>

<!-- ФИКСИРОВАННЫЕ БЛОКИ: СМЕНА ТЕМЫ И ...
<div class="mt-8 flex flex-col gap-4 bo
 <!-- Кнопки Конвертации / Экспорта
 <div class="flex flex-col gap-2">
 <div class="text-xs text-slate-
 <button onclick="downloadWord()
ont-bold flex items-center justify-center g
 Word
 </button>
 <a href="algebra.pdf" target="_
-sm font-bold flex items-center justify-cen
 PDF

 <button onclick="downloadMD()"
-bold flex items-center justify-center gap-
 Markdown
 </button>
 <button onclick="downloadHTML()
t-sm font-bold flex items-center justify-ce
 HTML
 </button>
 </div>

<!-- Смена темы -->
<div>
 <div class="text-xs text-slate-
 <div class="flex items-center g
 <button id="theme-dark" onc

```

```

9. Ко
10.
11.
12.
13.
14.
15.
17.
18.
19.
20.
21.
22.

 </div>
</header>

<!-- ===== WTF CONTAINER =====>
<div id="wtf-container" class="hidden">
 <h2 class="text-3xl font-black text-cen

 <div class="overflow-x-auto w-full pb-4">
 <div class="grid grid-cols-3 gap-4">
 <!-- Column headers -->
 <div class="bg-yellow-900/40 bo
оля и кольца, целые числа</div>
 <div class="bg-orange-900/40 bo
ольцо полиномов</div>
 <div class="bg-cyan-900/40 bord
сные числа</div>

 <!-- Row 1: База -->
 <a href="#question-1" onclick="setM
-4 border-yellow-500 hover:bg-slate-700 tra
 <div class="text-xs text-yellow
 <div class="font-bold text-slate

 <a href="#question-9" onclick="setM

```

```

 <!-- Empty 3rd row cell -->
 <div class="hidden md:block"></div>

 <!-- Row 4: Факторизация / Структура -->

-4 border-yellow-500 hover:bg-slate-700 transition
 <div class="text-xs text-yellow-500">
 <div class="font-bold text-slate-700">

-1-4 border-orange-500 hover:bg-slate-700 transition
 <div class="text-xs text-orange-500">
 <div class="font-bold text-slate-700">

-4 border-cyan-500 hover:bg-slate-700 transition
 <div class="text-xs text-cyan-500">
 <div class="font-bold text-slate-700">

 <!-- Row 5: Корни / Уравнения -->
 <div class="hidden md:block"></div>

-1-4 border-orange-500 hover:bg-slate-700 transition
 <div class="text-xs text-orange-500">
 <div class="font-bold text-slate-700">

-4 border-cyan-500 hover:bg-slate-700 transition
 <div class="text-xs text-cyan-500">
 <div class="font-bold text-slate-700">

 <!-- Row 6: Кратность -->
 <div class="hidden md:block"></div>

-1-4 border-orange-500 hover:bg-slate-700 transition
 <div class="text-xs text-orange-500">

```

```

 <a href="#question-21" onclick="setl
-l-4 border-indigo-500 hover:bg-slate-700 t
 <div class="text-xs text-indigo
 <div class="font-bold text-slate

 <a href="#question-22" onclick="setl
-l-4 border-fuchsia-500 hover:bg-slate-700
 <div class="text-xs text-fuchsia
 <div class="font-bold text-slate

 <!-- КЛАСТЕР 4: АЛГОРИТМЫ ВЫЖИВАНИЯ
 <div class="col-span-1 md:col-span-3
border-b border-red-500/30 pb-2">Кластер 4:

 <a href="#proof-algorithms" onclick=
:col-span-3 bg-red-900/20 p-4 border-l-4 bo
 <div class="text-xs text-red-500
 <div class="font-bold text-slate

</div>
</div>

<!-- ЛИКБЕЗ SECTION INSERTED HERE -->
<section id="proof-algorithms" class="m
 <div class="flex items-center mb-6">
 <div class="bg-red-500/20 p-3 r
 <svg class="w-8 h-8 text-re
rg/2000/svg">
 <path stroke-linecap="r
110 4m-6 8a2 2 0 100-4m0 4a2 2 0 110-4m0 4
 </svg>
 </div>
 <h2 class="text-3xl pr-4 border
 </div>

 <div class="prose prose-invert max-w
 <p class="text-lg">
```

```

 <ul class="list-disc pl-5 sp-0">

 , r_2).

 счет ограничений (размер остатка) докажи, что

 </div>

```

```

 <!-- Индукция -->
 <div class="bg-slate-800/80 p-6">
 <h3 class="text-xl text-yellow-400">
 <p class="text-sm text-slate-400">
 <ul class="list-disc pl-5 sp-0">

 валось.

 Просто перепиши формулу с буквой k .

 усок k , замени его на жареного карася из

 </div>

```

```

 <!-- Двойная делимость -->
 <div class="bg-slate-800/80 p-6">
 <h3 class="text-xl text-orange-400">
 <p class="text-sm text-slate-400">
 <ul class="list-disc pl-5 sp-0">

 вверх по уравнениям).

 рху вниз, показывая, что $\delta \vdots r_i$

 х. Твой d больше.

 </div>

```





```

import { GLOSSARY_BY_ID } from "../data/glossary";
import { annotateTerms } from "../hooks/useTermHints";
import { annotateTicketLinks } from "../ticketLinks";

/**
 * Выгрузка главы (или всего пособия) в самостоятельный
 *
 * Файл получается автономным: стили защиты внутрь, инт
 * его можно открыть с флешки, отправить в мессенджер и
 * Живые React-визуализации в статику не переносятся –
 * подставляется заметка со ссылкой на онлайн-версию, а
 * которые в приложении всплывают по наведению, собираю
 * в конце документа.
 */

/** Собирает CSS страницы: сначала пробуем правила, при
async function collectCss(): Promise<string> {
 const parts: string[] = [];
 for (const sheet of Array.from(document.styleSheets))
 try {
 const rules = (sheet as CSSStyleSheet).cssRules;
 parts.push(Array.from(rules).map((r) => r.cssText));
 } catch {
 const href = (sheet as CSSStyleSheet).href;
 if (!href) continue;
 try {
 const res = await fetch(href);
 if (res.ok) parts.push(await res.text());
 } catch {
 /* чужой домен – пропускаем */
 }
 }
 }
 return parts.join("\n");
}

const EXTRA_CSS = `
/* – правки для автономного файла – */
```

```

* Возвращает готовый HTML главы и список найденных терминов
*/
function prepareBody(html: string, opts: { selfId?: string;
 body: string;
 termIds: string[];
}) {
 const host = document.createElement("div");
 host.innerHTML = html;

 try {
 annotateTerms(host);
 } catch {
 /* если браузер не осилил регулярку – выгружаем текст как есть */
 }

 /*
 * Упоминания «билет 15» становятся якорями на главу 15
 * только для книги, где все главы на месте: в файле book.html
 * вёл бы в никуда, а битые ссылки – ровно то, от чего мы хотим избежать
 */
 if (opts.ticketAnchors) {
 try {
 annotateTicketLinks(host, { selfId: opts.selfId });
 host.querySelectorAll<HTMLAnchorElement>("a[data-goto]").forEach(
 a => {
 a.setAttribute("href", `#ch-${a.dataset.goto}`);
 a.removeAttribute("data-goto");
 }
);
 } catch {
 /* ссылки – улучшение, а не обязательная часть файла */
 }
 }

 const ids: string[] = [];
 host.querySelectorAll<HTMLElement>(".term-hint").forEach(
 el => {
 const id = el.dataset.termId;
 if (!id) return;
 if (!ids.includes(id)) ids.push(id);
 const link = document.createElement("a");
 }
);
}
```

```

 return `<!doctype html>
<html lang="ru">
<head>
<meta charset="utf-8">
<meta name="viewport" content="width=device-width, init
<title>${escapeHtml(opts.title)}</title>
<style>${opts.css}</style>
<style>${EXTRA_CSS}</style>
</head>
<body>
<div class="export-shell">
 <header class="export-head">
 <div style="font-size:.7rem;text-transform:uppercas
 <h1 style="color:#fff;font-size:1.6rem;font-weight:
 <div style="font-size:.72rem;color:#475569;margin-t
 </header>

 <div class="export-note">
 Это автономная копия: стили внутри файла, интернет
 Интерактивные тренажёры и анимации в статичный HTML
 они живут в онлайн-версии: <a href="${escapeHtml(op
 </div>

 <article class="prose prose-invert max-w-none">
${opts.body}
 </article>

${opts.gloss}

 <footer class="export-foot">Универсальное пособие по
</div>
</body>
</html>`;
}

/**
 * Картинки встраиваются в файл data-URL: автономная ко
 * иллюстрации офлайн (раньше src вел на ассет приложен

```

```

const base = import.meta.env.BASE_URL || "/";
const manifestRes = await fetch(`${base}export/viz-
if (!manifestRes.ok) return "";
const manifest = (await manifestRes.json()) as { fi
const mine = manifest.filter((m) => m.file === `${c
const figures: string[] = [];
for (const m of mine.slice(0, 4)) {
 const res = await fetch(`${base}export/viz-shots/
 if (!res.ok) continue;
 const blob = await res.blob();
 if (blob.size > 3_000_000) continue;
 const data = await new Promise<string>((resolve,
 const fr = new FileReader();
 fr.onload = () => resolve(String(fr.result));
 fr.onerror = () => reject(fr.error);
 fr.readAsDataURL(blob);
 });
 figures.push(
 `<figure class="export-fig">${m.name ? `Режим «${escapeHtml(
);
}
return figures.length
 ? `<div class="export-figs"><h3 class="export-fig
 : "";
} catch {
 return "";
}
}

```

```

function triggerDownload(html: string, filename: string
 const blob = new Blob([html], { type: "text/html;char
 const url = URL.createObjectURL(blob);
 const a = document.createElement("a");
 a.href = url;
 a.download = filename;
 document.body.appendChild(a);
 a.click();

```

```

 .map((c) => `<li style="margin:.2rem 0"><a href="#c
 .join("\n");

for (const c of chapters) {
 const { body: rawBody, termIds } = prepareBody(c.co
 // картинки глав встраиваются сразу: книга одна, ас
 // снимки демонстраций подтягиваются из /export/viz
 const body = (await vizFigures(c.id)) + (await inli
 termIds.forEach((t) => {
 if (!allTerms.includes(t)) allTerms.push(t);
 });
 bodies.push(`<div id="ch-${escapeHtml(c.id)}" style
<p style="text-align:right;font-size:.72rem;margin:0 0
 }

 const body = `<nav id="toc" style="background:#0f172a
 rem">
 <h2 style="color:#fff;font-size:1.05rem;font-weight:8
 <ol style="margin:0;padding-left:1.1rem;font-size:.85
</nav>
 ${bodies.join("\n")}`;

 const html = wrap({
 title: "Универсальное пособие по алгоритмам",
 subtitle: `Все темы, ${chapters.length} шт.`,
 css,
 body,
 gloss: glossarySection(allTerms, true),
 origin: window.location.origin,
 });
 return html;
}

/** Выгрузить всё пособие одним файлом. */
export async function downloadBookHtml(chapters: Chapte
 triggerDownload(await buildBookHtml(chapters), "posob
}
```

```

/**
 * Константы TreeWalker числами: глобальный `NodeFilter`
 * во всех окружениях (в jsdom его нет) – с числами разн
 */
const SHOW_TEXT = 4;
const FILTER_ACCEPT = 1;
const FILTER_REJECT = 2;

/** Внутри чего ссылки не ставим: код, уже существующие
const SKIP_SELECTOR = "code, pre, script, style, a, but

/** Одна страница на диапазон? «билеты 18–20» → mst, а
function chapterForRange(from: number, to: number): string {
 let found: string | undefined;
 for (let n = from; n <= to; n += 1) {
 const id = ticketToChapter.get(n);
 if (!id) return undefined;
 if (found && found !== id) return undefined;
 found = id;
 }
 return found;
}

function hrefFor(id: string, lite: boolean): string {
 return lite ? `?lite=1&topic=${encodeURIComponent(id)}
}

/**
 * Строковая версия – для текстовой (`?lite=1`) версии,
 * через `dangerouslySetInnerHTML`, а эффекты разметки
 * HTML режется на «тег» и «текст», меняются только тек
 * вне `pre/code/a`.
 */
export function linkifyTicketsHtml(html: string, opts:
 let inCode = 0;
 let inLink = 0;
 return html

```

```

 if (!parent) return FILTER_REJECT;
 if (parent.closest(SKIP_SELECTOR)) return FILTER_REJECT;
 const text = node.nodeValue ?? "";
 if (!/билет/i.test(text)) return FILTER_REJECT;
 return FILTER_ACCEPT;
 },
));

```

```

const targets: Text[] = [];
let current = walker.nextNode();
while (current) {
 targets.push(current as Text);
 current = walker.nextNode();
}

```

```

let created = 0;
for (const textNode of targets) {
 const text = textNode.nodeValue ?? "";
 TICKET_RE.lastIndex = 0;
 const frag = document.createDocumentFragment();
 let last = 0;
 let m: RegExpExecArray | null;
 while ((m = TICKET_RE.exec(text)) !== null) {
 const from = Number(m[1]);
 const to = m[2] ? Number(m[2]) : from;
 const id = chapterForRange(from, to);
 if (!id || id === opts.selfId) continue;
 if (m.index > last) frag.appendChild(document.createTextNode(
 const a = document.createElement("a");
 a.className = "ticket-link";
 a.dataset.goto = id;
 a.setAttribute("href", hrefFor(id, opts.lite ?? false));
 a.textContent = m[0];
 frag.appendChild(a);
 last = m.index + m[0].length;
 created += 1;
 }
 }
 if (last === 0) continue;
}

```



```
import { fileURLToPath } from "node:url";
import { build } from "esbuild";

const ROOT = path.resolve(path.dirname(fileURLToPath(import.meta.url)), "..");
const TMP = path.join(ROOT, "tmp", "audit");

fs.mkdirSync(TMP, { recursive: true });
const bundle = path.join(TMP, "content.bundle.mjs");

await build({
 entryPoints: [path.join(ROOT, "src/data/content.ts")],
 bundle: true,
 format: "esm",
 platform: "node",
 outfile: bundle,
 logLevel: "error",
});

const { chapters } = await import(`file://${bundle}`);

// Список интерактивов берём из реестра визуализаторов
const registry = fs.readFileSync(path.join(ROOT, "src/registry.json"), "utf-8");
const registryBody = registry.slice(registry.indexOf("["), registry.lastIndexOf("]"));
const vizIds = new Set([...registryBody.matchAll(/^\s{2}id: "([^"]+)"/)].map(m => m[1]));

const rows = chapters.map((c) => {
 const html = c.content ?? "";
 const analogies = (html.match(/Аналогия/gi) ?? []).length;
 return {
 id: c.id,
 title: c.title,
 num: Number.parseInt(c.title.match(/^(\\d+)\\.\\/)?.[1]),
 analogies,
 hasAnalogy: analogies > 0,
 inlineSvg: /<svg[\\s>]/i.test(html),
 image: /<img[\\s>]/i.test(html),
 interactive: vizIds.has(c.id),
 chars: html.length,
 };
});
```

ФАЙЛ 91/111: scripts/audit-terms.mjs

КАТЕГОРИЯ: Скрипты сборки и экспорта | СТРОК: 68 | РАЗМ

```
/**
 * Проверка покрытия глав всплывающими подсказками:
 * в каждой ли теме находятся термины из глоссария.
 *
 * node scripts/audit-terms.mjs
 */
import { build } from "esbuild";
import path from "node:path";

const ROOT = process.cwd();
await build({
 entryPoints: ["src/data/content.ts", "src/data/glossa
 bundle: true, format: "esm", platform: "node",
 outdir: "tmp/audit-terms", outExtension: { ".js": ".m
 external: ["react", "react-dom", "motion", "lucide-re
});

const { chapters } = await import(path.join(ROOT, "tmp/
const { GLOSSARY_LABELS, GLOSSARY } = await import(path

const esc = (s) => s.replace(/[\.*+?^${}()|[\]\$\s]/g, "\\
const re = new RegExp(
 `(?!<![\\p{L}\\p{N}_-])(${GLOSSARY_LABELS.map((l) => e
 "giu"
);
const map = new Map(GLOSSARY_LABELS.map((l) => [l.label

// те же лимиты, что и в src/hooks/useTermHints.ts
const MAX_PER_SURFACE = 2;
const MAX_PER_TERM = 8;

const bad = [];
const used = new Set();
```

```
/**
 * Компактный PDF пособия: public/export/guide_lite.pdf
 *
 * текст всех тем (без служебной разметки)
 * + снимки визуализаций, если они отрендерены (tmp/viz-
 * capture-viz-shots.mjs в CI через headless-браузер)
 * + иллюстрации страниц (jpg из src/assets)
 * + текстовое описание каждой демонстрации и переменные
 * Python-компилятор (работает и без браузера – локально)
 * – БЕЗ кода панели компилятора и без навигации приложения
 *
 * Запуск: npm run export:guide
 * Цель: 40–60 страниц (жёсткий предел – 100).
 *
 * Вёрстка целиком на собственном курсоре (cy): pdfkit
 * сам, длинные таблицы/код/боксы режутся на фрагменты
 */
import fs from "node:fs";
import path from "node:path";
import { build } from "esbuild";
import PDFDocument from "pdfkit";
import { ROOT, OUT_DIR, ensureDir, humanSize } from "../utils";
import { htmlToBlocks } from "../html-to-blocks.mjs";

const GUIDE_PDF_PATH = path.join(OUT_DIR, "guide_lite.pdf");
const SHOTS_DIR = path.join(ROOT, "tmp", "viz-shots");
const ENTRY = path.join(ROOT, "tmp", "guide-entry.ts");

/** Векторные схемы графов и таблицы состояний для PDF */
const VECTOR_VISUALIZERS = {
 dijkstra: {
 title: "Векторный граф релаксации (Дейкстра)",
 nodes: [
```

```

 { id: "D", label: "D", x: 310, y: 30, dist: "6",
 { id: "E", label: "E", x: 310, y: 90, dist: "7",
 { id: "F", label: "F", x: 400, y: 60, dist: "8",
],
 edges: [
 { u: "S", v: "A", w: 4, highlighted: true },
 { u: "S", v: "B", w: 5, highlighted: true },
 { u: "B", v: "C", w: -2, highlighted: true },
 { u: "A", v: "C", w: 1 },
 { u: "C", v: "D", w: 3, highlighted: true },
 { u: "C", v: "E", w: 4, highlighted: true },
 { u: "D", v: "F", w: 2 },
 { u: "E", v: "F", w: 1, highlighted: true },
 { u: "D", v: "A", w: -2 },
],
 table: [
 ["Проход (итерация)", "Рёбра в работе", "Изменени
 ["Пара 1", "S→A, S→B", "dist[A]=4, dist[B]=5", "н
 ["Пара 2", "B→C (-2)", "dist[C]=3", "нормальный х
 ["Пара 3", "C→D, C→E", "dist[D]=6, dist[E]=7", "н
 ["Пара 4", "E→F (1)", "dist[F]=8", "нормальный хо
 ["Итерация V-1", "все рёбра сошлись", "без измене
 ["Итерация V (контроль)", "все рёбра", "без измен
],
},
floyd: {
 title: "Матрица кратчайших путей (Фloyd–Уоршелл)",
 table: [
 ["из \\ в", "S (0)", "A (1)", "B (2)", "C (3)", "D (4)"],
 ["S (0)", "0", "4", "5", "3", "6"],
 ["A (1)", "∞", "0", "∞", "1", "4"],
 ["B (2)", "∞", "∞", "0", "-2", "1"],
 ["C (3)", "∞", "∞", "∞", "0", "3"],
 ["D (4)", "∞", "∞", "∞", "∞", "0"],
],
},
mst: {
 title: "Минимальный остов (Краскал / Прим / Борушка

```

```

const CANDIDATES = {
 regular: [
 "/usr/share/fonts/truetype/dejavu/DejaVuSans.ttf",
 "/usr/share/fonts/dejavu/DejaVuSans.ttf",
 "/usr/local/share/fonts/DejaVuSans.ttf",
 "/System/Library/Fonts/Supplemental/Arial Unicode.ttf",
 "C:/Windows/Fonts/arial.ttf",
],
 bold: [
 "/usr/share/fonts/truetype/dejavu/DejaVuSans-Bold.ttf",
 "/usr/share/fonts/dejavu/DejaVuSans-Bold.ttf",
 "/usr/local/share/fonts/DejaVuSans-Bold.ttf",
 "C:/Windows/Fonts/arialbd.ttf",
],
 oblique: [
 "/usr/share/fonts/truetype/dejavu/DejaVuSans-Oblique.ttf",
 "/usr/share/fonts/dejavu/DejaVuSans-Oblique.ttf",
],
 mono: [
 "/usr/share/fonts/truetype/dejavu/DejaVuSansMono.ttf",
 "/usr/share/fonts/dejavu/DejaVuSansMono.ttf",
 "/usr/local/share/fonts/DejaVuSansMono.ttf",
 "C:/Windows/Fonts/consola.ttf",
],
};
```

```
const pick = (list) => list.find((f) => fs.existsSync(f))
```

```
/**
```

```
 * Убираем символы, которых нет в DejaVu (эмодзи и пиктограммы)
 * на их месте появляется «пустой» глиф. Стрелки, математические
 * геометрические фигуры DejaVu знает — их оставляем.
 */
```

```
const UNSUPPORTED = /[\u{1F000}-\u{1FAFF}\u{2700}-\u{2700}
 22}]/gu;
```

```
/** Русское склонение счётных существительных: 1 страница
```

```
const plural = (n, one, few, many) => {
```

```
 const mod10 = Math.abs(n) % 10;
```

```

}

/* ----- СНИМКИ ВИЗУАЛИЗАЦИЙ -----

function shotsFor(chapterId) {
 const dirs = [SHOTS_DIR, path.join(ROOT, "public", "e
 for (const dir of dirs) {
 if (fs.existsSync(dir)) {
 const files = fs
 .readdirSync(dir)
 .filter((f) => /\.(png|jpe?g)$/i.test(f))
 .filter((f) => f === `${chapterId}.png` || f.st
 .sort();
 if (files.length > 0) {
 return files.map((f) => ({
 file: path.join(dir, f),
 name: f.replace(/\.(png|jpe?g)$/i, "").split(
 }));
 }
 }
 }
 return [];
}

```

/\* ----- отрисовка PDF -----

```

class Renderer {
 constructor(doc, fonts) {
 this.doc = doc;
 this.fonts = fonts;
 this.footerTitle = "";
 this.pages = 0;
 this.cy = TOP;
 }

 /**
 * Колонтитул текущей страницы (печатаем до перехода к
 *

```

```
}
```

```
/** Сколько места нужно, чтобы элемент не оторвался о
ensure(h, extra = 0) {
 if (this.cy + h + extra > BOTTOM) this.newPage();
}
```

```
height(text, { font, size, width, lineGap = 0 }) {
 return this.doc.font(font).fontSize(size).heightOfS
}
```

```
/**
```

```
* Режет текст на фрагменты не выше max пунктов: снач
* предложений, слишком длинное предложение – по слов
*
* Без этого pdfkit сам переносит остаток на новую стр
* создаётся в обход рендерера – без нижнего колонтиту
* из-за чего в файле оказывалось на одну страницу бо
*/
```

```
splitToFit(text, opts, max) {
 const measure = (t) => this.height(t, opts);
 const source = String(text);
 if (measure(source) <= max) return [source];
 const chunks = [];
 let current = "";
 const flush = () => {
 if (current.trim()) chunks.push(current.trim());
 current = "";
 };
 for (const sentence of source.split(/(?<=[.!?…;:])\
 const withSentence = current ? `${current} ${sent
 if (measure(withSentence) <= max) {
 current = withSentence;
 continue;
 }
 flush();
 if (measure(sentence) <= max) {
 current = sentence;
 }
 }
}
```

```

 this.doc.font(this.fonts.bold).fontSize(size).fillColor(
 this.cy = Math.max(this.cy, this.doc.y) + 3;
}

```

```

h3(rawText, { size = 11.2, color = "#334155" } = {})
 const text = plain(rawText);
 if (!text) return;
 const h = this.height(text, { font: this.fonts.bold
 this.ensure(h + 4, 16);
 this.cy += 4;
 this.doc.font(this.fonts.bold).fontSize(size).fillColor(
 this.cy = Math.max(this.cy, this.doc.y) + 2;
}

```

```

/** Абзац: режим по высоте страницы сами – автоперенос
para(rawText, { size = 10.0, indent = 0, color = "#1f
 const text = plain(rawText);
 if (!text) return;
 const font = mono ? this.fonts.mono : italic && thi
 const width = CONTENT_W - indent;
 const opts = { font, size, width, lineGap };
 const chunks = this.splitToFit(text, opts, BOTTOM -
 chunks.forEach((chunk, index) => {
 const last = index === chunks.length - 1;
 const h = this.height(chunk, opts);
 this.ensure(h + (last ? gapAfter : 0), 6);
 const y0 = this.cy;
 this.doc.font(font).fontSize(size).fillColor(col
 this.cy = Math.max(y0 + h, this.doc.y) + (last ?
 if (this.cy > BOTTOM) this.newPage();
 }));
}

```

```

bullet(rawText, { ordinal = null, depth = 0, size = 9
 const text = plain(rawText);
 if (!text) return;
 const indent = 16 + depth * 14;
 const marker = ordinal ? `${ordinal}.` : "*";

```



```

 h = ch;
 i += 1;
 }
 const body = chunk.join("\n");
 const boxH = h + 14;
 this.ensure(boxH);
 const y0 = this.cy;
 this.doc.roundedRect(PAGE.margin, y0, CONTENT_W, h);
 this.doc.font(font).fontSize(size).fillColor("#1f77b4");
 this.cy = y0 + boxH + (i < lines.length ? 2 : 7);
 if (i < lines.length) this.newPage();
}
}

vectorGraph(nodes = [], edges = [], { height = 130, title }) {
 this.ensure(height + 20, 10);
 const startY = this.cy;
 const boxW = CONTENT_W;

 this.doc
 .roundedRect(PAGE.margin, startY, boxW, height, 6)
 .fillAndStroke("#f8fafc", "#cbd5e1");

 if (title) {
 this.doc
 .font(this.fonts.bold)
 .fontSize(9.5)
 .fillColor("#1e293b")
 .text(title, PAGE.margin + 12, startY + 8);
 }

 // Рисуем графа
 for (const e of edges) {
 const fromNode = nodes.find((n) => n.id === e.u);
 const toNode = nodes.find((n) => n.id === e.v);
 if (!fromNode || !toNode) continue;

 const x1 = PAGE.margin + fromNode.x;

```

```

 .circle(nx, ny, r)
 .lineWidth(1.5)
 .fillAndStroke(fillColor, strokeColor);

 this.doc
 .font(this.fonts.bold)
 .fontSize(8.5)
 .fillColor("#0f172a")
 .text(String(n.label ?? n.id), nx - r, ny - 4, {
 align: "right", baseline: "middle"
 });

 if (n.dist !== undefined) {
 this.doc
 .font(this.fonts.mono)
 .fontSize(7)
 .fillColor("#059669")
 .text(`d=${n.dist}`, nx - 20, ny + r + 2, {
 align: "right", baseline: "middle"
 });
 }
}

this.cy = startY + height + 8;
}

/** Таблица: строки режутся по страницам, заголовок по ширине
table(rawRows, { size = 8.5 } = {}) {
 const rows = rawRows.map((r) => r.map((c) => plainText(c)));
 if (!rows.length) return;
 const cols = Math.max(...rows.map((r) => r.length));
 const widths = new Array(cols).fill(CONTENT_W / cols);
 const pad = 5;
 const header = rows[0];

 const rowHeight = (cells) => {
 let max = 0;
 for (let i = 0; i < cols; i++) {
 const h = this.height(cells[i] ?? "", { font: this.fonts.mono });
 max = Math.max(max, h);
 }
 return max + pad * 2;
 };
}

```

```

const width = CONTENT_W - 22;
const opts = { font, size: 9.5, width, lineGap: 1 }
// Длинная цитата режется по страницам заранее: рис
// текст уйдёт за BOTTOM и pdfkit добавит страницу
const chunks = this.splitToFit(text, opts, BOTTOM -
chunks.forEach((chunk) => {
 const h = this.height(chunk, opts);
 this.ensure(h + 10);
 const y0 = this.cy;
 this.doc.rect(PAGE.margin, y0, 3, h + 4).fill("#a
 this.doc.font(font).fontSize(9.5).fillColor("#374
 this.cy = y0 + h + 10;
 if (this.cy > BOTTOM) this.newPage();
}));
}

image(file, { maxWidth = CONTENT_W, maxHeight = 320,
 if (!fs.existsSync(file)) return false;
 let img;
 try {
 img = this.doc.openImage(file);
 } catch {
 return false;
 }
 const capH = caption ? 14 : 0;
 const scale = Math.min(maxWidth / img.width, (maxHe
 const w = img.width * scale;
 const h = img.height * scale;
 this.ensure(h + capH + 10, 20);
 const y0 = this.cy;
 const x0 = PAGE.margin + (CONTENT_W - w) / 2;
 this.doc.image(file, x0, y0, { width: w, height: h
 this.doc.rect(x0, y0, w, h).lineWidth(0.6).strokeCo
 this.cy = y0 + h + 4;
 if (caption) {
 this.doc.font(this.fonts.regular).fontSize(8).fil
 enter" }));
 this.cy = Math.max(this.cy, this.doc.y) + 6;

```

```
 }
 }

/* ----- сборка -----

async function main() {
 const fonts = {
 regular: pick(CANDIDATES.regular),
 bold: pick(CANDIDATES.bold),
 oblique: pick(CANDIDATES.oblique),
 mono: pick(CANDIDATES.mono),
 };
 if (!fonts.regular) throw new Error("Не найден TTF-шрифт");
 fonts.bold = fonts.bold ?? fonts.regular;
 fonts.mono = fonts.mono ?? fonts.regular;

 const { chapters, chapterTopics, PAGE_SYNC, VIZ_REGISTRY } = require('algvd0');
 ensureDir(OUT_DIR);

 const doc = new PDFDocument({
 size: [PAGE.width, PAGE.height],
 margin: PAGE.margin,
 autoFirstPage: false,
 info: {
 Title: "Универсальное пособие – алгоритмы и структуры данных",
 Author: "algvd0 • автоэкспорт",
 Subject: "Текст всех тем + визуализации, без кода",
 CreationDate: new Date(),
 },
 });
 const stream = fs.createWriteStream(GUIDE_PDF_PATH);
 doc.pipe(stream);

 const R = new Renderer(doc, fonts);
 const today = new Date().toISOString().slice(0, 10);
 const totalTopics = new Set(Object.values(chapterTopics));
 const vizCount = chapters.filter((c) => VIZ_REGISTRY[c.topic]).length;
```

```

 { size: 9.3, gapAfter: 2 }
 });
});

/* — темы — */
let shotsTotal = 0;
for (const chapter of chapters) {
 const topics = chapterTopics[chapter.id] ?? [];
 const shortTitle = chapter.title.replace(/^\\d+\\.--/);
 R.newPage(shortTitle);

 if (topics.length) {
 R.para(`Билет${topics.length > 1 ? "ы" : ""} ${topics[0]}
 : 2`);
 }
 R.h1(chapter.title.replace(/^\\d+\\.--/?\\s*/, ""), {

const chapterTitle = chapter.title.replace(/^\\d+\\.--/);
for (const b of htmlToBlocks(chapter.content)) {
 if (b.type === "p" && /^Билет\\s/i.test(b.text)) c
 if (b.type === "heading" && b.level <= 2 && b.tex

 if (b.type === "heading") {
 if (b.level <= 2) R.h2(b.text);
 else if (b.level === 3) R.h3(b.text);
 else R.para(b.text, { size: 10.2, color: "#334155" });
 } else if (b.type === "summary") {
 R.para("► " + b.text, { size: 9.8, color: "#373737" });
 } else if (b.type === "li") {
 R.bullet(b.text, { ordinal: b.ordinal, depth: b
 } else if (b.type === "code") {
 R.code(b.text);
 } else if (b.type === "table") {
 R.table(b.rows);
 } else if (b.type === "quote") {
 R.quote(b.text);
 } else if (b.type === "caption") {
 R.para(b.text, { size: 8.3, color: "#6b7280", i

```

```

 /* блиц: те же вопросы, что в приложении, но сразу
 const quiz = (quizzes && quizzes[chapter.id]) || []
 if (quiz.length) {
 R.h2("Блиц: вопросы по теме");
 quiz.forEach((q, qi) => {
 R.para(`${qi + 1}. ${q.question}`, { size: 9.6,
 q.options.forEach((option, oi) => {
 R.bullet(oi === q.correctIndex ? `${option} –
 });
 R.para(q.explanation, { size: 8.8, color: "#4b5
 });
 }
 }

 R.stampFooter();
 doc.end();
 await new Promise((resolve, reject) => {
 stream.on("finish", resolve);
 stream.on("error", reject);
 });

 const size = fs.statSync(GUIDE_PDF_PATH).size;
 console.log("[guide] PDF пособия собран");
 console.log(`      тем: ${chapters.length} • снимков: ${R.pages}`);
 console.log(`      выход: ${path.relative(ROOT, GUIDE_PDF_PATH)}`);

 // Самопроверка: страниц в файле должно быть ровно столько, сколько
 // рендерер. Расхождение означает, что pdftkit создал файл с лишними
 // в файл без нижнего колонтитула и без нумерации.
 const pdfLib = await import("pdf-lib");
 const physical = (await pdfLib.PDFDocument.load(fs.readFileSync(GUIDE_PDF_PATH))).getPages().length;
 console.log(`      страниц: ${R.pages} (рендер) / ${physical} (файл)`);
 if (R.pages > 100) console.warn(`      ВНИМАНИЕ: ${R.pages} страниц – слишком много`);
 if (physical !== R.pages) {
 console.error(`      ОШИБКА: pdftkit добавил ${physical - R.pages} страниц`);
 process.exitCode = 1;
 }
}

```

```
ensureDir(OUT_DIR);

const doc = new PDFDocument({
 size: [A4.width, A4.height],
 margin: 0,
 autoFirstPage: false,
 info: {
 Title: "Универсальное пособие – полный исходный код",
 Author: "CI: rebuild-pdf",
 Subject: "Экспорт всего кода проекта (код -> txt)",
 CreationDate: new Date(),
 },
});

const stream = fs.createWriteStream(PDF_PATH);
doc.pipe(stream);

for (const file of pages) {
 doc.addPage({ size: [A4.width, A4.height], margin: 0 });
 doc.image(path.join(PAGES_DIR, file), 0, 0, { width: A4.width });
}

doc.end();
await new Promise((resolve, reject) => {
 stream.on("finish", resolve);
 stream.on("error", reject);
});

console.log("[3/3] png -> pdf");
console.log(`    страниц: ${pages.length}`);
console.log(`    выход:    ${path.relative(ROOT, PDF_PATH)}`);
}

main().catch((err) => {
 console.error("Ошибка сборки PDF:", err.message);
 process.exit(1);
});
```

```
const slug = (s) =>
 s
 .toLowerCase()
 .replace(/[^a-zA-яё0-9]+/gi, "-")
 .replace(/^-+|-+$/g, "")
 .slice(0, 24);

async function vizIds() {
 const bundle = await build({
 stdin: {
 contents: `export { activeVizIds } from "../../src";`,
 resolveDir: path.join(ROOT, "scripts", "export"),
 loader: "ts",
 },
 bundle: true,
 write: false,
 format: "esm",
 platform: "node",
 logLevel: "warning",
 });
 const mod = await import("data:text/javascript;base64:" + btoa(bundle));
 return mod.activeVizIds;
}

fs.mkdirSync(OUT_DIR, { recursive: true });

const ids = await vizIds();
console.log(`[export:shots] страниц со визуализациями: ${ids.length}`);

const browser = await puppeteer.launch({
 headless: true,
 args: ["--no-sandbox", "--disable-setuid-sandbox", "--no-dev-shm-usage"],
});
const page = await browser.newPage();
await page.setViewport({ width: 1400, height: 1000, deviceScaleFactor: 1 });

let done = 0;
let failed = 0;
```



```
const manifest = fs
 .readdirSync(OUT_DIR)
 .filter((f) => /\.(png|jpe?g)$/i.test(f))
 .sort()
 .map((f) => ({ file: f, name: f.replace(/\.(png|jpe?g)/, ".json") }));
fs.writeFileSync(path.join(OUT_DIR, "manifest.json"), JSON.stringify(manifest));

await browser.close();
console.log(`[export:shots] снимков: ${done}, пропусков: ${missed}`);
// Пропущенные страницы не повод ронять пайплайн: PDF д
// текстовое описание демонстрации.
process.exit(0);
```

ФАЙЛ 95/111: scripts/export/collect-code.mjs

КАТЕГОРИЯ: Скрипты сборки и экспорта | СТРОК: 196 | РАЗМ: 1024

```
/**
 * ШАГ 1: КОД -> TXT
 *
 * Собирает ВСЕ исходный код репозитория в один текст
 * public/export/full_code_all_pages.txt
 *
 * Список файлов берётся из git (чтобы ничего не забыть
 * с фоллбэком на обход файловой системы, если git недо
 */
import fs from "node:fs";
import path from "node:path";
import { execFileSync } from "node:child_process";
import { ROOT, OUT_DIR, TXT_PATH, HR, SUBHR, ensureDir,

/** Расширения, которые считаем «кодом» и включаем в да
const CODE_EXT = new Set([
 ".ts", ".tsx", ".js", ".jsx", ".mjs", ".cjs",
 ".css", ".scss", ".html", ".json", ".md", ".yaml", ".y
```

```

let files;
try {
 files = execFileSync("git", ["ls-files", "--cached"], {
 cwd: ROOT,
 encoding: "utf8",
 })
 .split("\n")
 .filter(Boolean);
} catch {
 files = [];
 const walk = (dir) => {
 for (const e of fs.readdirSync(dir, { withFileTypes: true })) {
 const abs = path.join(dir, e.name);
 const rel = path.relative(ROOT, abs).split(path.sep);
 if (EXCLUDE_DIRS.some((d) => rel === d || rel.startsWith(d + path.sep))) continue;
 if (e.isDirectory()) walk(abs);
 else files.push(rel);
 }
 };
 walk(ROOT);
}

return files
 .filter((rel) => !EXCLUDE_DIRS.some((d) => rel.startsWith(d)))
 .filter((rel) => !EXCLUDE_FILES.has(path.basename(rel)))
 .filter((rel) => CODE_EXT.has(path.extname(rel)))
 .filter((rel) => fs.existsSync(path.join(ROOT, rel)))
 .sort((a, b) => {
 const ia = ORDER.indexOf(categoryOf(a));
 const ib = ORDER.indexOf(categoryOf(b));
 return ia !== ib ? ia - ib : a.localeCompare(b);
 }));
}

```

/\*\* Достаём заголовки глав из данных пособия (без запуска)

```

function extractChapters(files) {
 const chapters = [];
 const seen = new Set();

```

```
push();
```

```
push(HR);
```

```
push("
```

ОГЛАВЛЕНИЕ: ГЛАВЫ И БИЛЕ

```
push(HR);
```

```
chapters.forEach((c, i) => {
```

```
 push(` ${String(i + 1).padStart(3, " ")}: ${c.title}
`);
```

```
push();
```

```
push(HR);
```

```
push("
```

ОГЛАВЛЕНИЕ: ФАЙЛЫ ИСХОДН

```
push(HR);
```

```
let current = null;
```

```
files.forEach((rel, i) => {
```

```
 const cat = categoryOf(rel);
```

```
 if (cat !== current) {
```

```
 current = cat;
```

```
 push();
```

```
 push(`  # ${cat}`);
```

```
 }
```

```
 push(`  ${String(i + 1).padStart(3, " ")}: ${rel}`
);
```

```
push();
```

```
push();
```

```
current = null;
```

```
files.forEach((rel, i) => {
```

```
 const cat = categoryOf(rel);
```

```
 if (cat !== current) {
```

```
 current = cat;
```

```
 push(HR);
```

```
 push(`РАЗДЕЛ: ${cat.toUpperCase()}`);
```

```
 push(HR);
```

```
 push();
```

```
 }
```

```
 const content = fs.readFileSync(path.join(ROOT, rel
```

```
 const lines = content.split("\n");
```

```
*/
import fs from "node:fs";
import path from "node:path";
import { fileURLToPath } from "node:url";
import { execFileSync } from "node:child_process";

export const ROOT = path.resolve(path.dirname(fileURLToPath(import.meta.url)), "..");

/** Куда кладём финальные артефакты (эта папка отдаётся клиенту) */
export const OUT_DIR = path.join(ROOT, "public", "export");

/** Промежуточные PNG-страницы (в .gitignore, в CI уезжают) */
export const PAGES_DIR = path.join(ROOT, "tmp", "export");

export const TXT_PATH = path.join(OUT_DIR, "full_code_and_comments.txt");
export const PDF_PATH = path.join(OUT_DIR, "full_code_and_comments.pdf");

/** Параметры вёрстки страницы (подобраны под A4 @150dpi) */
export const PAGE = {
 /** DPI раstra. Можно переопределить: EXPORT_DENSITY=100 */
 density: Number(process.env.EXPORT_DENSITY ?? 150),
 /** 1-bit ч/б страницы весят в 2-4 раза меньше. EXPORT_GRAYSCALE=1 */
 monochrome: process.env.EXPORT_GRAYSCALE !== "1",
 size: "A4",
 font: "DejaVu-Sans-Mono",
 fontFile: "/usr/share/fonts/truetype/dejavu/DejaVuSansMono.ttf",
 pointsize: 8,
 /** Максимум символов в строке (дальше – жёсткий перенос) */
 cols: 138,
 /** Строк содержимого на странице (+2 служебные: шапка и подвал) */
 rows: 76,
};

export const HR = "=".repeat(PAGE.cols);
export const SUBHR = "-".repeat(PAGE.cols);

export function ensureDir(dir) {
 fs.mkdirSync(dir, { recursive: true });
}
```

```

* (, <code>,) не «уезжает» в конец абзаца.
*/

/** Содержимое этих тегов не нужно в текстовой версии.
const SKIP = new Set([
 "svg", "script", "style", "button", "iframe", "video",
 "path", "circle", "rect", "line", "polyline", "polygon",
 "defs", "marker", "tspan", "ellipse", "use", "clipPath",
]);

/** Пустые элементы (без закрывающего тега). */
const VOID = new Set(["br", "hr", "img", "input", "meta"]);

/** Блочные теги: их содержимое собирается в отдельный блок.
const BLOCK = new Set([
 "h1", "h2", "h3", "h4", "h5", "h6", "p", "li", "pre",
 "summary", "figcaption", "table", "tr", "td", "th", "thead",
 "article", "ul", "ol", "details", "figure", "header",
]);

const HEADING = new Set(["h1", "h2", "h3", "h4", "h5", "h6"]);

/**
 * Разбор HTML в дерево: { tag, attrs, children }, текст
 * Намеренно простой: страницы пособия генерируются нами
 */
export function parseHtml(html) {
 const root = { tag: "#root", attrs: {}, children: [] };
 const stack = [root];
 const re = /<!--[\s\S]*?-->|<\/([a-zA-Z0-9]+)\s*>|<([a-zA-Z0-9]+)\s*/g;
 let m;
 while ((m = re.exec(html)) !== null) {
 const [full, closeTag, openTag, attrsStr, selfClose] = m;
 if (full.startsWith("<!--")) continue;

 if (closeTag) {
 const tag = closeTag.toLowerCase();
 // закрываем до совпадения (прощает незакрытые

```

```

 .replace(/»/g, "»")
 .replace(/…/g, "...")
 .replace(/×/g, "×")
 .replace(/≤/g, "≤")
 .replace(/≥/g, "≥")
 .replace(/≠/g, "≠")
 .replace(/∞/g, "∞")
 .replace(/→/g, "→")
 .replace(/←/g, "←")
 .replace(/&/g, "&")
 .replace(/&#(\d+)/g, (_, d) => String.fromCharCode

const clean = (s) => decodeEntities(s).replace(/\s+/g,

const isEl = (n) => n.tag !== "#text";
const isBlockEl = (n) => isEl(n) && BLOCK.has(n.tag) &&

/** Есть ли у узла блочные потомки (тогда его текст «ра
const hasBlockChild = (node) => node.children.some(isBl

/** Текст инлайнового поддеревя в порядке документа. */
function inlineText(node) {
 let out = "";
 for (const c of node.children) {
 if (c.tag === "#text") {
 out += c.text;
 } else if (SKIP.has(c.tag)) {
 continue;
 } else if (BLOCK.has(c.tag)) {
 continue;
 } else if (c.tag === "br") {
 out += "\n";
 } else {
 out += inlineText(c);
 }
 }
 return clean(out);
}

```

```

const out = [];

const push = (block) => {
 const prev = out[out.length - 1];
 if (prev && prev.type === block.type && prev.text === block.text)
 out.push(block);
};

const emitListItem = (li, ordinal, depth) => {
 const text = collectText(li);
 if (text) push({ type: "li", text, ordinal: ordinal });
 // вложенные списки внутри li
 li.children.forEach((c) => {
 if (c.tag === "ul" || c.tag === "ol") emitList(c, depth + 1);
 });
};

const emitList = (list, depth) => {
 let i = 0;
 for (const li of list.children) {
 if (li.tag !== "li") continue;
 i += 1;
 emitListItem(li, list.tag === "ol" ? i : null, depth);
 }
};

const walk = (node, listDepth = 0) => {
 if (node.tag !== "#root" && SKIP.has(node.tag)) return;

 if (node.tag === "table") {
 const rows = tableRows(node);
 if (rows.length) push({ type: "table", rows });
 return;
 }
 if (node.tag === "pre") {
 const text = decodeEntities(collectRawPre(node));
 if (text.trim()) push({ type: "code", text: text });
 return;
 }
};

```

```
 else if (node.tag === "blockquote") push({ type: "q", text });
 else if (node.tag === "figcaption") push({ type: "caption", text });
 else push({ type: "p", text });
 });

 root.children.forEach((c) => walk(c, 0));
 return out;
}
```

```
/** То же самое, но простым текстом (для .txt-дампа и вёрстки)
export function htmlToPlainText(html) {
 const lines = [];
 for (const b of htmlToBlocks(html)) {
 if (b.type === "heading") lines.push("", "#".repeat(b.level));
 else if (b.type === "summary") lines.push("", "▶ ");
 else if (b.type === "li") lines.push(" ".repeat(b.level));
 else if (b.type === "code") lines.push("", ...b.text);
 else if (b.type === "table") {
 lines.push("");
 b.rows.forEach((r) => lines.push("| " + r.join(" ") + "|"));
 lines.push("");
 } else if (b.type === "quote") lines.push("> " + b.text);
 else if (b.type === "caption") lines.push(" " + b.text);
 else lines.push(b.text, "");
 }
 return lines.join("\n").replace(/\n{3,}/g, "\n\n").trim();
}
```

---

ФАЙЛ 98/111: scripts/export/render-pages.mjs

КАТЕГОРИЯ: Скрипты сборки и экспорта | СТРОК: 117 | РАЗМЕР: 1.1 КБ

---

```
/**
 * ШАГ 2: TXT -> PNG
 *
 * Разбивает full code all pages.txt на страницы формата
```



```

 return pages;
}

async function main() {
 if (!fs.existsSync(TXT_PATH)) {
 throw new Error(`Нет ${path.relative(ROOT, TXT_PATH)}
 }

 const im = imageMagickCmd();
 const pages = paginate(fs.readFileSync(TXT_PATH, "utf8"));

 fs.rmSync(PAGES_DIR, { recursive: true, force: true });
 ensureDir(PAGES_DIR);

 const total = pages.length;
 const jobs = pages.map((lines, idx) => {
 const num = String(idx + 1).padStart(4, "0");
 const header = `Универсальное пособие • полный исходный код
 Math.max(1, PAGE.cols - 44 - `стр. ${idx + 1} / ${total}`);
 const body = [header, "-".repeat(PAGE.cols), ...lines];
 const txtFile = path.join(PAGES_DIR, `page-${num}.txt`);
 const pngFile = path.join(PAGES_DIR, `page-${num}.png`);
 fs.writeFileSync(txtFile, body + "\n", "utf8");
 return { txtFile, pngFile, num };
 });

 const args = (job) => [
 "-density", String(PAGE.density),
 "-page", PAGE.size,
 "-font", fs.existsSync(PAGE.fontFile) ? PAGE.fontFile : "DejaVuSansMono",
 "-pointsize", String(PAGE.pointsize),
 `text:${job.txtFile}`,
 "-background", "white",
 "-flatten",
 "-colorspace", "Gray",
 ...(PAGE.monochrome ? ["-monochrome"] : []),
 "-strip",
];
}

```

ФАЙЛ 99/111: scripts/verify/browser-checks.tsx

КАТЕГОРИЯ: Скрипты сборки и экспорта | СТРӨК: 1065 | РА

```
/**
 * DOM-харнес: настоящее приложение в jsdom + настоящий
 * Монтирует App, кликает кнопки, печатает в редакторе
 * визуализациям матрицы чужих размерностей и ловит пад
 */
import { act } from "react";
import { createRoot } from "react-dom/client";
import type { Root } from "react-dom/client";
import App from "../../src/App";
import { chapters, chapterTopics, SECTIONS, sectionOf }
import { quizzes } from "../../src/data/quizzes";
import { VIZ_REGISTRY } from "../../src/components/vizR
import { VizLiveStatus } from "../../src/components/Viz
import { PAGE_SYNC, buildInitTemplate } from "../../src
import { VizChapterContext, emitVizState, clearVizState
import { buildDebugRunner, type DebugResult } from "../

type Py = { runPythonAsync: (src: string) => Promise<un

export interface TestResult {
 group: string;
 name: string;
 ok: boolean;
 ms: number;
 detail: string;
 consoleErrors: string[];
}

const results: TestResult[] = [];
let consoleErrors: string[] = [];
const realConsoleError = console.error;

/* -----
```

```

 const hay = [b.getAttribute("aria-label"), b.getAttribute("aria-labelledby")];
 return typeof match === "string" ? hay.includes(match) : false;
 });
}

async function click(el: Element | undefined | null, label: string) {
 if (!el) throw new Error(`не найдена ${label}`);
 await act(async () => {
 el.dispatchEvent(new window.MouseEvent("click", { bubbles: true }));
 await sleep(20);
 });
}

/** Панель компилятора – все поиски внутри неё, чтобы не искать в документе
function panel(): HTMLElement | null {
 return document.querySelector<HTMLElement>('aside[aria-label="Compiler Panel"]');
}

function panelText(): string {
 return (panel()?.textContent ?? "").replace(/\s+/g, " ");
}

function textarea(): HTMLTextAreaElement | undefined {
 return panel()?.querySelector<HTMLTextAreaElement>("textarea");
}

function typeInto(ta: HTMLTextAreaElement, value: string) {
 const setter = Object.getOwnPropertyDescriptor(window, "document").value;
 if (!setter) throw new Error("нет нативного setter'a для document");
 act(() => {
 setter.call(ta, value);
 ta.dispatchEvent(new window.Event("input", { bubbles: true }));
 });
}

async function key(k: string, init: KeyboardEventInit = {}) {
 await act(async () => {
 document.dispatchEvent(new window.KeyboardEvent("keydown", {
 key: k,
 code: k.toUpperCase().replace(/-/g, "219"),
 ctrl: false,
 alt: false,
 meta: false,
 shift: false,
 bubbles: true,
 cancelable: true,
 }));
 });
}

```

```

 root = null;
 }
 if (host) {
 host.remove();
 }
 host = freshHost();
 window.history.replaceState({}, "", `/?topic=${encodeURIComponent(topic)}`);
 root = createRoot(host);
 await act(async () => {
 root!.render(<App />);
 });
 await settle(80);
 return host;
}

```

```

/** Смонтировать одну визуализацию так, как это делает
 * @param vizId ID визуализации
 */
async function mountViz(vizId: string) {
 if (root) {
 await act(async () => {
 root?.unmount();
 });
 root = null;
 }
 if (host) host.remove();
 host = freshHost();
 const entry = VIZ_REGISTRY[vizId];
 if (!entry) throw new Error(`в реестре нет визуализации с ID ${vizId}`);
 const Comp = entry.Component;
 root = createRoot(host);
 await act(async () => {
 root!.render(
 <VizChapterContext.Provider value={vizId}>
 <VizLiveStatus demoTitle={entry.title} />
 <Comp />
 </VizChapterContext.Provider>
);
 });
 await settle(60);
}

```

```

 const d = await fn();
 detail = d ?? "";
 } catch (error) {
 ok = false;
 detail = error instanceof Error ? `${error.message}`
 }
 const ms = Date.now() - t0;
 const errs = consoleErrors.filter((e) => !/not wrapped/);
 if (errs.length > 0) {
 ok = false;
 detail = `${detail}\nconsole.error (${errs.length})
 }
 results.push({ group, name, ok, ms, detail, consoleError});
 const mark = ok ? "PASS" : "FAIL";
 realConsoleError === console.error;
 process.stdout.write(`${mark} [${group}] ${name}${ok ? " " : "\n"}`);
 if (!ok) process.stdout.write(detail.split("\n").map((line) => `
 return ok;
}

/* -----
/* сценарии
/* -----

/** Фильтры прогона: HAR_CHAPTERS=segment-trees,floyd и
const onlyChapters = (process.env.HAR_CHAPTERS || "").split(",");
const skipGroups = new Set((process.env.HAR_SKIP || "").split(","));
/** HAR_SYNC=floyd,dijkstra – прогнать только эти записи
const onlySync = new Set((process.env.HAR_SYNC || "").split(","));

export async function runSuite({ py }: { py: Py }): Promise<void> {
 const chapterList = onlyChapters.length ? chapters.filter((c) =>
 const want = (group: string) => !skipGroups.has(group);
 // перехват console.error приложения (React пишет сюда)
 console.error = (...args: unknown[]) => {
 consoleErrors.push(args.map((a) => (a instanceof Error ? a.message : a)));
 };

```

```

 if (/Ошибка|Не удалось/.test(msg)) throw new Error(msg);
 // после настоящего прогона визуализация обязана быть
 const linked = await waitFor(() => /данные из ком/);
 if (!linked) return `${msg}; визуализация осталась прежней`;
 return `${msg} → визуализация переключилась на данные`;
 });
}

```

/\* — 2. Флойд: матрицы ЛЮБОЙ размерности из кода полн

```

const floydCases: { name: string; vars: Record<string, any> }[] = [
 { name: "5x5 (меньше демо-7x7)", vars: { n: 5, k: 1, i: 0, j: 0,
 [9, 6, 9, 9, 0] }, expect: "5x5" },
 { name: "10x10 (больше демо – старый краш nodeNames)", vars: { n: 10, k: 1, i: 0, j: 0,
 from({ length: 10 }, (_, c) => (r === c ? 0 : (r + c) % 50)) }, expect: "10x10" },
 { name: "3x3 с бесконечностями", vars: { n: 3, k: 0, i: 0, j: 0,
 "3x3" }, expect: "3x3" },
 { name: "не квадратная 4x6", vars: { n: 4, k: 1, i: 0, j: 0,
 [17, 18, 19, 0, 20] }, expect: "Демо-схема графа" },
 { name: "словарь словарей d[0][1]", vars: { n: 4, k: 1, i: 0, j: 0,
 2: { 0: 9, 1: 9, 2: 0, 3: 1 }, 3: { 0: 3, 1: 9, 2: 0, 3: 1 } }, expect: "словарь словарей" },
 { name: "ключи-пары \"{i, j}\"", vars: { n: 3, k: 0, i: 0, j: 0,
 "(1, 2)": 2, "(2, 0)": 1, "(2, 1)": 9, "(2, 2)": 0 }, expect: "ключи-пары" },
 { name: "плоский список – это НЕ матрица", vars: { n: 10, k: 1, i: 0, j: 0,
 [10, 11, 12, 0] }, expect: "плоский список" },
 { name: "80x80 (обрезка до 64)", vars: { n: 80, k: 1, i: 0, j: 0,
 (_, c) => (r === c ? 0 : 1 + ((r + c) % 50)) }, expect: "80x80" },
 { name: "пустая матрица", vars: { n: 0, k: 0, i: 0, j: 0,
 [10, 11, 12, 0] }, expect: "4x4" },
];

```

```

for (const c of floydCases) {
 await test("В. Флойд: размерности", c.name, async () => {
 await mountViz("floyd");
 await act(async () => {
 emitVizState("floyd", { line: 3, func: "<module>";
 await sleep(30);
 });
 const txt = (host?.textContent ?? "").replace(/\\s/g, " ");
 });
}

```



```

const declared = sync.variables.map((v) => v.name)
const ident = (t: string) => /^[A-Za-z_][A-Za-z0-9_]*$/
const missing = declared
 .map((raw) => raw.split(/[/\[\\],,]\s+или\s+/).map(s => s.trim()))
 .filter((alts) => alts.some(ident))
 .filter((alts) => !alts.some((a) => ident(a) && a.startsWith('_')))
 .map((alts) => alts.join(" / "));
if (missing.length) throw new Error(`код не создан: ${missing.join(", ")}`);

if (!VIZ_REGISTRY[chapterId]) return `шагов ${result.steps.length}`;
await mountViz(chapterId);
const finalVars = result.finalValues ?? result.steps[result.steps.length - 1].finalValues;
await act(async () => {
 emitVizState(chapterId, { line: result.steps[result.steps.length - 1].line,
 ed: Object.keys(finalVars) });
 await sleep(40);
});
const txt = (host?.textContent ?? "").replace(/\\s/g, " ");
if (!/данные из компилятора/.test(txt)) throw new Error(`некорректный код`);
const defects = domDefects(host);
if (defects.length) throw new Error(`артефакты в DOM: ${defects.join(", ")}`);
return `шагов ${result.steps.length}${result.steps[result.steps.length - 1].finalText}
 `).join(", ");
});
}

```

/\* — 6. Поведение редактора: пустой код, ошибки, гла...

```

const ch = "dijkstra";

```

```

await test("F. редактор", "только комментарии: редактор", async () => {
 await mountApp(ch);
 await click(findButton("Боковая панель Python"), "поиск");
 const ta = textarea();
 if (!ta) throw new Error("нет редактора");
 typeInto(ta, "# просто комментарий\n# и ещё один\n");
 const done = await waitFor(() => bodyText().include("комментарий"));
 if (!done) throw new Error(`не дождался сообщения "комментарий"`);
});

```



```

 await mountApp(ch);
 await click(findButton("Боковая панель Python"), "п");
 const ta = textarea();
 if (!ta) throw new Error("нет редактора");
 typeInto(ta, "");
 const done = await waitFor(() => bodyText().include("Python"));
 if (!done) throw new Error(`на пустой редактор панель не появилась`);
 if (!textarea()) throw new Error("редактор исчез после клика");
 if (!/встроенное демо/.test(bodyText())) throw new Error("не встроенное демо");
 if (allButtons().length < 5) throw new Error(`кнопки не появились`);
 return `пустой код: кнопок ${allButtons().length}, панель появилась`;
 });

```

```

await test("F. редактор", "глаз: эталон попадает в редактор", async () => {
 await mountApp(ch);
 await click(findButton("Боковая панель Python"), "п");
 const ta = textarea();
 if (!ta) throw new Error("нет редактора");
 const init = ta.value;
 const editorButtons = allButtons();
 const ref = PAGE_SYNC[ch]?.code ?? "";
 if (!ref) throw new Error("у страницы нет эталонного кода");
 const done0 = await waitFor(() => RUN_DONE.some((m) => m === "done"));
 if (!done0) throw new Error(`инициализация не выполнена`);

```

```

 // — глаз ВКЛ: эталон лежит В РЕДАКТОРЕ, и редактор не пустой
 await click(findButton("Вставить эталонный код"), "в");
 const trace = await waitFor(() => !!findButton(/Остаток кода/));
 if (!trace) throw new Error(`эталон не запустился. панель не появилась`);
 if (!textarea()) throw new Error("редактор исчез во время клика");
 if (/Ошибка:/.test(panelText())) throw new Error(`эталон не вставился`);
 const ta2 = textarea();
 const firstRefLine = ref.split("\n").find((l) => l.startsWith("#"));
 if (!ta2?.value.includes(firstRefLine)) {
 throw new Error(`выполняется не тот текст, что лежит в эталоне`);
 }
 const mustBe = [

```

```

 await mountApp(ch);
 await click(findButton("Боковая панель Python"), "п
 const ta = textarea();
 if (!ta) throw new Error("нет редактора");
 typeInto(ta, "# MY-MARKER-14\nx = 41\nx += 1\nprint
 await waitFor(() => RUN_DONE.some((m) => bodyText())
 if (!bodyText().includes("42")) throw new Error("св
 // переходим на другую страницу, не закрывая панель
 await mountApp("floyd");
 await click(findButton("Боковая панель Python"), "п
 const ta2 = textarea();
 if (!ta2) throw new Error("на новой странице нет ре
 if (ta2.value.includes("MY-MARKER-14")) throw new E
 if (!ta2.value.includes("# «")) throw new Error("на
 await mountApp(ch);
 await click(findButton("Боковая панель Python"), "п
 const ta3 = textarea();
 if (!ta3?.value.includes("MY-MARKER-14")) throw new
 return "у каждой страницы свой код, возврат сохраня
 });

```

```

await test("F. редактор", "редактор живёт и во время
 await mountApp(ch);
 await click(findButton("Боковая панель Python"), "п
 const done = await waitFor(() => bodyText().include
 if (!done) throw new Error(`трасса не собралась. Па
 if (!textarea()) throw new Error("во время прогона
 if (!findButton("/Остановить прогон трассы/")) throw
 // Листинг сворачивается (освобождая место редактор
 // и редактор остаются на месте.
 const codeLines = () => panel()?.querySelectorAll("
 if (codeLines() === 0) throw new Error("в раскрытой
 await click(findButton("Свернуть листинг трассы"),
 await settle(80);
 if (!textarea()) throw new Error("после сворачивани
 if (codeLines() !== 0) throw new Error(`листинг не
 if (!findButton("В конец трассы")) throw new Error(
 await click(findButton("Показать листинг трассы"),

```

```

 , "}}`);
 if (namelessListing.length) throw new Error(`в листинге нет
 ("", "}}`);
 const required = [/Остановить прогон трассы/, /Свернуть
 начало трассы/, /В конец трассы/, /автопроход/i];
 const btns = allButtons().join(" | ");
 const missing = required.filter((re) => !re.test(btns));
 if (missing.length) throw new Error(`в листинге нет
 return `редактор: ${editorBtns} кнопок, листинг: ${
 });

 await test("Н. доступность", "бейдж честно различает
 await mountApp(ch);
 // 1) демо: компилятор ещё ничего не присылал
 if (!/встроенное демо/.test(bodyText())) throw new Error("демо не
 if (/переменных пока нет/.test(bodyText())) throw new Error("перемен
 await click(findButton("Боковая панель Python"), "панель Python");
 const listing = await waitFor(() => !!findButton(/Остановить прогон тр
 if (!listing) throw new Error("панель не показала трассу");
 // 2) код выполнен, отладчик на первой строке: переменных пока нет
 const awaiting = await waitFor(() => /переменных пока нет/.test(bodyText
 if (!awaiting) throw new Error(`после запуска бейдж не показывает
 `${(bodyText().match(/встроенное демо|данные из компилятора/) || []).
 // 3) дошли до конца трассы – картинка обязана жить
 await click(findButton("В конец трассы"), "в конец трассы");
 const linked = await waitFor(() => /данные из компилятора/.test(bodyText
 if (!linked) throw new Error("в конце трассы визуализация не появилась");
 // 4) сняли связь (Esc + пустой код) – снова демо
 await key("Escape");
 await settle(80);
 const ta = textarea();
 if (!ta) throw new Error("редактор не вернулся по EventSource");
 typeInto(ta, "");
 const cleared = await waitFor(() => /встроенное демо/.test(bodyText()));
 if (!cleared) throw new Error("после очистки кода бейдж не показывает");
 if (/переменных пока нет/.test(bodyText())) throw new Error("переменн
 return "демо → выполнено/переменных нет → данные из компилятора";
 });

```

```

 }
 if (!menu.querySelector("#download-html-btn")) throw
 const py = document.querySelector<HTMLButtonElement>
 if (!py) throw new Error("нет кнопки панели Python")
 if (!py.getAttribute("aria-label")) throw new Error
 if (!py.getAttribute("title")) throw new Error("у к
 return "4 загрузки на месте, href ведут на файлы, к
 });

```

```

await test("I. ссылки", "упоминание «билет 5» кликабе
 await mountApp("sparse-table");
 const links = Array.from(document.querySelectorAll<
 if (links.length === 0) throw new Error("на страниц
 const toPrefix = links.find((a) => a.dataset.goto =
 if (!toPrefix) throw new Error(`нет ссылки на билет
 if (!/билет 5/i.test(toPrefix.textContent ?? "")) th
 `);
 const href = toPrefix.getAttribute("href") ?? "";
 if (!href.includes("topic=prefix-sums-2d")) throw n
 await click(toPrefix, "ссылка на билет 5");
 await settle(120);
 const txt = bodyText();
 if (!/Двумерная матрица префиксных сумм/.test(txt))
 throw new Error(`клик по «билет 5» не открыл стра
 }
 return `ссылок на странице: ${links.length}, перехо
});

```

```

await test("I. ссылки", "содержание: разделы по поряд
 await mountApp("bridges-code");
 const sidebar = document.querySelector("#sidebar-ro
 if (!sidebar) throw new Error("нет сайдбара содержа
 const headers = Array.from(sidebar.querySelectorAll
 .map((el) => (el.textContent ?? "").trim()))
 .filter((t) => SECTIONS.some((sec) => sec.title =
 if (headers.length !== SECTIONS.length) {
 throw new Error(`в содержании ${headers.length} p
 }

```

```

 }
 if (!txt.includes("2n") || !txt.includes("2n+1")) throw new Error("во введе
 if (!txt.includes("≤ 1")) throw new Error("во введе
 if (/Обзор: Кратчайшие пути/.test(txt)) throw new Error("во введе
 const goto = Array.from(document.querySelectorAll("a[href='#goto']"));
 if (!goto.includes("dijkstra")) throw new Error(`обзор: кратчайшие пути: ди
 if (domDefects(document.querySelector("#chapter-viz"))) throw new Error("артефакты в DOM введения");
 }
 return "введение = куча/очередь/бинпоиск/сортировка";
});

```

```

await test("I. блиц", "вопросы отвечают, объясняют и
 await mountApp("sparse-table");
 const list = quizzes["sparse-table"] ?? [];
 if (list.length === 0) throw new Error("у билета 2
 const section = Array.from(document.querySelectorAll("div[id='table']"));
 (el.getAttribute("aria-label") ?? "").includes("Блиц");
);
 if (!section) throw new Error("блиц не отрисован на
 const ask = (label: string | RegExp) =>
 Array.from(section.querySelectorAll("button")).find(b =>
 const hay = [b.getAttribute("aria-label"), b.textContent];
 return typeof label === "string" ? hay.includes(label) : hay.some(s => s.match(label));
);
 const shown = () => (section.textContent ?? "").replace(/<div id="table">/, "").includes(list[0].question.slice(0, 24));
 if (!shown().includes(list[0].question.slice(0, 24))) throw new Error(`первый вопрос не показан: ${list[0].question}`);
 const wrong = list[0].options.findIndex((_, i) => i !== 0);
 await click(ask(list[0].options[wrong].slice(0, 20)));
 await settle(80);
 if (!/Неверно/.test(shown())) throw new Error("на не
 if (!shown().includes(list[0].explanation.slice(0, 24))) throw new Error("после ответа нет разбора – польза");
 }
 const nextBtn = ask("Следующий вопрос");
 if (!nextBtn) throw new Error("нет кнопки «Следующий вопрос»");
}

```



```

 "a = [5, 1, 9, 3, 7, 2]",
 "n = len(a)",
 "LOG = 3",
 "i, j = 0, 0",
 "st = []",
 "for row in a:",
 " st.append([row] + [None] * (LOG - 1))",
 "for j in range(1, LOG):",
 " for i in range(n - (1 << j) + 1):",
 " st[i][j] = min(st[i][j - 1], st[i + (1 << j) - 1][j - 1])",
 "print(n, st[0][2], st[1][1])",
].join("\n")
);
const ready = await waitFor(() => bodyText().includes("Готово"));
if (!ready) throw new Error(`своя таблица не построена`);
if (!/6 1 1/.test(panelText())) throw new Error(`отсутствует таблица`);
await click(findButton("В конец трассы"), "в конец трассы");
const linked = await waitFor(() => /данные из компилятора/);
if (!linked) throw new Error("демонстрация не принята");
const defects = domDefects(document.querySelector("#defects"));
if (defects.length) throw new Error(`артефакты в демонстрации`);
return "свой массив из 6 чисел: st[0][2] = 1, st[1][1] = 1";
});

```

```

await test("К. свой код", "Косарайю: эталон сам считает");
await mountApp("scc-kosaraju");
await click(findButton("Боковая панель Python"), "поиск");
if (!textarea()) throw new Error("нет редактора");
await click(findButton("Вставить эталонный код"), "вставить");
const ready = await waitFor(() => bodyText().includes("Готово"));
if (!ready) throw new Error(`эталон Косарайю не выполнен`);
if (/Ошибка:/.test(panelText())) throw new Error(`эталон не выполнен`);
const out = panelText();
if (!/порядок выхода:/.test(out)) throw new Error(`неверный порядок`);
// граф на экране: 0→1→2→0, 2→3, 3→4 – ровно две SCC
if (!/компонент: 2/.test(out)) throw new Error(`в графе не 2 компонента`);
if (!textarea()) throw new Error("редактор исчез, попробуйте еще раз");
await click(findButton("В конец трассы"), "в конец трассы");

```

```

 });

 await test("G. lite-режим", "HTML-книга: оглавление",
 const { buildBookHtml } = await import("../src/buildBookHtml.js");
 const html = await buildBookHtml(chapters);
 if (!html.includes('<nav id="toc"')) throw new Error("нет оглавления");
 const missing = chapters.filter((c) => !html.includes(c.title));
 if (missing.length) throw new Error(`в книге потеряны главы: ${missing.map(c => c.title).join(", ")}`);
 const tocLinks = (html.match(/href="#ch-/g) ?? []).length;
 if (tocLinks < chapters.length) throw new Error(`ссылки на главы: ${chapters.length - tocLinks} шт.`);
 if (!/Словарик терминов пособия/.test(html)) throw new Error("не найден словарь терминов");
 if (!/↑ к оглавлению/.test(html)) throw new Error("не найден ссылка на оглавление");
 if (html.length < 200_000) throw new Error(`книга слишком маленькая: ${html.length} байт`);
 return `${(html.length / 1024).toFixed(0)} КБ, глав: ${chapters.length}`;
 });

 console.error = realConsoleError;
 return results;
 }

 /** Сводка для оркестратора. */
 export function summary() {
 const byGroup = new Map<string, { ok: number; fail: number }>();
 for (const r of results) {
 const g = byGroup.get(r.group) ?? { ok: 0, fail: 0 };
 g[r.ok ? "ok" : "fail"]++;
 byGroup.set(r.group, g);
 }
 return { total: results.length, passed: results.filter(r => r.ok).length, failed: results.filter(r => !r.ok).length };
 }

 /** Инициализация страницы из vizSync – нужна для проверки */
 export const initTemplateOf = (id: string) => buildInitTemplateOf(id);

```





```

// 3. упоминания «билет N» — только внутри текста, не
const textOnly = html
 .replace(/<pre[\s\S]*?<\/pre>/gi, " ")
 .replace(/<code[\s\S]*?<\/code>/gi, " ")
 .replace(/<[>]*>/g, " ");
for (const m of textOnly.matchAll(TICKET_RE)) {
 const from = Number(m[1]);
 const to = m[2] ? Number(m[2]) : from;
 mentionsHere += 1;
 for (let n = from; n <= to; n += 1) {
 ok(n >= 1 && n <= 24 && covered.has(n), `«${chapter
 }
}
ticketMentions += mentionsHere;

// 4. явные переходы
for (const m of html.matchAll(/data-goto="([^\"]+)"/g))
 explicitLinks += 1;
 ok(chapterIds.has(m[1]), `«${chapter.id}»: data-goto
}
for (const m of html.matchAll(/href="\?(?:lite=1&)?to
 explicitLinks += 1;
 ok(chapterIds.has(decodeURIComponent(m[1])), `«${ch
}

// 5. якоря
for (const m of html.matchAll(/href="#([^\"]+)"/g)) {
 ok(html.includes(`id="${m[1]}"`), `«${chapter.id}»:
}

if (mentionsHere === 0 && !/data-goto=/.test(html)) w
}

/* 6–7. глоссарий */
const demoKinds = new Set(MINI_DEMO_KINDS);
for (const term of GLOSSARY) {
 if (term.simulator) {
 ok(registryIds.includes(term.simulator), `глоссарий

```

Универсальное пособие • полный исходный код

```

console.log(` упоминаний «билет N» в тексте: ${ticketM
console.log(` визуализаций в реестре: ${registryIds.le
 GLOSSARY.length}`);
console.log(` блиц есть у ${Object.keys(quizzes).length
if (NOTES.length) console.log(` заметки:\n ${NOTES.
if (noQuiz.length) console.log(` без блица: ${noQuiz.j

if (FAILS.length) {
 console.log(`\nПРОВАЛЕНО ${FAILS.length}:`);
 FAILS.slice(0, 40).forEach((f) => console.log(` x ${f
 process.exit(1);
}
console.log("\nвсе ссылки целы: 0 битых переходов");

```

ФАЙЛ 101/111: scripts/verify/check-reference-code.mjs  
КАТЕГОРИЯ: Скрипты сборки и экспорта | СТРОК: 133 | РАЗМ

```

/**
 * Контракт «эталонный код ↔ страница ↔ визуализация».
 *
 * npm run verify:code
 *
 * Каждый референсный Python-код из PAGE_SYNC выполняет
 * (pyodide из node_modules) под тем же трейсером, что
 * проверяется:
 * 1. код не падает и не обрывается по лимиту шагов;
 * 2. трасса не пустая (иначе панель показала бы пусто
 * 3. каждая переменная, обещанная странице, реально
 * иначе визуализация не может за ним следовать;
 * 4. в снимках нет «мусорных» значений, которые урон
 */
import * as esbuild from "esbuild";
import { writeFileSync, mkdirSync } from "fs";
import { loadPyodide } from "pyodide";
```

```

const alts = (raw) => raw.split(/[\/[\]],)|\s+или\s+/).map(a => a
```

```
const rows = [];
let problems = 0;
for (const [key, entry] of Object.entries(mod.sync)) {
 if (only.length && !only.some((o) => key.includes(o)))
 printed.length = 0;
 let result;
 try {
 result = JSON.parse(String(await py.runPythonAsync(
 }) catch (error) {
 problems++;
 rows.push({ key, steps: 0, status: `СБОЙ ЗАПУСКА: $
 continue;
 }
 const seen = new Set();
 for (const step of result.steps) for (const name of 0)
 for (const name of Object.keys(result.finalValues ??

const issues = [];
if (result.error) issues.push(`ошибка Python: ${result
if (result.truncated) issues.push("трасса оборвана по
if (result.steps.length === 0) issues.push("нет ни од
const missing = (mod.declared[key] ?? [])
 .map(alts)
 .filter((a) => a.some(ident))
 .filter((a) => !a.some((x) => ident(x) && seen.has(x)))
 .map((a) => a.join(" / "));
if (missing.length) issues.push(`код не создаёт обеща
/*
 * Обещанная переменная не должна быть пустой на ВСЕХ
 * объявил и не наполнил – визуализации нечего показы
 * «защитый» результат вместо работы алгоритма. Именно
 * Косарайю: order был дан готовым списком, а comp та
 */
const isEmpty = (v) =>
 Array.isArray(v) ? v.length === 0
 : typeof v === "string" ? v.length === 0
```

```

process.exit(problems > 0 ? 1 : 0);

```

ФАЙЛ 102/111: scripts/verify/dom-env.mjs

КАТЕГОРИЯ: Скрипты сборки и экспорта | СТРОК: 107 | РАЗМЕР: 1.5 КБ

```

/**
 * Браузерное окружение для проверок: jsdom + те API, к
 * Создаётся ПОСЛЕ загрузки Pyodide (иначе pyodide решит
 */
import { JSDOM, VirtualConsole } from "jsdom";

const OBSERVER_STUB = class {
 observe() {}
 unobserve() {}
 disconnect() {}
 takeRecords() {
 return [];
 }
};

export function createDom({ url = "http://localhost/?to
 const virtualConsole = new VirtualConsole();
 const jsdomErrors = [];
 virtualConsole.on("jsdomError", (error) => {
 jsdomErrors.push(error?.message || String(error));
 onError?.(error);
 });
 // console.error/warn приложения собираем отдельно —
 const consoleErrors = [];
 virtualConsole.on("error", (...args) => consoleErrors
 virtualConsole.on("warn", () => {});

 const dom = new JSDOM(
 `<!doctype html><html lang="ru"><head><meta charset
 <body><div id="root"></div></body></html>`,
```

```

 "HTMLAnchorElement",
 "HTMLDivElement",
 "HTMLSelectElement",
 "Element",
 "Node",
 "Event",
 "CustomEvent",
 "MouseEvent",
 "KeyboardEvent",
 "InputEvent",
 "getComputedStyle",
 "requestAnimationFrame",
 "cancelAnimationFrame",
 "DOMParser",
 "SVGElement",
 "Text",
 "DocumentFragment",
 "NodeList",
 "HTMLCollection",
 "Range",
 "MutationObserver",
 "localStorage",
 "sessionStorage",
 "Blob",
 "File",
 "FileReader",
];
 for (const key of copy) {
 try {
 Object.defineProperty(globalThis, key, { value: w
 } catch {
 /* read-only в Node – не критично */
 }
 }
 globalThis.IS_REACT_ACT_ENVIRONMENT = true;

 return { dom, window, document: window.document, cons
}

```

```

const failures = [];
function check(name, actual, expected) {
 const a = JSON.stringify(actual);
 const e = JSON.stringify(expected);
 if (a === e) passed++;
 else failures.push(`${name}\n          получено: ${a}\n`);
}
const m = (v) => liveMatrix(v);
const shape = (v) => {
 const r = m(v);
 return r ? { rows: r.rows, cols: r.cols, label: r.size };
};
const cells = (v) => {
 const r = m(v);
 if (!r) return null;
 return Array.from({ length: r.rows }, (_, i) => Array);
};

/* — liveMatrix: формы данных -----
check("квадратная 3x3 (список списков)", shape([[1, 2, 3], [4, 5, 6], [7, 8, 9]]));
check("не квадратная 2x4", shape([[1, 2, 3, 4], [5, 6, 7, 8]]));
check("значения ячеек 2x2", cells([[0, 1], [2, 3]]), [[0, 1], [2, 3]]);
check("рванные строки: cols = максимум", shape([[1, 2, 3], [4, 5, 6], [7, 8, 9]]));
check("рванные строки: дырка = null", cells([[1, 2, 3], [4, 5, 6], [7, 8, 9]]));
check("at() вне диапазона → null", m([[1, 2], [3, 4]]).at(10));
check("at() с отрицательным индексом → null", m([[1, 2], [3, 4]]).at(-10));
check("список словарей с числовыми ключами", cells([{ 0: 1, 1: 2 }, { 0: 3, 1: 4 }]));
check("словарь списков", cells({ 0: [1, 2], 1: [3, 4] }));
check("словарь словарей", cells({ 0: { 0: 7, 1: 8 }, 1: { 0: 9, 1: 10 } }));
check('ключи-пары "(i, j)" после снапшота tuple', cells([["0", "1"], ["2", "3"]]));
check('ключи-пары без скобок "0, 1"', cells({ "0, 0": 5 }));

/* — liveMatrix: что матрицей НЕ является -----
check("плоский список чисел → null", m([1, 2, 3, 4, 5]));
check("словарь по именам → null", m({ dist: 1, prev: 2 }));
check("пустой список → null", m([]), null);
check("пустой словарь → null", m({}), null);
check("undefined → null", m(undefined), null);

```

```
/**
 * Зонд состояния панели Python во времени: что происходит
 * Запуск: node tmp/harness/run-probe.mjs
 */
import { act } from "react";
import { createRoot } from "react-dom/client";
import App from "../../src/App";

const sleep = (ms: number) => new Promise((r) => setTimeout(r, ms));
async function settle(ms = 40) {
 await act(async () => {
 await sleep(ms);
 });
}

function panel(): HTMLElement | null {
 return document.querySelector<HTMLElement>('aside[aria-label="panel"]');
}

function editor(): HTMLTextAreaElement | undefined {
 return panel()?.querySelector<HTMLTextAreaElement>("text-area");
}

function footer(): string {
 const divs = Array.from(panel()?.querySelectorAll("div"));
 const f = divs.find((d) => d.className.includes("text-area"));
 return (f?.textContent ?? "").replace(/\s+/g, " ");
}

function state(tag: string) {
 const p = panel();
 const ed = editor();
 const txt = (p?.textContent ?? "").replace(/\s+/g, " ");
 const step = txt.match(/var[^\s]*\s*\d+\s*\//i);
 process.stdout.write(
 `
 [${tag.padEnd(9)}] редактор: ${ed ? "ДА" : "нет"}
 step ? step[0] : "-" | футер: ${footer()}\n`
);
 process.stdout.write(`
 бейдж страницы: ${tag}`);
}
```



```

 if (/Трасса готова|Нет шагов|Пустой код|Ошибка/.test(
 }

```

```

process.stdout.write("\n=== 2. вводим код ТОЛЬКО с кодами
const ed = editor();
if (!ed) {
 process.stdout.write(" редактора нет – жмём Esc и
 await act(async () => {
 document.dispatchEvent(new window.KeyboardEvent("
 await sleep(30);
 });
}
const ed2 = editor();
if (!ed2) throw new Error("редактор так и не появился
typeInto(ed2, "# только комментариев\n# и ещё\n");
state("ввели");
for (const t of [200, 800, 2000, 6000]) {
 await settle(t);
 state(`${t}${t}`);
}

```

```

process.stdout.write("\n=== 3. вводим рабочий код x=4
const ed3 = editor();
if (!ed3) throw new Error("редактор пропал после комм
typeInto(ed3, "x = 41\nx += 1\nprint(x)\n");
state("ввели");
for (const t of [200, 800, 2000, 6000]) {
 await settle(t);
 state(`${t}${t}`);
}

```

```

process.stdout.write("\n=== 4. очищаем редактор полно
const ed4 = editor();
if (ed4) {
 typeInto(ed4, "");
 state("очистили");
 for (const t of [200, 800, 2000]) {
 await settle(t);
 }
}

```

Разбор данных, которые присылает компилятор, в матрицу размерности: список списков, рванные строки, словарь слов, `"(0, 3)"` (так снапшотятся tuple-ключи Python), `"inf"`, числа-строки, обрезка до 64x64. Отдельно – что матрицей список и словарь по именам (`dist`, `prev`): иначе визуализация чужие данные за сетку. Плюс `pickLiveVar` (приоритет имен, (индексы пользователя вне диапазона демо).

### `verify:code` – `check-reference-code.mjs`

Каждый референсный код из `PAGE\_SYNC` выполняется настоящим `sys.settrace`-трейсером, что и панель компилятора. Протрасса не оборвана по лимиту шагов, шагов больше нуля (пустой листинг), и \*\*каждая переменная, обещанная строкой кодом\*\* – иначе визуализация физически не может за ним следовать. Предупреждения интерпретатора, которые иначе по «Результат».

### `verify:dom` – `browser-checks.tsx`

Настоящее приложение монтируется в jsdom (`App` целиком глава, визуализация, панель Python), сценарии кликают к редакторе.

| Группа | Что делает |

| --- | --- |

| А. страницы (22 × 3) | каждая глава монтируется; бейдж автозапуска инициализации настоящим Python не роняет `"[object Object]"`; нет кнопок без имени |

| В. Флойд: размерности (10) | 5x5, 10x10 (больше демо – й список, 80x80 (обрезка), пустая матрица, чужое имя |

| С. матричные страницы (6) | префиксные суммы 2D и раз

| D. графы путей (2) | Дейкстра/Форд-Беллман с вершинами грывания гаснут и оживают обратно |

| Е. эталонный код (46) | прогон кода в CPython → финал е показать мусор |

| F. редактор (6) | код из одних комментариев, синтаксис ляет эталон и возвращает инициализацию; код не перете

| G. lite-режим (1) | `?lite=1` отдаёт текст без навига

| H. доступность (2) | в панели подписаны все кнопки в

```

import { createDom } from "../dom-env.mjs";

const OUT_DIR = "tmp/verify";
const t0 = Date.now();
const jsdomErrors = [];

log("гружаю CPython (pyodide из node_modules)...");
const py = await loadPyodide({ indexURL: "node_modules/" });
log(` Python готов за ${Date.now() - t0} мс`);

log("поднимаю jsdom...");
const { window, document, consoleErrors } = createDom({
 window.__algv0PythonRuntime = py;

log("собираю приложение esbuild'ом.");
mkdirSync(OUT_DIR, { recursive: true });
await esbuild.build({
 entryPoints: ["scripts/verify/browser-checks.tsx"],
 bundle: true,
 outfile: `${OUT_DIR}/bundle.mjs`,
 format: "esm",
 platform: "browser",
 target: "es2022",
 jsx: "automatic",
 sourcemap: true,
 // Vite-специфичные глобалы, которых нет вне dev-серв
 define: {
 "process.env.NODE_ENV": '"development"',
 "import.meta.env.BASE_URL": '"/',
 "import.meta.env.MODE": '"development"',
 "import.meta.env.DEV": "true",
 "import.meta.env.PROD": "false",
 "import.meta.env.SSR": "false",
 },
 loader: { ".css": "empty", ".png": "dataurl", ".jpg":
 logLevel: "warning",
});
const mod = await import(`${process.cwd()}/${OUT_DIR}/b
```

---

ФАЙЛ 107/111: scripts/verify/run-probe.mjs

КАТЕГОРИЯ: Скрипты сборки и экспорта | СТРОК: 45 | РАЗМ

---

```
/**
 * Зонд панели Python: печатает состояние редактора/тра
 * Не тест, а диагностика — когда нужно понять, что пан
 * ввода кода (автозапуск, переход в листинг, сообщения
 *
 * npm run verify:probe
 */
import * as esbuild from "esbuild";
import { mkdirSync } from "fs";
import { loadPyodide } from "pyodide";
import { createDom } from "../dom-env.mjs";

const OUT_DIR = "tmp/verify";
mkdirSync(OUT_DIR, { recursive: true });
const log = (m) => process.stdout.write(`[verify:probe]

log("гружаю CPython...");
const py = await loadPyodide({ indexURL: "node_modules/
log("поднимаю jsdom...");
const { window, document } = createDom();
window.__algv0PythonRuntime = py;

await esbuild.build({
 entryPoints: ["scripts/verify/probe-compiler.tsx"],
 bundle: true,
 outfile: `${OUT_DIR}/probe-bundle.mjs`,
 format: "esm",
 platform: "browser",
 target: "es2022",
 jsx: "automatic",
 sourcemap: true,
```

```
- main
Позволяет проверить Pages прямо из рабочей ветки
- "arena/**"
workflow_dispatch:

permissions:
 contents: read
 pages: write
 id-token: write

concurrency:
 group: pages
 cancel-in-progress: true

jobs:
 build:
 name: Build static site
 runs-on: ubuntu-latest
 steps:
 - name: Checkout
 uses: actions/checkout@v6

 - name: Setup Node.js
 uses: actions/setup-node@v7
 with:
 node-version: 24
 cache: npm

 - name: Install dependencies
 run: npm ci --no-audit --no-fund

 - name: Build for the repository subpath
 env:
 VITE_BASE: /algv0/
 run: npm run build

 - name: Make GitHub Pages fallback
 run: cp dist/index.html dist/404.html
```

-----  
name: Rebuild code PDF

# Пайплайн пересборки PDF при каждом коммите:

```
#
исходный код → full_code_all_pages.txt (script)
|
|→ page-0001.png ... page-NNNN.png (artifacts)
|
|→ full_code_all_page.pdf
#
Готовые TXT и PDF коммитаются обратно в ветку (их отдаёт GitHub)
промежуточные PNG сохраняются как artifacts запуска.
```

on:

push:

branches:

- "\*\*"

paths-ignore:

# чтобы коммит самого workflow не запускал бесконечно

- "public/export/\*\*"

- "\*\*/\*.md"

workflow\_dispatch:

inputs:

density:

description: "DPI растеризации страниц (по умолчанию 300)"

required: false

default: "150"

concurrency:

group: rebuild-pdf- $\{\{\text{github.ref}\}\}$

cancel-in-progress: true

permissions:

contents: write

jobs:

rebuild-pdf:

name: код → txt → png → pdf

- ```
EXPORT_DENSITY: ${github.event.inputs.density}
run: npm run export:png
```
- name: Step 3 – png → pdf
run: npm run export:pdf
 - name: Step 4 – собрать приложение и поднять preview
run: |
 # headless-браузер для снимков: chrome ставится автоматически
 # этот вызов – страховка, если кэш оказался пустым
 npx puppeteer browsers install chrome || true
 npm run build
 mkdir -p tmp
 nohup npx vite preview --host 127.0.0.1 --port 4173 &
 for i in \$(seq 1 40); do
 if curl -sf http://127.0.0.1:4173/ > /dev/null; then
 sleep 1
 fi
 done
 - name: Step 5 – снимки визуализаций (headless-браузер)
run: npm run export:shots
continue-on-error: true
 - name: Step 6 – PDF-конспект со снимками
run: |
 npm run export:guide
 # снимки публикуются вместе с приложением: HTML-снимки
 # в автономный файл прямо из /export/viz-shots
 mkdir -p public/export/viz-shots
 cp tmp/viz-shots/*.png tmp/viz-shots/manifest.json
 - name: Type-check приложения
run: npx tsc --noEmit
continue-on-error: true
 - name: Summary
run: |
 {

```
fi
git commit -m "chore(export): пересборка full
git push origin "HEAD:${GITHUB_REF_NAME}"
```

ФАЙЛ 110/111: .github/workflows/verify.yml
КАТЕГОРИЯ: CI / автоматизация | СТРОК: 71 | РАЗМЕР: 2.3

name: Verify

Проверки приложения на каждом пуше и PR:

```
#
#   verify:unit   – разбор «живых» данных компилятора (l
#   verify:code   – все эталонные Python-коды страниц вы
#                   под трейсером панели и создают обеща
#   verify:dom     – приложение монтируется в jsdom: стра
#                   матрицы чужих размерностей, кнопки,
#
# Headless-браузер не нужен: jsdom + pyodide из node_mo
```

on:

```
  push:
    branches:
      - "**"
    paths-ignore:
      - "public/export/**"
      - "**/*.md"
  pull_request:
  workflow_dispatch:
```

concurrency:

```
  group: verify-${{ github.ref }}
  cancel-in-progress: true
```

permissions:

```
  contents: read
```



```
with:
  name: verify-report
  path: tmp/verify/report.json
  if-no-files-found: ignore
```

РАЗДЕЛ: ПРОЧЕЕ

ФАЙЛ 111/111: .ai_skills/universal_guide_authoring/SKIL
КАТЕГОРИЯ: Прочее | СТРОК: 252 | РАЗМЕР: 13.3 KB

Universal Guide Authoring – шаблоны страниц пособия

Обязательный справочник для ****создания и правки контента репозитории**. На него ссылаются `AGENTS.md` и `GEMINI.md` собираем ****только**** из блоков ниже – не придумываем новый «голый» markdown.

Контент живёт в `src/data/\*\*/\*.ts` как массив объектов `src/data/content.ts` (при одинаковом `id` побеждает бо.

1. Объект главы

```
```ts
import { Chapter } from "../../types";

export const myChapters: Chapter[] = [
 {
 id: "dijkstra", // = ключ VIZ_REGISTRY и
 title: "14. Графы. Поиск кратчайшего пути. Дейкстра",
 type: "html", // всегда "html": страни
 category: "Графы. Пути", // группа в навигации
```

Цвет левой полосы и заголовка блока – семантика раздела

Полоса / цвет	Смысл раздела
---	---
<code>`border-blue-500` + `text-blue-400`</code>	определение, «о»
<code>`border-emerald-500` + `text-emerald-400`</code>	алгоритм,
<code>`border-indigo-500` + `text-indigo-400`</code>	теория, док
<code>`border-amber-500` + `text-amber-400`</code>	ловушки, част
<code>`border-rose-500` + `text-rose-400`</code>	ограничения, «к

---

## ## 3. Библиотека блоков (точные классы)

### ### 3.1 Определение / строгое правило – всегда по центру

```
```html
<div class="bg-slate-900 p-4 rounded-lg mb-6 text-center">
  <p class="text-lg text-emerald-300 italic mb-2">Строг
  <p class="text-xl font-mono text-white">if d[v] > d[u]
</div>
```
```

### ### 3.2 Обычный абзац

```
```html
<p class="text-slate-300 text-sm mb-4">
  Текст с <b>акцентом</b> и инлайн-кодом
  <span class="font-mono text-emerald-300">dist[v]</span>
</p>
```
```

Инлайн-«пилюля» для имени переменной/константы:

```
```html
<span class="bg-slate-950 px-2 py-0.5 rounded text-xs f
```
```

«Когда не работает» – маккуп тот же (пунктирная rose-ра

### ### 3.5 Инфо-блок с таблицей

```
```html
<div class="bg-slate-800/70 p-5 rounded-lg border borde
  <p class="text-sm font-bold text-white mb-2">  Заголо
  <p class="text-slate-300 text-sm mb-3">Пояснение пере
  <table class="w-full text-xs text-left border-collapse
    <thead>
      <tr class="text-slate-500 border-b border-slate-700
        <th class="py-1.5 pr-2 font-bold">Колонка</th>
        <th class="py-1.5 pr-2 font-bold">Колонка</th>
      </tr>
    </thead>
    <tbody>
      <tr class="border-b border-slate-800 align-top">
        <td class="py-2 pr-3 font-bold">Значение</td>
        <td class="py-2 pr-3 font-mono">O(log n)</td>
      </tr>
    </tbody>
  </table>
</div>
```
```

### ### 3.6 Спойлер ``<details>`` – **\*\*обязателен\*\*** для кода, и

```
```html
<details class="bg-slate-900/40 rounded-lg border borde
  <summary class="p-4 font-bold text-slate-400 outline-n
    e-700/50">
    ▶ Код: Дейкстра на куче (Python)
  </summary>
  <div class="p-5 text-sm text-slate-300 space-y-4">
    <pre class="bg-slate-950 p-4 rounded overflow-x-auto
...</code></pre>
    <p class="text-slate-300 text-sm">Комментарий после
  </div>
```

Каждая страница обязана иметь и визуализацию, и синхронизацию.

- ``src/components/vizRegistry.tsx`` – запись с ключом `**demoId`` (``title``, опциональный ``hint``, ``Component``).
- ``src/data/vizSync.ts`` – запись ``PAGE_SYNC[id]`` или ``PAGE_SYNC[demoId]`, где ``demoId`` – второй аргумент ``emitVizDemo(demoId, title)`` (вкладки одной демонстрации).
- Поля записи: ``vizTitle``, ``stepNote`` (что считается одной страницей), ``variables: [{ name, role, range }]``, ``code`` (референс на код).
- Переменные, которые читает визуализация, должны быть одного из типов (``int``, ``list``, ``dict``, ``str``): экземпляры классов снотификатора до графики не доезжают.
- Стартовый шаблон в редакторе компилятора – ``buildInit``, инициализация переменных до строки ``# --- тело алгоритма``.

Проверка перед коммитом: ``npm run dev`` → страница открывшись, подставляет код, шаг демонстрации подсвечивает нужные переменные.

6. Экспорт в PDF (``npm run export:guide``)

Текст страниц парсится ``scripts/export/html-to-blocks.mjs``, результат сохраняется в ``public/export/guide_lite.pdf`` (текст + визуализации, без кода компилятора и без навигации). Чтобы экспорт был полным:

- заголовки – только ``h2/h3/h4``; списки – ``ul/ol/li``; теги кода – ``pre``; определения – обычный ``p`` (см. 3.1);
- ****смысл не должен держаться на эмодзи****: в PDF используются только латинские буквы, эмодзи-глифы – пиктограммы вырезаются, остаётся «Операции на отрезках» доедет как «Операции на отрезках», а вот «Операции на отрезках / Операции на отрезках» потеряет смысл, пиктограммы не вырезаются;
- спойлеры в PDF раскрываются полностью – не прячьте ту часть, которую хотите скрыть, минимум, но и не дублируйте его в основном тексте;
- картинки страниц берутся из ``CHAPTER_IMAGES`` (``scripts/export/html-to-blocks.mjs``), снимки демонстраций – из ``tmp/viz-shots/*.png`` (их референсы в ``src/data/vizSync.ts``); локально без браузера в ``npm run export:shots`` в CI).